

BGéSZC Ganz Ábrahám Két Tanítási Nyelvű Technikum



KoliPortál

Vizsgaremek

Szakma megnevezése:
Szoftverfejlesztő és -tesztelő

Készítette: Kercsó Norbert

Pallai Norbert Zoltán

Témavezető: Szalainé Török Edit

Méhes József György

Csukárdi Rajmund

Szabó Rózsa Terézia

2026

Tartalomjegyzék

1	Bevezetés	11
1.1	Projekt célja	11
1.2	Téma kiválasztásának indoklása	11
1.3	Célközönség	12
1.4	Csoport tagjai és munkamegosztás	13
1.4.1	Közös fejlesztések	13
1.4.2	Fejlesztői egyéni hozzájárulásai	13
2	Fejlesztői dokumentáció	15
2.1	Alkalmazás áttekintése	15
2.2	Felhasznált szoftverek.....	15
2.3	Felhasznált programozási nyelvek	16
2.4	GITHUB	17
2.5	Adatbázis	18
2.5.1	Adatbázis szerkezete.....	18
2.5.2	Táblák részletes leírása	18
2.5.2.1	Felhasználók	18
2.5.2.2	DiakAdatok.....	19
2.5.2.3	Szerepkorok.....	19
2.5.2.4	Szobák	19
2.5.2.5	SzobaBeosztások.....	19
2.5.2.6	Penzugyek	20
2.5.2.7	FizetesTipusok	20
2.5.2.8	KarbantartásiKeresek	20
2.5.2.9	KarbantartásStatuszok	20
2.5.2.10	Kollegium.....	21
2.6	Fő funkciók.....	21
2.6.1	Lakók kezelése	21

2.6.2	Szobabeosztás és kapacitáskezelés.....	21
2.6.3	Fizetések nyilvántartása.....	21
2.6.4	Karbantartási kérések	22
2.6.5	Adminisztráció és jogosultságkezelés	22
2.7	Képernyőtervek	22
2.7.1	Kezdőlap.....	22
2.7.2	Bejelentkezés	23
2.7.3	Adminisztrátor felület – Áttekintés.....	25
2.7.4	Adminisztrátor felület – Felhasználók	25
2.7.5	Adminisztrátor felület – Regisztrálás	26
2.7.6	Adminisztrátor felület – Szobák.....	27
2.7.7	Adminisztrátor felület – Karbantartás	28
2.7.8	Adminisztrátor felület – Beállítások.....	29
2.7.9	Diák/Lakó felület – Áttekintés.....	30
2.7.10	Diák/Lakó felület – Diák/Lakó adatai	31
2.7.11	Diák/Lakó felület – Befizetések.....	31
2.7.12	Diák/Lakó felület – Hibajelentés.....	32
2.8	Projekt struktúra.....	33
2.8.1	Fő könyvtárak.....	34
2.8.1.1	KoliPortal.API (Háttérrendszer / Backend)	34
2.8.1.2	KoliPortal.Lib (Közös Osztálykönyvtár / Shared Library)	35
2.8.1.3	KoliPortal.Web (Kliensoldal / Frontend)	35
2.9	Adatmodellek	35
2.9.1	DiakAdatok.....	36
2.9.2	Felhasznalok	36
2.9.3	FizetesTipusok.....	37

2.9.4	KarbantartasiKeresek.....	37
2.9.5	KarbantartasiStatuszok	37
2.9.6	Kollegium	37
2.9.7	Login	38
2.9.8	Penzugyek	38
2.9.9	Szerepkorok	38
2.9.10	SzobaBeosztasok.....	39
2.9.11	Szobak.....	39
2.9.12	KoliPortalDBContext	39
2.10	Interface.....	40
2.10.1	IDiakAdatok.....	40
2.10.2	IFelhasznalok	41
2.10.3	IFizetesiTipusok.....	41
2.10.4	IKarbantartasiKeresek.....	41
2.10.5	IKarbantartasiStatuszok	42
2.10.6	IKollégium	42
2.10.7	IPenzugyek	42
2.10.8	ISzerepkorok	43
2.10.9	ISzobaBeosztasok.....	43
2.10.10	ISzobak.....	43
2.11	Service	44
2.11.1	DiakAdatokService	46
2.11.2	FelhasznalokService	47
2.11.3	FizetesTipusokService	47
2.11.4	KarbantartasiKeresekService	48
2.11.5	KarbantartasiStatuszokService	48

2.11.6	KollegiumService	49
2.11.7	PenzugyekService	49
2.11.8	SzerepkorService.....	50
2.11.9	SzobaBeosztasokService	50
2.11.10	SzobakService	51
2.11.11	JWTTokenService	51
2.12	Controllers	53
2.12.1	DiakAdatokController	53
2.12.2	FelhasznalokController	54
2.12.3	FizetesTipusokController	55
2.12.4	KarbantartasiKeresekController	56
2.12.5	KarbantartasStatuszokController.....	58
2.12.6	KollegiumController.....	58
2.12.7	PenzugyekController.....	59
2.12.8	SzerepkorokController	59
2.12.9	SzobaBeosztasokController	60
2.12.10	SzobakController.....	61
2.12.11	AuthController	61
2.13	API/appsetting.json.....	63
2.14	API/Program.cs	64
2.15	Koliportal.Lib/Service.....	66
2.15.1	DiakAdatokService	67
2.15.2	FelhasznalokService	68
2.15.3	FizetesTipusokService	68
2.15.4	KarbantartasiKeresekService	69
2.15.5	KarbantartasStatuszokService	70

2.15.6	KollegiumService	70
2.15.7	PenzugyekService	71
2.15.8	SzerepkorokService.....	71
2.15.9	SzobaBeosztasokService	72
2.15.10	SzobakService	72
2.16	Kezdolap.razor	73
2.16.1	Útválasztás és Keretbeágyazás	73
2.16.2	Navigáció és Hero Szekció.....	73
2.16.3	Funkciók Bemutatása.....	74
2.16.4	Kapcsolat és Lábléc.....	75
2.17	Login.razor	75
2.17.1	Direktívák és Szolgáltatás-injekciók	75
2.17.2	Felhasználói Felület és Kétirányú Adatkötés.....	76
2.17.3	Az Üzleti Logika és az API Kommunikáció.....	78
2.17.4	Szerepkör-alapú Útválasztás	79
2.18	Adatvedelem.razor	80
2.19	AdminLayout.razor	81
2.19.1	Függőségek és Alaposztály	81
2.19.2	Felhasználói Felület és Navigáció.....	82
2.19.3	Kliensoldali Útvonalvédelem	84
2.20	LakoLayout.razor.....	85
2.20.1	Alapinjektálások és Függőségek	85
2.20.2	Dedikált Felhasználói Felület	86
2.20.3	Szigorított Kliensoldali Útvonalvédelem.....	87
2.21	MainLayout.razor.....	88
2.22	Admin/Attekintes.razor.....	88

2.22.1	Direktívák, Injektálások és Renderelési Mód.....	89
2.22.2	Állapotkezelés és Vizuális Visszajelzések	90
2.22.3	Változók és Segédosztályok deklarálása	92
2.22.4	Az Adatelemző Motor	93
2.23	Felhasználók.razor	94
2.23.1	Komponens Direktívák és Injektálások	95
2.23.2	Vizuális Visszajelzések és Állapotkezelés	95
2.23.3	Kereső, Szűrő és Oldalméret-választó Szekció	96
2.23.4	Dinamikus Adattábla.....	97
2.23.5	Lapozó Sáv	98
2.23.6	Komplex Szerkesztő Modális Ablak.....	98
2.23.7	Állapotkezelés és Memóriastruktúra	100
2.23.8	Hitelesítés és Aszinkron Adatbetöltés	101
2.23.9	Kétlépcsős Szűrési Algoritmus	102
2.23.10	Matematikai Lapozás	103
2.23.11	A Modális Ablak Állapotkezelése	104
2.23.12	Szigorú Kaszkádolt Törlés	105
2.23.13	Komplex Mentés és Szobabeosztási Logika	106
2.23.14	Kliensoldali Segédmetódusok.....	108
2.23.15	Adatcsomagoló Modell.....	110
2.24	Regisztrálás.razor	110
2.24.1	Környezet és Alapadatok Betöltése	110
2.24.2	Dinamikus és Feltételes Felhasználói Felület.....	112
2.24.3	Kliensoldali Validációs Motor	114
2.24.4	Komplex Mentési Folyamat	115
2.24.5	Kliensoldali Segédmetódusok és Kapacitásvizsgálat	117

2.25	Szobak.razor.....	118
2.25.1	Alapok és Injektált Szolgáltatások.....	118
2.25.2	Dinamikus Szűrő és Keresőfelület.....	119
2.25.3	Az Adattábla és az Állapotjelzők.....	120
2.25.4	Biztonság és Aszinkron Adatbetöltés.....	121
2.25.5	A Szűrési Algoritmus Motorja.....	122
2.25.6	Matematikai Lapozás.....	123
2.25.7	Intelligens Segédmetódusok.....	124
2.26	Karbantartas.razor	124
2.26.1	Függőségek és Alapkonfiguráció.....	125
2.26.2	Kétpaneles Adatmegjelenítés.....	126
2.26.3	Állapotkezelő Modális Ablak.....	128
2.26.4	Adatbetöltés és LINQ Alapú Szortírozás	129
2.26.5	Aszinkron Mentés, Törlés és UX Finomhangolások.....	131
2.26.6	Dinamikus Segédmetódusok	132
2.27	Penzugyek.razor.....	132
2.27.1	Komponens Direktívák és Szolgáltatások.....	133
2.27.2	Valós Idejű Statisztikák és Szűrőfelület.....	133
2.27.3	Dinamikus Adattábla és Formázás.....	134
2.27.4	A Tömeges Díjkiíró Modális Ablak	135
2.27.5	Adatbetöltés és Memória-statisztikák.....	136
2.27.6	Állapotfüggő Rendezés	138
2.27.7	A Tömeges Díjkiírás Algoritmus.....	139
2.28	Beallitas.razor	140
2.28.1	Keretrendszer és Felhasználói Felület.....	140
2.28.2	Validációs Motor és API Kommunikáció	141

2.28.3	Biztonsági Kapu és Adatcsomagoló Osztály.....	142
2.29	Lako/Attekintes.razor	143
2.29.1	Keretrendszer és Moduláris Függőségek.....	143
2.29.2	Személyre Szabott UI és Dinamikus KPI Kártyák.....	144
2.29.3	Szigorú Biztonság és Kliensoldali Identitás	146
2.29.4	Segédmetódusok	147
2.30	Adataim.razor	147
2.30.1	Függőségek és Felhasználói Környezet	148
2.30.2	Adatvédelem a Felhasználói Felületen	148
2.30.3	Identitás Alapú Adatlekérés.....	150
2.30.4	Szigorú Kliensoldali Validáció és Mentési Algoritmus	151
2.31	Befizetések.razor.....	152
2.31.1	Keretrendszer és Felhasználói Felület.....	152
2.31.2	Intelligens Adattábla és Gyorsszűrők.....	154
2.31.3	Fizetési Szimuláció	155
2.31.4	Biztonság és Kliensoldali Identitás.....	156
2.32	Hibajelentes.razor	157
2.32.1	Keretrendszer és Moduláris Függőségek.....	157
2.32.2	Hibabejelentő Űrlap és Visszajelzések.....	158
2.32.3	Személyes Archívum	159
2.32.4	Biztonság és Kliensoldali Identitás.....	160
2.32.5	Mentési Logika és Adatfeldolgozás.....	161
2.33	Web/Program.cs	162
2.34	Fejlesztési lehetőségek	163
3	Felhasználói dokumentáció	165
3.1	A program célja és rövid ismertetése	165

3.2	Rendszerkövetelmények	165
3.2.1	Hardverkövetelmények.....	165
3.2.2	Szoftverkövetelmények.....	165
3.3	Telepítés és indítás lépéseinek ismertetése	166
3.4	Regisztráció nélkül elérhető funkciók (Kezdőlap)	166
3.5	Bejelentkezés a rendszerbe és beviteli szabályok.....	167
3.6	A Diák (Lakó) felhasználói felület részletes bemutatása.....	169
3.6.1	Áttekintés.....	170
3.6.2	Pénzügyek	170
3.6.3	Hibajelentés modul és beviteli szabályok.....	171
3.7	Az Adminisztrátor felhasználói felület részletes bemutatása.....	174
3.7.1	Adminisztrátori Navigációs Oldalsáv.....	174
3.7.2	Áttekintés.....	176
3.7.3	Felhasználók	178
3.7.4	Regisztrálás	187
3.7.5	Szobák.....	193
3.7.6	Karbantartás	196
3.7.7	Penzügyek.....	198
3.7.8	Beállítások.....	203
4	Összefoglalás.....	205
4.1	Elkészült projektünk értékelése	205
4.1.1	Kihívások és tanulságok.....	205
4.1.2	Erős alapok és fejlesztési lehetőségek.....	205
4.1.3	A projekt utóélete.....	206
4.2	Köszönetnyilvánítás	206
5	Irodalomjegyzék.....	207

5.1	Szakmai Fórumok és Problémamegoldó Közösségek (StackOverflow)	207
5.2	Frontend, UI Design és Webes technológiák (W3Schools & Bootstrap)	207
5.3	Hivatalos Microsoft Dokumentációk	208
5.4	C# Backend és ASP.NET Core Web API.....	208
5.5	Adatbázis-kezelés: Entity Framework Core.....	208
5.6	Frontend Logika és Blazor WebAssembly/Server	209

1 BEVEZETÉS

1.1 Projekt célja

A projekt elsődleges célja egy olyan egységes kollégiumimenedzsment rendszer kifejlesztése, amely hatékonyan kezeli a kollégium lakóinak mindennapi életét és az adminisztrációs feladatokat.

Az általunk fejlesztett program kifejezetten a kollégiumosok életének könnyebbé tételére fókuszál, és lehetőséget biztosít a kollégium vezetőségének adminisztrációs feladataira. Weboldalunk kiemelt figyelmet fordít az átláthatóságra, így felhasználóbarátabb és hatékonyabb élményt kínál a felhasználók számára.

1.2 Téma kiválasztásának indoklása

Egy szoftveres projekt megkezdése előtt elengedhetetlen a már létező megoldások vizsgálata. Jelenleg a magyarországi oktatási intézmények, technikumok és egyetemek túlnyomó többsége komplex, integrált tanulmányi rendszereket (leggyakrabban a Neptun egységes tanulmányi rendszert és annak kollégiumi modulját) használja a diákok adminisztrációjára. Ezen felül a kisebb intézményekben még mindig gyakori a papíralapú, vagy decentralizált (pl. külön Excel táblázatokban vezetett) nyilvántartás. A KoliPortál megtervezését ezen rendszerek hiányosságainak és gyakorlati korlátainak felismerése indokolta.

Miben más, és miért volt szükség a KoliPortál elkészítésére?

1. **Fókuszáltság és funkcionális letisztultság:** A nagy, integrált tanulmányi rendszerek (mint a Neptun) moduláris felépítésűek, de elsődlegesen az oktatásszervezésre lettek optimalizálva. Emiatt a kollégiumi funkcióik gyakran eldugottak, a felhasználói felületük (UI) pedig elavult és túlzsúfolt. A KoliPortál ezzel szemben egy "cél-szoftver": kizárólag azokat a funkciókat (szobabeosztás, kollégiumi díjak, hibabejelentés) tartalmazza, amelyekre egy kollégium mindennapi üzemeltetése során ténylegesen szükség van. Ez a letisztultság drasztikusan csökkenti a betanulási időt mind az adminisztrátorok, mind a diákok számára.

2. **Központosított és azonnali hibabejelentés:** Számos intézményben a műszaki problémák (eldugult lefolyó, kiégett izzó) bejelentése még mindig egy, a portán elhelyezett papíralapú füzetben történik, vagy e-mailek formájában vész el az adminisztrációban. A KoliPortál egyik legnagyobb innovációja a dedikált, kategóriákra bontott digitális hibajegy-rendszer. A diákok mobiltelefonról, másodpercek alatt leadhatják a problémát, amit az adminisztráció (és a karbantartás) valós időben, strukturáltan lát, így a hibaelhárítási folyamat nyomon követhetővé és mérhetővé válik.
3. **Pénzügyi átláthatóság a diákok felé:** A diákok számára gyakran okoz frusztrációt, hogy nem látják át egyértelműen az aktuális kollégiumi tartozásaikat vagy a fizetési határidőket. A rendszerünk Lakói moduljában a felhasználók egy letisztult "műszerfal" (dashboard) azonnal látják a kintlévőségeiket és a fizetési előzményeiket, ami növeli a fizetési hajlandóságot és csökkenti az intézmény adminisztrációs terheit (pl. kevesebb felszólítást kell küldeni).
4. **Agilis szobakezelés:** Míg a meglévő rendszerekben egy szobacsere vagy kiköltözés adminisztrálása gyakran több lépcsős, bürokratikus folyamat, a KoliPortálban egyetlen szerkesztőablakból, vizuálisan követve a szabad ágyak számát (telítettségi százalékokkal támogatva) oldható meg a be- és kiköltöztetés, méghozzá úgy, hogy a rendszer a háttérben automatikusan kezeli a lakhatási előzményeket (history).
5. **Modern, reszponzív technológia:** Ellentétben a régebbi, sok esetben csak asztali gépre optimalizált rendszerekkel, a KoliPortál a "Mobile-First" (mobilra tervezett) elvet követi a Bootstrap 5 keretrendszer használatával. Mivel a diákok ma már az ügyintézéseik 90%-át okostelefonon végzik, alapvető elvárás volt egy reszponzív, bármilyen képernyőméreten tökéletesen használható webes applikáció létrehozása.

1.3 Célközönség

A KoliPortál célközönsége alapvetően két nagy csoportra, a kollégiumot üzemeltető személyzetre és az ott lakó diákokra bontható. Az intézmény vezetősége és az adminisztrátorok számára a szoftver egy átlátható, központosított felületet kínál a szobabeosztások, a pénzügyi kötelezettségek, a felhasználói adatok hatékony kezelésére, valamint a felmerülő műszaki problémák és hibaelhárítási folyamatok valós idejű

menedzselésére. Végül, de nem utolsósorban a kollégista diákok egy modern, mobilbarát felületet kapnak, ahol nyomon követhetik saját fizetési egyenlegüket, saját személyes adataikat, és pillanatok alatt, papírmunka nélkül adhatnak le hibajelentéseket.

1.4 Csoport tagjai és munkamegosztás

A projektet **Kercsó Norbert** és **Pallai Norbert Zoltán** készítette. A fejlesztés során a feladatokat igyekeztünk egyenlően elosztani.

1.4.1 Közös fejlesztések

A projekt kezdeti fázisában közösen fektettük le az alkalmazás alapjait, hogy mindkét modul zökkenőmentesen tudjon együttműködni. Közös munkánk eredményei:

- **Rendszerarchitektúra és adatbázis-tervezés:** A relációs adatbázis sémájának, a táblák kapcsolatainak (felhasználók, szobák, beosztások, pénzügyek, hibajegyek) és a C# adatmodelleknek a megtervezése.
- **Autentikáció és biztonság:** A JWT (JSON Web Token) alapú bejelentkezési rendszer közös implementálása, amely a tokenből kinyert jogosultság (Role) alapján irányítja a felhasználókat a megfelelő (Lakó vagy Admin) felületre.
- **Alapvető UI/UX elvek lefektetése:** A Blazor komponensek struktúrájának és a közös CSS stíluslapoknak (pl. gombok, kártyák, táblázatok dizájnja) a kialakítása a konzisztens felhasználói élmény érdekében.

1.4.2 Fejlesztői egyéni hozzájárulásai

Kercsó Norbert – Adminisztrátori modul fejlesztése: Kercsó Norbert feladata a komplex háttérfolyamatok, a rendszer globális beállításainak és a felhasználók adatainak menedzselését végző adminisztrátori felületek megalkotása volt. Az adminisztrátori felület teljes körű megvalósítása.

- **Felhasználók teljes körű kezelése (/felhasznalok és /regisztralas):** Fejlett, lapozható és szűrhető (összes/bent lakó/kiköltözött) listanézet kialakítása. Megvalósította az új diákok, adminok és karbantartók regisztrációját, a szobabeosztások dinamikus módosítását (várható kiköltözés automatikus számításával), valamint a biztonságos, kaszkádolt adatbázis-törlést.

- **Szobakapacitás és telítettség felügyelete (/szobak):** A kollégiumi férőhelyek valós idejű, vizuális (telítettséget százalékosan és színkóddal jelző) áttekintését biztosító oldal lefejlesztése, szabad ágyak szerinti szűrési lehetőséggel.
- **Pénzügyi adminisztráció (/penzugyek):** Globális pénzügyi statisztikák (összes kintlévőség vs. befizetett tételek) megjelenítése. Létrehozta a "Tömeges díjkiírás" funkciót, amely egyetlen gombnyomással generál fizetendő tételeket minden aktív lakó számára a megadott jogcím és határidő alapján.
- **Rendszerbeállítások kezelése:** Az alkalmazás működését befolyásoló globális "kapcsolók" (pl. automatikus értesítések, karbantartási mód) logikájának integrálása.

Pallai Norbert Zoltán – Lakó (Diák) modulok fejlesztése:

Pallai Norbert felelt a rendszer azon felületeinek kialakításáért, amelyeket a kollégisták a mindennapok során használnak. Célja egy letisztult, mobilbarát és felhasználóközpontú élmény megteremtése volt. A diákok számára készült profiloldal és saját adatok kezelése.

- **Személyes profil és adatkezelés (/adataim):** A saját adatok és az aktuális szobabeosztás megjelenítése, valamint az e-mail cím és a telefonszám szerkesztésének megvalósítása biztonságos validációval (pl. formátum-ellenőrzés).
- **Pénzügyi modul lakóknak (/befizetesekLako):** Egy áttekinthető műszerfal (dashboard) fejlesztése, ahol a diákok láthatják az aktuális tartozásaikat és a már befizetett tételeiket. Implementálta a lapozható (paginált) táblázatot és a számlák "Fizetés" gombbal történő szimulált kiegyenlítését.
- **Hibabejelentő rendszer (/hibajelentes):** A karbantartási kérések rögzítésére szolgáló űrlap elkészítése. Olyan életszerű, specifikus kategóriákat (pl. Vízvezeték és szaniter, Internet és hálózat) épített be, amelyek segítik a problémák gyors kategorizálását, majd ezek mentését a közös adatbázisba.

2 FEJLESZTŐI DOKUMENTÁCIÓ

2.1 Alkalmazás áttekintése

A KoliPortál egy kollégiumi menedzsment és adminisztrációs weboldal, amely C# és ASP.NET Core technológiával készült. Az alkalmazás .NET 8 keretrendszert használ, és két fő felhasználói szerepkört támogat: lakó és adminisztrátor.

2.2 Felhasznált szoftverek

A fejlesztési folyamat során az alábbi szoftvereket használtuk különböző feladatokra:

- **Microsoft Windows 11**

A fejlesztéshez használt elsődleges operációs rendszer. Ez a modern és stabil környezet biztosította a megbízható alapot a teljes fejlesztési munkafolyamathoz. Mivel a projekt a Microsoft technológiai ökoszisztémájára (.NET, C#, MS SQL) épül, a Windows 11 garantálta a legmagasabb szintű kompatibilitást és a fejlesztői eszközök (Visual Studio 2022, SQL Server) optimális, teljesítményvesztés nélküli futtatását. Emellett a beépített biztonsági funkciók és a natív lokális webservertámogatás (Kestrel/IIS Express) gördülékennyé tették a kódok folyamatos tesztelését.

- **Visual Studio 2022**

A Blazor WebAssembly Standalone App és az ASP.NET Core Web API fejlesztése C# nyelven történt, Visual Studio 2022-ben. Ez a fejlesztői környezet számos beépített eszközt biztosít a hatékony backend- és frontend-fejlesztéshez, például kódlépésenkénti hibakeresést, NuGet csomagkezelést, valamint a .NET projektek könnyű kezelhetőségét.

- **SQL Server Management Studio 21**

Az adatbázisok létrehozásához, kezeléséhez és karbantartásához az SQL Server Management Studio 21 verzióját használtuk Microsoft SQL Server (MSSQL) adatbázis-kezelővel. Az MSSQL választása azért történt, mert erős integrációt biztosít a .NET alapú alkalmazásokkal, kiváló teljesítményt nyújt nagyobb adatmennyiségek kezelésekor tranzakciókezelési funkciókkal rendelkezik. Ezzel a szoftverrel hatékonyan kezelhetők az SQL szerverek, lekérdezések

futtathatók, adatbázisok módosíthatók, valamint optimalizálhatók az adatok tárolására és lekérdezésére szolgáló folyamatok.

2.3 Felhasznált programozási nyelvek

- **C#:** A Blazor WebAssembly alkalmazás és az ASP.NET Core Web API fejlesztéséhez használt, objektumorientált programozási nyelv, amely hatékony és biztonságos backend megoldásokat biztosít. Beépített eszközeivel hihetetlenül gyorsan és biztonságosan tudtuk leprogramozni a komplex funkciókat (pl. a JWT alapú felhasználó-azonosítást, a szobabeosztások dátumkalkulációit és a jelszavak titkosítását).
- **Blazor (Interactive Server mód) és Razor szintaxis:** A frontend (kliensoldal) fejlesztéséhez a Microsoft Blazor technológiáját választottuk. A Blazor legnagyobb előnye, hogy a felhasználói felület interaktivitását nem JavaScriptben, hanem ugyanabban a C# nyelven írhattuk meg, mint a szerveroldalt. A .razor fájlokban a HTML kód és a C# logika zökkenőmentesen keverhető, így egyetlen, egységes kódbázisban tudtunk dolgozni, ami felgyorsította a fejlesztést és a hibakeresést.
- **HTML5, CSS3 és Bootstrap 5:** A weboldal vizuális vázát és formázását a standard HTML5 és CSS3 nyelvekkel végeztük, kiegészítve a Bootstrap 5 keretrendszerrel. Bootstrap beépített "Grid" (rácsos) rendszere és előre megírt osztályai (pl. kártyák, gombok, felugró ablakok) lehetővé tették, hogy hetekig tartó CSS írás helyett napok alatt egy modern, letisztult, és ami a legfontosabb: teljesen reszponzív (mobilbarát) felületet hozzunk létre.
- **MSSQL (Microsoft SQL Server Query Language):** Az adatbázisok kezeléséhez, lekérdezések futtatásához és adatok manipulálásához használt SQL dialektus a Microsoft SQL Server környezetben. A kollégiumi rendszer adatai (diákok, szobák, pénzügyek) szorosan összefüggnek. Az SQL nyelv és a relációs adatbázis (az idegen kulcsok és megszorítások révén) garantálja, hogy ne lehessen érvénytelen adatokat rögzíteni (például ne lehessen valakit olyan szobába beosztani, ami nem is létezik).
- **JavaScript (JS Interop):** Bár a Blazor miatt a logika 95%-a C#-ban íródott, minimális szinten használtunk JavaScriptet is. Erre kizárólag azért volt szükség, hogy a böngésző natív funkcióit elérjük, például a bejelentkezéskor kapott

biztonsági JWT tokenet el tudjuk menteni a böngésző helyi tárhelyébe (localStorage).

2.4 GITHUB

A KoliPortál projekt zökkenőmentes lebonyolítása és a párhuzamos munkavégzés összehangolása érdekében felhő alapú verziókövetést alkalmaztunk a **GitHub** platformján. Az egységes, hivatalos hozzáférés és az egyszerű azonosítás biztosítása céljából a projektben részt vevő fejlesztők a saját, **@ganziskola.hu** végződésű intézményi e-mail címükkel csatlakoztak a fejlesztői környezethez.

A közös munkához egy dedikált kódtárat (repository-t) hoztunk létre **KoliPortal** néven, amely a bírálók számára is megtekinthető az alábbi hivatkozáson: <https://github.com/pallainorbertzoltan/KoliPortal>

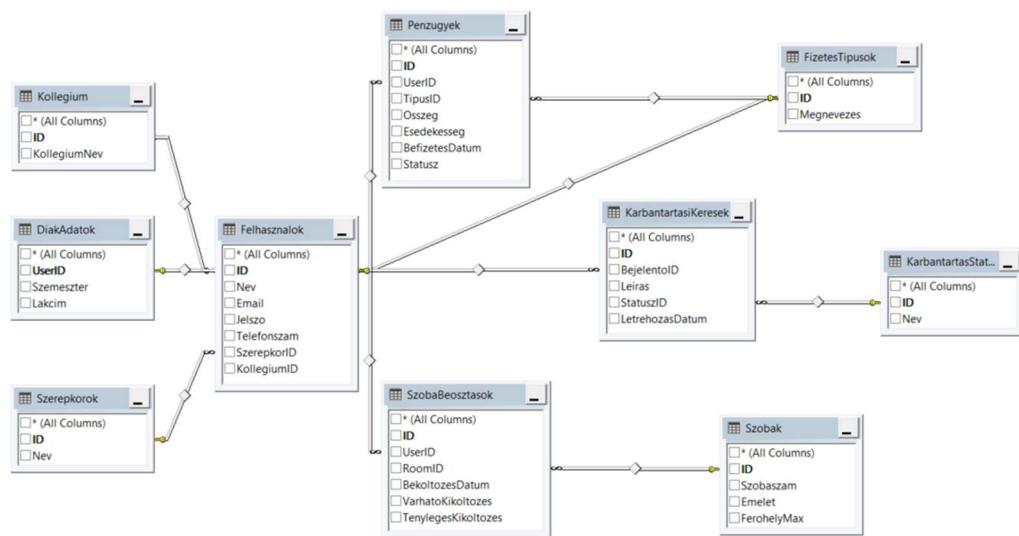
Ebben a központi tárhelyben minden csapattag rögzítésre került megfelelő jogosultságokkal. A repository egyetlen, biztonságos helyen fogja össze a projekt megvalósítása során létrejött összes állományt: az alkalmazás teljes C# és Blazor forráskódját, az adatbázis-szkripteket, valamint a hivatalos dokumentációkat.

A fejlesztési munkafolyamat során a fejlesztők a saját lokális környezetükben (saját számítógépükön) dolgoztak a rájuk bízott modulokon. Az elkészült és tesztelt kódváltoztatásokat (commitokat) rendszeresen feltöltötték (push) a közös GitHub tárhelyre. Ez a strukturált megközelítés garantálta a kódváltozások másodpercre pontos nyomon követhetőségét, a folyamatos verziókövetést, és lehetővé tette a KoliPortál különböző logikai részeinek (az Admin és a Lakó moduloknak) a hibamentes, csapatszintű integrációját.

2.5 Adatbázis

2.5.1 Adatbázis szerkezete

A rendszer adatait úgy terveztük meg, hogy minden logikusan, külön csoportokban legyen tárolva. A tervezés során a normalizálás elveit követtük, hogy minimalizáljuk az adatismétlődést. Az adatbázis központi eleme a Felhasználók tábla, amelyhez a többi funkcionális modul (lakhatás, pénzügyek, karbantartás) kapcsolódik. A statikus adatokat (szerepkörök, státuszok) külön táblákban helyeztük el a könnyebb karbantarthatóság érdekében.



2.5.2 Táblák részletes leírása

2.5.2.1 Felhasználók

A rendszer összes felhasználójának közös adatait tároló tábla.

Osztóp	Típus	Leírás
ID	INT (IDENTITY)	A felhasználó egyedi azonosítója
Nev	NVARCHAR(100)	A felhasználó teljes neve
Email	NVARCHAR(100)	E-mail cím, egyben a belépési azonosító
Jelszo	NVARCHAR(MAX)	Titkosított jelszó (Hash)
Telefonszam	NVARCHAR(20)	Kapcsolattartási telefonszám
SzerepkorID	INT	Hivatkozás a Szerepkorok táblára

2.5.2.2 DiakAdatok

A kollégiumi lakók kiegészítő adatai:

Oszlop	Típus	Leírás
UserID	INT (IDENTITY)	Kapcsolat a Felhasznalok táblához
Szemeszter	NVARCHAR(50)	A hallgató aktuális féléve
Lakcim	NVARCHAR(255)	A diák állandó lakcíme

2.5.2.3 Szerepkorok

A felhasználói jogosultság szintek:

Oszlop	Típus	Leírás
ID	INT (IDENTITY)	Egyedi azonosító, elsődleges kulcs
Nev	NVARCHAR(50)	Szerepkör neve (pl. Admin, Lakó)

2.5.2.4 Szobak

A kollégium szobáinak nyilvántartása:

Oszlop	Típus	Leírás
ID	INT (IDENTITY)	A szoba azonosítója
Szobaszam	NVARCHAR(10)	A szoba ajtószáma (pl. "101")
Emelet	INT	Az emelet száma
FerohelyMax	INT	A szoba maximális kapacitása (fő)

2.5.2.5 SzobaBeosztások

A diákok és szobák összerendelése:

Oszlop	Típus	Leírás
ID	INT (IDENTITY)	A beosztás azonosítója
UserID	INT (FK)	A beköltöző diák azonosítója
RoomID	INT (FK)	A kiválasztott szoba azonosítója
BekoltozesDatum	DATE	A lakhatás kezdete
VarhatoKikoltozes	DATE	Szerződés szerinti vége
TenylegesKikoltozes	DATE	Korábbi kiköltözés esetén kitöltve

2.5.2.6 Penzugyek

Fizetési kötelezettségek és befizetések:

Oszlop	Típus	Leírás
ID	INT (IDENTITY)	Tranzakció azonosítója
UserID	INT (FK)	A fizetésre kötelezett diák azonosítója
TipusID	INT (FK)	Fizetés típusának az azonosítója
Osszeg	INT	Fizetendő összeg
Esedekesseg	DATE	Fizetési határidő
BefizetesDatum	DATE	A befizetés dátuma
Statusz	NVARCHAR(20)	Állapot (pl. Fizetett)

2.5.2.7 FizetesTipusok

Segéd tábla a fizetési jogcímekhez:

Oszlop	Típus	Leírás
ID	INT (IDENTITY)	Egyedi azonosító
Megnevezes	NVARCHAR(50)	Típus neve (pl. Lakbér)

2.5.2.8 KarbantartasiKeresek

Hibabejelentések kezelése:

Oszlop	Típus	Leírás
ID	INT (IDENTITY)	Hibajegy azonosítója
BejelentőID	INT (FK)	A hibát bejelentő felhasználó
Leiras	NVARCHAR(MAX)	A probléma részletes leírása
StatuszID	INT (FK)	Hivatkozás a KarbantartasStatuszok táblára
LetrehozásDatum	DATE	A bejelentés ideje

2.5.2.9 KarbantartasStatuszok

A hibajegy lehetséges állapotai:

Oszlop	Típus	Leírás
ID	INT (IDENTITY)	Egyedi azonosító
Nev	NVARCHAR(50)	Státusz neve (pl. Új, Kész).

2.5.2.10 Kollegium

Oszlop	Típus	Leírás
ID	INT (IDENTITY)	Egyedi azonosító
KollegiumNev	NVARCHAR(50)	Kollégium neve

2.6 Fő funkciók

2.6.1 Lakók kezelése

Hallgatók nyilvántartását és adatainak menedzselését végzi.

- **Személyes adatok kezelése:** A rendszerben rögzítésre kerülnek a lakók alapvető információi, mint a név, évfolyam és elérhetőségek. Ez biztosítja a pontos nyilvántartást és a kapcsolattartás lehetőségét.
- **Szoba-hozzárendelés:** A lakók profiljához közvetlenül kapcsolódik a lakhatási információ. Az adminisztrátorok itt rendelik össze a diákot a kiválasztott szobával.
- **Időbeli nyilvántartás:** A rendszer pontosan tárolja a beköltözés dátumát és a várható kiköltözés idejét, ami elengedhetetlen a férőhelyek tervezhetősége szempontjából.

2.6.2 Szobabeosztás és kapacitáskezelés

Ez a funkció biztosítja az épület erőforrásainak (szobák) logikus kezelését.

- **Szobák paraméterei:** Minden szobáról tároljuk a fizikai jellemzőket: szobaszám, emelet, és a maximális férőhelyek száma.
- **Lakók hozzárendelése:** Az adminisztrátori felületen keresztül történik a lakók elhelyezése az adott szobákba.
- **Automatikus kapacitás ellenőrzés:** A rendszer hozzárendeléskor automatikusan összeveti az aktuális létszámot a maximális férőhellyel, és hibaüzenetet ad, ha a szoba betelt.

2.6.3 Fizetések nyilvántartása

A pénzügyi modul átláthatóvá teszi a kollégiumi díjak kezelését.

- **Fizetési típusok:** A rendszer képes kezelni különböző fizetési jogcímeket, mint például a havi lakbér vagy az esetleges közös költségek.

- **Befizetések rögzítése:** Az adminisztrátorok rögzíthetik a beérkezett összegeket. A rendszer nyilvántartja a fizetési státuszt (pl. rendezett vagy elmaradt), amit a diákok is nyomon követhetnek.

2.6.4 Karbantartási kérések

- **Hibabejelentés:** A lakók saját felületükön keresztül rögzíthetik a felmerülő problémákat (pl. csöpögő csap, elektromos hiba).
- **Feladatkezelés:** A karbantartók láthatják a rájuk bízott, vagy a rendszerben lévő nyitott feladatokat, és elvállalhatják azokat.
- **Visszajelzés:** A munka elvégzése után a státusz módosul, amiről a lakó visszajelzést kap a felületen, így látja, hogy a probléma megoldódott.

2.6.5 Adminisztráció és jogosultságkezelés

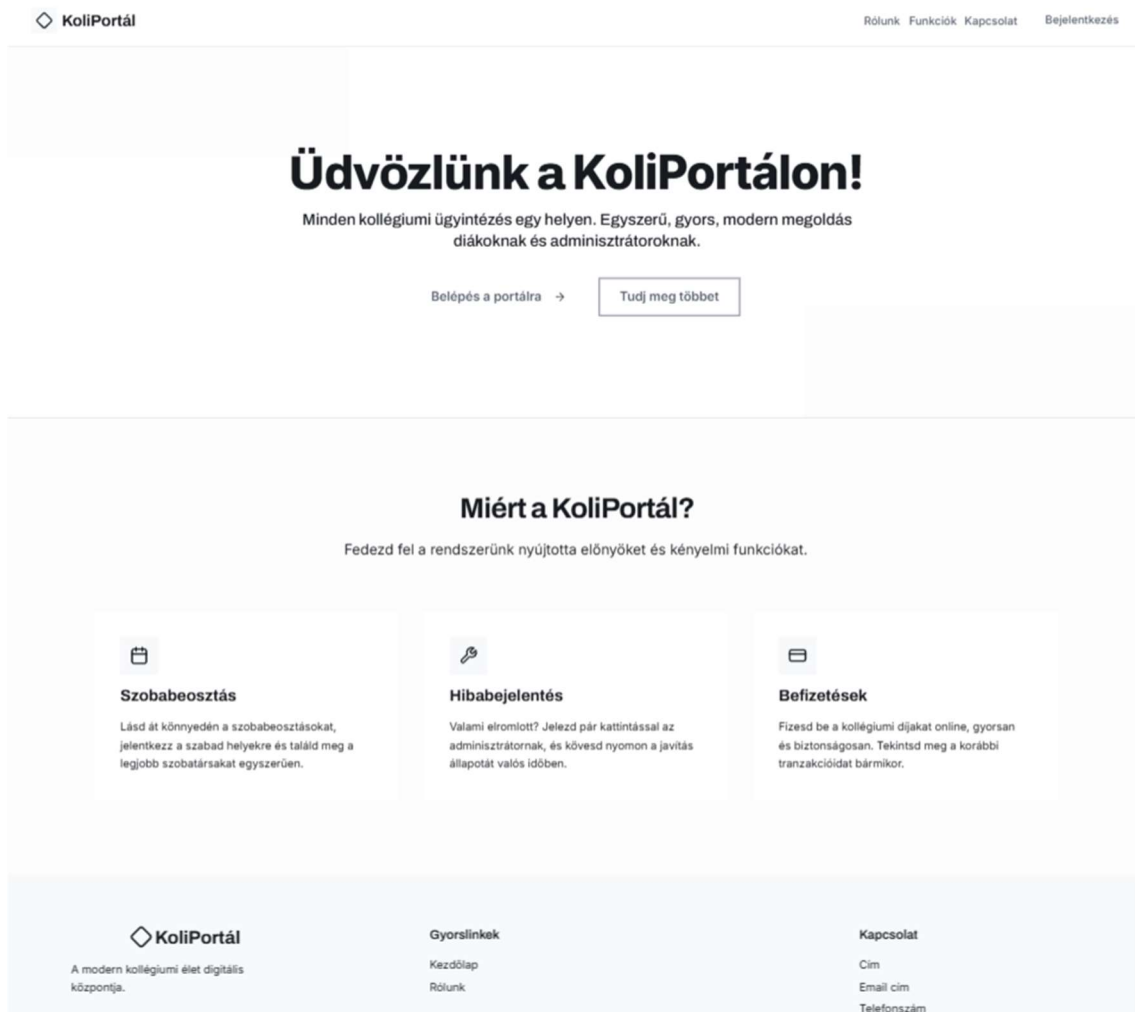
- **Adminisztrátor:** Teljes körű hozzáféréssel rendelkezik a rendszer minden eleméhez (felhasználók, szobák, pénzügyek, karbantartás szerkesztése).
- **Lakó:** Korlátozott jogosultságú felhasználó. Kizárólag a saját adatait tekintheti meg vagy szerkesztheti, karbantartási kérést küldhet be, valamint ellenőrizheti saját fizetési státuszát.

2.7 Képernyőtervek

A drótvázak tervezésekor kiemelt figyelmet fordítottunk az átláthatóságra és a felhasználóbarát elrendezésre. Munkánk során az interneten található modern weboldalak letisztult stílusából indultunk ki, és azokat formáltuk át a saját igényeinkre. Az alábbiakban bemutatjuk ezeket a terveket szerepkörökre bontva.

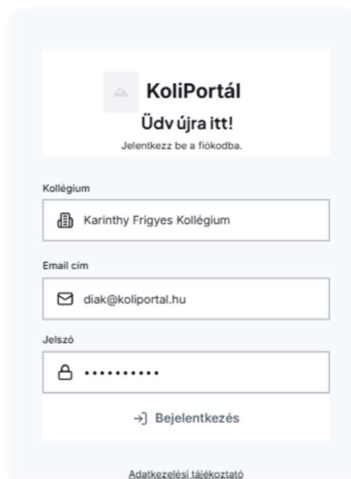
2.7.1 Kezdőlap

Ez a képernyő funkcionál a KoliPortál rendszer elsődleges belépési pontjaként (érkezési oldal). Elsődleges célja a rendszer iránt érdeklődők tájékoztatása, valamint a meglévő felhasználók (diákok és adminisztrátorok) gyors és egyértelmű továbbirányítása a bejelentkezési felületre.



2.7.2 Bejelentkezés

Ez a képernyő képezi a KoliPortál biztonsági kapuját: itt történik a felhasználók (diákok és adminisztrátorok) azonosítása (autentikációja), és ez biztosítja a zárt, jogosultságokhoz kötött funkciókhoz való hozzáférést.



Rendszer tesztelése és Hitelesítő adatok

A vizsgabizottság számára a rendszer funkcióinak teljes körű teszteléséhez az adatbázist előre feltöltöttük mintaadatokkal. Kérjük, az alkalmazás elindítása után az alábbi tesztfiókok valamelyikét használják a bejelentkezéshez:

Adminisztrátori hozzáférés (Teljes jogosultság):

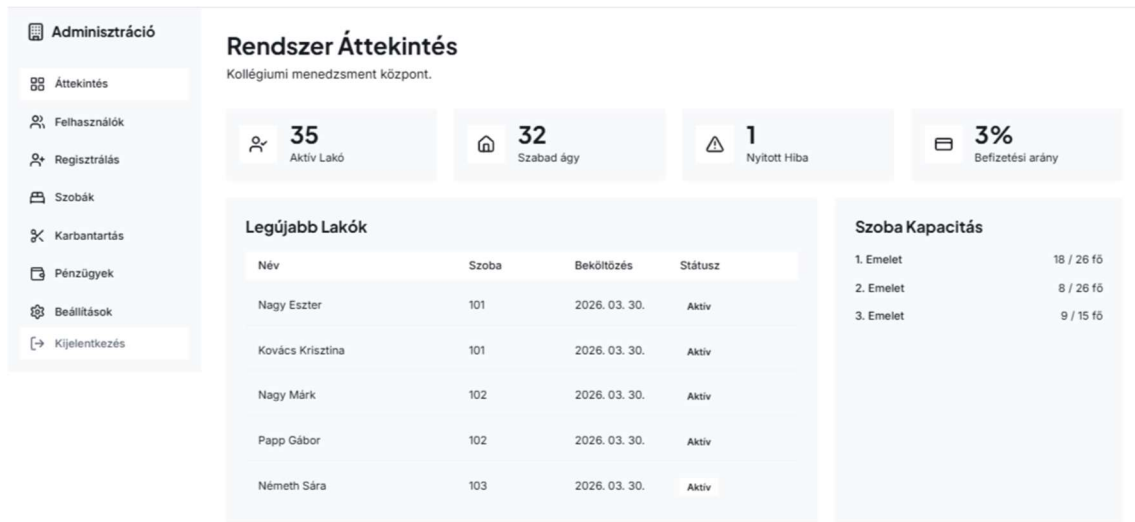
- **E-mail cím:**
 - pallai.norbert.zoltan@koliportal.hu
 - VAGY
 - kercso.norbert@koliportal.hu
- **Jelszó:** 123456 (Mindkettő adminisztrátori felhasználónál ez a jelszó)

Diák / Lakó hozzáférés (Korlátozott jogosultság):

- **E-mail cím:** nagy.eszter@koliportal.hu
- **Jelszó:** ne1234

2.7.3 Adminisztrátor felület – Áttekintés

Ez a képernyő az adminisztrátorok elsődleges érkezési oldala, egy központi vezérlőpult (Dashboard), amely megmutatja a kollégium aktuális állapotát. Célja, hogy a legfontosabb metrikák azonnal, görgetés nélkül leolvashatóak legyenek.



2.7.4 Adminisztrátor felület – Felhasználók

Ez az adatbázis-kezelő felület felel a kollégiumi lakók és a személyzet adatainak listázásáért és kezeléséért. Kialakításánál a nagytömegű adatok gyors kereshetősége és átláthatósága volt a fő szempont.

Felhasználók Kezelése					
Kollégisták adatbázisa.					
Összes	Bent lakók	Kiköltözöttek	Keresés (név, email, szoba)...		SOR/OLDAL: 5 ▾
NÉV	KOLLÉGIUM	SZOBA	EMAIL	SZEREPKÖR	MŰVELET
Balázs Botond	Karinthy Frigyes Kollégium	303	balazs.botond@koliportal.hu	LAKÓ	
Balogh Márton	Karinthy Frigyes Kollégium	202	balogh.marton@koliportal.hu	LAKÓ	
Farkas Zsófia	Karinthy Frigyes Kollégium	201	farkas.zsofia@koliportal.hu	LAKÓ	
Fekete András	Karinthy Frigyes Kollégium	105	fekete.andras@koliportal.hu	LAKÓ	
Fekete Noémi	Karinthy Frigyes Kollégium	306	fekete.noemi@koliportal.hu	LAKÓ	
< Előző 1 / 8 Következő >					

2.7.5 Adminisztrátor felület – Regisztrálás

Ez a moduláris űrlap (Form) szolgál az új fiókok (diákok, adminisztrátorok) rendszerbe történő felvételére. A tervezés során az egyértelműség és az adatbeviteli hibák minimalizálása volt a cél.

Új Felhasználó Regisztráció

Kollégista, adminisztrátor vagy karbantartó hozzáadása a rendszerhez.
Kérjük, töltsön ki minden kötelező mezőt.

...

Vezetéknév *

Pl. Kovács

Keresztnév *

Pl. János

Email cím *

pelda@email.com

Jelszó *

Jelszó megerősítés *

☐ Legalább 8 karakter

Szerepkör *

LAKÓ (Hallgató) ▾

Diák extra adatok

Szemeszter *

Válassz szemesztert... ▾

Szoba beosztás

Szoba beosztása ▾

Lakcím

Pl. 1051 Budapest, Fő utca 1.

Telefonszám

Pl. +36203988762

☐ Formátum: + és max. 11 számjegy (szóköz nélkül).

Mégse

Létrehozás

* Kötelezően kitöltendő mezőket jelöl.

2.7.6 Adminisztrátor felület – Szobák

A férőhelyek és a szobabeosztások vizuális ellenőrzésére szolgáló felület. Kialakítása azonnali áttekintést nyújt a rendszer adminisztrátorainak a kapacitásról.

Szobák Állapota

Kapacitás és lakók áttekintése.

Összes Szabad

Q Keresés szobaszámra...

SOR/OLDAL:

Szoba	Emelet	Kapacitás	Lakók	Telítettség
101	1. emelet	2 / 2 fő	  Nagy Eszter, Kovács Krisztina	100% (2/2)
102	1. emelet	2 / 2 fő	  Nagy Márk, Papp Gábor	100% (2/2)
103	1. emelet	2 / 2 fő	  Németh Sára, Maros Lili	100% (2/2)
104	1. emelet	0 / 2 fő	-	0% (0/2)
105	1. emelet	2 / 3 fő	  Fekete András, Juhász Péter	67% (2/3)

< Előző

1 / 6

Következő >

2.7.7 Adminisztrátor felület – Karbantartás

Ez a felület a kollégiumban felmerülő műszaki problémák (hibajegyek) nyomon követésére és menedzselésére szolgál, gyakorlatilag egy egyszerűsített ticketing rendszer.

Karbantartás

Hibajegyek felügyelete, feladatok ütemezése és javítási előzmények nyomon követése.

Aktuális Feladatok

Itt látod a folyamatban lévő vagy új hibajegyeket. A befejezéshez kattints a pipa ikonra.

Azonosító	Bejelentő / Helyszín	Leírás	Állapot	Művelet
1	Nagy Eszter 101 szoba	[Vízvezeték és szaniter] Az első emeleti mosdóban csöpög az egyik csap BEKÜLDVE: 2024. 03. 30.	🔄 FOLYAMATBAN	🕒 ✎ ⋮
3	Kovács Péter Konyha - B szárny	[Elektromos] Villóg a neoncső a főzőhelyiség felett BEKÜLDVE: 2024. 04. 01.	🔄 FOLYAMATBAN	🕒 ✎ ⋮
4	Szabó Anna 205 szoba	[Bútorzat] Kilazult az íróasztal lába BEKÜLDVE: 2024. 04. 02.	🕒 ÚJ	🕒 ✎ ⋮

Előzmények

Lezárt feladatok listája és archívum. Használd a visszakereséshez.

Azonosító	Bejelentő / Helyszín	Leírás	Állapot	Művelet
2	Nagy Eszter 101 szoba	[Fűtés] Sajnos az egész első emeleten nincsen fűtés LEZÁRVA: 2024. 03. 29.	🕒 KÉSZ	✎ ⋮
5	Horváth László Folyosó - 3. emelet	[Biztonság] Meglazult a tűzoltó készülék tartója LEZÁRVA: 2024. 03. 26.	🕒 KÉSZ	✎ ⋮

2.7.8 Adminisztrátor felület – Beállítások

A globális szoftverkonfigurációs felület, ahol az adminisztrátor a rendszer automatizmusait és alapvető működését szabályozhatja.

Rendszerbeállítások

Globális rendszerbeállítások és értesítések kezelése.

ÉRTESTÉSEK ÉS RENDSZER

Új hibajegy értesítés

Email küldése, ha egy lakó új karbantartási igényt ad le.

☒

Készenléti fizetés riasztás

Automatikus figyelmeztetés a lejárt tartozásokról.

☒

Karbantartási mód

A diákok nem tudnak bejelentkezni, amíg a rendszer frissül.

☐

2.7.9 Diák/Lakó felület – Áttekintés

Ez a felület a kollégiumi lakók személyes műszerfala (Dashboardja). Célja, hogy bejelentkezés után a diák azonnal, egyetlen pillantással átlássa a saját kollégiumi státuszát és legfontosabb teendőit.

Üdvözlünk a KoliPortál oldalán!

Itt áttekintheted a kollégiumi státuszodat.

101

Jelenlegi szobád

Nincs tartozás

Fizetendő összeg

1 db

Folyamatban lévő hibajelentésed

Legutóbbi Befizetéseid

BEFIZETÉS DÁTUMA	JÓOCÍM	ÖSSZE	STÁTUSZ
2024. 03. 30.	Kollégiumi térítési díj	12 000 Ft	

Hamarosan: Közösségi események és falújság

Hamarosan: Menzarendelés és heti menü

2.7.10 Diák/Lakó felület – Diák/Lakó adatai

Ez a profilkezelő oldal, ahol a diák megtekintheti és – bizonyos korlátok között – módosíthatja a rendszerben tárolt személyes elérhetőségeit.

The screenshot shows a web form titled "Saját Adataim" (My Data) with the subtitle "Személyes információk és szoba beosztás." (Personal information and room assignment). The form contains four input fields: "Teljes Név" (Full Name) with the value "Nagy Eszter", "Szoba száma" (Room Number) with the value "101", "E-mail cím" (Email Address) with the value "nagy.eszter@kolportal.hu", and "Telefonszám" (Phone Number) with the value "+36302134563". Below the phone number field, there is a small text note: "Formátum: + és max. 11 számjegy (pl. +36301234567)". At the bottom of the form, there is a button labeled "Adatok mentése" (Save Data) with a floppy disk icon.

2.7.11 Diák/Lakó felület – Befizetések

A diákok pénzügyi áttekintő felülete. Itt követhetik nyomon a kötelező kollégiumi díjakat és az esetleges egyéb térítéseket.

Saját Befizetések

Kollégiumi díjak és egyéb tranzakciók áttekintése, rendezése.

SOR/OLDAL: 5

Határidő / Fizetve	Jogcím	Összeg	Státusz	Művelet
Fizetve: 2026. 03. 30.	Kollégiumi térítési díj	12 000 Ft	Fizetve	✓

2.7.12 Diák/Lakó felület – Hibajelentés

Ezen a képernyőn a lakók interakcióba léphetnek az adminisztrátor személyzettel. A felület kettős funkciót lát el: új problémák rögzítését és a korábbi bejelentések nyomon követését biztosítja.

Hibabejelentés

Műszaki probléma jelzése a karbantartóknak és a saját jegyeid követése.


Új hiba bejelentése

Típus

Leírás

Részletezd a problémát pontosan (pl. 204-es szoba, csöpög a csap a fürdőben)...


Jelentés küldése

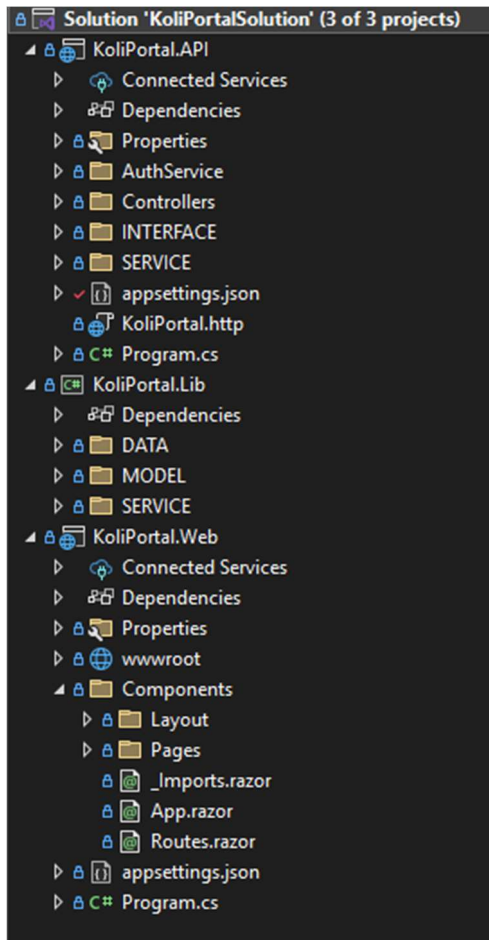

Nyitott hibajegyeim

Dátum	Leírás	Státusz
2026. 03. 30.	[Vízvezeték és szaniter] Az első emeleti mosdóban csöpög az egyik csap	FOLYAMATBAN

2.8 Projekt struktúra

A szoftver egy modern, három különálló projektből álló megoldásra épül. Ez a rétegzett architektúra biztosítja a kód átláthatóságát, skálázhatóságát és a funkciók logikai szétválasztását.

2.8.1 Fő könyvtárak



2.8.1.1 KoliPortal.API (Háttérrendszer / Backend)

Ez a projekt felel a kiszolgáló oldali üzleti logikáért és a kliens kéréseinek feldolgozásáért (RESTful API).

- **Controllers:** Itt találhatóak a végpontok (Endpointok), amelyek fogadják a frontend felől érkező HTTP (GET, POST, PUT, DELETE) kéréseket, és irányítják az adatforgalmat.
- **AuthService:** A felhasználók hitelesítéséért és az azonosításhoz szükséges JWT (JSON Web Token) generálásáért felelős szolgáltatások mappája.
- **INTERFACE & SERVICE:** Itt valósul meg az üzleti logika. Az INTERFACE mappa tartalmazza a metódusok szerződéseit (absztrakcióját), a SERVICE mappa pedig azok tényleges implementációját. Ez a felépítés elengedhetetlen a modern Függőséginjektálás (Dependency Injection) alkalmazásához.

- **appsettings.json:** A backend konfigurációs fájlja, amely a biztonságos adatbázis-kapcsolati karakterláncot (Connection String) és a token titkosítási kulcsait tárolja.

2.8.1.2 KoliPortal.Lib (Közös Osztálykönyvtár / Shared Library)

Ez a köztes réteg (Class Library) tartalmazza azokat a fájlokat, amelyeket az API és a Web projekt is közösen használ, elkerülve ezzel a kódDuplikációt.

- **MODEL:** Az adatszerkezeteket (Entitásokat és DTO - Data Transfer Object osztályokat) definiáló mappa. Ezek az osztályok (pl. Felhasználók, Szobák) képezik le az adatbázis tábláit a C# kód számára.
- **DATA:** Az adatelérési réteg (Data Access Layer), amely jellemzően az adatbázis kontextust (DbContext) és az Entity Framework Core beállításait foglalja magában.
- **SERVICE:** Olyan közös szolgáltatások, amelyeket a frontend és a backend is közvetlenül meghívhat.

2.8.1.3 KoliPortal.Web (Kliensoldal / Frontend)

Ez a projekt felel a grafikus felhasználói felületért (UI), amely Blazor Web technológiára épül.

- **wwwroot:** A statikus webes állományok (CSS stíluslapok, JavaScript fájlok, képek, Bootstrap ikonok) gyűjtőhelye.
- **Components / Pages:** Itt helyezkednek el a tényleges felhasználói felületek (Razor fájlok, pl. a korábban bemutatott Login.razor, Szobak.razor). A logika szerint külön mappákba rendezve az adminisztrátori és a diák funkciók.
- **Components / Layout:** Az oldalakat összefogó keretrendszerek (pl. oldalsó menüsáv, navigáció) helye, biztosítva az egységes megjelenést a különböző nézetek (AdminLayout, LakoLayout) között.

2.9 Adatmodellek

A KoliPortal.Lib projektben található MODEL mappa felel a rendszer adatstruktúrájának definiálásáért. Ezek az osztályok (Entitások és DTO-k) jelentik a kapcsolatot a relációs adatbázis (SQL) táblái és a C# objektumorientált üzleti logikája között, lehetővé téve a

zökkenőmentes ORM (Object-Relational Mapping) működést, például az Entity Framework Core használatát.

A kódképeken jól látható, hogy a modellek kialakításánál tudatosan alkalmaztuk a C# modern adatszerkezeti megoldásait: a [Key] adatrögzítő (Data Annotation) attribútum jelöli az elsődleges kulcsokat, a kérdőjellel ellátott típusok (pl. DateTime?, string?) pedig a nem kötelező, adatbázisban NULL értéket is felvevő mezőket reprezentálják.

2.9.1 DiakAdatok

```
namespace KoliPortal.Lib.MODEL
{
    21 references
    public class DiakAdatok
    {
        [Key]
        8 references
        public int UserID { get; set; }
        4 references
        public string? Szemeszter { get; set; }
        4 references
        public string? Lakcim { get; set; }
    }
}
```

2.9.2 Felhasznalok

```
namespace KoliPortal.Lib.MODEL
{
    public class Felhasznalok
    {
        [Key]
        18 references
        public int ID { get; set; }
        12 references
        public string? Nev { get; set; }
        11 references
        public string? Email { get; set; }
        4 references
        public string? Jelszo { get; set; }
        5 references
        public string? Telefonszam { get; set; }
        6 references
        public int SzerepkorID { get; set; }
        4 references
        public int KollegiumID { get; set; }
    }
}
```

2.9.3 FizetesTipusok

```
namespace KoliPortal.Lib.MODEL
{
    10 references
    public class FizetesTipusok
    {
        [Key]
        5 references
        public int ID { get; set; }
        8 references
        public string? Megnevezes { get; set; }
    }
}
```

2.9.4 KarbantartasiKeresek

```
namespace KoliPortal.Lib.MODEL
{
    28 references
    public class KarbantartasiKeresek
    {
        [Key]
        12 references
        public int ID { get; set; }
        12 references
        public int BejelentoID { get; set; }
        7 references
        public string? Leiras { get; set; }
        16 references
        public int StatuszID { get; set; }
        9 references
        public DateTime LetrehozasiDatum { get; set; }
    }
}
```

2.9.5 KarbantartasiStatuszok

```
namespace KoliPortal.Lib.MODEL
{
    9 references
    public class KarbantartasiStatuszok
    {
        [Key]
        3 references
        public int ID { get; set; }
        3 references
        public string? Nev { get; set; }
    }
}
```

2.9.6 Kollegium

```
namespace KoliPortal.Lib.MODEL
{
    8 references
    public class Kollegium
    {
        [Key]
        2 references
        public int ID { get; set; }
        2 references
        public string KollegiumNev { get; set; }
    }
}
```

2.9.7 Login

```
namespace KoliPortal.Lib.MODEL
{
    3 references
    public class Login
    {
        2 references
        public string Email { get; set; } = string.Empty;
        2 references
        public string Jelszo { get; set; } = string.Empty;
    }
}
```

2.9.8 Penzugyek

```
namespace KoliPortal.Lib.MODEL
{
    public class Penzugyek
    {
        [Key]
        public int ID { get; set; }

        public int UserID { get; set; }

        public int TipusID { get; set; }
        7 references
        public decimal Osszeg { get; set; }
        10 references
        public DateTime Esedekesseg { get; set; }
        21 references
        public DateTime? BefizetesDatum { get; set; }
        10 references
        public string? Statusz { get; set; }
    }
}
```

2.9.9 Szerepkorok

```
namespace KoliPortal.Lib.MODEL
{
    public class Szerepkorok
    {
        [Key]
        4 references
        public int ID { get; set; }
        5 references
        public string? Nev { get; set; }
    }
}
```

2.9.10 SzobaBeosztások

```
namespace KoliPortal.Lib.MODEL
{
    27 references
    public class SzobaBeosztasok
    {
        [Key]
        3 references
        public int ID { get; set; }
        18 references
        public int UserID { get; set; }
        14 references
        public int RoomID { get; set; }
        5 references
        public DateTime? BekoltozesDatum { get; set; }
        3 references
        public DateTime? VarhatoKikoltozes { get; set; }
        15 references
        public DateTime? TenylegesKikoltozes { get; set; }
    }
}
```

2.9.11 Szobak

```
namespace KoliPortal.Lib.MODEL
{
    19 references
    public class Szobak
    {
        [Key]
        17 references
        public int ID { get; set; }
        13 references
        public string? Szobaszam { get; set; }
        2 references
        public int Emelet { get; set; }
        12 references
        public int FerohelyMax { get; set; }
    }
}
```

2.9.12 KoliPortalDbContext

Az KoliPortalDbContext osztály az adatbázis-kapcsolatot és adatmodelleket kezeli. Ez az osztály az Entity Framework Core DbContext osztály kiterjesztése, amely a szolgáltatások és a vezérlők számára biztosítja az adatbázis-hozzáférést.

```
namespace KoliPortal.Lib.DATA
{
    25 references
    public class KoliportalDBContext : DbContext
    {
        5 references
        public DbSet<DiakAdatok> DiakAdatok { get; set; }
        8 references
        public DbSet<Felhasznalok> Felhasznalok { get; set; }
        1 reference
        public DbSet<FizetesTipusok> FizetesTipusok { get; set; }
        5 references
        public DbSet<KarbantartasiKeresek> KarbantartasiKeresek { get; set; }
        1 reference
        public DbSet<KarbantartasStatuszok> KarbantartasStatuszok { get; set; }
        5 references
        public DbSet<Penzugyek> Penzugyek { get; set; }
        1 reference
        public DbSet<Szerepkorok> Szerepkorok { get; set; }
        5 references
        public DbSet<SzobaBeosztasok> SzobaBeosztasok { get; set; }
        1 reference
        public DbSet<Szobak> Szobak { get; set; }
        1 reference
        public DbSet<Kollegium> Kollegium { get; set; }

        0 references
        public KoliportalDBContext()
        {
        }

        0 references
        public KoliportalDBContext(DbContextOptions options) : base(options)
        {
        }
    }
}
```

2.10 Interface

A **KoliPortal.API.INTERFACE** névterületen belül található interfészek a rendszer üzleti logikájának "szerződéseit" (contracts) jelentik. Ezek az állományok nem tartalmaznak konkrét végrehajtható kódot, csupán előírják, hogy az adatbázissal kommunikáló szolgáltatásoknak (Services) milyen metódusokat kell kötelezően megvalósítaniuk.

2.10.1 IDiakAdatok

A hallgató-specifikus kiegészítő információk (szemeszter, lakcím) teljes körű (CRUD) kezelését végző metódusokat definiálja.

```
namespace KoliPortal.API.INTERFACE
{
    4 references
    public interface IDiakAdatok
    {
        2 references
        Task<List<DiakAdatok>> GetAll();
        2 references
        Task<DiakAdatok> Add(DiakAdatok entity);
        2 references
        Task Update(DiakAdatok entity);
        2 references
        Task Delete(int id);
    }
}
```

2.10.2 IFelhasznalok

A rendszer központi profiljainak (diákok, adminok, karbantartók) létrehozásáért, módosításáért, törléséért és listázásáért felelős fő szerződés.

```
namespace KoliPortal.API.INTERFACE
{
    public interface IFelhasznalok
    {
        2 references
        Task<List<Felhasznalok>> GetAll();
        2 references
        Task<Felhasznalok> Add(Felhasznalok entity);
        2 references
        Task Update(Felhasznalok entity);
        2 references
        Task Delete(int id);
    }
}
```

2.10.3 IFizetesTipusok

Csak olvasható adatelérést (GetAll) biztosít a rendszerben elérhető tranzakciós kategóriák (pl. kollégiumi térítési díj) legördülő listákba történő betöltéséhez.

```
namespace KoliPortal.API.INTERFACE
{
    4 references
    public interface IFizetesTipusok
    {
        2 references
        Task<List<FizetesTipusok>> GetAll();
    }
}
```

2.10.4 IKarbantartasiKeresek

A műszaki hibajegyek teljes életciklusát (beküldés, szerkesztés, törlés, listázás) lefedő szolgáltatások előírása.

```
namespace KoliPortal.API.INTERFACE
{
    public interface IKarbantartasiKeresek
    {
        2 references
        Task<List<KarbantartasiKeresek>> GetAll();
        2 references
        Task<KarbantartasiKeresek> Add(KarbantartasiKeresek entity);
        2 references
        Task Update(KarbantartasiKeresek entity);
        2 references
        Task Delete(int id);
    }
}
```

2.10.5 IKarbantartasiStatuszok

A hibajegyekhez rendelhető lehetséges munkafolyamat-állapotok (pl. Új, Folyamatban, Kész) lekérdezését teszi lehetővé.

```
namespace KoliPortal.API.INTERFACE
{
    4 references
    public interface IKarbantartasStatuszok
    {
        2 references
        Task<List<KarbantartasStatuszok>> GetAll();
    }
}
```

2.10.6 IKollégium

A rendszerben regisztrált kollégiumi épületek/intézmények listázását végző metódus absztrakciója a több-kollégiumos működéshez.

```
namespace KoliPortal.API.INTERFACE
{
    4 references
    public interface IKollegium
    {
        2 references
        Task<List<Kollegium>> GetAll();
    }
}
```

2.10.7 IPenzugyek

A diákokhoz tartozó konkrét befizetések, tartozások felvitelét, szerkesztését, törlését és lekérdezését szabályozó interfész.

```
namespace KoliPortal.API.INTERFACE
{
    public interface IPenzugyek
    {
        2 references
        Task<List<Penzugyek>> GetAll();
        2 references
        Task<Penzugyek> Add(Penzugyek entity);
        2 references
        Task Update(Penzugyek entity);
        2 references
        Task Delete(int id);
    }
}
```

2.10.8 ISzerepkorok

A jogosultsági szintek adatbázisból való lekérdezésének szerződése, amely a felhasználók létrehozásánál és a jogosultság-ellenőrzéseknél kap szerepet.

```
namespace KoliPortal.API.INTERFACE
{
    4 references
    public interface ISzerepkorok
    {
        2 references
        Task<List<Szerepkorok>> GetAll();
    }
}
```

2.10.9 ISzobaBeosztasok

A lakók szobához rendelését, valamint a be- és kiköltözések idejét menedzselő teljes körű üzleti logika alapja.

```
namespace KoliPortal.API.INTERFACE
{
    4 references
    public interface ISzobaBeosztasok
    {
        2 references
        Task<List<SzobaBeosztasok>> GetAll();
        2 references
        Task<SzobaBeosztasok> Add(SzobaBeosztasok entity);
        2 references
        Task Update(SzobaBeosztasok entity);
        2 references
        Task Delete(int id);
    }
}
```

2.10.10 ISzobak

A kollégiumi szobák és azok alapvető kapacitás-adatainak (férőhely, emelet) lekérdezését biztosítja a kliens számára.


```
namespace KoliPortal.API.INTERFACE
{
    4 references
    public interface ISzobak
    {
        2 references
        Task<List<Szobak>> GetAll();
    }
}
```

2.11 Service

A **KoliPortal.API.SERVICE** névterületen található osztályok alkotják a szoftver üzleti logikai rétegét (Business Logic Layer). Ezek az osztályok implementálják a korábban bemutatott interfészeket, és végzik el a tényleges adatszerveket az Entity Framework Core segítségével.

A Service réteg feladata és technológiai háttere:

Ennek a rétegnek a fő célja a "Separation of Concerns" (felelősségek szétválasztása) elv megtartása. A felhasználói felület (UI) vagy az API végpontok soha nem kommunikálnak közvetlenül az adatbázissal; minden kérést ezeken a szolgáltatásokon keresztül indítanak.

A képeken jól látható a Függőséginjektálás (Dependency Injection) alkalmazása: minden Service osztály a konstruktorán keresztül, egy privát, csak olvasható (private readonly) változóba kapja meg a KoliportalDbContext példányt. Továbbá a rendszer teljes mértékben aszinkron (async / await) technológiát használ, ami garantálja, hogy a szerver több ezer párhuzamos kérés esetén sem akad meg (non-blocking) az adatbázis-műveletekre várva.

1. Teljes adatkezelést (CRUD) végző szolgáltatások: Ezek az osztályok valósítják meg az adatok létrehozását (AddAsync), lekérdezését (ToListAsync), módosítását (Update) és törlését (Remove).

- **DiakAdatokService:** A hallgatók extra adatainak (pl. lakcím, évfolyam) aszinkron adatbázis-műveleteit kezeli.
- **FelhasználókService:** A rendszer legkritikusabb szolgáltatása, amely a fiókok felvitelét és teljes körű menedzselését végzi.
- **KarbantartásiKeresekService:** A hibajegyek bekerülését, azok adatainak frissítését (pl. státuszváltás) és lekérdezését hajtja végre.

- **PenzugyekService:** A pénzügyi modul tranzakcióinak aszinkron mentéséért és listázásáért felelős szolgáltatás.
- **SzobaBeosztasokService:** A beköltözések és kiköltözések rögzítését, valamint a szobákhoz rendelt diákok adatbázis-kapcsolatainak módosítását végzi.

2. Csak olvasható (Read-Only / Lookup) szolgáltatások: Ezek az osztályok kizárólag a GetAll() aszinkron metódust implementálják, feladatuk a kliensoldali űrlapok (legördülő listák) dinamikus adatokkal történő feltöltése.

- **FizetesTipusokService:** Lekérdezi az adatbázisból a rögzíthető pénzügyi kategóriákat.
- **KarbantartasStatuszokService:** Visszaadja a hibajegyekhez rendelhető állapotokat (Új, Folyamatban, Kész).
- **KollegiumService:** Az elérhető kollégiumi épületek/intézmények listáját szolgáltatja.
- **SzerepkorokService:** A jogosultsági szinteket (Admin, Lakó) kéri le a regisztrációs űrlapok számára.
- **SzobakService:** Visszaadja a rendszerben konfigurált szobák listáját, a hozzájuk tartozó maximális férőhelyekkel együtt.

2.11.1 DiakAdatokService

```
namespace KoliPortal.API.SERVICE
{
    2 references
    public class DiakAdatokService : IDiakAdatok
    {
        private readonly KoliportalDBContext _context;
        0 references
        public DiakAdatokService(KoliportalDBContext context)
        {
            _context = context;
        }

        2 references
        public async Task<DiakAdatok> Add(DiakAdatok entity)
        {
            _context.DiakAdatok.AddAsync(entity);
            await _context.SaveChangesAsync();
            return entity;
        }

        2 references
        public async Task Delete(int id)
        {
            var entity = await _context.DiakAdatok.FindAsync(id);
            if (entity != null)
            {
                _context.DiakAdatok.Remove(entity);
                await _context.SaveChangesAsync();
            }
        }

        2 references
        public async Task<List<DiakAdatok>> GetAll()
        {
            return await _context.DiakAdatok.ToListAsync();
        }

        2 references
        public async Task Update(DiakAdatok entity)
        {
            _context.DiakAdatok.Update(entity);
            await _context.SaveChangesAsync();
        }
    }
}
```

2.11.2 FelhasznalokService

```
namespace KoliPortal.API.SERVICE
{
    2 references
    public class FelhasznalokService : IFelhasznalok
    {
        private readonly KoliportalDBContext _context;
        0 references
        public FelhasznalokService(KoliportalDBContext context)
        {
            _context = context;
        }
        2 references
        public async Task<Felhasznalok> Add(Felhasznalok entity)
        {
            _context.Felhasznalok.AddAsync(entity);
            await _context.SaveChangesAsync();
            return entity;
        }

        2 references
        public async Task Delete(int id)
        {
            var entity = await _context.Felhasznalok.FindAsync(id);
            if (entity != null)
            {
                _context.Felhasznalok.Remove(entity);
                await _context.SaveChangesAsync();
            }
        }

        2 references
        public async Task<List<Felhasznalok>> GetAll()
        {
            return await _context.Felhasznalok.ToListAsync();
        }

        2 references
        public async Task Update(Felhasznalok entity)
        {
            _context.Felhasznalok.Update(entity);
            await _context.SaveChangesAsync();
        }
    }
}
```

2.11.3 FizetesTipusokService

```
namespace KoliPortal.API.SERVICE
{
    2 references
    public class FizetesTipusokService : IFizetesTipusok
    {
        private readonly KoliportalDBContext _context;
        0 references
        public FizetesTipusokService(KoliportalDBContext context)
        {
            _context = context;
        }
        2 references
        public async Task<List<FizetesTipusok>> GetAll()
        {
            return await _context.FizetesTipusok.ToListAsync();
        }
    }
}
```

2.11.4 KarbantartasiKeresekService

```
namespace KoliPortal.API.SERVICE
{
    2 references
    public class KarbantartasiKeresekService : IKarbartartasiKeresek
    {
        private readonly KoliportalDBContext _context;
        0 references
        public KarbantartasiKeresekService(KoliportalDBContext context)
        {
            _context = context;
        }
        2 references
        public async Task<KarbantartasiKeresek> Add(KarbantartasiKeresek entity)
        {
            _context.KarbantartasiKeresek.AddAsync(entity);
            await _context.SaveChangesAsync();
            return entity;
        }

        2 references
        public async Task Delete(int id)
        {
            var entity = await _context.KarbantartasiKeresek.FindAsync(id);
            if(entity != null)
            {
                _context.KarbantartasiKeresek.Remove(entity);
                await _context.SaveChangesAsync();
            }
        }

        2 references
        public async Task<List<KarbantartasiKeresek>> GetAll()
        {
            return await _context.KarbantartasiKeresek.ToListAsync();
        }

        2 references
        public async Task Update(KarbantartasiKeresek entity)
        {
            _context.KarbantartasiKeresek.Update(entity);
            await _context.SaveChangesAsync();
        }
    }
}
```

2.11.5 KarbantartasiStatuszokService

```
namespace KoliPortal.API.SERVICE
{
    2 references
    public class KarbantartasStatuszokService : IKarbartartasStatuszok
    {
        private readonly KoliportalDBContext _context;
        0 references
        public KarbantartasStatuszokService(KoliportalDBContext context)
        {
            _context = context;
        }

        2 references
        public async Task<List<KarbantartasStatuszok>> GetAll()
        {
            return await _context.KarbantartasStatuszok.ToListAsync();
        }
    }
}
```

2.11.6 KollegiumService

```
namespace KoliPortal.API.SERVICE
{
    2 references
    public class KollegiumService : IKollegium
    {
        private readonly KoliportalDbContext _context;

        0 references
        public KollegiumService(KoliportalDbContext context)
        {
            _context = context;
        }

        2 references
        public async Task<List<Kollegium>> GetAll()
        {
            return await _context.Kollegium.ToListAsync();
        }
    }
}
```

2.11.7 PenzugyekService

```
namespace KoliPortal.API.SERVICE
{
    2 references
    public class PenzugyekService : IPenzugyek
    {
        private readonly KoliportalDbContext _context;

        0 references
        public PenzugyekService(KoliportalDbContext context)
        {
            _context = context;
        }

        2 references
        public async Task<Penzugyek> Add(Penzugyek entity)
        {
            _context.Penzugyek.AddAsync(entity);
            await _context.SaveChangesAsync();
            return entity;
        }

        2 references
        public async Task Delete(int id)
        {
            var entity = await _context.Penzugyek.FindAsync(id);
            if (entity != null)
            {
                _context.Penzugyek.Remove(entity);
                await _context.SaveChangesAsync();
            }
        }

        2 references
        public Task<List<Penzugyek>> GetAll()
        {
            return _context.Penzugyek.ToListAsync();
        }

        2 references
        public async Task Update(Penzugyek entity)
        {
            _context.Penzugyek.Update(entity);
            await _context.SaveChangesAsync();
        }
    }
}
```

2.11.8 SzerepkorService

```
namespace KoliPortal.API.SERVICE
{
    public class SzerepkorokService : ISzerepkorok
    {
        private readonly KoliportalDBContext _context;

        public SzerepkorokService(KoliportalDBContext context)
        {
            _context = context;
        }

        2 references
        public async Task<List<Szerepkorok>> GetAll()
        {
            return await _context.Szerepkorok.ToListAsync();
        }
    }
}
```

2.11.9 SzobaBeosztasokService

```
namespace KoliPortal.API.SERVICE
{
    public class SzobaBeosztasokService : ISzobaBeosztasok
    {
        private readonly KoliportalDBContext _context;

        public SzobaBeosztasokService(KoliportalDBContext context)
        {
            _context = context;
        }

        public async Task<SzobaBeosztasok> Add(SzobaBeosztasok entity)
        {
            _context.SzobaBeosztasok.AddAsync(entity);
            await _context.SaveChangesAsync();
            return entity;
        }

        public async Task Delete(int id)
        {
            var entity = await _context.SzobaBeosztasok.FindAsync(id);
            if (entity != null)
            {
                _context.SzobaBeosztasok.Remove(entity);
                await _context.SaveChangesAsync();
            }
        }

        public async Task<List<SzobaBeosztasok>> GetAll()
        {
            return await _context.SzobaBeosztasok.ToListAsync();
        }

        2 references
        public async Task Update(SzobaBeosztasok entity)
        {
            _context.SzobaBeosztasok.Update(entity);
            await _context.SaveChangesAsync();
        }
    }
}
```

2.11.10 SzobakService

```
namespace KoliPortal.API.SERVICE
{
    2 references
    public class SzobakService : ISzobak
    {
        private readonly KoliportalDBContext _context;
        0 references
        public SzobakService(KoliportalDBContext context)
        {
            _context = context;
        }

        2 references
        public async Task<List<Szobak>> GetAll()
        {
            return await _context.Szobak.ToListAsync();
        }
    }
}
```

2.11.11 JWTTokenService

A rendszer biztonsági architektúrájának (Authentication & Authorization) legfontosabb eleme a JWTTokenService osztály. Feladata egy szabványos, biztonságos és digitálisan aláírt JSON Web Token (JWT) generálása a felhasználó sikeres bejelentkezését követően. Mivel az osztály önálló Service-ként funkcionál, a kód tökéletesen követi a felelősségek szétválasztásának (Separation of Concerns) elvét.

A KoliPortál állapotmentes REST API architektúrára épül. Ez azt jelenti, hogy a szerver memóriája nem tárol adatokat arról, hogy ki van éppen bejelentkezve. Ehelyett a JWTTokenService által kiállított token a kliens (a Blazor weboldal) elmenti, és minden egyes védett végpont meghívásakor (pl. szobabeosztás lekérése) csatolja a HTTP kérés fejlécéhez (Authorization: Bearer). A szerver kizárólag ezen token alapján azonosítja a felhasználót.

A token generálás lépései és a kód működése:

1. **Biztonságos konfiguráció beolvasása (IConfiguration):** A konstruktoron keresztül injektált _configuration objektum segítségével a szolgáltatás beolvassa a token generálásához szükséges paramétereket (Kibocsátó/Issuer, Célközönség/Audience, Titkos Kulcs/Key, Lejárat) az appsettings.json fájlból. Ez szigorú biztonsági alapkövetelmény: a kriptográfiai titkos kulcs sosem szerepelhet beleégetve (hardcoded) a forráskódba.
2. **Adatok csomagolása (Claims listázása):** A token egy úgynevezett "Payload" (rakomány) részt is tartalmaz. A kód egy Claim (állítás) listába csomagolja a bejelentkezett felhasználó alapadatait (Email, egyedi ID, Teljes név). Ennek a

technikának a hatalmas előnye a **teljesítmény-optimalizálás**: mivel a token már tartalmazza a diák nevét és azonosítóját, a frontend azonnal ki tudja rajzolni a felhasználói felületet anélkül, hogy ehhez egy újabb, felesleges adatbázis-lekérdezést kellene indítania.

3. **Szerepkör-alapú hozzáférés-vezérlés**: A rendszer az összeállított listához hozzáadja a ClaimTypes.Role azonosítót (pl. "Admin" vagy "Lakó"). Az API végpontokon lévő biztonsági ellenőrzések kizárólag ebből a Claim-ből dolgoznak.
4. **Kriptográfiai aláírás (HMAC-SHA256)**: A token legfontosabb védelmi vonala. A kód a szerver titkos kulcsa (Jwt:Key) és a robusztus HmacSha256 titkosítási algoritmus segítségével digitálisan aláírja a tokenet. Ha egy rosszindulatú felhasználó a böngészőjében vissza is fejté a tokenet, és megpróbálja átírni a saját szerepkörét "Diák"-ról "Admin"-ra, a token digitális aláírása azonnal érvénytelenné válik. Amikor ezt a módosított tokenet elküldi a szervernek, az API az aláírás sérülése miatt megtagadja a hozzáférést (401 Unauthorized), garantálva a KoliPortál maximális biztonságát.

```
public class JwtTokenService
{
    private readonly IConfiguration _configuration;

    // references
    public JwtTokenService(IConfiguration configuration)
    {
        _configuration = configuration;
    }

    /// <summary>
    /// A felhasználó és a szerepkör alapján létrehozza a JWT tokenet.
    /// </summary>
    // reference
    public string CreateToken(Felhasznalok user, string szerepkorNev)
    {
        var issuer = _configuration["Jwt:Issuer"];
        var audience = _configuration["Jwt:Audience"];
        var key = _configuration["Jwt:Key"];
        var expires = _configuration["Jwt:ExpiresMinutes"];

        // Allítás a felhasználó adataiból. Beletesszük a nevét, az emailjét és az ID-ját a tokenbe.
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Email, user.Email),
            new Claim(ClaimTypes.NameIdentifier, user.ID.ToString()),
            new Claim(ClaimTypes.GivenName, user.Nev), // Eltároljuk a teljes nevét is
            new Claim(JwtRegisteredClaimNames.Sub, user.ID.ToString()),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
        };

        // A Szerepkör hozzáadása, hogy az API tudja, hogy ő Admin, Marbantartó vagy Diák
        claims.Add(new Claim(ClaimTypes.Role, szerepkorNev));

        // Titkosítás a kulcs alapján
        var signingKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(key));
        var creds = new SigningCredentials(signingKey, SecurityAlgorithms.HmacSha256);

        // A Token "megsütése" (generálása)
        var token = new JwtSecurityToken(
            issuer: issuer,
            audience: audience,
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(double.Parse(expires)),
            signingCredentials: creds
        );

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

2.12 Controllers

A KoliPortal.API.Controllers névterületen található osztályok alkotják a szoftver RESTful API végpontjait. Ezek az osztályok fogadják a kliens (Blazor WebAssembly/Server) felől érkező HTTP kéréseket (GET, POST, PUT, DELETE), és visszaküldik a megfelelő válaszokat.

Controllerek, Interfészek és Service-ek szétválasztása:

A modern szoftverfejlesztésben alapkövetelmény a laza csatolás. A Controller feladata kizárólag a HTTP forgalom irányítása; nem tartalmazhat adatbázis-mentési vagy számítási logikát. Ezért a Controllerek konstruktorukon keresztül (Dependency Injection) kizárólag az Interfészeket (pl. IFelhasználok) kapják meg, nem pedig a konkrét Service osztályokat. Ez a megoldás lehetővé teszi, hogy a backend üzleti logikája a Controllerek érintése nélkül is lecserélhető vagy tesztelhető legyen, garantálva a tiszta és karbantartható kódot.

2.12.1 DiakAdatokController

A hallgatók kiegészítő adatainak (szemeszter, lakcím) hálózati forgalmát (CRUD műveletek) irányítja. Fogadja a kliens JSON formátumú adatait, és továbbítja azokat a Service rétegnek.

```
namespace MoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class DiakAdatokController : ControllerBase
    {
        private readonly IDiakAdatok _service;
        0 references
        public DiakAdatokController(IDiakAdatok service)
        {
            _service = service;
        }

        [HttpGet]
        0 references
        public async Task<ActionResult<List<DiakAdatok>>> GetAll()
        {
            var lista = await _service.GetAll();
            return Ok(lista);
        }

        [HttpPost]
        0 references
        public async Task<ActionResult<DiakAdatok>> Create([FromBody] DiakAdatok diakAdatok)
        {
            await _service.Add(diakAdatok);
            return Ok(diakAdatok.UserID);
        }

        [HttpPut]
        0 references
        public async Task<ActionResult> Update([FromBody] DiakAdatok diakAdatok)
        {
            await _service.Update(diakAdatok);
            return NoContent();
        }

        [HttpDelete("{id}")]
        0 references
        public async Task<ActionResult> Delete(int id)
        {
            await _service.Delete(id);
            return NoContent();
        }
    }
}
```

Végpontok:

- **GET /api/DiakAdatok** - Összes diák specifikus kiegészítő adatának (szemeszter, lakcím) lekérése.
- **POST /api/DiakAdatok** - Új diákadat profil létrehozása és rögzítése.
- **PUT /api/DiakAdatok** - Egy meglévő diákadat információinak frissítése.
- **DELETE /api/DiakAdatok/{id}** - Diákadat végleges törlése azonosító alapján.

2.12.2 FelhasználokController

A felhasználói profilok végpontjait biztosítja. Ez a vezérlő dolgozza fel a regisztrációs adatokat és a profilmódosítási kéréseket a megfelelő HTTP metódusokon keresztül.

```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class FelhasznalokController : ControllerBase
    {
        private readonly IFelhasznalok _service;

        0 references
        public FelhasznalokController(IFelhasznalok service)
        {
            _service = service;
        }

        [HttpGet]
        0 references
        public async Task<ActionResult<List<Felhasznalok>>> GetAll()
        {
            var lista = await _service.GetAll();
            return Ok(lista);
        }

        [HttpPost]
        0 references
        public async Task<ActionResult<Felhasznalok>> Create([FromBody] Felhasznalok felhasznalok)
        {
            await _service.Add(felhasznalok);
            return Ok(felhasznalok.ID);
        }

        [HttpPut]
        public async Task<ActionResult> Update([FromBody] Felhasznalok felhasznalok)
        {
            await _service.Update(felhasznalok);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public async Task<ActionResult> Delete(int id)
        {
            await _service.Delete(id);
            return NoContent();
        }
    }
}
```

Végpontok:

- **GET /api/Felhasznalok** - Az összes regisztrált felhasználó (diák, admin, karbantartó) lekérése.
- **POST /api/Felhasznalok** - Új felhasználó regisztrálása a rendszerbe.
- **PUT /api/Felhasznalok** - Felhasználói profil adatainak (pl. név, email, szerepkör) frissítése.
- **DELETE /api/Felhasznalok/{id}** - Felhasználó törlése azonosító alapján.

2.12.3 FizetesTipusokController

Csak olvasható (GET) végpontot biztosít a kliens számára. Feladata a pénzügyi tranzakciók típusainak (pl. kollégiumi díj) szolgáltatása a frontend űrlapjaihoz.

```
namespace MoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]

    public class FizetesTipusokController : ControllerBase
    {
        private readonly IFizetesTipusok _service;
        // references
        public FizetesTipusokController(IFizetesTipusok service)
        {
            _service = service;
        }

        [HttpGet]
        // references
        public async Task<ActionResult<List<FizetesTipusok>>> GetAll()
        {
            var lista = await _service.GetAll();
            return Ok(lista);
        }
    }
}
```

Végpont:

- **GET /api/FizetesTipusok** - A rendszerben elérhető fizetési kategóriák (pl. kollégiumi díj, kártérítés) lekérése az űrlapokhoz.

2.12.4 KarbantartasiKeresekController

A hibajegy-rendszer kommunikációs csatornája. Itt futnak be a diákok új hibabejelentései (POST), illetve az adminok állapotfrissítései (PUT).

```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]

    public class KarbantartasiKeresekController : ControllerBase
    {
        private readonly IKarbantartasiKeresek _service;
        0 references
        public KarbantartasiKeresekController(IKarbantartasiKeresek service)
        {
            _service = service;
        }

        [HttpGet]
        0 references
        public async Task<ActionResult<List<KarbantartasiKeresek>>> GetAll()
        {
            var list = await _service.GetAll();
            return Ok(list);
        }

        [HttpPost]
        0 references
        public async Task<ActionResult<KarbantartasiKeresek>> Create([FromBody] KarbantartasiKeresek karbantartasiKeresek)
        {
            await _service.Add(karbantartasiKeresek);
            return Ok(karbantartasiKeresek.ID);
        }

        [HttpPut]
        0 references
        public async Task<ActionResult> Update([FromBody] KarbantartasiKeresek karbantartasiKeresek)
        {
            await _service.Update(karbantartasiKeresek);
            return NoContent();
        }

        [HttpDelete("{id}")]
        0 references
        public async Task<ActionResult> Delete(int id)
        {
            await _service.Delete(id);
            return NoContent();
        }
    }
}
```

Végpontok:

- **GET /api/KarbantartasiKeresek** - Az összes leadott műszaki hibabejelentés lekérése.
- **POST /api/KarbantartasiKeresek** - Új karbantartási kérés (hibajegy) rögzítése a diákok által.
- **PUT /api/KarbantartasiKeresek** - Hibajegy adatainak vagy aktuális állapotának (státuszának) frissítése.
- **DELETE /api/KarbantartasiKeresek/{id}** - Tévesen leadott hibajegy törlése azonosító alapján.

2.12.5 KarbantartasStatuszokController

Egy egyszerű lekérdező végpont, amely JSON formátumban adja át a hibajegyek lehetséges állapotait (Új, Folyamatban, Kész) a felhasználói felületnek.

```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class KarbantartasStatuszokController : ControllerBase
    {
        private readonly IKarbantartasStatuszok _service;
        public KarbantartasStatuszokController(IKarbantartasStatuszok service)
        {
            _service = service;
        }
        [HttpGet]
        public async Task<ActionResult<List<KarbantartasStatuszok>>> GetAll()
        {
            var list = await _service.GetAll();
            return Ok(list);
        }
    }
}
```

Végpont:

- **GET /api/KarbantartasStatuszok** - A hibajegyekhez rendelhető lehetséges állapotok (Új, Folyamatban, Kész) lekérése.

2.12.6 KollegiumController

A rendszerben rögzített kollégiumok listáját teszi elérhetővé az API-n keresztül. Alapvető fontosságú a regisztrációs folyamat legördülő menüjének betöltéséhez.

```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class KollegiumController : ControllerBase
    {
        private readonly IKollegium _service;
        public KollegiumController(IKollegium service)
        {
            _service = service;
        }
        [HttpGet]
        public async Task<ActionResult<List<Kollegium>>> GetAll()
        {
            var list = await _service.GetAll();
            return Ok(list);
        }
    }
}
```

Végpont:

- **GET /api/Kollegium** - A rendszerben rögzített kollégiumi épületek/intézmények listájának lekérése.

2.12.7 PenzugyekController

A pénzügyi modul HTTP kéréseit kezeli. Biztosítja a befizetések biztonságos hálózati továbbítását a kliens és az adatbázis között.

```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]

    public class PenzugyekController : ControllerBase
    {
        private readonly IPenzugyek _service;

        public PenzugyekController(IPenzugyek service)
        {
            _service = service;
        }

        [HttpGet]

        public async Task<ActionResult<List<Penzugyek>>> GetAll()
        {
            var lista = await _service.GetAll();
            return Ok(lista);
        }

        [HttpPost]

        public async Task<ActionResult<Penzugyek>> Create([FromBody] Penzugyek penzugyek)
        {
            await _service.Add(penzugyek);
            return Ok(penzugyek.ID);
        }

        [HttpPut]

        public async Task<ActionResult> Update([FromBody] Penzugyek penzugyek)
        {
            await _service.Update(penzugyek);
            return NoContent();
        }

        [HttpDelete("{id}")]
        // references
        public async Task<ActionResult> Delete(int id)
        {
            await _service.Delete(id);
            return NoContent();
        }
    }
}
```

Végpontok:

- **GET /api/Penzugyek** - Az összes pénzügyi tranzakció (befizetések, tartozások) lekérése.
- **POST /api/Penzugyek** - Új pénzügyi tétel (fizetési kötelezettség vagy teljesítés) rögzítése.
- **PUT /api/Penzugyek** - Egy meglévő tranzakció adatainak módosítása.
- **DELETE /api/Penzugyek/{id}** - Pénzügyi tétel törlése azonosító alapján.

2.12.8 SzerepkorokController

Olvasható végpontot biztosít a rendszer jogosultsági szintjeihez (Admin, Lakó). A kliensoldal ezen keresztül tudja listázni a választható szerepköröket.


```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]

    public class SzerepkorokController : ControllerBase
    {
        private readonly ISzerepkorok _service;
        public SzerepkorokController(ISzerepkorok service)
        {
            _service = service;
        }
        [HttpGet]
        public async Task<ActionResult<List<Szerepkorok>>> GetAll()
        {
            var lista = await _service.GetAll();
            return Ok(lista);
        }
    }
}
```

Végpont:

- **GET /api/Szerepkorok** - A kiosztható jogosultsági szintek (pl. Adminisztrátor, Lakó) lekérése felhasználó létrehozásához.

2.12.9 SzobaBeosztasokController

A kollégiumi be- és kiköltöztetések komplex folyamatait kiszolgáló végpontokat tartalmazza. Szabványosítja a kliens és a szobakezelő üzleti logika közötti adatcserét.

```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]

    public class SzobaBeosztasokController : ControllerBase
    {
        private readonly ISzobaBeosztasok _service;

        public SzobaBeosztasokController(ISzobaBeosztasok service)
        {
            _service = service;
        }

        [HttpGet]
        public async Task<ActionResult<List<SzobaBeosztasok>>> GetAll()
        {
            var lista = await _service.GetAll();
            return Ok(lista);
        }

        [HttpPost]
        public async Task<ActionResult<SzobaBeosztasok>> Create([FromBody] SzobaBeosztasok szobaBeosztasok)
        {
            await _service.Add(szobaBeosztasok);
            return Ok(szobaBeosztasok.ID);
        }

        [HttpPut]
        public async Task<ActionResult> Update([FromBody] SzobaBeosztasok szobaBeosztasok)
        {
            await _service.Update(szobaBeosztasok);
            return NoContent();
        }

        [HttpDelete]
        public async Task<ActionResult> Delete(int id)
        {
            await _service.Delete(id);
            return NoContent();
        }
    }
}
```

Végpontok:

- **GET /api/SzobaBeosztasok** - Az összes jelenlegi és korábbi szobabeosztás lekérése.
- **POST /api/SzobaBeosztasok** - Új szobabeosztás rögzítése (diák beköltöztetése egy szobába).
- **PUT /api/SzobaBeosztasok** - Szobabeosztás frissítése (pl. tényleges kiköltözés dátumának beállítása).
- **DELETE /api/SzobaBeosztasok/{id}** - Szobabeosztás adminisztratív törlése azonosító alapján.

2.12.10 SzobakController

A kollégiumi szobák adatainak (férőhelyek, emeletek) lekérdezését biztosító GET végpont. A frontend kapacitás-számoló felületei hívják meg a betöltéskor.

```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]

    public class SzobakController : ControllerBase
    {
        private readonly ISzobak _service;
        // references
        public SzobakController(ISzobak service)
        {
            _service = service;
        }
        [HttpGet]
        // references
        public async Task<ActionResult<List<Szobak>>> GetAll()
        {
            var lista = await _service.GetAll();
            return Ok(lista);
        }
    }
}
```

Végpont:

- **GET /api/Szobak** - A kollégiumi szobák alapadatainak (szobaszám, emelet, maximális férőhely) lekérése a kapacitás-számításhoz.

2.12.11 AuthController

A KoliPortal.API.Controllers névterületen található AuthController a rendszer biztonsági kapuja. Ennek az osztálynak a kizárólagos feladata a felhasználók azonosítása (Authentication), a biztonságos regisztráció lebonyolítása, és a munkamenetek (Session) felépítéséhez szükséges JWT (JSON Web Token) generálása.

Ez az osztály garantálja, hogy a rendszer zárt és védett maradjon. A modern webes szabványoknak megfelelően a KoliPortál "állapotmentes" hitelesítést használ. Ez azt

jelenti, hogy a szerver nem tárolja a memóriájában, hogy ki van bejelentkezve, hanem a sikeres belépéskor kiállít egy titkosított tokent (JWT), amit a kliens minden további kéréséhez csatol. Az AuthController választja el a nyilvános végpontokat a védett (csak bejelentkezve elérhető) üzleti logikától.

A kód egyik legfontosabb algoritmus az adatbiztonságot garantálja. A rendszer soha nem tárolja a jelszavakat nyílt szöveggént (plain text) az adatbázisban.

- **Regisztrációkor:** A Register metódusban a BCrypt.Net.BCrypt.HashPassword() algoritmus fut le. Ez a kapott jelszót egy egyirányú, matematikailag visszafejthetetlen "hash" formátummá alakítja, védve a rendszert a támadásoktól.
- **Bejelentkezéskor:** A Login metódusban a BCrypt.Verify() algoritmus veszi át az irányítást. Mivel a hash visszafejthetetlen, ez az algoritmus a felhasználó által éppen beírt jelszót hasheli le újra, és ezt az eredményt hasonlítja össze az adatbázisban tárolt lenyomattal. Ha egyezik, a belépés sikeres.

```
namespace KoliPortal.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class AuthController : ControllerBase
    {
        private readonly KoliportalDbContext _context;
        private readonly JWTTokenService _jwtService;

        0 references
        public AuthController(KoliportalDbContext context, JWTTokenService jwtService)
        {
            _context = context;
            _jwtService = jwtService;
        }

        [HttpPost("Login")]
        0 references
        public async Task<IActionResult> Login([FromBody] Login loginAdat)
        {
            var user = await _context.Felhasznalok
                .FirstOrDefaultAsync(x => x.Email == loginAdat.Email);

            // BCrypt verifikáció
            if (user == null || !BCrypt.Net.BCrypt.Verify(loginAdat.Jelszo, user.Jelszo))
            {
                return Unauthorized("Hibás email vagy jelszó!");
            }

            var Token = _jwtService.CreateToken(user, user.SzerepkoID.ToString());
            return Ok(new { Token });
        }

        [HttpPost("Register")]
        // Itt 'Felhasznalok' modellt várunk, nem 'Login'-t
        0 references
        public async Task<IActionResult> Register([FromBody] Felhasznalok ujUser)
        {
            // Ellenőrizzük, foglalt-e az email
            if (await _context.Felhasznalok.AnyAsync(x => x.Email == ujUser.Email))
            {
                return BadRequest("Ez az email már használatban van!");
            }

            // Jelszó hashelése BCrypt-tel (itt a Blazor által küldött sima jelszót titkosítjuk)
            ujUser.Jelszo = BCrypt.Net.BCrypt.HashPassword(ujUser.Jelszo);

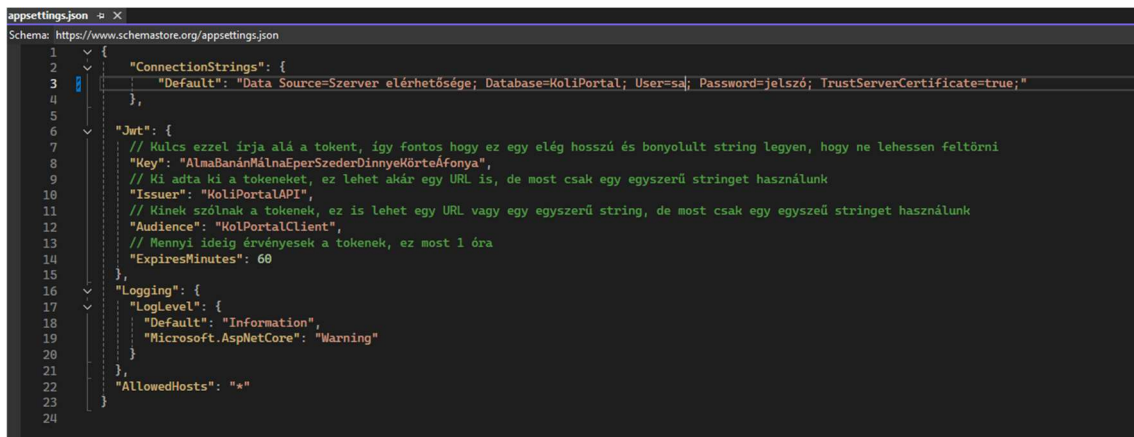
            // Mentés az adatbázisba.
            // A Nev és a SzerepkoID már benne van az 'ujUser' objektumban a Blazorból
            _context.Felhasznalok.Add(ujUser);
            await _context.SaveChangesAsync();

            return Ok(new { Message = "Sikeres regisztráció!" });
        }
    }
}
```

Végpontok:

- **POST /api/Auth/Login** - Hitelesíti a felhasználót az e-mail és jelszó alapján. Sikeres azonosítás esetén egy érvényes JWT tokennel tér vissza, amely tartalmazza a jogosultsági szintet is.
- **POST /api/Auth/Register** - Új fiók létrehozása. Ellenőrzi az adatok egyediségét, titkosítja a jelszót, majd elmenti az új felhasználót az adatbázisba.

2.13 API/appsetting.json



A .NET Core architektúrában az appsettings.json fájl funkcionál a backend (API) "vezérlőpultjaként". Ez a központi konfigurációs állomány tartalmazza azokat a környezeti változókat és titkosítási kulcsokat, amelyeket az alkalmazás futásidőben (runtime) olvas be. Ennek a megközelítésnek a legnagyobb előnye, hogy a rendszer alapvető paraméterei (pl. adatbázis-szerver címe, token lejárat ideje) anélkül módosíthatók, hogy a teljes C# forráskódot újra kellene fordítani.

A KoliPortál konfigurációs fájlja két fő, üzletileg kritikus blokkra oszlik:

1. **Adatbázis Kapcsolat (ConnectionStrings)** Ebben a blokkban található a Default nevű kapcsolati karakterlánc (Connection String). Ez adja meg a korábban bemutatott KoliportalDbContext számára, hogy fizikailag hol található az SQL Server, mi az adatbázis neve (KoliPortal), és milyen hitelesítő adatokkal (User=sa; Password=...) kell csatlakozni.
2. **JWT Hitelesítési Paraméterek (Jwt)** Ez a szekció szolgáltatja a paramétereket a hitelesítésért felelős JWTTokenService számára. A fájlban elhelyezett magyar nyelvű fejlesztői kommentek is jól mutatják ezen értékek funkcióját:

- **Key (Titkos Kulcs):** Egy hosszú, komplex karaktersorozat, amely a tokenek digitális aláírására szolgál (HMAC-SHA256 algoritmussal). Minél bonyolultabb ez a kulcs, annál lehetetlenebb a tokenek feltörése (Brute Force támadás).
- **Issuer (Kibocsátó):** Azonosítja, hogy melyik szerver állította ki a tokenet (KoliPortalAPI). A validáció során a rendszer ellenőrzi, hogy a token tényleg saját magától származik-e.
- **Audience (Célközönség):** Meghatározza, hogy kiknek (milyen kliensalkalmazásoknak) készült a token (KoliPortalClient).
- **ExpiresMinutes (Lejáratási idő):** A munkamenet biztonsági korlátja. Jelenleg 60 percre van állítva. Ha a felhasználó egy órán át nem végez műveletet, a token lejár, és a rendszer automatikusan kilépteti, megakadályozva az örízetlenül hagyott gépeken történő visszaéléseket (Session Hijacking elleni védelem).

2.14 API/Program.cs

```
builder.Services.AddDbContext<KoliportalDbContext>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
// A token tulajdonságainak beállítása a konfigurációs fájlból például a JWT kulcs, issuer, audience és lejáratási idő beállítása a JWTTokenService szolgáltatásban.
var jwtSettings = builder.Configuration.GetSection("Jwt");

// Jwt Scheme beállítása a hitelesítéshez, amely meghatározza, hogy a JWT tokeneket hogyan kell kezelni és érvényesíteni az alkalmazásban.

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme; // Alapvető hitelesítési sémá beállítása a JWT Bearer sémá használatára.
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme; // Alapvető kihívás sémá beállítása a JWT Bearer sémá használatára.
})
.AddJwtBearer(options => options.TokenValidationParameters = new Microsoft.IdentityModel.Tokens.TokenValidationParameters
{
    ValidateIssuer = true, // Ki állította ki
    ValidateAudience = true, // Kinek szól
    ValidateLifetime = true, // Lejárati idő
    ValidateIssuerSigningKey = true, // Aláírás érvényes-e

    ValidIssuer = jwtSettings["Issuer"], // A JWT token érvényességének ellenőrzése a megadott issuer értéke alapján.
    ValidAudience = jwtSettings["Audience"], // A JWT token érvényességének ellenőrzése a megadott audience értéke alapján.
    IssuerSigningKey = new Microsoft.IdentityModel.Tokens.SymmetricSecurityKey(System.Text.Encoding.UTF8.GetBytes(jwtSettings["Key"])), // A JWT token aláírásának érvényességének ellenőrzése
    ClockSkew = TimeSpan.Zero // Az időeltolódás beállítása a token érvényességének ellenőrzésekor, hogy ne legyen megengedett időeltolódás a token lejáratási idejéhez képest.
});
```

A szoftver belépési pontján (a Program.cs állományban) történik a rendszer alapvető szolgáltatásainak regisztrálása a beépített Függőséginjektálási (Dependency Injection - DI) konténerbe. Ez a kódrészlet két kritikus infrastrukturális elemet konfigurál: az adatbázis-kapcsolatot és a végpontok védelméért felelős JWT hitelesítési sémát.

1. Adatbázis Kontextus Regisztrációja (DbContext) A legelső sor (builder.Services.AddDbContext...) regisztrálja a korábban bemutatott KoliportalDbContext osztályt.

- **Működése:** A kód dinamikusan kiolvassa a "Default" nevű kapcsolati karakterláncot az appsettings.json fájlból, és utasítja az Entity Framework

Core-t, hogy SQL Server szolgáltatót (UseSqlServer) használjon az adatbázis-műveletekhez. Ezzel a megoldással a teljes rendszerben elég ezt az egyetlen regisztrációt megtenni, a Service osztályok pedig automatikusan megkapják a kész kapcsolatot a konstruktorukon keresztül.

2. A JWT Hitelesítés (Authentication) Felépítése A kód további része a rendszer biztonsági kapujának, a JWT (JSON Web Token) alapú hitelesítésnek a felépítését végzi. Ez az a middleware (köztesréteg), ami a vezérlők (Controllers) előtt áll, és minden bejövő hálózati kérést megvizsgál.

- **Hitelesítési Séma beállítása (AddAuthentication):** A kód explicit módon kijelöli, hogy az alkalmazás a modern és állapotmentes JwtBearerDefaults.AuthenticationScheme-t használja alapértelmezettként. Ez jelzi a .NET-nek, hogy a HTTP kérések fejlécében (Header) egy "Bearer" típusú tokent kell keresnie.
- **Token Validációs Paraméterek (TokenValidationParameters):** Ahhoz, hogy egy bejövő token a szerver érvényesnek (bejelentkezettnek) fogadjon el, szigorú biztonsági ellenőrzéseken kell átesnie. A kód négy fő paraméter vizsgálatát kényszeríti ki (mindegyik true értékre van állítva):
 - ValidateIssuer: Ellenőrzi, hogy a token tényleg a KoliPortál API állította-e ki (az appsettings.json-ben lévő érték alapján).
 - ValidateAudience: Biztosítja, hogy a token a megfelelő kliensnek szól.
 - ValidateLifetime: Ellenőrzi a token lejáratát. Ha a beállított 60 perc letelt, a szerver automatikusan eldobja a kérést.
 - ValidateIssuerSigningKey: A legfontosabb lépés; kriptográfiailag ellenőrzi a token digitális aláírását a szerver titkos kulcsa (Key) alapján. Ha a token módosították, az aláírás sérül, és a hitelesítés elbukik.

```
builder.Services.AddScoped<JWTTokenService>();
builder.Services.AddScoped<IDiakAdatok, DiakAdatokService>();
builder.Services.AddScoped<IFelhasznalok, FelhasznalokService>();
builder.Services.AddScoped<IFizetesTipusok, FizetesTipusokService>();
builder.Services.AddScoped<IKarbantartasiKeresek, KarbantartasiKeresekService>();
builder.Services.AddScoped<IKarbantartasStatuszok, KarbantartasStatuszokService>();
builder.Services.AddScoped<IKollegium, KollegiumService>();
builder.Services.AddScoped<IPenzugyek, PenzugyekService>();
builder.Services.AddScoped<ISzerepkorok, SzerepkorokService>();
builder.Services.AddScoped<ISzobaBeosztasok, SzobaBeosztasokService>();
builder.Services.AddScoped<ISzobak, SzobakService>();
```


A modern .NET Core alkalmazások lelke a beépített Függőséginjektálási (Dependency Injection - DI) konténer. Ebben a kódrészletben (Program.cs) történik meg a rendszer üzleti logikájának és a hozzájuk tartozó interfészeknek a központi regisztrációja.

```
app.UseAuthentication();
app.UseAuthorization(); /
```

app.UseAuthentication(); (Hitelesítés): Ez a réteg felel a felhasználó azonosításáért, ellenőrizve a bejövő kérésekben lévő JWT tokenet, hogy megállapítsa, **kicsoda** próbálja elérni a rendszert.

app.UseAuthorization(); (Jogosultság-ellenőrzés): Ez a réteg a már azonosított felhasználó szerepköre (pl. Diák vagy Admin) alapján dönti el, hogy az illetőnek **van-e joga** végrehajtani a kért műveletet a védett végpontokon.

2.15 Koliportal.Lib/Service

A **KoliPortal.Lib.SERVICE** névterületen elhelyezkedő osztályok alkotják a szoftver kliensoldali hálózati rétegét. Míg a backend szolgáltatások közvetlenül az adatbázissal kommunikálnak, ezek a frontend szolgáltatások jelentik a hidat a felhasználói felület (Blazor Razor komponensek) és a védett REST API végpontok között.

Ennek a fontosságai az alábbi pontokban olvasható:

- **Felelősségek szétválasztása (Clean Code):** A Blazor weboldalak (UI) kódja mentes marad a bonyolult hálózati kérésektől és JSON szeriálizációs logikától. A weboldal csupán meghívja a megfelelő szolgáltatást, amely elrejti a HTTP kommunikáció komplexitását.
- **Központosított Biztonság és Hitelesítés:** A kódképeken jól látható, hogy minden egyes metódus bemeneti paraméterként várja a felhasználó JWT tokenjét (string token). A szolgáltatás a kérés elküldése előtt ezt a tokenet automatikusan hozzáfűzi a HTTP fejlécéhez (`DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token)`). Ezzel garantált, hogy egyetlen védett API hívás sem indul el hitelesítő adatok nélkül a hálózaton.
- **Kód-újrahasznosítás (DRY elv):** Mivel ezek az API hívó osztályok a közös (KoliPortal.Lib) projektben kaptak helyet, a jövőben a webes felület mellé

könnyedén fejleszthető lenne egy mobilalkalmazás (pl. .NET MAUI) is. A mobilalkalmazás újra tudná használni ezeket a hálózati szolgáltatásokat anélkül, hogy az API kommunikációt újra le kellene programozni.

Az osztályok a .NET beépített **HttpClient** szolgáltatását használják aszinkron módon (async/await). A JSON adatok küldését és fogadását a modern, teljesítményoptimalizált `GetFromJsonAsync`, `PostAsJsonAsync` és `PutAsJsonAsync` kiterjesztésekkel végzik, míg az esetleges szerveroldali hibákra (pl. 401 Unauthorized, 404 Not Found) az `EnsureSuccessStatusCode()` metódus hívása figyel, amely hiba esetén azonnal megszakítja a folyamatot, elkerülve a hibás adatok feldolgozását a kliensoldalon.

2.15.1 DiakAdatokService

A hallgatók specifikus adatainak (lakcím, szemeszter) HTTP kéréseit (GET, POST, PUT, DELETE) csomagolja be, biztosítva a JWT token alapú biztonságos adatátvitelt a profiloldalak számára.

```
namespace KoliPortal.Lib.SERVICE
{
    8 references
    public class DiakAdatokService
    {
        private readonly HttpClient _httpClient;
        0 references
        public DiakAdatokService(HttpClient client)
        {
            _httpClient = client;
        }

        1 reference
        public async Task<List<DiakAdatok>> GetDiakAdatok(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            return await _httpClient.GetFromJsonAsync<List<DiakAdatok>>("api/DiakAdatok") ?? new List<DiakAdatok>();
        }

        1 reference
        public async Task UpdateDiakAdatok(DiakAdatok adat, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PutAsJsonAsync("api/DiakAdatok", adat);
            response.EnsureSuccessStatusCode();
        }

        2 references
        public async Task CreateDiakAdatok(DiakAdatok adat, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PostAsJsonAsync("api/DiakAdatok", adat);
            response.EnsureSuccessStatusCode();
        }

        2 references
        public async Task DeleteDiakAdatok(int id, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.DeleteAsync($"api/DiakAdatok/{id}");
            response.EnsureSuccessStatusCode();
        }
    }
}
```


2.15.2 FelhasználokService

A regisztrációs és profilkezelési UI komponensek hálózati motorja. Feladata a felhasználói adatbázis tartalmának lekérése, valamint az új vagy módosított profiladatok JSON formátumban történő továbbítása az API felé.

```
namespace KoliPortal.Lib.SERVICE
{
    16 references
    public class FelhasznalokService
    {
        private readonly HttpClient _httpClient;
        0 references
        public FelhasznalokService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        7 references
        public async Task<List<Felhasznalok>> GetFelhasznalok(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            return await _httpClient.GetFromJsonAsync<List<Felhasznalok>>("api/Felhasznalok") ?? new List<Felhasznalok>();
        }

        2 references
        public async Task UpdateFelhasznalok(Felhasznalok adat, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PutAsJsonAsync("api/Felhasznalok", adat);
            response.EnsureSuccessStatusCode();
        }

        1 reference
        public async Task DeleteFelhasznalok(int id, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.DeleteAsync($"api/Felhasznalok/{id}");
            response.EnsureSuccessStatusCode();
        }

        0 references
        public async Task CreateFelhasznalok(Felhasznalok adat, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PostAsJsonAsync("api/Felhasznalok", adat);
            response.EnsureSuccessStatusCode();
        }
    }
}
```

2.15.3 FizetesTipusokService

Egy dedikált, csak olvasható (GET) hálózati szolgáltatás, amely a pénzügyi űrlapok legördülő listáit tölti fel a szerverről lekérdezett választható fizetési kategóriákkal.

```
namespace KoliPortal.Lib.SERVICE
{
    8 references
    public class FizetesTipusokService
    {
        private readonly HttpClient _httpClient;
        0 references
        public FizetesTipusokService(HttpClient client)
        {
            _httpClient = client;
        }

        3 references
        public async Task<List<FizetesTipusok>> GetFizetesTipusok(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            return await _httpClient.GetFromJsonAsync<List<FizetesTipusok>>("api/FizetesTipusok") ?? new List<FizetesTipusok>();
        }
    }
}
```

2.15.4 KarbantartasiKeresekService

A hibabejelentő felület kommunikációs csatornája. Ezen keresztül küldi el a Blazor kliens a diákok új hibajegyeit az API-nak, és ezen keresztül kéri le a karbantartók számára a nyitott feladatok listáját.

```
namespace KoliPortal.Lib.SERVICE
{
    12 references
    public class KarbantartasiKeresekService
    {
        private readonly HttpClient _httpClient;
        0 references
        public KarbantartasiKeresekService(HttpClient client)
        {
            _httpClient = client;
        }

        5 references
        public async Task<List<KarbantartasiKeresek>> GetKarbantartasiKeresek(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            return await _httpClient.GetFromJsonAsync<List<KarbantartasiKeresek>>("api/KarbantartasiKeresek") ?? new List<KarbantartasiKeresek>();
        }

        2 references
        public async Task DeleteKarbantartasiKeresek(int id, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.DeleteAsync($"api/KarbantartasiKeresek/{id}");
            response.EnsureSuccessStatusCode();
        }

        1 reference
        public async Task UpdateKarbantartasiKeresek(KarbantartasiKeresek adat, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PutAsJsonAsync($"api/KarbantartasiKeresek", adat);
            response.EnsureSuccessStatusCode();
        }

        1 reference
        public async Task CreateKarbantartasiKeresek(KarbantartasiKeresek adat, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PostAsJsonAsync("api/KarbantartasiKeresek", adat);
            response.EnsureSuccessStatusCode();
        }
    }
}
```

2.15.5 KarbantartasStatuszokService

A hibajegyek adminisztrációs felületét kiszolgáló lekérdező (GET) osztály, amely a feladatokhoz rendelhető állapotokat (Új, Folyamatban, Kész) hozza le a szerverről.

```
namespace KoliPortal.Lib.SERVICE
{
    3 references
    public class KarbantartasStatuszokService
    {
        private readonly HttpClient _httpClient;
        1 reference
        public KarbantartasStatuszokService(HttpClient client)
        {
            _httpClient = client;
        }
        2 references
        public async Task<List<KarbantartasStatuszok>> GetKarbantartasStatuszok(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);
            return await _httpClient.GetFromJsonAsync<List<KarbantartasStatuszok>>("api/KarbantartasStatuszok") ?? new List<KarbantartasStatuszok>();
        }
    }
}
```

2.15.6 KollegiumService

A regisztrációs űrlap hálózati függősége; az API-tól kéri le a választható kollégiumi intézmények listáját.

```
namespace KoliPortal.Lib.SERVICE
{
    4 references
    public class KollegiumService
    {
        private readonly HttpClient _httpClient;
        0 references
        public KollegiumService(HttpClient client)
        {
            _httpClient = client;
        }
        1 reference
        public async Task<List<Kollegium>> GetKollegium(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);
            return await _httpClient.GetFromJsonAsync<List<Kollegium>>("api/Kollegium") ?? new List<Kollegium>();
        }
    }
}
```

2.15.7 PenzugyekService

A pénzügyi modul teljes körű (CRUD) HTTP kliense. Biztosítja, hogy a diákok befizetései és tartozásai szinkronba kerüljenek a backend adatbázissal.

```
namespace KoliPortal.Lib.SERVICE
{
    12 references
    public class PenzugyekService
    {
        private readonly HttpClient _httpClient;
        0 references
        public PenzugyekService(HttpClient client)
        {
            _httpClient = client;
        }

        5 references
        public async Task<List<Penzugyek>> GetPenzugyek(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            return await _httpClient.GetFromJsonAsync<List<Penzugyek>>("api/Penzugyek") ?? new List<Penzugyek>();
        }

        1 reference
        public async Task DeletePenzugyek(int id, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.DeleteAsync($"api/Penzugyek/{id}");
            response.EnsureSuccessStatusCode();
        }

        1 reference
        public async Task CreatePenzugyek(Penzugyek penzugyek, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PostAsJsonAsync("api/Penzugyek", penzugyek);
            response.EnsureSuccessStatusCode();
        }

        1 reference
        public async Task UpdatePenzugyek(Penzugyek penzugyek, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PutAsJsonAsync($"api/Penzugyek", penzugyek);
            response.EnsureSuccessStatusCode();
        }
    }
}
```

2.15.8 SzerepkorokService

A felhasználókezelő felületek lekérdező szolgáltatása. Segítségével a UI megkapja a szerveren konfigurált jogosultsági szinteket (Admin, Lakó) az új fiókok kiosztásához.

```
namespace KoliPortal.Lib.SERVICE
{
    public class SzerepkorokService
    {
        private readonly HttpClient _httpClient;

        public SzerepkorokService(HttpClient httpClient)
        {
            _httpClient = httpClient;
        }

        1 reference
        public async Task<List<Szerepkorok>> GetSzerepkorok(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            return await _httpClient.GetFromJsonAsync<List<Szerepkorok>>("api/Szerepkorok") ?? new List<Szerepkorok>();
        }
    }
}
```

2.15.9 SzobaBeosztasokService

A kapacitáskezelő és beköltöztető felületek hálózati hidja. Feladata a szobákhoz rendelt hallgatói kapcsolatok létrehozásának, módosításának és törlésének elküldése az API-nak.

```
namespace KoliPortal.Lib.SERVICE
{
    18 references
    public class SzobaBeosztasokService
    {
        private readonly HttpClient _httpClient;
        0 references
        public SzobaBeosztasokService(HttpClient client)
        {
            _httpClient = client;
        }

        8 references
        public async Task<List<SzobaBeosztasok>> GetSzobaBeosztasok(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            return await _httpClient.GetFromJsonAsync<List<SzobaBeosztasok>>("api/SzobaBeosztasok") ?? new List<SzobaBeosztasok>();
        }

        2 references
        public async Task UpdateSzobaBeosztasok(SzobaBeosztasok adat, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);
            var response = await _httpClient.PutAsJsonAsync("api/SzobaBeosztasok", adat);
            response.EnsureSuccessStatusCode();
        }

        3 references
        public async Task CreateSzobaBeosztasok(SzobaBeosztasok adatok, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.PostAsJsonAsync("api/SzobaBeosztasok", adatok);
            response.EnsureSuccessStatusCode();
        }

        2 references
        public async Task DeleteSzobaBeosztasok(int id, string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            var response = await _httpClient.DeleteAsync($"api/SzobaBeosztasok/{id}");
            response.EnsureSuccessStatusCode();
        }
    }
}
```

2.15.10 SzobakService

A szobabeosztásokhoz és az áttekintő műszerfalhoz (Dashboard) szükséges alapadatokat (szobaszám, max. férőhely) lekérdező aszinkron hálózati szolgáltatás.

```
namespace KoliPortal.Lib.SERVICE
{
    16 references
    public class SzobakService
    {
        private readonly HttpClient _httpClient;
        0 references
        public SzobakService(HttpClient client)
        {
            _httpClient = client;
        }

        7 references
        public async Task<List<Szobak>> GetSzobak(string token)
        {
            _httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);

            return await _httpClient.GetFromJsonAsync<List<Szobak>>("api/Szobak") ?? new List<Szobak>();
        }
    }
}
```


2.16 Kezdolap.razor

A Kezdolap.razor komponens a KoliPortál belépési pontja (Landing Page). Feladata a szoftver értékajánlatának bemutatása, a funkciók ismertetése, valamint a felhasználók átirányítása a bejelentkezési felületre. Mivel ez egy publikus, hitelesítést nem igénylő oldal, vizuálisan és logikailag is teljesen elkülönül a belső adminisztrációs moduloktól.

2.16.1 Útválasztás és Keretbeágyazás

```
@page "/"
@layout MainLayout
@* Vagy a saját üres layoutod, ha van olyan, ami nem tartalmazza az oldalsó menüt *@
```

- **@page "/"**: Ez a direktíva jelzi a Blazor útválasztójának (Router), hogy ez a szoftver gyökérkönyvtára, azaz ez töltődik be alapértelmezetten a webcím (pl. koliportal.hu/) megnyitásakor.
- **@layout MainLayout**: Ahogy azt az elrendezéseknél már definiáltuk, ez a parancs utasítja a komponenst, hogy a belső menüsávok keretek helyett a teljesen "üres vásznat" biztosító MainLayout-ot használja. Így a kezdőlap 100%-ban kitöltheti a képernyőt.

2.16.2 Navigáció és Hero Szekció

```


<nav class="landing-navbar">
  <div class="nav-brand">
    <a href="#rolunk" style="text-decoration: none; color: inherit; display: flex; align-items: center; gap: 0.5rem;" @onclick:preventDefault>
      <i class="bi bi-building"></i> <span>KoliPortál</span>
    </a>
  </div>

  <div class="d-flex align-items-center">
    <div class="nav-links d-none d-md-flex me-2">
      <a href="#rolunk" @onclick:preventDefault>Rólunk</a>
      <a href="#funkciok" @onclick:preventDefault>Funkciók</a>
      <a href="#kapcsolat" @onclick:preventDefault>Kapcsolat</a>
    </div>
    <div class="nav-actions">
      <a href="/login" class="btn btn-primary login-btn" style="text-decoration: none;">Bejelentkezés</a>
    </div>
  </div>
</nav>

<header class="hero-section" id="rolunk">
  <div class="hero-content text-center">
    <h1 class="hero-title">Üdvözlünk a <span class="text-highlight">KoliPortálon!</span></h1>
    <p class="hero-subtitle">
      Minden kollégiumi ügyintézés egy helyen. Egyszerű, gyors, <br class="d-none d-md-block" />
      modern megoldás diákoknak és adminisztrátoroknak.
    </p>
    <div class="hero-buttons">
      <a href="/login" class="btn btn-primary btn-lg action-btn" style="text-decoration: none;">
        Belépés a portálra <i class="bi bi-arrow-right ms-2"></i>
      </a>
      <a href="#funkciok" class="btn btn-outline-secondary btn-lg outline-btn">
        Tudj meg többet
      </a>
    </div>
  </div>
</header>


```

- **Landing Navbar**: Egy dedikált, reszponzív navigációs sáv, amely horgonylinkekkel ([href="#rolunk"](#)) navigál az egyoldalas lapon belül. A jobb felső

sarokban kapott helyet a legfontosabb "Call to Action" (Cselekvésre ösztönző) gomb, amely a /login oldalra, azaz a bejelentkezéshez vezet.

- **Hero Section:** A webdizájnban "Hero"-nak nevezzük a hajtás feletti (görgetés nélkül látható) legelső, nagyméretű vizuális blokkot. Itt jelenik meg a fő üzenet ("Üdvözlünk a KoliPortálon!"). A Bootstrap keretrendszer gombosztályainak (btn-primary btn-lg) használatával a felület azonnal egyértelművé teszi a látogató számára a következő lépést ("Belépés a portálra").

2.16.3 Funkciók Bemutatása

```
<section class="features-section" id="funkciok">
  <div class="container">
    <div class="text-center mb-5">
      <h2 class="section-title">Miért a KoliPortál?</h2>
      <p class="section-subtitle">Fedezd fel a rendszerünk nyújtotta előnyöket és kényelmi funkciókat.</p>
    </div>

    <div class="row g-4 justify-content-center">

      <div class="col-12 col-md-4">
        <div class="feature-card">
          <div class="icon-wrapper">
            <i class="bi bi-calendar-check"></i>
          </div>
          <h3>Szobabeosztás</h3>
          <p>Lásd át könnyedén a szobabeosztásokat, jelentkezz a szabad helyekre és találd meg a legjobb szobátársakat egyszerűen.</p>
        </div>
      </div>

      <div class="col-12 col-md-4">
        <div class="feature-card">
          <div class="icon-wrapper">
            <i class="bi bi-tools"></i>
          </div>
          <h3>Hibabejelentés</h3>
          <p>Valami elromlott? Jelezd pár kattintással az adminisztrátornak, és kövesd nyomon a javítás állapotát valós időben.</p>
        </div>
      </div>

      <div class="col-12 col-md-4">
        <div class="feature-card">
          <div class="icon-wrapper">
            <i class="bi bi-receipt"></i>
          </div>
          <h3>Befizetések</h3>
          <p>Fizesd be a kollégiumi díjakat online, gyorsan és biztonságosan. Tekintsd meg a korábbi tranzakcióidat bármikor.</p>
        </div>
      </div>

    </div>
  </div>
</section>
```

- **Reszponzív Rácsszerkezet (Grid System):** A kód a Bootstrap rácsrendszerét (row g-4) használja. A col-12 col-md-4 osztályok biztosítják, hogy a három információs kártya asztali nézetben egymás mellett (három oszlopban), míg mobiltelefonon egymás alatt (egyetlen oszlopba törve) jelenjen meg, garantálva a tökéletes olvashatóságot minden eszközön.
- **Kártya Dizájn (Card UI):** A Szobabeosztás, Hibabejelentés és Befizetések modulokat modern, ikonokkal (bi-tools, bi-receipt) ellátott kártyákon jeleníti meg a kód, amely vizuálisan vonzó és könnyen feldolgozható.

2.16.4 Kapcsolat és Lábléc

```
<footer class="landing-footer" id="kapcsolat">
  <div class="container">
    <div class="row gy-4">
      <div class="col-12 col-md-4 brand-col">
        <div class="footer-brand">
          <i class="bi bi-building"></i> KoliPortál
        </div>
        <p class="footer-desc">A modern kollégiumi élet digitális központja.</p>
      </div>
      <div class="col-12 col-md-4 links-col">
        <h4 class="footer-title">Gyorslinkek</h4>
        <ul class="footer-list">
          <li><a href="#rolunk">Kezdőlap</a></li>
          <li><a href="#rolunk">Rólunk</a></li>
        </ul>
      </div>
      <div class="col-12 col-md-4 contact-col">
        <h4 class="footer-title">Kapcsolat</h4>
        <ul class="footer-list">
          <li><i class="bi bi-geo-alt"></i> 1234 Budapest, Egyetem tér 1.</li>
          <li><i class="bi bi-envelope"></i> koliportal@gmail.com</li>
          <li><i class="bi bi-telephone"></i> +36 30 123 4567</li>
        </ul>
      </div>
    </div>
  </div>
</footer>
</div>
```

Az oldal legalsó szekciója (landing-footer). A funkciókat bemutató részhez hasonlóan ez is egy többoszlopos Bootstrap rácsra épül. Három logikai blokkot tartalmaz:

1. **Brand:** A KoliPortál logója és rövid szlogenje.
2. **Gyorslinkek:** Belső navigációs segédlet.
3. **Kapcsolat:** Itt található a kollégium hivatalos elérhetőségei (Cím, E-mail, Telefonszám), amelyeket beszédes Bootstrap ikonok (bi-geo-alt, bi-telephone) tesznek könnyen felismerhetővé.

2.17 Login.razor

A Login.razor komponens a KoliPortál biztonsági kapuja. Felelőssége a felhasználói hitelesítő adatok (e-mail és jelszó) bekérése, azok továbbítása az API felé, majd sikeres azonosítás esetén a JWT token eltárolása és a felhasználó megfelelő (Admin vagy Diák) felületre történő átirányítása.

2.17.1 Direktívák és Szolgáltatás-injekciók

```
@page "/login"
@rendermode InteractiveServer
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@using System.IdentityModel.Tokens.Jwt @* Ez a csomag kell a Token visszafejtéséhez *@
@inject AuthControllerService AuthController
@inject NavigationManager Nav
@inject IJSRuntime JS
```

A fájl fejléce határozza meg a komponens futtatási környezetét és függőségeit:

- **@page "/login"**: Kijelöli az útvonalat.
- **@rendermode InteractiveServer**: Létfontosságú direktíva, amely engedélyezi a Blazor szerveroldali interaktivitását, így a gombnyomások és az űrlapküldések azonnal, oldalfrissítés nélkül dolgozhatnak fel.
- **Injektálások (@inject)**: A kód behívja a hálózati kommunikációhoz szükséges AuthService-t, a navigációt végző NavigationManager-t, valamint az IJSRuntime-ot a böngésző helyi tárhelyének (Local Storage) kezeléséhez.

2.17.2 Felhasználói Felület és Kétirányú Adatkötés

```
<div class="login-container">
  <div class="login-card">
    <div style="text-align: left; margin-bottom: 10px;">
      <a href="/" style="text-decoration: none; color: #64748b; font-size: 0.9rem; transition: color 0.2s; onmouseover="this.style.color='#8d8368'" onmouseout="this.style.color='#64748b'">
        <i class="bi bi-arrow-left"></i> Vissza a kezdőlapra
      </a>
    </div>
    <div class="card-header">
      <div class="logo-container">
        <i class="bi bi-building logo-icon" style="font-size: 2.5rem;"></i>
        <h1 class="logo-text">KöliPortál</h1>
      </div>
      <h2 class="welcome-text">Üdv újra itt!</h2>
      <p class="subtitle-text">Jelentkez be a fiókodba.</p>
    </div>
    @if (!string.IsNullOrEmpty(hibaUzenet))
    {
      <div class="alert alert-danger">
        @hibaUzenet
      </div>
    }
  </div>
</div>
```

```
@ DIV HELYETI FORM LEÍRÁS ÉS MÓDOKAT AZ ÖSSZEÍRÁS
<form class="form-container" @onsubmit="HandleLogin">
  <div class="form-group">
    <label for="Mollégium" Mollégium</label>
    <div class="input-wrapper" style="position: relative;">
      <i class="bi bi-buildings input-icon" style="font-size: 1.2rem; color: #94a3b8; position: absolute; left: 14px; top: 50%; transform: translateY(-50%);"></i>
      <input type="text" value="" class="form-control" style="width: 100%; height: 46px; padding: 10px 14px 10px 42px; border: 1px solid #e5e9ec; border-radius: 8px; font-size: 0.95rem; color: #1a233a; background-color: #ffffff; font-family: inherit; cursor: not-allowed; -webkit-appearance: none; -moz-appearance: none; appearance: none;" />
    </div>
  </div>
  <div class="form-group">
    <label for="Email cím" Email cím</label>
    <div class="input-wrapper" style="position: relative;">
      <i class="bi bi-envelope input-icon" style="font-size: 1.2rem; color: #94a3b8; position: absolute; left: 14px; top: 50%; transform: translateY(-50%);"></i>
      <input type="email" id="email" placeholder="dia@kollportál.hu" @bind="BeviteliAdatok.Email" style="width: 100%; height: 46px; padding: 10px 14px 10px 42px; border: 1px solid #e5e9ec; border-radius: 8px; font-size: 0.95rem; color: #1a233a; background-color: #ffffff; font-family: inherit; box-sizing: border-box;" />
    </div>
  </div>
  <div class="form-group">
    <label for="password" Jelszó</label>
    <div class="input-wrapper" style="position: relative;">
      <i class="bi bi-lock input-icon" style="font-size: 1.2rem; color: #94a3b8; position: absolute; left: 14px; top: 50%; transform: translateY(-50%);"></i>
      <input type="password" id="password" placeholder="*****" @bind="BeviteliAdatok.Jelszo" style="width: 100%; height: 46px; padding: 10px 14px 10px 42px; border: 1px solid #e5e9ec; border-radius: 8px; font-size: 0.95rem; color: #1a233a; background-color: #ffffff; font-family: inherit; box-sizing: border-box;" />
    </div>
  </div>
  <button type="submit" class="btn-submit" disabled="@toltesAlatt" style="width: 100%; height: 46px; margin-top: 10px; background: #8d8368; color: white; border: none; border-radius: 8px; font-weight: 600; font-size: 1rem; cursor: pointer;">
    @if (@toltesAlatt)
    {
      <span class="spinner-border spinner-border-sm me-2"></span>
      <span>Működik várj...</span>
    }
    else
    {
      <i class="bi bi-box-arrow-in-right me-2"></i>
      <span>Bejelentkezés</span>
    }
  </button>
  <div class="privacy-footer">
    <a href="/adatevelen" class="privacy-link">Adatkezelési tájékoztató</a>
  </div>
</form>
</div>
```

A vizuális felület tervezésekor a modern megjelenés mellett a hibatűrés és a felhasználói visszajelzések (UX) kapták a főszerepet.

- **Kétirányú adatkötés (@bind)**: Az e-mail és jelszó beviteli mezők a `@bind="BeviteliAdatok.Email"` szintaxissal közvetlenül össze vannak kötve a C# kód háttérváltozóival. Amint a felhasználó gépel, az adatok azonnal szinkronizálódnak a memóriában.

- **Állapotfüggő UI (Töltés jelző):** A "Bejelentkezés" gomb tartalmaz egy apró, de fontos logikát: `@if (toltesAlatt)`. Amikor az API hívás folyamatban van, a gomb felirata "Kérlek várj..."-ra változik, kap egy forgó animációt (spinner), és a gomb inaktívvá (disabled) válik. Ez a megoldás elegánsan megakadályozza, hogy a türelmetlen felhasználók többször is rákattintsanak a gombra, miközben a szerver dolgozik.
- **Dinamikus Hibajelzés:** Egy dedikált HTML blokk (alert-danger) csak akkor jelenik meg az űrlap felett, ha a `hibaUzenet` változó értéke nem üres, azonnali vizuális visszajelzést adva az elgépelésekről vagy a hálózati hibákról.

2.17.3 Az Üzleti Logika és az API Kommunikáció

```
@code {
    // Saját, ideiglenes tároló a form adatainak
    public class LoginDto
    {
        public string Email { get; set; } = "";
        public string Jelszo { get; set; } = "";
    }

    private LoginDto beviteliAdatok = new LoginDto();
    private string hibaUzenet = "";
    private bool toltesAlatt = false;

    private async Task HandleLogin()
    {
        hibaUzenet = "";

        if (string.IsNullOrEmpty(beviteliAdatok.Email) || string.IsNullOrEmpty(beviteliAdatok.Jelszo))
        {
            hibaUzenet = "Kérlek tölts ki minden mezőt!";
            return;
        }

        toltesAlatt = true;

        try
        {
            // Áttöltjük a beírt adatokat az API által várt modellbe
            var loginCsomag = new KoliPortal.Lib.MODEL.Login
            {
                Email = beviteliAdatok.Email,
                Jelszo = beviteliAdatok.Jelszo
            };

            // Elküldjük a kérést
            string kapottToken = await AuthController.LoginAsync(loginCsomag);

            if (!string.IsNullOrEmpty(kapottToken))
            {
                // Eltesszük a tokenet a böngésző "zsebébe", hogy a többi oldal is lássa
                await JS.InvokeVoidAsync("localStorage.setItem", "authToken", kapottToken);

                // Kinyerjük a szerepkört a tokenből
                string szerepkor = SzerepkorKinyerese(kapottToken);

                // Szerepkör alapján átírányítja a felhasználót
                if (szerepkor == "1" || szerepkor.ToLower() == "admin")
                {
                    Nav.NavigateTo("/adminAttekintes");
                }
                else if (szerepkor == "2" || szerepkor.ToLower() == "lako")
                {
                    Nav.NavigateTo("/lakoAttekintes");
                }
            }
            else
            {
                hibaUzenet = "Hibás email vagy jelszó!";
            }
        }
        catch (Exception ex)
        {
            hibaUzenet = "Hiba történt a szerverhez való kapcsolódáskor.";
        }
        finally
        {
            toltesAlatt = false;
        }
    }
}
```

A <form @onsubmit="HandleLogin"> esemény hatására lefutó aszinkron metódus a rendszer biztonsági "motorja". A folyamat lépései:

1. **Kliensoldali Validáció:** A rendszer elsőként leellenőrzi (string.IsNullOrEmpty), hogy mindkét mezőt kitöltötték-e. Ha nem, hálózati forgalom generálása nélkül hibaüzenetet ad.

2. **DTO (Data Transfer Object) Képzés:** A helyi űrlap adatait becsomagolja a közös MODEL.Login objektumba, ami pontosan az a formátum, amit a backend API elvár.
3. **API Hívás:** Az `await AuthController.LoginAsync(...)` sor elküldi az adatokat a szervernek. A rendszer itt blokkolatlanul (aszinkron) várakozik a válasza.
4. **Token Mentése:** Sikeres válasz (kapott JWT token) esetén a C# kód JavaScript Interop-ot (`JS.InvokeVoidAsync`) használva elmenti a token a böngésző `localStorage`-ébe, létrehozva ezzel a hitelesített munkamenetet (Session).

2.17.4 Szerepkör-alapú Útválasztás

```
// JWT TOKEN VISSZAFEJTŐ
private string SzerepkörKinyerese(string jwtCsomag)
{
    try
    {
        // Létrehozunk egy "olvasót", ami a Microsoft beépített eszköze JWT tokenekhez.
        // Ez tudja, hogyan kell a három részből álló, kódolt szöveget biztonságosan szétbontani.
        var handler = new JwtSecurityTokenHandler();

        // Az olvasóval feldolgoztatjuk a nyers, kódolt token (pl. "eyJhbGciOiI...").
        // Eredményül kapunk egy 'token' objektumot, amiben már olvasható formában vannak az adatok (Claims).
        var token = handler.ReadJwtToken(jwtCsomag);

        // Heresés az adatok (Claims) között.
        // A FirstOrDefault végigmegy a tokenben lévő összes adaton, és megkeresi a szerepkört.
        // A .NET kétféleképpen mentheti el a szerepkört:
        // - Vagy egyszerűen "role" néven.
        // - Vagy a hivatalos XML séma URL-jével
        // A || miatt mindkét esetben megtaláljuk!
        var szerepkorAdat = token.Claims.FirstOrDefault(x =>
            x.Type == "role" ||
            x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role");

        // Ha találtunk szerepkört, visszaadjuk annak az értékét (pl. az "1"-es számot szöveggént).
        // A '?' megvéd a hibától: ha a szerepkorAdat null (nem talált semmit), nem omlik össze,
        // hanem a '?' utáni üres stringet (") adja vissza.
        return szerepkorAdat?.Value ?? "";
    }
    catch
    {
        // Ha a token formátuma teljesen rossz vagy hibás, biztonsági okokból üres szöveget adunk vissza.
        return "";
    }
}
```

A bejelentkezés utolsó, kritikus fázisa a felhasználó megfelelő helyre történő irányítása:

- A `SzerepkörKinyerese` metódus (a `JwtSecurityTokenHandler` segítségével) a kliens memóriájában olvassa ki a kapott tokenből a felhasználó jogosultsági szintjét (Role Claim).
- Egy logikai elágazás (if-else) dönti el az irányt: ha a szerepkör "1" vagy "admin", a `NavigationManager` a rendszergazdai műszerfalra (`/adminAttekintes`) viszi a felhasználót. Ha "2" vagy "lako", akkor a diák felületre (`/lakoAttekintes`).
- A teljes folyamat egy try-catch-finally blokkba van ágyazva. Bármilyen váratlan hiba (pl. megszakadt internetkapcsolat) esetén a program nem omlik össze, hanem

barátságos hibaüzenetet ad, a finally blokk pedig garantálja, hogy a töltésjelző animáció (toltesAlatt = false) mindenképpen eltűnjön a képernyőről.

2.18 Adatvedelem.razor

```

@page "/Adatvedelem"
@rendermode InteractiveServer
@inject NavigationManager Nav

<div class="privacy-container">
  <div class="privacy-card">
    <div class="card-header text-center">
      <div class="logo-container mb-3">
        <img alt="Shield icon" class="bi bi-shield-check" style="font-size: 3em; color: #008368;"/>
      </div>
      <h1 class="welcome-text">Adatvédelmi Tájékoztató</h1>
      <p class="subtitle-text">Hatályos: 2026. április 1-től</p>
    </div>
    <div class="card-body privacy-content">
      <p>A KoliPortál (továbbiakban: Rendszer) üzemeltetője kiemelt figyelmet fordít a felhasználók (kollégisták, adminisztrátorok) személyes adatainak védelmére.</p>
      <h3>1. Az Adatkezelő adatai</h3>
      <ul>
        <li>Név: Marinho Frigyes Kollégium Intézményfenntartója</li>
        <li>Email: adatvedelem@koliportal.hu</li>
        <li>Rendszer: KoliPortál Kollégiumi Nyilvántartó Rendszer</li>
      </ul>
      <h3>2. A kezelt adatok köre</h3>
      <p>A rendszer az alábbi személyes és adminisztratív adatokat kezeli a regisztráció és a használat során:</p>
      <ul>
        <li>Azonosítási adatok: Név, Email cím, Titkosított jelszó</li>
        <li>Kapcsolattartási adatok: Telefonszám, Lakcím</li>
        <li>Tanulmányi adatok: Évfolyam</li>
        <li>Kollégiumi adatok: Szobaszám (Kollégium épülete, szobaszám), beöltöztetés és várható kiöltöztetés dátuma</li>
        <li>Pénzügyi adatok: Kollégiumi díjak, tartozások, befizetések állapota</li>
        <li>Karbantartási adatok: A lakó által leadott hibajegyek leírása és státusza</li>
      </ul>
      <h3>3. Az adatkezelés célja és jogalapja</h3>
      <p>Cél: A kollégiumi adminisztráció, szobabesorítások nyomon követése, a lakók tájékoztatása, a kollégiumi díjak számlázása és a karbantartási folyamatok (hibajegyek) menedzselése.</p>
      <h3>4. Adatbiztonság és adattárolás ideje</h3>
      <p>A Rendszer a jelszavakat egyirányú, visszafordíthatatlan titkosítással tárolja. A személyes adatokat harmadik fél részére marketing célból nem adjuk ki. Az adatokat a kollégiumi jogviszony fennállása alatt, illetve a jogviszony megszűnését követően a számviteli és adójogszabályokban meghatározott kötelező ideig (jelenleg 8 év) őrzük meg a pénzügyi adatok esetében.</p>
      <h3>5. Az érintettek jogai</h3>
      <p>Ön jogosult kérelmezni az adatkezelőnél a rá vonatkozó személyes adatokhoz való hozzáférést, azok helyesbítését, törlését vagy kezelésének korlátozását. A felhasználó bizonyos adatait (pl. név, telefonszám, lakcím) kizárólag az intézmény adminisztrátorán keresztül módosíthatja vagy töröltheti a rendszerből.</p>
    </div>
    <div class="card-footer text-center mt-4">
      <button class="btn btn-outline-secondary px-4 py-2 fw-bold" style="border-radius: 8px;" @onclick="VisszaBejelentkezéshez">
        <i class="bi bi-arrow-left me-2"/> Vissza a Bejelentkezéshez
      </button>
    </div>
  </div>
</div>

@code {
  private void VisszaBejelentkezéshez()
  {
    Nav.NavigateTo("/login");
  }
}

```

A modern szoftverfejlesztés elengedhetetlen része a felhasználói adatok védelme és az átlátható tájékoztatás (GDPR megfelelés). Az Adatvedelem.razor komponens ezt a célt szolgálja: egy publikus, bejelentkezés nélkül is elérhető felületet biztosít, ahol a felhasználók megismerhetik jogaikat és a rendszer adatkezelési elveit.

Bár technológiailag ez egy statikusabb, szöveggözpontú oldal, a felépítése és a tartalma tökéletesen tükrözi a KoliPortál mögöttes architektúráját:

1. Vizuális Kialakítás (UI) és Hozzáférhetőség

A kód az InteractiveServer renderelési módot és a beépített Blazor navigációt (NavigationManager) használja. A szöveges tartalom egy középre zárt, elegáns Bootstrap kártyán (privacy-card) jelenik meg. Ez a letisztult, "jogi dokumentumhoz" illő dizájn megkönnyíti a hosszú szövegek olvasását mobiltelefonon és asztali gépen egyaránt.

2. Rendszerszintű Transzparencia (A tartalom logikája)

A HTML blokkokba (<h4>, ,) rendezett tájékoztató nem csupán egy generikus "Lorem Ipsum" szöveg, hanem pontosan leképezi az általunk tervezett adatbázis-struktúrát:

- **Kezelt adatok köre:** A listában felsorolt elemek (Azonosítási, Kapcsolattartási, Tanulmányi, Kollégiumi, Pénzügyi, Karbantartási adatok) egy az egyben megfelelnek a backend MODEL rétegében bemutatott entitásoknak (pl. DiakAdatok, Penzugyek, KarbantartasiKeresek).
- **Adatbiztonság és Titkosítás:** Kifejezetten tájékoztatja a felhasználót arról, hogy a jelszavakat a rendszer egyirányú, visszafejthetetlen titkosítással (BCrypt hash) tárolja, így azokhoz még a rendszergazdák sem férhetnek hozzá. Ezzel a fejlesztés maximálisan eleget tesz a "Privacy by Design" (beépített adatvédelem) elvnek.

3. Zökkenőmentes Navigáció

A dokumentum alján található gomb (@onclick="VisszaABejelentkezeshez") a @code blokkban lévő egyszerű, egyetlen soros C# metódust hívja meg. A NavigationManager segítségével azonnal, az oldal újratöltése nélkül (SPA élmény) irányítja vissza a látogatót a belépési felületre (/login), amint végzett az olvasással.

2.19 AdminLayout.razor

Az **AdminLayout.razor** komponens a KoliPortál adminisztrációs felületének "csomagolóanyaga" (Wrapper). Ez a fájl felel azért, hogy a rendszergazdák minden menüpont alatt ugyanazt a bal oldali navigációs sávot és egységes dizájnt lássák, anélkül, hogy a keretet minden egyes aloldalra (pl. Szobák, Felhasználók) újra be kellene másolni.

2.19.1 Függőségek és Alaposztály

```
@inherits LayoutComponentBase
@using System.IdentityModel.Tokens.Jwt
@inject IJSRuntime JS
@inject NavigationManager Nav
```

A fájl elején található deklarációk határozzák meg a komponens képességeit:

- **@inherits LayoutComponentBase:** Ez mondja meg a Blazor keretrendszernek, hogy ez nem egy hagyományos weboldal, hanem egy mesteroldal (Layout), amely képes más oldalakat magába foglalni.
- **@inject IJSRuntime JS:** Lehetővé teszi a C# kód számára a JavaScript interakciót (JS Interop). Erre a böngésző helyi tárhelyének (localStorage) eléréséhez van szükségünk, ahol a bejelentkezési tokent tároljuk.
- **@inject NavigationManager Nav:** A Blazor beépített útválasztója, amellyel programozottan tudjuk átirányítani a felhasználókat (pl. visszadobni őket a login oldalra).

2.19.2 Felhasználói Felület és Navigáció

```
<div class="dashboard-container">
  <header class="mobile-top-bar d-md-none">
    <button class="hamburger-btn" type="button" data-bs-toggle="collapse" data-bs-target="#sidebarMenu">
      <i class="bi bi-list"></i>
    </button>
  </header>

  <aside class="sidebar collapse d-md-flex" id="sidebarMenu">
    <div class="sidebar-header d-none d-md-flex">
      <div class="brand">
        <i class="bi bi-person-fill-gear icon"></i>
        Adminisztráció
      </div>
    </div>

    <nav class="sidebar-nav">
      <NavLink class="nav-item" href="/adminAttekintes" Match="NavLinkMatch.All">
        <i class="bi bi-grid-fill icon"></i> Áttekintés
      </NavLink>
      <NavLink class="nav-item" href="/felhasznalok">
        <i class="bi bi-people icon"></i> Felhasználók
      </NavLink>
      <NavLink class="nav-item" href="/regisztralas">
        <i class="bi bi-person-plus icon"></i> Regisztrálás
      </NavLink>
      <NavLink class="nav-item" href="/szobak">
        <i class="bi bi-door-open icon"></i> Szobák
      </NavLink>
      <NavLink class="nav-item" href="/karbantartas">
        <i class="bi bi-tools icon"></i> Karbantartás
      </NavLink>
      <NavLink class="nav-item" href="/penzugyek">
        <i class="bi bi-cash-coin icon"></i> Pénzügyek
      </NavLink>
      <NavLink class="nav-item" href="/beallitasok">
        <i class="bi bi-gear icon"></i> Beállítások
      </NavLink>
    </nav>

    <div class="sidebar-footer">
      @* Javascript, kijelentkezés nyomására kijelentkeztet *@
      <button class="logout-btn" onclick="localStorage.removeItem('authToken'); window.location.href='/login';">
        <i class="bi bi-box-arrow-right"></i>
        <span class="logout-text" aria-hidden="true"></span> Kijelentkezés
      </button>
    </div>
  </aside>

  <main class="main-content">
    @Body
  </main>
</div>
```

Ez a blokk tartalmazza a vizuális struktúrát, felkészítve a reszponzív (mobilos és asztali) megjelenésre.

- **Oldalsó menü (Sidebar) és NavLink:** A bal oldali sávban a Blazor beépített `<NavLink>` komponenseit használjuk a hagyományos `<a>` (anchor) tagek helyett. A NavLink hatalmas előnye, hogy automatikusan rárakja az "active" CSS osztályt arra a menüpontra, amelyik oldalon éppen tartózkodunk, vizuális visszajelzést adva a felhasználónak.
- **Kijelentkezés:** Az oldal alján található Kijelentkezés gomb egy rendkívül elegáns, egysoros JavaScript hívást használ (`onclick="localStorage.removeItem('authToken'); window.location.href='/login';"`) a munkamenet azonnali megszakítására és az átirányításra.
- **A @Body direktíva:** Ez a Layout legfontosabb eleme. Ez az a dinamikus helyőrző (placeholder), ahová a Blazor beilleszti az éppen megnyitott oldalt (pl. Felhasználók Kezelése) tartalmát.

2.19.3 Kliensoldali Útvonalvédelem

```
@code {
    // Az oldal betöltésekor lefutó védelem az illetéktelenek ellen
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            try
            {
                // Megnézzük a token
                var token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

                // Ha nincs token (nincs bejelentkezve), vissza a login oldalra
                if (string.IsNullOrEmpty(token))
                {
                    Nav.NavigateTo("/login");
                    return;
                }

                // Kinyerjük a szerepkört
                string szerepkor = SzerepkorKinyerese(token);

                // Ha a szerepkör NEM Admin (nem 1-es az ID-ja), akkor is repül!
                if (szerepkor != "1" && szerepkor.ToLower() != "admin")
                {
                    Nav.NavigateTo("/login");
                    return;
                }
            }
            catch
            {
                // Bármilyen hiba esetén biztonsági kidobás
                Nav.NavigateTo("/login");
            }
        }
    }

    // JWT Token visszafejtő
    private string SzerepkorKinyerese(string jwtCsomag)
    {
        try
        {
            var handler = new JwtSecurityTokenHandler();
            var token = handler.ReadJwtToken(jwtCsomag);

            var szerepkorAdat = token.Claims.FirstOrDefault(x =>
                x.Type == "role" ||
                x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role");

            return szerepkorAdat?.Value ?? "";
        }
        catch
        {
            return "";
        }
    }
}
```

Bár a rendszer valódi védelmét az API végpontok (backend) adják, a kiváló felhasználói élmény (UX) megköveteli, hogy a jogosulatlan felhasználók be se tudják tölteni az adminisztrációs oldalakat a böngészőben. Ezt a védelmet az `OnAfterRenderAsync` életrajzi-metódusba írt algoritmus biztosítja **minden egyes oldalnál**.

1. **Token ellenőrzése:** Az oldal legelső betöltésekor (`if (firstRender)`) a kód kiolvassa a `localStorage`-ból a JWT tokenet. Ha a token üres (a felhasználó nincs bejelentkezve), azonnal átirányítja a `/login` felületre.
2. **Szerepkör kinyerése:** A `SzerepkorKinyerese` metódus a `JwtSecurityTokenHandler` segítségével a kliensoldalon, a böngésző

memóriájában bontja ki a JSON Web Token. Egy LINQ lekérdezéssel (FirstOrDefault) megkeresi a token rakományában (Claims) a role (szerepkör) azonosítót.

3. **Logikai validáció és Hibakezelés:** Az algoritmus megvizsgálja, hogy a kinyert szerepkör megfelel-e az adminisztrátori jogosultságnak (ID: 1, vagy név: "admin"). Ha nem, vagy ha a token manipulált/olvashatatlan (amit a try-catch blokk hibaként elkap), a rendszer biztonsági okokból kilépteti az illetőt.

Ez a megoldás garantálja, hogy egy egyszerű "Diák" jogosultsággal rendelkező lakó még az URL közvetlen beírásával se láthassa meg a rendszergazdai műszerfalat.

2.20 LakoLayout.razor

A LakoLayout.razor komponens a kollégiumi lakók (diákok) számára dedikált mesteroldal (Wrapper). Célja, hogy a diákok számára egy letisztult, csak a számukra releváns funkciókat tartalmazó keretrendszert biztosítson, fizikailag és logikailag is elszeparálva az adminisztrációs felülettől.

2.20.1 Alapinjektálások és Függőségek

```
@inherits LayoutComponentBase
@using System.IdentityModel.Tokens.Jwt
@Inject IJSRuntime JS
@Inject NavigationManager Nav
```

Az adminisztrátori felülethez hasonlóan, ez a komponens is a LayoutComponentBase ősosztályból származik le. A működéséhez elengedhetetlen a NavigationManager injektálása az átirányításokhoz, valamint az IJSRuntime, amely a böngésző helyi memóriájával (Local Storage) való JavaScript szintű kommunikációt biztosítja.

2.20.2 Dedikált Felhasználói Felület

```
<link href="https://cdn.jsdelivr.net/npm/remixicon@3.5.0/fonts/remixicon.css" rel="stylesheet">

<div class="dashboard-container">
  <header class="mobile-top-bar d-md-none">
    <button class="hamburger-btn" type="button" data-bs-toggle="collapse" data-bs-target="#sidebarMenu">
      <i class="bi bi-list"></i>
    </button>
  </header>

  <aside class="sidebar collapse d-md-flex" id="sidebarMenu">
    <div class="sidebar-header d-none d-md-flex">
      <div class="brand">
        <i class="ri-building-2-fill"></i>
        KoliPortál
      </div>
    </div>

    <nav class="sidebar-nav">
      <NavLink class="nav-item" href="/lakoAttekintes" Match="NavLinkMatch.All">
        <i class="bi bi-grid-fill icon"></i> Áttekintés
      </NavLink>
      <NavLink class="nav-item" href="/adataim">
        <i class="bi bi-people icon"></i> Adataim
      </NavLink>
      <NavLink class="nav-item" href="/befizetesekLako">
        <i class="bi bi-person-plus icon"></i> Befizetések
      </NavLink>
      <NavLink class="nav-item" href="/hibajelentes">
        <i class="bi bi-door-open icon"></i> Hibajelentés
      </NavLink>
    </nav>

    <div class="sidebar-footer">
      <!-- Javascript, kijelentkezés nyomására kijelentkeztet -->
      <button class="logout-btn" onclick="localStorage.removeItem('authToken'); window.location.href='/login';">
        <i class="bi bi-box-arrow-right"></i>
        <span class="logout-text" aria-hidden="true"></span> Kijelentkezés
      </button>
    </div>
  </aside>

  <main class="main-content">
    <@Body>
  </main>
</div>
```

A felhasználói élmény (UX) egyik legfontosabb alapszabálya, hogy a felhasználó elől el kell rejtetni azokat a menüpontokat, amelyekhez nincs jogosultsága.

- **Diák-specifikus Navigáció:** A bal oldali menüsáv (<nav class="sidebar-nav">) kizárólag a diákok számára engedélyezett útvonalakat tartalmazza: Áttekintés (/lakoAttekintes), Személyes adatok (/adataim), Pénzügyek (/befizetesekLako) és Hibabejelentés (/hibajelentes).
- **Ikonográfia:** A kódban látható egy külső stíluslap (RemixIcon) behívása is, amely vizuálisan gazdagítja a felületet, modern és intuitív megjelenést kölcsönözve a navigációs sávnak.
- **Kijelentkezés:** A rendszer itt is egy gyors, kliensoldali JavaScript utasítással törli a munkamenetet (a token eltávolításával), és biztonságosan kijelentkezteti a hallgatót.

2.20.3 Szigorított Kliensoldali Útvonalvédelem

```
@code {
    // Az oldal betöltésekor lefutó védelem az illetéktelenek ellen
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            try
            {
                // Megnézzük a token
                var token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

                // Ha nincs token (nincs bejelentkezve), vissza a login oldalra
                if (string.IsNullOrEmpty(token))
                {
                    Nav.NavigateTo("/login");
                    return;
                }

                // Kinyerjük a szerepkört
                string szerepkor = SzerepkorKinyerese(token);

                // Ha a szerepkör NEM Lako (nem 2-es az ID-je), akkor is repül!
                if (szerepkor != "2" && szerepkor.ToLower() != "lako")
                {
                    Nav.NavigateTo("/login");
                    return;
                }
            }
            catch
            {
                // Bármilyen hiba esetén biztonsági kidobás
                Nav.NavigateTo("/login");
            }
        }
    }

    // JWT Token visszafejtő
    private string SzerepkorKinyerese(string jwtCsomag)
    {
        try
        {
            var handler = new JwtSecurityTokenHandler();
            var token = handler.ReadJwtToken(jwtCsomag);

            var szerekorAdat = token.Claims.FirstOrDefault(x =>
                x.Type == "role" ||
                x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role");

            return szerekorAdat?.Value ?? "";
        }
        catch
        {
            return "";
        }
    }
}
```

Bár a token-ellenőrzés algoritmus megegyezik az AdminLayout-ban bemutatottal, az üzleti logika és a validációs feltétel itt teljesen más.

1. **Azonosítás és Kibontás:** A rendszer az oldal betöltésekor (OnAfterRenderAsync) kiolvassa a JWT token, majd a SzerepkorKinyerese metódussal dekódolja annak belső tartalmát (Claims).
2. **Szigorú Szerepkör-ellenőrzés:** Az algoritmus lelke a következő feltétel: `if (szerepkor != "2" && szerepkor.ToLower() != "lako")`. Ez a logikai kapu biztosítja, hogy ezt a felületet *kizárólag* a "Lakó" (2-es ID-jú) szerepkörrel rendelkezők tölthessék be.

3. **Kereszt-jogosultsági védelem:** Ennek a szigorú elválasztásnak köszönhetően nemcsak az illetéktelen (kijelentkezett) látogatókat dobja ki a rendszer a login oldalra, hanem megakadályozza a jogosultságok keveredését is. A Blazor keretrendszer így garantálja, hogy egy Admin ne ragadhasson be a Diák felületre, és a Diák ne láthassa az Admin felületet.

2.21 MainLayout.razor

```
@inherits LayoutComponentBase  
  
@Body
```

Míg az AdminLayout és a LakoLayout összetett navigációs sávokat és jogosultság-ellenőrző logikákat tartalmaz, a **MainLayout.razor** egy teljesen letisztult, transzparens "csomagolóanyagként" (Wrapper) funkcionál. Célja, hogy egy teljesen üres vásznat biztosítson a speciális oldalak számára.

- **@inherits LayoutComponentBase:** Ez a Blazor direktíva kötelező minden mesteroldal esetében. Ez mondja meg a fordítónak, hogy ez a fájl egy elrendezési komponens, és rendelkezik a beágyazáshoz szükséges alapvető tulajdonságokkal.
- **@Body:** Az a dinamikus helyőrző, ahová a Blazor beilleszti az adott oldal tartalmát, anélkül, hogy bármilyen menüt, fejléceket vagy formázást tenne köré.

Egy biztonságos webalkalmazásban vannak olyan felületek, amelyeket a felhasználóknak hitelesítés (bejelentkezés) nélkül is látniuk kell, és ahol kifejezetten nemkívánatos a menürendszer megjelenítése. A KoliPortál esetében a MainLayout felel a **Bejelentkezési (Login) oldal** és az esetleges **Hibaoldalak** (pl. 404 Not Found) megjelenítéséért. Mivel ez a Layout nem tartalmaz sem oldalsó sávot (Sidebar), sem biztonsági ellenőrző logikát, tökéletes és gyors keretet ad az induló képernyőknek, megakadályozva, hogy egy be nem jelentkezett diák vizuálisan is hozzáférjen az alkalmazás belső navigációs elemeihez.

2.22 Admin/Attekintes.razor

Az AdminAttekintes.razor komponens a rendszergazdák elsődleges érkezési oldala. Feladata, hogy a bejelentkezést követően azonnali, valós idejű statisztikákat (KPI-okat) és listákat nyújtson a kollégium aktuális állapotáról. Mivel a felület több különböző adatbázistáblából dolgozik, a kód felépítése a hatékony adatlekérésre és a kliensoldali adatfeldolgozásra optimalizált.

A komponens logikailag és vizuálisan az alábbi blokkokra tagolódik:

2.22.1 Direktívák, Injektálások és Renderelési Mód

```
@page "/AdminAttekintes"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@layout AdminLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject SzobakService Szobak
@inject KarbantartasiKeresekService KarbantartasiKeresek
@inject SzobaBeosztasokService SzobaBeosztasok
@inject FelhasznalokService FelhasznalokService
@inject PenzugyekService Penzugyek
@inject IJSRuntime JS
@inject NavigationManager Nav
```

A fájl fejléce beállítja a komponens környezetét:

- **Útvonal és Elrendezés:** A `@page "/AdminAttekintes"` definiálja az URL-t, a `@layout AdminLayout` pedig biztosítja, hogy a korábban bemutatott adminisztrátori menüsávossal keretbe töltsön be az oldal.
- **Prerender kikapcsolása:** A `@rendermode @(new InteractiveServerRenderMode(prerender: false))` egy kritikus szoftverfejlesztési döntés. Mivel az oldal a böngésző helyi tárhelyéből (Local Storage) olvassa ki a JWT token, a szerveroldali előrenderelést (prerendering) ki kell kapcsolni, hogy elkerüljük a JavaScript hívásokból fakadó hibákat a betöltés pillanatában.
- **Szolgáltatások (Dependency Injection):** A kód szinte az összes korábban megírt hálózati szolgáltatást (SzobakService, PenzugyekService, stb.) injektálja, hiszen a statisztikákhoz minden modul adataira szükség van.

2.22.2 Állapotkezelés és Vizuális Visszajelzések

```
<header class="page-header">
  <h1>Rendszer Áttekintés</h1>
  <p>Kollégiumi menedzsment központ.</p>
</header>

@if (betoltesAlatt)
{
  <div class="text-center mt-5">
    <div class="spinner-border text-success" role="status">
      <span class="visually-hidden">Betöltés...</span>
    </div>
    <p class="mt-2 text-muted">Adatok lekérése az API-tól...</p>
  </div>
}
else
{
  <div class="stats-grid">
    <div class="stat-card">
      <div class="stat-icon purple">
        <i class="bi bi-person-check-fill icon-large"></i>
      </div>
      <div class="stat-details">
        <h3>@aktívLakokSzama</h3>
        <p>Aktív Lakó</p>
      </div>
    </div>

    <div class="stat-card">
      <div class="stat-icon blue">
        <i class="bi bi-house-door-fill icon-large"></i>
      </div>
      <div class="stat-details">
        <h3>@szabadAgyakSzama</h3>
        <p>Szabad ágy</p>
      </div>
    </div>

    <div class="stat-card">
      <div class="stat-icon orange">
        <i class="bi bi-exclamation-triangle-fill icon-large"></i>
      </div>
      <div class="stat-details">
        <h3>@nyitottHibakSzama</h3>
        <p>Nyitott Hiba</p>
      </div>
    </div>

    <div class="stat-card">
      <div class="stat-icon green">
        <i class="bi bi-cash-coin icon-large"></i>
      </div>
      <div class="stat-details">
        <h3>@befizetesiArany%</h3>
        <p>Befizetési arány</p>
      </div>
    </div>
  </div>
}
```

```

<div class="content-grid">
  <div class="card recent-residents-card">
    <div class="table-scroll-container">
      <div class="card-header">
        <h2>Legújabb Lakók</h2>
      </div>

      <table class="data-table">
        <thead>
          <tr>
            <th>Név</th>
            <th>Szoba</th>
            <th>Beköltözés</th>
            <th>Státusz</th>
          </tr>
        </thead>
        <tbody>
          @if (legujabbLakok != null && legujabbLakok.Any())
          {
            @foreach (var beosztas in legujabbLakok)
            {
              <tr>
                <td>@GetFelhasznaloNev(beosztas.UserID)</td>
                <td>@GetSzobaSzam(beosztas.RoomID)</td>
                <td>@beosztas.BekoltozesDatum?.ToShortDateString()</td>
                <td>
                  <span class="status-badge aktiv">Aktiv</span>
                </td>
              </tr>
            }
          }
          else
          {
            <tr>
              <td colspan="4" class="text-center text-muted">Nincsenek aktiv lakók a rendszerben.</td>
            </tr>
          }
        </tbody>
      </table>
    </div>
  </div>
</div>

```

```

<div class="card capacity-card">
  <div class="table-scroll-container">
    <div class="card-header">
      <h2>Szoba Kapacitás</h2>
    </div>

    <div class="capacity-list">
      @if (emeletKapacitasok != null && emeletKapacitasok.Any())
      {
        @foreach (var emelet in emeletKapacitasok)
        {
          <div class="capacity-item">
            <span class="floor-name">@emelet.EmeletNev</span>
            <span class="floor-capacity">@emelet.AktivLakokSzama / @emelet.MaxFerohely fõ</span>
          </div>
        }
      }
      else
      {
        <p class="text-muted text-center mt-3">Nincsenek szoba adatok.</p>
      }
    </div>
  </div>
</div>

```

A felhasználói felület a HTML, a Bootstrap és a Razor C# szintaxisának ötvözete.

- **Aszinkron Betöltés (Loading State):** Az @if (betoltesAlatt) logika egy kiváló UX (User Experience) megoldás. Amíg a gigantikus adatmennyiség letöltődik az API-tól, a felhasználó egy forgó töltésjelzőt (Spinner) lát, elkerülve a lefagyott képernyő érzetét.

- **Dinamikus KPI Kártyák:** A kód a statisztikai változókat (pl. @aktivLakokSzama, @szabadAgyakSzama) közvetlenül a HTML kártyákba köti, így az értékek kiszámítása után a felület automatikusan frissül.
- **Relációs adatok feloldása a nézetben:** A táblázatokban lévő @foreach ciklusoknál látható a GetFelhasznaloNev() és GetSzobaSzam() metódusok hívása. Mivel az adatbázis csak azonosítókat (ID) tárol (pl. UserID), ezek a segédmetódusok felelnek azért, hogy a képernyőre már a diákok valós nevei és a szobák tényleges számai kerüljenek kiírásra.

2.22.3 Változók és Segédosztályok deklarálása

```
@code {
    private bool betoltesAlatt = true;
    private string token = "";

    // Statisztika változók
    private int aktivLakokSzama = 0;
    private int szabadAgyakSzama = 0;
    private int nyitottHibakSzama = 0;
    private int befizetesiArany = 0;

    // Lista változók
    private List<SzobaBeosztasok> legujabbLakok = new();
    private List<EmeletKapacitas> emeletKapacitasok = new();

    // Nevek megkereséséhez
    private List<KoliPortal.Lib.MODEL.Felhasznalo> osszesFelhasznalo = new();
    private List<KoliPortal.Lib.MODEL.Szobak> osszesSzoba = new();
}
```

```
private string GetFelhasznaloNev(int userId)
{
    var user = osszesFelhasznalo.FirstOrDefault(u => u.ID == userId);
    return user != null ? user.Nev : $"User #{userId}";
}

private string GetSzobaSzam(int roomId)
{
    var szoba = osszesSzoba.FirstOrDefault(s => s.ID == roomId);
    return szoba != null ? szoba.Szobaszam : $"Szoba #{roomId}";
}

public class EmeletKapacitas
{
    public string EmeletNev { get; set; } = "";
    public int MaxFerohely { get; set; }
    public int AktivLakokSzama { get; set; }
}
```

A @code blokk elején találhatók a lokális memóriát képző változók:

- **Statisztikai számlálók:** Egyszerű int és double típusú változók a KPI értékek tárolására.
- **Memória Listák:** A letöltött adatokat C# List<T> típusú memóriatárolókba mentjük, hogy a bonyolult szűréseket már villámgyorsan, a kliens memóriájában végezhessük el, ne terheljük feleslegesen az adatbázis-szervert.

- **Kliensoldali DTO (EmeletKapacitas):** A kód egy egyedi, belső osztályt is definiál. Ennek feladata a legkomplexebb adatszerkezet (az emeletenkénti telítettség) összefogása egyetlen típusos objektumba.

2.22.4 Az Adatelemző Motor

```
protected override async Task OnAfterRenderAsync(bool elsoBetoltes)
{
    if (elsoBetoltes)
    {
        try
        {
            token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

            if (string.IsNullOrEmpty(token))
            {
                Nav.NavigateTo("/");
                return;
            }

            var handler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
            var jwtToken = handler.ReadJwtToken(token);
            var szerepkor = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;
            if (szerepkor != "1" && szerepkor?.ToLower() != "admin")
            {
                Nav.NavigateTo("/");
                return;
            }

            // Lekérjük az adatokat a API-ből
            osszesSzoba = await Szobak.GetSzobak(token) ?? new List<KoliPortal.Lib.MODEL.Szoba>();
            beosztasok = await SzobaBeosztasok.GetSzobaBeosztasok(token) ?? new List<SzobaBeosztasok>();
            var hibak = await KarbantartasiKeresek.GetKarbantartasiKeresek(token) ?? new List<KarbantartasiKereselo>();
            osszesFelhasznalo = await FelhasznaloService.GetFelhasznalok(token) ?? new List<KoliPortal.Lib.MODEL.Felhasznalok>();
            var penzugyekLista = await Penzugyek.GetPenzugyek(token) ?? new List<KoliPortal.Lib.MODEL.Penzugyek>();

            // Aktiv lakók
            var aktivBeosztasok = beosztasok.Where(b => b.TenylegesKikoltozes == null).ToList();
            aktivLakokSzama = aktivBeosztasok.Count();

            // Szabad ágyak
            int osszesFerohely = osszesSzoba.Sum(s => s.FerohelyMax);
            szabadAgyakSzama = osszesFerohely - aktivLakokSzama;

            // Nyitott hibák
            myitottHibakSzama = hibak.Count(h => h.StatuszID != 3);

            // Befizetési arány számítása
            if (penzugyekLista.Any())
            {
                // Hány olyan tétel van, aminek van befizetési dátuma VAGY a státusza "fizetve"
                int befizetettTetelek = penzugyekLista.Count(p => p.BefizetesDatum.HasValue || (p.Statusz ?? "").ToLower() == "fizetve");

                // Százalékszámítás és kerekítés
                double arany = (double)befizetettTetelek / penzugyekLista.Count * 100;
                befizetesiArany = (int)Math.Round(arany);
            }
            else
            {
                befizetesiArany = 100; // Ha senkinek nincs kiírva díj, akkor 100%
            }

            // Legújabb lakók a táblázathoz
            legujabbLakok = aktivBeosztasok
                .OrderByDescending(b => b.BeoltozesDatum)
                .Take(5)
                .ToList();

            // Emelet kapacitások
            emeletKapacitasok = osszesSzoba
                .GroupBy(s => s.Emelet)
                .Select(g => new EmeletKapacitas
                {
                    EmeletNev = $"{g.Key}. Emelet",
                    MaxFerohely = g.Sum(s => s.FerohelyMax),
                    AktivLakokSzama = aktivBeosztasok.Count(b => g.Select(sz => sz.ID).Contains(b.RoomID))
                })
                .OrderBy(e => e.EmeletNev)
                .ToList();
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine($"Hiba az adatok lekérésekor: {ex.Message}");
        }
        finally
        {
            betoltesAllat = false;
            StateHasChanged();
        }
    }
}
```

Ez a blokk a műszerfal "agya". Az oldal legelső betöltésekor lefutó életciklus-metódus először elvégzi a biztonsági token-ellenőrzést, majd – ha a felhasználó Adminisztrátor – letölti az adatokat és elindítja a számításokat.

A letöltött nyers adatok feldolgozását modern, funkcionális C# LINQ (Language Integrated Query) kifejezések végzik. Ezek az algoritmusok matematikai és logikai műveleteket hajtanak végre a memóriában:

1. **Aktív lakók szűrése:** A `.Where(b => b.TenylegesKikoltozes == null)` feltétel kiválogatja azokat a szobabeosztásokat, ahol a diák még nem költözött ki, majd a `.Count` megszámolja őket.
2. **Pénzügyi arányszámítás:** A kód százalékos értéket (`double`) számol a befizetett tételek és az összes tétel hányadosából, majd a `Math.Round` metódussal kerekíti azt, elkerülve a végtelen tizedestörtek megjelenítését a UI-on.
3. **Toplista generálása:** A legújabb lakók listáját a `.OrderByDescending(b => b.BekoltozesDatum)` metódus időrendben csökkenő sorrendbe teszi, a `.Take(5)` paranccsal pedig levágja a lista legfelső, 5 legfrissebb elemét.
4. **Komplex Csoportosítás (Kapacitás számítás):** A legfejlettebb algoritmus az emeletek kapacitásának számítása. A kód a `.GroupBy(s => s.Emelet)` segítségével a szobákat emeletek szerint virtuális dobozokba (csoportokba) rendezi. Ezután minden egyes dobozra külön-külön kiszámolja az összes férőhelyet (`Sum(s => s.FerohelyMax)`), majd egy beágyazott lekérdezéssel (`Count(...)`) megszámolja, hogy az adott emelet szobáiban hány aktív lakó tartózkodik éppen. A végeredményt az `EmeletKapacitas` objektumok listájaként adja vissza.

A folyamat legvégén a `betoltesAlatt = false` és a `StateHasChanged()` utasítások jelzik a Blazornak, hogy a számítások befejeződtek, a betöltés jelzőt el lehet tüntetni, és a képernyőt újra lehet rajzolni a friss adatokkal.

2.23 Felhasznalok.razor

A `Felhasznalok.razor` komponens az adminisztrátorok legfőbb eszköze a kollégiumi lakók és munkatársak adatainak kezelésére. Ez a felület nemcsak listázza a felhasználókat, hanem integrálja a szobabeosztási, tanulmányi és jogosultsági modulokat is. A kód felépítése a modern webfejlesztés legjobb gyakorlatait (komponens alapú tervezés, reszponzív adatrács, modális ablakok) követi.

2.23.1 Komponens Direktívák és Injektálások

```
@page "/felhasznalok"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@layout AdminLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))

@inject FelhasznalokService Felhasznalo
@inject SzerepkorokService Szerepkorok
@inject SzobaBeosztasokService SzobaBeosztasok
@inject SzobakService Szoba
@inject DiakAdatokService DiakAdatok
@inject KarbantartasiKeresekService KarbantartasiKeresek
@inject PenzugyekService Penzugyek
@inject KollegiumService Kollegiumok
@inject IJSRuntime JS
@inject NavigationManager Nav
```

A fájl fejléce megmutatja a modul komplexitását. A `@rendermode InteractiveServer` itt is elengedhetetlen a valós idejű kereséshez és a modális ablakok kezeléséhez. A komponens szinte az összes létező szolgáltatást (FelhasznalokService, SzobakService, DiakAdatokService, stb.) injektálja, mivel egy felhasználó szerkesztésekor a rendszernek le kell kérdeznie a szabad szobákat, a meglévő beosztásokat és a kollégiumok listáját is.

2.23.2 Vizuális Visszajelzések és Állapotkezelés

```
<header class="page-header text-center">
  <h1>Felhasználók Kezelése</h1>
  <p>Kollégisták adatbázisa.</p>
</header>

@if (sikerMentes)
{
  <div class="alert alert-success m-3">
    <i class="bi bi-check-circle-fill"></i> <strong>Siker:</strong> @sikerUzenet
  </div>
}

@if (betoltesAlatt)
{
  <div class="text-center mt-5">
    <div class="spinner-border text-primary" role="status"></div>
    <p class="mt-2 text-muted">Felhasználók betöltése...</p>
  </div>
}
```

A felhasználói élmény (UX) maximalizálása érdekében a felület két fontos állapotjelzőt használ:

- **Sikerüzenet (@if (sikerMentes)):** Egy feltételesen megjelenő Bootstrap Alert, amely a szerkesztések vagy törlések után ad vizuális megerősítést a felhasználónak.
- **Töltésjelző (@if (betoltesAlatt)):** Amíg az API-tól megérkezik a masszív felhasználói lista, a felület egy elegáns animációt mutat, blokkolva a türelmetlen kattintásokat.

2.23.3 Kereső, Szűrő és Oldalméret-választó Szekció

```

else
{
  <div class="content-wrapper">
    @* --- SZŰRŐK, KERESŐ ÉS OLDALMÉRET VÁLASZTÓ --- *@
    <div class="d-flex justify-content-between align-items-center mb-3 flex-wrap gap-3">
      <div class="filter-controls m-0">
        <button class="filter-btn @(szurtMod == 0 ? "active" : "")" @onclick="() => Szures(0)">Összes</button>
        <button class="filter-btn @(szurtMod == 1 ? "active" : "")" @onclick="() => Szures(1)">Bent lakók</button>
        <button class="filter-btn @(szurtMod == 2 ? "active" : "")" @onclick="() => Szures(2)">Kiköltözöttek</button>
      </div>
      <div class="d-flex align-items-center gap-4">
        <div class="input-group input-group-sm shadow-sm" style="width: 250px;">
          <span class="input-group-text bg-white border-end-0" style="font-size: 0.8em; padding: 0 10px;">Keresés</span>
          <input type="text" class="form-control border-start-0 ps-0" placeholder="Keresés (név, email, szoba)..." value="@KeresesSzoveg" @oninput="KeresesValtozas" />
        </div>
        <div class="d-flex align-items-center gap-2">
          <span class="text-muted small fw-bold text-uppercase">Sor/oldal:</span>
          <select class="form-select form-select-sm shadow-sm page-size-select" value="@elemekOldalankent" @onchange="OldalMeretValtozas">
            <option value="5">5</option>
            <option value="10">10</option>
          </select>
        </div>
      </div>
    </div>
  </div>
}

```

Ez a blokk a nagyméretű adatbázisok kezelésének elengedhetetlen eszköze.

- **Gyorsszűrők:** Három gomb ("Összes", "Bent lakók", "Kiköltözöttek"), amelyek a `@szurtMod` változóhoz kötve azonnal szűkítik a találatokat az aktuális lakókra vagy az archív adatokra.
- **Valós idejű Kereső:** Az `<input>` mező `@oninput="KeresesValtozas"` eseménykezelője a legmodernebb megoldás: ahogy a rendszergazda gépel (pl. nevet vagy szobaszámot), az oldal azonnal, Enter lenyomása nélkül szűri a táblázatot.
- **Oldalméret:** Egy legördülő menü (`@onchange="OldalMeretValtozas"`), ahol szabályozható a képernyőn egyszerre megjelenő sorok száma (5 vagy 10).

2.23.4 Dinamikus Adattábla

```

@* TÁBLÁZAT *@
<div class="card shadow-sm mb-3">
  <div class="table-scroll-container">
    <table class="data-table">
      <thead>
        <tr>
          <th>Név</th>
          <th>Kollégium</th>
          <th>Szoba</th>
          <th>Email</th>
          <th>Szerepkör</th>
          <th>Művelet</th>
        </tr>
      </thead>
      <tbody>
        @if (megjelenítettFelhasznalok != null && megjelenítettFelhasznalok.Any())
        {
          @foreach (var user in megjelenítettFelhasznalok)
          {
            <tr>
              <td class="fw-bold">@user.Nev</td>
              <td>@GetKollegiumNev(user.KollegiumID)</td>
              <td>
                @{
                  var szobaStatusz = GetAktualisSzoba(user.ID);
                  if (szobaStatusz == "Kiköltözött")
                  {
                    <span class="status-badge kikoltozott">Kiköltözött</span>
                  }
                  else if (szobaStatusz == "Nincs szobája")
                  {
                    <span class="status-badge nincs-szoba">Nincs szobája</span>
                  }
                  else
                  {
                    <strong class="status-badge van-szoba">@szobaStatusz</strong>
                  }
                }
              </td>
              <td>@user.Email</td>
              <td>
                <span class="role-badge @GetSzerekorCss(user.SzerekorID)">
                  @GetSzerekorNev(user.SzerekorID)
                </span>
              </td>
              <td>
                <button class="action-btn" title="Szerkesztés" @onclick="() => MegnyitSzerkesztes(user.ID)">
                  <i class="bi bi-pencil"></i>
                </button>
              </td>
            </tr>
          }
        }
        else
        {
          <tr>
            <td colspan="6" class="text-center text-muted p-5">Nincsenek a szűrésnek megfelelő felhasználók.</td>
          </tr>
        }
      </tbody>
    </table>
  </div>
</div>

```

A tartalom központja egy reszponzív Bootstrap táblázat (table-scroll-container), amely a megjelenítettFelhasznalok listát iterálja végig.

- A táblázat nemcsak nyers adatokat ír ki, hanem segédmetódusokkal (GetKollegiumNev, GetAktualisSzoba) oldja fel a relációs azonosítókat olvasható szöveggé.
- **Állapotjelzők (Badges):** A kód dinamikus CSS osztályokat rendel a státuszokhoz (pl. piros a "Kiköltözött", zöld a "Van szobája"), így az adminisztrátor vizuálisan, másodpercek alatt átlátja a diákok helyzetét.
- Az utolsó oszlop tartalmazza a Művelet gombot, ami a Megnyitszerkesztes(user.ID) hívással aktiválja a felugró ablakot.

2.23.5 Lapozó Sáv

```

@* LAPOZÓ SÁV *@
@if (összesOldal > 1)
{
    <div class="d-flex justify-content-center align-items-center gap-3 mt-4 mb-5">
        <button class="btn btn-outline-secondary rounded-pill px-4 fw-bold shadow-sm" @onclick="ElozoOldal" disabled="@{(jelenlegiOldal == 1)}">
            <i class="bi bi-chevron-left me-1"></i> Előző
        </button>

        <span class="text-muted fw-bolder bg-white border px-4 py-2 rounded-pill shadow-sm">@jelenlegiOldal / @összesOldal</span>

        <button class="btn btn-outline-secondary rounded-pill px-4 fw-bold shadow-sm" @onclick="KovetkezoOldal" disabled="@{(jelenlegiOldal == összesOldal)}">
            Következő <i class="bi bi-chevron-right ms-1"></i>
        </button>
    </div>
}

```

Ha a találatok száma meghaladja a beállított oldalméretet, megjelenik a lapozósáv. Az "Előző" és "Következő" gombok automatikusan inaktívvá válnak (`disabled="..."`), ha a felhasználó a legelső vagy a legutolsó oldalon tartózkodik, megelőzve az indexelési hibákat a kódban.

2.23.6 Komplex Szerkesztő Modális Ablak

```

@if (isModalOpen)
{
    <div class="custom-modal-backdrop">
        <div class="custom-modal">
            <div class="modal-header">
                <h3>Felhasználó Szerkesztése</h3>
                <button type="button" class="btn-close" @onclick="BezárModal"></button>
            </div>
            <div class="modal-body">
                @if (editModel != null)
                {
                    <div class="form-group mb-3">
                        <label>Név</label>
                        <input type="text" class="form-control" @bind="editModel.Nev" />
                    </div>

                    <div class="form-group mb-3">
                        <label>Kollégium</label>
                        <select class="form-select" @bind="editModel.KollegiumID">
                            <option value="0" disabled>Válassz kollégiumot...</option>
                            @foreach (var koli in kollegiumLista)
                            {
                                <option value="@koli.ID">@koli.KollegiumNev</option>
                            }
                        </select>
                    </div>

                    <div class="form-group mb-3">
                        <label>Email</label>
                        <input type="email" class="form-control" @bind="editModel.Email" />
                    </div>
                }
            </div>
        </div>
    </div>
}

```

```

@if (MarbantarTo(editModel.SzerekorID))
{
    <div class="form-group mb-3">
        <label>Szerekor</label>
        <select class="form-select" @bind="editModel.SzerekorID">
            @foreach (var item in szerekorokLista)
            {
                <option value="@item.ID">@item.Nev</option>
            }
        </select>
    </div>
}

@if (Diak(editModel.SzerekorID))
{
    <hr />
    <h5>Diák adatok</h5>

    <div class="form-group mb-3">
        <label>Évfolyam</label>
        <select class="form-select" @bind="editModel.Szemeszter">
            <option value="" disabled selected>Válassz szemesztert...</option>
            <option value="1 szemeszter">1 szemeszter</option>
            <option value="2 szemeszter">2 szemeszter</option>
            <option value="3 szemeszter">3 szemeszter</option>
            <option value="4 szemeszter">4 szemeszter</option>
            <option value="5 szemeszter">5 szemeszter</option>
            <option value="6 szemeszter">6 szemeszter</option>
            <option value="7 szemeszter">7 szemeszter</option>
            <option value="8 szemeszter">8 szemeszter</option>
            <option value="9 szemeszter">9 szemeszter</option>
            <option value="10 szemeszter">10 szemeszter</option>
        </select>
    </div>

    <div class="form-group mb-3">
        <label>Lakcim</label>
        <input type="text" class="form-control" @bind="editModel.Lakcim" />
    </div>

    <hr />
    <h5>Szoba beosztás</h5>

    <div class="form-group mb-3">
        <label>Jelenlegi szoba</label>
        <select class="form-select" @bind="editModel.RoomID">
            @if (editModel.EredetiRoomID != 0)
            {
                <option value="@editModel.EredetiRoomID">@GetSzobaSzam(editModel.EredetiRoomID) (Jelenlegi szobája)</option>
                <option value="-1" class="text-danger fw-bold">Kiköltöztetés</option>
            }
            else
            {
                <option value="0">Nincs szobába beosztva</option>
            }

            @foreach (var item in GetSzabadSzobak(editModel.EredetiRoomID))
            {
                <option value="@item.ID">@item.Szobaszam (Szabad helyek: @item.FerohelyMax - GetAktivLakokSzam(item.ID))</option>
            }
        </select>
    </div>
}

<div class="modal-footer">
    @if (GetSzerekorNev(editModel.SzerekorID).ToLower().Contains("admin"))
    {
        <span class="text-danger me-auto admin-warning-text">
            <i class="bi bi-shield-lock"></i> Admin nem törölhető!
        </span>
    }
    else
    {
        <button class="btn btn-danger me-auto" @onclick="TorlesGomb">Törlés</button>

        <button class="btn btn-secondary" @onclick="BezarModal">Mégse</button>
        <button class="btn btn-success" @onclick="MentesGomb">Mentés</button>
    }
</div>

```

Ez a Blazor komponens fénypontja. Amikor az @if (isModalOpen) igazra vált, a képernyőt egy sötétítő réteg fedi be, és megjelenik a szerkesztő űrlap.

- **Kétirányú adatkötés:** Az űrlap elemei (@bind="editModel...") azonnal frissítik a memóriában lévő ideiglenes másolatot.

- **Szerepkör-alapú dinamikus űrlap (Feltételes UI):** Zseniális megoldás! A kód megvizsgálja a felhasználó típusát. Ha az illető "Diák", akkor automatikusan megjelennek a tanulmányi (Évfolyam) és lakcím mezők, valamint a **Szobabeosztás kezelő**. Itt a rendszergazda egyetlen legördülő menüből áthelyezheti a diákot egy másik szobába (ahol látja a szabad férőhelyeket is), vagy beállíthatja "Kiköltöztetésre".
- **Biztonsági Fék (Admin védelem):** A modális ablak alján (Lábléc) egy szigorú logikai ellenőrzés található. Ha a szerkesztett felhasználó "admin" jogosultságú, a Törlés gomb eltűnik, és helyette egy figyelmeztető szöveg jelenik meg (Admin nem törölhető!). Ez védi a rendszert attól, hogy a rendszergazdák véletlenül kizárják magukat a szoftverből.

2.23.7 Állapotkezelés és Memóriastruktúra

```
@code {
    private string token = "";
    private bool betoltesAlatt = true;
    private string hibaUzenet = "";
    private bool sikeresMentes = false;
    private string sikerUzenet = "";

    private int szurtMod = 0;
    private string keresoSzoveg = "";
    private List<KoliPortal.Lib.MODEL.Felhasznalok> szurtFelhasznalok = new();
    private List<KoliPortal.Lib.MODEL.Felhasznalok> megjelenitettFelhasznalok = new();

    // LAPOZÁS VÁLTOZÓI
    private int jelenlegiOldal = 1;
    private int elemekOldalankent = 10;
    private int osszesOldal = 1;

    private bool isModalOpen = false;
    private EditViewModel editModel = new();

    private List<KoliPortal.Lib.MODEL.Felhasznalok> felhasznalokLista = new();
    private List<Szerepkorok> szerepkorokLista = new();
    private List<SzobaBeosztasok> beosztasokLista = new();
    private List<KoliPortal.Lib.MODEL.Szobak> szobakLista = new();
    private List<DiakAdatok> diakAdatokLista = new();
    private List<KarbantartasiKeresek> karbantartasiKeresekLista = new();
    private List<KoliPortal.Lib.MODEL.Penzugyek> penzugyekLista = new();
    private List<Kollegium> kollegiumLista = new();
}
```

A kód eleje inicializálja az oldal működéséhez szükséges változókat.

- **Adat-rétegzés (Data Layering):** A rendszer profi módon választja szét a nyers, API-ból kapott adatokat (felhasznalokLista) a képernyőn aktuálisan látható adatoktól (szurtFelhasznalok, megjelenitettFelhasznalok). Ez biztosítja, hogy egy új keresés indításakor ne kelljen újra a szerverhez fordulni, hanem a memóriában meglévő alaplistából lehessen szűrni.
- **DTO (Data Transfer Object):** Az EditViewModel editModel egy speciális, csak a kliensen létező ideiglenes osztály. Feladata, hogy a különböző

adatbázistáblákból (Felhasználó, DiákAdat, SzobaBeosztás) származó adatokat egyetlen lapos (flat) objektummá gyűrja össze a szerkesztő űrlap (Modal) számára.

2.23.8 Hitelesítés és Aszinkron Adatbetöltés

```
protected override async Task OnAfterRenderAsync(bool elsőBetöltés)
{
    if (elsőBetöltés)
    {
        token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

        if (string.IsNullOrEmpty(token))
        {
            Nav.NavigateTo("/login");
            return;
        }

        var handler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
        var jwtToken = handler.ReadJwtToken(token);
        var szerep = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

        if (szerep != "1" && szerep?.ToLower() != "admin")
        {
            Nav.NavigateTo("/login");
            return;
        }

        await AdatokBetöltése();
        StateHasChanged();
    }
}
```

```
private async Task AdatokBetöltése()
{
    try
    {
        betöltésAlatt = true;
        felhasználókLista = await Felhasználó.GetFelhasználók(token) ?? new();
        szerepekLista = await Szerepek.GetSzerepek(token) ?? new();
        beosztásokLista = await SzobaBeosztások.GetSzobaBeosztások(token) ?? new();
        szobákLista = await Szoba.GetSzobák(token) ?? new();
        diákAdatokLista = await DiákAdatok.GetDiákAdatok(token) ?? new();
        karbantartásiKérésekLista = await KarbantartásiKérések.GetKarbantartásiKérések(token) ?? new();
        penzugyekLista = await Penzugyek.GetPenzugyek(token) ?? new();
        kollegiumLista = await Kollegiumok.GetKollegiumok(token) ?? new();

        FrissítSzurot();
    }
    catch (Exception ex)
    {
        Console.Error.WriteLine($"Hiba az adatok lekérésekor: {ex.Message}");
    }
    finally
    {
        betöltésAlatt = false;
    }
}

private void Szures(int mód)
{
    szurtMód = mód;
    jelenlegiOldal = 1;
    FrissítSzurot();
}

private void KeresesValtozas(ChangeEventArgs e)
{
    keresoSzoveg = e.Value?.ToString() ?? "";
    jelenlegiOldal = 1;
    FrissítSzurot();
}

private void FrissítSzurot()
{
    // 1. Alap szűrés (Gombok: Összes / Bent lakók / Kiköltözötték)
    if (szurtMód == 1)
    {
        szurtFelhasznalok = felhasználókLista
            .Where(x => beosztásokLista.Any(b => b.UserID == x.ID && b.TenylegesKikoltozas == null))
            .ToList();
    }
    else if (szurtMód == 2)
    {
        szurtFelhasznalok = felhasználókLista
            .Where(x => beosztásokLista.Any(b => b.UserID == x.ID) && !beosztásokLista.Any(b => b.UserID == x.ID && b.TenylegesKikoltozas == null))
            .ToList();
    }
    else
    {
        szurtFelhasznalok = felhasználókLista.ToList();
    }
}
```

- **Biztonsági kapu (OnAfterRenderAsync):** Mint minden védett oldalon, itt is lefut a kliensoldali token-ellenőrzés. Ha a bejelentkezett személy nem Admin (Role != 1), a rendszer könyörtelenül visszadobja a bejelentkezési képernyőre.
- **Masszív Adatlekérés (AdatokBetoltese):** Ez a metódus 7 különböző API végpontot hív meg párhuzamosan (Felhasználók, Szerepkörök, Beosztások, Szobák, stb.). A letöltés befejeztével azonnal meghívja a FrissitSzurot() metódust, hogy a táblázat inicializálódjon, majd leveszi a töltésjelzőt (betoltesAlatt = false).

2.23.9 Kétlépcsős Szűrési Algoritmus

```
private void FrissitSzurot()
{
    // 1. Alap szűrés (Gombok: Összes / Bent lakók / Kiköltözöttek)
    if (szurtMod == 1)
    {
        szurtFelhasznalok = felhasznalokLista
            .Where(x => beosztasokLista.Any(b => b.UserID == x.ID && b.TenylegesKikoltozes == null))
            .ToList();
    }
    else if (szurtMod == 2)
    {
        szurtFelhasznalok = felhasznalokLista
            .Where(x => beosztasokLista.Any(b => b.UserID == x.ID) && !beosztasokLista.Any(b => b.UserID == x.ID && b.TenylegesKikoltozes == null))
            .ToList();
    }
    else
    {
        szurtFelhasznalok = felhasznalokLista.ToList();
    }
}
```

```
// 2. Szöveges keresés (Név, Email, vagy Szobaszám alapján)
if (!string.IsNullOrEmpty(keresoSzoveg))
{
    var kereso = keresoSzoveg.ToLower();
    szurtFelhasznalok = szurtFelhasznalok.Where(x =>
        x.Nev.ToLower().Contains(kereso) ||
        x.Email.ToLower().Contains(kereso) ||
        GetAktualisSzoba(x.ID).ToLower().Contains(kereso)
    ).ToList();
}

LapozasFrissitese();
```

A táblázat adatkészletének előállítás egy rendkívül gyors, memóriában futó kétlépcsős folyamat:

1. **Állapotszűrő (Gombok):** A kód elsőként a @szurtMod alapján válogat. A .Any(b => b.UserID == x.ID && b.TenylegesKikoltozes == null) LINQ kifejezés egy zseniális relációs vizsgálat: megkeresi, hogy a diákhhoz tartozik-e olyan szobabeosztás a másik listában, aminek nincs még kiköltözési dátuma (vagyis jelenleg is bent lakik).
2. **Szöveges Kereső:** Ha a felhasználó gépelt a keresőbe (keresosZoveg), a rendszer tovább szűkíti a találatokat. A .Contains() metódus kisbetűsítve (ToLower()) vizsgálja a nevet, az e-mailt, ÉS a dinamikusan feloldott szobaszámot is, így akár szobaszámra is lehet keresni a lakók között.

2.23.10 Matematikai Lapozás

```
// LAPOZÁS LOGIKÁJA
private void LapozasFrissitese()
{
    osszesOldal = (int)Math.Ceiling((double)szurtFelhasznalok.Count / elemekOldalankent);
    if (osszesOldal == 0) osszesOldal = 1;

    if (jelenlegiOldal > osszesOldal) jelenlegiOldal = osszesOldal;

    megjelenitettFelhasznalok = szurtFelhasznalok
        .OrderBy(u => u.Nev)
        .Skip((jelenlegiOldal - 1) * elemekOldalankent)
        .Take(elemekOldalankent)
        .ToList();
}

private void ElozoOldal()
{
    if (jelenlegiOldal > 1)
    {
        jelenlegiOldal--;
        LapozasFrissitese();
    }
}

private void KovetkezoOldal()
{
    if (jelenlegiOldal < osszesOldal)
    {
        jelenlegiOldal++;
        LapozasFrissitese();
    }
}

private void OldalMeretValtozas(EventArgs e)
{
    if (int.TryParse(e.Value?.ToString(), out int ujMeret))
    {
        elemekOldalankent = ujMeret;
        jelenlegiOldal = 1;
        LapozasFrissitese();
    }
}
```

Mivel egy kollégiumban több száz lakó is lehet, a böngésző memóriáját és a UI-t lapozással védjük.

- A rendszer kiszámolja a maximális oldalszámot ($\text{Math.Ceiling}(\text{összes} / \text{oldalméret})$).
- Az aktuálisan megjelenített 5 vagy 10 sort a LINQ `.Skip().Take()` párosa vágja ki a szűrt listából. Például a 3. oldalon, 10-esével listázva: átugrik (Skip) 20 elemet, és kiveszi (Take) a következő 10-et.

2.23.11 A Modális Ablak Állapotkezelése

```
private void MegnyitSzerkesztes(int userId)
{
    var user = felhasznalokLista.FirstOrDefault(x => x.ID == userId);
    if (user == null) return;

    var diakAdat = diakAdatokLista.FirstOrDefault(x => x.UserID == userId);
    var aktivBeosztas = beosztasokLista.FirstOrDefault(x => x.UserID == userId && x.TenylegesKikoltozes == null);

    int eredetiSzoba = aktivBeosztas?.RoomID ?? 0;

    editModel = new EditViewModel
    {
        UserID = user.ID,
        Nev = user.Nev,
        Email = user.Email,
        KollegiumID = user.KollegiumID,
        SzerepkorID = user.SzerepkorID,
        Szemeszter = diakAdat?.Szemeszter ?? "",
        Lakcim = diakAdat?.Lakcim ?? "",
        EredetiRoomID = eredetiSzoba,
        RoomID = eredetiSzoba
    };

    isModalOpen = true;
}

private void BezarModal()
{
    isModalOpen = false;
}
```

A `MegnyitSzerkesztes(int userId)` metódus az adatok összeszereléséért felel. Megkeresi a kiválasztott felhasználó alapadatait, kiegészítő adatait (`diakAdat`) és az aktív szobabeosztását. Ezeket beletölti az `editModel`-be, majd az `isModalOpen = true` paranccsal "felhúzza a függönyt" a UI-on.

2.23.12 Szigorú Kaszkádolt Törlés

```
private async Task TorlesGomb()
{
    if (GetSzerekorNev(editModel.SzerekorID).ToLower().Contains("admin"))
    {
        hibaUzenet = "Adminisztrátor fiók nem törölhető!";
        isModalOpen = false;
        return;
    }

    bool isConfirmed = await JS.InvokeAsync<bool>("confirm", $"Birtosan véglegesen törölni szeretné '{editModel.Nev}' felhasználót? FIGYELEM: A hozzá tartozó összes adat is törölődni fog!");

    if (isConfirmed)
    {
        try
        {
            var penzugyL = penzugyekLista.Where(x => x.UserID == editModel.UserID).ToList();
            foreach (var item in penzugyL)
            {
                await Penzugyek.DeletePenzugyek(item.ID, token);
            }

            var diakAdat = diakAdatokLista.FirstOrDefault(x => x.UserID == editModel.UserID);
            if (diakAdat != null)
            {
                await DiakAdatok.DeleteDiakAdatok(diakAdat.UserID, token);
            }

            var beosztasok = beosztasokLista.Where(x => x.UserID == editModel.UserID).ToList();
            foreach (var item in beosztasok)
            {
                await SzobaBeosztasok.DeleteSzobaBeosztasok(item.ID, token);
            }

            var erintettKeresek = karbantartasiKeresekLista
                .Where(x => x.Bejelentoid == editModel.UserID)
                .ToList();

            foreach (var keres in erintettKeresek)
            {
                await KarbantartasiKeresek.DeleteKarbantartasiKeresek(keres.ID, token);
            }

            await Felhasznalo.DeleteFelhasznalok(editModel.UserID, token);

            isModalOpen = false;
            await AdatokBetoltese();

            sikerUzenet = "Felhasználó és adatai véglegesen törölve!";
            sikeresMentes = true;

            _ = Task.Run(async () =>
            {
                await Task.Delay(3000);
                sikeresMentes = false;
                await InvokeAsync(StateHasChanged);
            });
        }
        catch (Exception ex)
        {
            hibaUzenet = $"Hiba a törlés során: {ex.Message}";
        }
    }
}
```

A **TorlesGomb()** metódus egy professzionális adatbázis-menedzsment technikát alkalmaz. Mielőtt egy felhasználót törölnénk, egy böngészős megerősítést (confirm) kérünk. Mivel a relációs adatbázisok nem engedik a fő rekord törlését, ha ahhoz más táblákban adatok kötődnek (Foreign Key Constraint), a C# kód lépésről lépésre, manuálisan törli ki a diákhoz tartozó:

1. Pénzügyi befizetéseket
2. Kiegészítő diákadatokat
3. Szobabeosztásokat
4. Leadott hibajegyeket Csak miután a terep tiszta, akkor hívja meg a `Felhasznalo.DeleteFelhasznalok` API végpontot a profil végleges törléséhez.

2.23.13 Komplex Mentés és Szobabeosztási Logika

```
private async Task MentésGomb()
{
    try
    {
        var eredetiUser = felhasznalokLista.FirstOrDefault(x => x.ID == editModel.UserID);
        if (eredetiUser != null)
        {
            eredetiUser.Nev = editModel.Nev;
            eredetiUser.Email = editModel.Email;
            eredetiUser.SzerepkoID = editModel.SzerepkoID;
            eredetiUser.KollegiumID = editModel.KollegiumID;
            await Felhasznalo.UpdateFelhasznalok(eredetiUser, token);
        }

        if (!Diak(editModel.SzerepkoID))
        {
            var voltDiakAdat = diakAdatokLista.FirstOrDefault(x => x.UserID == editModel.UserID);
            if (voltDiakAdat != null)
            {
                await DiakAdatok.DeleteDiakAdatok(editModel.UserID, token);
            }

            var voltBeosztasok = beosztasokLista.Where(x => x.UserID == editModel.UserID).ToList();
            foreach (var beosztas in voltBeosztasok)
            {
                await SzobaBeosztasok.DeleteSzobaBeosztasok(beosztas.ID, token);
            }
        }
    }
}

else
{
    var eredetiDiakAdat = diakAdatokLista.FirstOrDefault(x => x.UserID == editModel.UserID);
    if (eredetiDiakAdat != null)
    {
        eredetiDiakAdat.Szemeszter = editModel.Szemeszter;
        eredetiDiakAdat.Lakcim = editModel.Lakcim;
        await DiakAdatok.UpdateDiakAdatok(eredetiDiakAdat, token);
    }
    else if (!string.IsNullOrEmpty(editModel.Szemeszter) || !string.IsNullOrEmpty(editModel.Lakcim))
    {
        var ujDiakAdat = new DiakAdatok { UserID = editModel.UserID, Szemeszter = editModel.Szemeszter, Lakcim = editModel.Lakcim };
        await DiakAdatok.CreateDiakAdatok(ujDiakAdat, token);
    }

    var aktivBeosztas = beosztasokLista.FirstOrDefault(x => x.UserID == editModel.UserID && x.TenylegesKikoltozes == null);

    if (editModel.RoomID == -1 && aktivBeosztas != null)
    {
        aktivBeosztas.TenylegesKikoltozes = DateTime.Now;
        await SzobaBeosztasok.UpdateSzobaBeosztasok(aktivBeosztas, token);
    }
    else if (editModel.RoomID > 0)
    {
        if (aktivBeosztas != null && editModel.RoomID != editModel.EredetiRoomID)
        {
            aktivBeosztas.TenylegesKikoltozes = DateTime.Now;
            await SzobaBeosztasok.UpdateSzobaBeosztasok(aktivBeosztas, token);

            int varhatoKikoltozes = 6;

            switch (editModel.Szemeszter)
            {
                case "1 szemeszter": varhatoKikoltozes = 6; break;
                case "2 szemeszter": varhatoKikoltozes = 12; break;
                case "3 szemeszter": varhatoKikoltozes = 18; break;
                case "4 szemeszter": varhatoKikoltozes = 24; break;
                case "5 szemeszter": varhatoKikoltozes = 30; break;
                case "6 szemeszter": varhatoKikoltozes = 36; break;
                case "7 szemeszter": varhatoKikoltozes = 42; break;
                case "8 szemeszter": varhatoKikoltozes = 48; break;
                case "9 szemeszter": varhatoKikoltozes = 54; break;
                case "10 szemeszter": varhatoKikoltozes = 60; break;
                default: varhatoKikoltozes = 6; break;
            }

            var ujBeosztas = new SzobaBeosztasok
            {
                UserID = editModel.UserID,
                RoomID = editModel.RoomID,
                BekoltozesDatum = DateTime.Now,
                VarhatoKikoltozes = DateTime.Now.AddMonths(varhatoKikoltozes)
            };
            await SzobaBeosztasok.CreateSzobaBeosztasok(ujBeosztas, token);
        }
    }
}
```

```

else if (aktivBeosztas == null)
{
    int varhatoKikoltozes = 6;

    switch (editModel.Szemeszter)
    {
        case "1 szemeszter": varhatoKikoltozes = 6; break;
        case "2 szemeszter": varhatoKikoltozes = 12; break;
        case "3 szemeszter": varhatoKikoltozes = 18; break;
        case "4 szemeszter": varhatoKikoltozes = 24; break;
        case "5 szemeszter": varhatoKikoltozes = 30; break;
        case "6 szemeszter": varhatoKikoltozes = 36; break;
        case "7 szemeszter": varhatoKikoltozes = 42; break;
        case "8 szemeszter": varhatoKikoltozes = 48; break;
        case "9 szemeszter": varhatoKikoltozes = 54; break;
        case "10 szemeszter": varhatoKikoltozes = 60; break;
        default: varhatoKikoltozes = 6; break;
    }

    var ujBeosztas = new SzobaBeosztasok
    {
        UserID = editModel.UserID,
        RoomID = editModel.RoomID,
        BekoltozesDatum = DateTime.Now,
        VarhatoKikoltozes = DateTime.Now.AddMonths(varhatoKikoltozes)
    };
    await SzobaBeosztasok.CreateSzobaBeosztasok(ujBeosztas, token);
}

isModalOpen = false;
await AdatokBetoltese();

sikerUzenet = "Sikeres mentés és adatok frissítése!";
sikerMentes = true;

_ = Task.Run(async () =>
{
    await Task.Delay(3000);
    sikeresMentes = false;
    await InvokeAsync(StateHasChanged);
});
}
catch (Exception ex)
{
    hibaUzenet = $"Hiba a mentés során: {ex.Message}";
}
}

```

A **MentesGomb()** a legösszetettebb üzleti logika az egész rendszerben.

- **Alapadatok:** Először frissíti a felhasználó nevét, emailjét és szerepkörét.
- **Szerepkörváltás:** Ha egy Diákot visszaminősítenek (vagy felminősítenek Adminná), a rendszer automatikusan "takarít": törli a specifikus diákadatait és az aktív szobabeosztásait, mivel ezekre már nem lesz jogosult.
- **Szobakezelés (Kiköltözés és Átköltözés):** * Ha a legördülő menüben a "-1" (Kiköltöztetés) értéket választják, a rendszer lezárja a meglévő szobabeosztást (TenylegesKikoltozes = DateTime.Now), anélkül, hogy újat hozna létre.
 - Ha új szobát kap a diák, a rendszer lezárja a régit, és létrehoz egy újat.
- **Várható kiköltözés számítása:** Új szobabeosztás létrehozásakor a kód egy switch szerkezettel elemzi a diák szemeszterét. Minden szemeszter 6 hónap kollégiumi tartózkodást jelent (pl. 2. szemeszter = 12 hónap). Az algoritmus a

`DateTime.Now.AddMonths(varhatoKikoltozes)` függvénnyel automatikusan kiszámítja és elmenti, hogy a hallgatónak papírforma szerint mikor kell elhagynia a kollégiumot.

2.23.14 Kliensoldali Segédmetódusok

```
private string GetKollegiumNev(int kolid)
{
    var kolid = kollegiumLista.FirstOrDefault(x => x.ID == kolid);
    return kolid != null ? kolid.KollegiumNev : "Nincs megadva";
}

private bool Diak(int szerepId)
{
    var nev = GetSzerepNev(szerepId).ToLower();
    return !nev.Contains("admin") && !nev.Contains("karbantarto");
}

private bool Karbantarto(int szerepId)
{
    var nev = GetSzerepNev(szerepId).ToLower();
    return !nev.Contains("admin");
}

private List<KoliPortal.Lib.MODEL.Szoba> GetSzabadSzobak(int kiveveSzobaId)
{
    var szabadok = new List<KoliPortal.Lib.MODEL.Szoba>();
    foreach (var szoba in szobakLista)
    {
        if (szoba.ID == kiveveSzobaId) continue;

        int aktivLakok = GetAktivLakokSzam(szoba.ID);
        if (aktivLakok < szoba.FerohelyMax)
        {
            szabadok.Add(szoba);
        }
    }
    return szabadok.OrderBy(s => s.Szobaszam).ToList();
}

private int GetAktivLakokSzam(int roomId)
{
    return beosztasokLista.Count(x => x.RoomID == roomId && x.TenylegesKikoltozes == null);
}

private string GetSzobaSzam(int roomId)
{
    var szoba = szobakLista.FirstOrDefault(x => x.ID == roomId);
    return szoba != null ? szoba.Szobaszam : "Ismeretlen";
}

private string GetAktualisSzoba(int userId)
{
    var aktivBeosztas = beosztasokLista.FirstOrDefault(x => x.UserID == userId && x.TenylegesKikoltozes == null);
    if (aktivBeosztas != null)
    {
        return GetSzobaSzam(aktivBeosztas.RoomID);
    }

    bool voltSzobaja = beosztasokLista.Any(x => x.UserID == userId);
    if (voltSzobaja)
    {
        return "Kiköltözött";
    }

    return "Nincs szobája";
}

private string GetSzerepNev(int szerepId)
{
    var szerep = szerepekLista.FirstOrDefault(x => x.ID == szerepId);
    return szerep != null ? szerep.Nev : "Ismeretlen";
}

private string GetSzerepCss(int szerepId)
{
    var nev = GetSzerepNev(szerepId).ToLower();
    if (nev.Contains("admin")) return "role-admin";
    else if (nev.Contains("karbantarto")) return "role-maintenance";
    else return "role-student";
}
```

Ez a blokk tartalmazza a memóriában lévő listák lekérdezésére és a UI formázására szolgáló metódusokat. Ahelyett, hogy a Razor HTML kódját bonyolult C# logikákkal zsúfolnád tele, ezek a metódusok kiszervezik és újrafelhasználhatóvá teszik a gyakori feladatokat:

- **Relációs Azonosítók (Foreign Keys) feloldása:** A `GetKollegiumNev`, `GetSzerepkorNev` és `GetSzobaSzam` metódusok felelnek azért, hogy a nyers azonosítók (ID-k) a felhasználó számára olvasható szövegeket generáljanak (pl. a `RoomID: 12` helyett "101-es szoba"). Mindezt villámgyorsan, a kliens memóriájában lévő listákból (`.FirstOrDefault()`), felesleges hálózati kérések nélkül.
- **Jogosultsági Logika (Role Checks):** A `Diak()` és `Karbantarto()` metódusok leegyszerűsítik a jogosultság-vizsgálatot. A HTML részben elég meghívni a `@if (Diak(szerepkor))` feltételt, hogy a rendszer eldöntse, meg kell-e jeleníteni a tanulmányi mezőket.
- **Komplex Állapotvizsgálat (GetAktualisSzoba):** Ez a metódus állapítja meg a táblázat számára a diák kollégiumi státuszát. Logikája háromrétű:
 1. Ha van aktív beosztása, visszaadja a szobaszámot.
 2. Ha nincs aktív, de a múltban volt (`voltsZobaja`), akkor "Kiköltözött" státuszt ad.
 3. Ha sosem volt szobája, "Nincs szobája" értékkel tér vissza.
- **Dinamikus Kapacitásszámítás (GetSzabadSzobak):** A szobabeosztás szerkesztő legördülő menüjének "motorja". Végigiterál az összes létező szobán, a `GetAktivLakokSzam` segítségével megszámlolja a bentlakókat, és **csak azokat** a szobákat adja hozzá a listához, ahol `aktivLakok < szoba.FerohelyMax`. Külön figyelmet érdemel, hogy a paraméterként kapott jelenlegi szobát (`kiveveSzobaId`) kihagyja a listából, hogy a diákok ne lehessen ugyanabba a szobába "áthelyezni", ahol már eleve bent lakik.
- **UI Dinamika (GetSzerepkorCss):** A jogosultság nevéből automatikusan generálja a megfelelő CSS osztály nevét (`role-admin`, `role-student`), így a táblázatban a színes állapotjelzők (Badges) minden beavatkozás nélkül, automatikusan a megfelelő stílusban jelennek meg.

2.23.15 Adatcsomagoló Modell

```
public class EditViewModel
{
    public int UserID { get; set; }
    public string Nev { get; set; } = "";
    public string Email { get; set; } = "";
    public int KollegiumID { get; set; }
    public int SzerepkorID { get; set; }
    public string Szemeszter { get; set; } = "";
    public string Lakcim { get; set; } = "";
    public int EredetiRoomID { get; set; }
    public int RoomID { get; set; }
}
```

kód legvégén látható az EditViewModel osztály deklarációja. Ez egy klasszikus **DTO** (Data Transfer Object) tervezési minta alkalmazása a kliensoldalon.

- **Miért van rá szükség?** Egy diák adatainak szerkesztésekor a háttérben három különböző adatbázistábla (Felhasználó, DiákAdat, Szobabeosztás) adatait kell módosítani. Ahelyett, hogy a modális ablak (Modal) HTML kódjában három különböző objektumra hivatkoznánk, a rendszer ezt az egyetlen, "lapos" (flat) osztályt használja. Ez fogja össze a nevet, emailt, szemesztert és a szobainformációkat, rendkívül tisztává és karbantarthatóvá téve az űrlap adatkötéseit (@bind).

2.24 Regisztralas.razor

A **Regisztralas.razor** komponens az adminisztrációs felület beépített regisztrációs modulja. Felelőssége új rendszergazdák, karbantartók vagy diákok (lakók) hozzáadása a rendszerhez. Mivel a diákok esetében nemcsak egy alapfiókot, hanem kiegészítő adatokat (lakcím, tanulmányi félév) és szobabeosztást is azonnal létre kell hozni, a komponens egy többlépcsős, komplex tranzakciót szimulál a kliensoldalon.

2.24.1 Környezet és Alapadatok Betöltése

```
@page "/regisztralas"
@using MoliPortal.Lib.MODEL
@using MoliPortal.Lib.SERVICE
@layout AdminLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject SzerepkorokService Szerepkorok
@inject AuthControllerService AuthController
@inject SzobakService Szobak
@inject SzobaBeosztasokService SzobaBeosztasok
@inject DiakAdatokService DiakAdatok
@inject FelhasznalokService Felhasznalok
@inject NavigationManager Nav
@inject IJSRuntime JS
```

```
@code {
    private string token = "";
    private bool betoltesAlatt = true;
    private string hibaUzenet = "";

    private string vezetekNev = "";
    private string keresztnév = "";
    private string email = "";
    private string jelszo = "";
    private string jelszoUjra = "";
    private string választottTelefonszam = "";
    private int választottSzerepId = 0;

    private string választottSzemeszter = "";
    private string választottLakcim = "";
    private int választottSzobaId = 0;
}
```

```
protected override async Task OnAfterRenderAsync(bool elsőBetoletes)
{
    if (elsőBetoletes)
    {
        try
        {
            token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

            if (string.IsNullOrEmpty(token))
            {
                Nav.NavigateTo("/");
                return;
            }

            var handler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
            var jwtToken = handler.ReadJwtToken(token);
            var szerepId = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

            if (szerepId != "1" && szerepId?.ToLower() != "admin")
            {
                Nav.NavigateTo("/");
                return;
            }

            szerepIdLista = await SzerepIdok.GetSzerepIdok(token) ?? new();
            szobakLista = await Szobak.GetSzobak(token) ?? new();
            beosztasokLista = await SzobaBeosztasok.GetSzobaBeosztasok(token) ?? new();
        }
        catch (Exception ex)
        {
            hibaUzenet = $"Hiba a betöltéskor: {ex.Message}";
        }
        finally
        {
            betoltesAlatt = false;
            StateHasChanged();
        }
    }
}
```

- **Injektálások és Direktívák:** A komponens a megszokott AdminLayout keretbe töltődik, és az InteractiveServer mód biztosítja az azonnali űrlap-reakciókat. Számos hálózati szolgáltatást injektál (AuthController, DiakAdatokService, SzobakService), mivel a teljes regisztrációs folyamat ezek összehangolt munkáját igényli.
- **Biztonsági Kapu (OnAfterRenderAsync):** Az oldal betöltésekor a rendszer dekódolja a böngészőben lévő JWT tokent. Szigorú ellenőrzést végez: ha a felhasználó nem "1"-es (Admin) jogosultságú, azonnal visszairányítja a kezdőlapra.
- **Függőségek letöltése:** Sikeres azonosítás után a kód előtölti a regisztrációhoz szükséges legördülő listákat (szerepek, szobák, jelenlegi beosztások).

2.24.2 Dinamikus és Feltételes Felhasználói Felület

```

else
{
    <div class="content-wrapper">
    <div class="card form-card shadow-sm p-4" style="max-width: 600px; margin: 0 auto;">
        @if (!string.IsNullOrEmpty(hibaUzenet))
        {
            <div class="alert alert-danger">
                <i class="bi bi-exclamation-triangle-fill me-2"></i>@hibaUzenet
            </div>
        }

        <form @onsubmit="FelhasznaloLeltrehozasa">
            <div class="form-row d-flex gap-3 mb-3">
                <div class="form-group flex-fill">
                    <label>Vezetéknév <span class="text-danger">*</span></label>
                    <input type="text" class="form-control" @bind="vezetekNev" placeholder="Pl. Kovács" required />
                </div>
                <div class="form-group flex-fill">
                    <label>Keresztnév <span class="text-danger">*</span></label>
                    <input type="text" class="form-control" @bind="keresztNev" placeholder="Pl. János" required />
                </div>
            </div>

            <div class="form-group mb-3">
                <label>Email cím <span class="text-danger">*</span></label>
                <input type="email" class="form-control" @bind="email" placeholder="pelda@email.com" required />
            </div>

            <div class="form-row d-flex gap-3 mb-3">
                <div class="form-group flex-fill">
                    <label>Jelszó <span class="text-danger">*</span></label>
                    <input type="password" class="form-control" @bind="jelszo" placeholder="Legalább 6 karakter" required />
                </div>
                <div class="form-group flex-fill">
                    <label>Jelszó megerősítés <span class="text-danger">*</span></label>
                    <input type="password" class="form-control" @bind="jelszoUjra" placeholder="Jelszó újra" required />
                </div>
            </div>

            <div class="form-group mb-3">
                <label>Szerepkör <span class="text-danger">*</span></label>
                <select class="form-select" @bind="valasztottSzerepkorId" required>
                    <option value="0" disabled selected>Válassz szerepkört...</option>
                    @foreach (var role in szerepkorokLista)
                    {
                        <option value="@role.ID">@role.Nev</option>
                    }
                </select>
            </div>
        </form>
    </div>
}

```

```

@if (IsLako(valasztottSzerepkorId))
{
  <hr class="my-4" />
  <h5 class="mb-3">Diák extra adatok</h5>

  <div class="form-row d-flex gap-3 mb-3">
    <div class="form-group flex-fill">
      <label>Szemeszter <span class="text-danger">*</span></label>
      <select class="form-select" @bind="valasztottSzemeszter" required>
        <option value="" disabled selected>Válassz szemesztert...</option>
        <option value="1" szemeszter">1 szemeszter</option>
        <option value="2" szemeszter">2 szemeszter</option>
        <option value="3" szemeszter">3 szemeszter</option>
        <option value="4" szemeszter">4 szemeszter</option>
        <option value="5" szemeszter">5 szemeszter</option>
        <option value="6" szemeszter">6 szemeszter</option>
        <option value="7" szemeszter">7 szemeszter</option>
        <option value="8" szemeszter">8 szemeszter</option>
        <option value="9" szemeszter">9 szemeszter</option>
        <option value="10" szemeszter">10 szemeszter</option>
      </select>
    </div>

    <div class="form-group flex-fill">
      <label>Szoba beosztás</label>
      <select class="form-select" @bind="valasztottSzobaId">
        <option value="0" selected>Szoba beosztása</option>
        @foreach (var szoba in GetSzabadSzobak())
        {
          <option value="@szoba.ID"@szoba.Szobaszam (Szabad hely: @(<math>szoba.FerohelyMax - GetAktivLakokSzam(szoba.ID)</math>))</option>
        }
      </select>
    </div>
  </div>

  <div class="form-group mb-3">
    <label>Lakcím</label>
    <input type="text" class="form-control" @bind="valasztottLakcim" placeholder="Pl. 1051 Budapest, Fő utca 1." />
  </div>

  <div class="form-group mb-3">
    <label>Telefonszám</label>
    <input type="tel" class="form-control" @bind="valasztottTelefonszam" placeholder="Pl. +36203988762" maxlength="12">
    <div class="form-text">Formátum: + és max. 11 számjegy (szóköz nélkül).</div>
  </div>

  <div class="form-actions d-flex justify-content-end gap-2 mt-4">
    <button type="button" class="btn btn-secondary" @onclick="Megse">Megse</button>
    <button type="submit" class="btn btn-success px-4">Létrehozás</button>
  </div>
</form>
</div>

```

Az űrlap a modern webes irányelveknek megfelelően reagál a felhasználói interakciókra:

- **Alap űrlap:** Kétirányú adatkötéssel (@bind) gyűjti be a vezetéknév, keresztnév, e-mail és jelszó adatokat a C# változókba.
- **Állapotfüggő UI (Feltételes renderelés):** A @if (IsLako(valasztottSzerepkorId)) logikai blokk egy elegáns UX megoldás. Ha az adminisztrátor a "Lakó" szerepkört választja ki a legördülő menüben, a DOM-ban (dokumentumobjektum-modell) azonnal és animáció nélkül megjelennek a diák-specifikus extra mezők: Szemeszter, Szoba beosztás, Lakcím és Telefonszám. Ha Karbantartót vagy Admint választ, ezek a mezők rejtve maradnak, elkerülve a felesleges adatbekérést.

2.24.3 Kliensoldali Validációs Motor

```
private async Task FelhasznaloLeltrehozasa()
{
    hibajzenet = "";

    if (valasztottSzerepkoId == 0)
    {
        hibajzenet = "Kérlek, válassz egy szerepkört!";
        return;
    }

    // Telefonszám ellenőrzése (ha Lakó és van telefonszám)
    if (IsLako(valasztottSzerepkoId) && !string.IsNullOrWhiteSpace(valasztottTelefonszam))
    {
        string tel = valasztottTelefonszam.Trim();

        if (!tel.StartsWith("+"))
        {
            hibajzenet = "A telefonszámnak '+' jellel kell kezdődnie!";
            return;
        }

        if (tel.Length < 2)
        {
            hibajzenet = "A '+' jel után számokat is meg kell adni!";
            return;
        }

        if (tel.Length > 12)
        {
            hibajzenet = "A telefonszám maximum 11 számjegyet tartalmazhat a '+' jel után!";
            return;
        }

        for (int i = 1; i < tel.Length; i++)
        {
            if (!char.IsDigit(tel[i]))
            {
                hibajzenet = "Érvénytelen telefonszám! A '+' jel után csak számok állhatnak.";
                return;
            }
        }
    }

    if (IsLako(valasztottSzerepkoId) && string.IsNullOrEmpty(valasztottSzemeszter))
    {
        hibajzenet = "Diák regisztrálásakor kötelező megadni az szemesztert!";
        return;
    }

    if (jelszo != jelszoUjra)
    {
        hibajzenet = "A megadott jelszavak nem egyeznek!";
        return;
    }

    if (jelszo.Length < 6)
    {
        hibajzenet = "A jelszónak legalább 6 karakter hosszúnak kell lennie!";
        return;
    }
}
```

A **felhasznaloLeltrehozasa()** aszinkron metódus első szakasza egy robusztus, többlépcsős adatellenőrző algoritmus, amely hálózati forgalom generálása nélkül, azonnal kiszűri a felhasználói hibákat:

- **Kötelező mezők:** Ellenőrzi, hogy választottak-e szerepkört, illetve diák esetében megadták-e a szemesztert.
- **Jelszó ellenőrzés:** Biztosítja, hogy a két jelszómező tartalma egyezzen, és hossza elérje a minimális 6 karaktert.
- **Szigorú Telefonszám-validáció:** Ha a diák megadott telefonszámot, a kód manuálisan, karakterről karakterre elemzi azt. Megköveteli, hogy + jellel kezdődjön, minimum 2, maximum 12 karakter hosszú legyen, és a + jel után kizárólag számjegyeket (`char.IsDigit`) tartalmazzon. Bármilyen hiba esetén a folyamat megszakad, és az alert-danger dobozban megjelenik a pontos hibaüzenet.

2.24.4 Komplex Mentési Folyamat

```
try
{
    var ujUser = new KoliPortal.Lib.MODEL.Felhasznalok
    {
        Nev = $"{vezetekNev} {keresztNev}",
        Email = email,
        Jelszo = jelszo,
        SzerepkoID = választottSzerepkoID,
        Telefonszam = választottTelefonszam,
        KollegiumID = 1 // Jelenleg fix érték, később ha tovább lesz fejlesztve akkor lehet majd választani kollégiumot regisztrációkor
    };

    bool sikeres = await AuthController.RegisterAsync(ujUser, token);

    if (sikeres)
    {
        if (IsLako(valasztottSzerepkoID))
        {
            var osszesUser = await Felhasznalok.GetFelhasznalok(token) ?? new();
            var frissUser = osszesUser.FirstOrDefault(u => u.Email == email);
        }
    }
}
```

```
try
{
    string veglegesLakcim = string.IsNullOrEmpty(valasztottLakcim) ? "Nincs megadva" : választottLakcim;

    var ujDiakAdat = new DiakAdatok
    {
        UserID = frissUser.ID,
        Szemeszter = választottSzemeszter,
        Lakcim = veglegesLakcim
    };
    await DiakAdatok.CreateDiakAdatok(ujDiakAdat, token);

    if (valasztottSzobaID > 0)
    {
        // Alapértelmezetten 6 hónapot adunk
        int varhatoKikoltozes = 6;

        switch (valasztottSzemeszter)
        {
            case "1 szemeszter":
                varhatoKikoltozes = 6;
                break;
            case "2 szemeszter":
                varhatoKikoltozes = 12;
                break;
            case "3 szemeszter":
                varhatoKikoltozes = 18;
                break;
            case "4 szemeszter":
                varhatoKikoltozes = 24;
                break;
            case "5 szemeszter":
                varhatoKikoltozes = 30;
                break;
            case "6 szemeszter":
                varhatoKikoltozes = 36;
                break;
            case "7 szemeszter":
                varhatoKikoltozes = 42;
                break;
            case "8 szemeszter":
                varhatoKikoltozes = 48;
                break;
            case "9 szemeszter":
                varhatoKikoltozes = 54;
                break;
            case "10 szemeszter":
                varhatoKikoltozes = 60;
                break;
            default:
                varhatoKikoltozes = 6;
                break;
        }

        var ujBeosztas = new SzobaBeosztasok
        {
            UserID = frissUser.ID,
            RoomID = választottSzobaID,
            BekoltozesDatum = DateTime.Now,
            VarhatoKikoltozes = DateTime.Now.AddMonths(varhatoKikoltozes)
        };
        await SzobaBeosztasok.CreateSzobaBeosztasok(ujBeosztas, token);
    }
}
```



```

        catch (Exception Hiba)
        {
            hibaUzenet = $"A fiók létrejött, de a plusz adatokat nem sikerült menteni! ({Hiba.Message})";
            return;
        }
    }
    Nav.NavigateTo("/adminAttekintes");
}
else
{
    hibaUzenet = "Hiba történt! Lehet, hogy ez az email cím már foglalt.";
}
}
catch (Exception ex)
{
    hibaUzenet = $"Rendszerhiba: {ex.Message}";
}
}

```

Ha a validáció sikeres, egy try-catch blokkban elindul a tényleges adatbázis-szimulációs tranzakció:

1. **Alapfiók létrehozása:** A kód összefűzi a vezeték- és keresztnévet, majd a MODEL.Felhasznalok osztályba csomagolva elküldi az AuthController.RegisterAsync végpontnak. Ez létrehozza az alap profilt titkosított jelszóval.
2. **Az új azonosító (ID) visszakeresése:** Mivel az API állapotmentes, a kliensnek meg kell tudnia, mi lett az új felhasználó generált ID-ja. Ehhez az `osszesUser.FirstOrDefault(...)` LINQ lekérdezéssel, az egyedi e-mail cím alapján kikeresi az imént létrehozott fiókot.
3. **Kiegészítő adatok mentése (Diák esetén):** * Létrehoz egy DiakAdatok objektumot a laccímmel és évfolyammal, majd elmenti.
 - A switch szerkezettel a kiválasztott szemeszter alapján dinamikusan kiszámolja a várható kiköltözés idejét (pl. 2 szemeszter = 12 hónap).
 - Létrehoz egy SzobaBeosztasok objektumot az új ID-val, a választott szoba azonosítójával és az aktuális dátummal (`DateTime.Now`), majd ezt is elküldi a szervernek.

Hibakezelés (Exception Handling): A folyamat mesterien használja a beágyazott try-catch blokkokat. Ha maga a regisztráció (az e-mail cím) sikertelen (mert pl. már létezik), az egyedi hibaüzenetet ad. Ha a regisztráció sikerült, de a hálózat megszakad a szobabeosztás mentése közben, a belső catch blokk figyelmezteti az adminisztrátort: *"A fiók létrejött, de a plusz adatokat nem sikerült menteni!"*, megakadályozva az adatok inkonzisztenciáját, majd visszairányítja az áttekintő nézetre.

2.24.5 Kliensoldali Segédmetódusok és Kapacitásvizsgálat

```
private bool IsLako(int roleId)
{
    var role = szerepkorokLista.FirstOrDefault(x => x.ID == roleId);
    return role != null && !role.Nev.ToLower().Contains("admin") && !role.Nev.ToLower().Contains("karbantarto");
}

private List<KoliPortal.Lib.MODEL.Szobak> GetSzabadSzobak()
{
    var szabadok = new List<KoliPortal.Lib.MODEL.Szobak>();
    foreach (var szoba in szobakLista)
    {
        if (GetAktivLakokSzam(szoba.ID) < szoba.FerohelyMax)
        {
            szabadok.Add(szoba);
        }
    }
    return szabadok.OrderBy(s => s.Szobaszam).ToList();
}

private int GetAktivLakokSzam(int roomId)
{
    return beosztasokLista.Count(x => x.RoomID == roomId && x.TenylegesKikoltozes == null);
}

private void Megse()
{
    Nav.NavigateTo("/adminAttekintes");
}
```

A komponens végén található metódusok a dinamikus űrlap és a szobabeosztási logika alapkövei. Ezek a funkciók a memóriába letöltött listákon dolgoznak, így azonnali válaszüdejű, hálózati késleltetés nélküli felhasználói élményt nyújtanak:

- **Jogosultságvizsgáló (IsLako):** Ez a metódus a lelke az űrlap feltételes megjelenítésének (Conditional UI). Keresést végez a szerepkörök listájában, és ha a kiválasztott szerepkör neve nem tartalmazza az "admin" vagy "karbantarto" szavakat, akkor igaz (true) értékkel tér vissza. Ekkor nyílnak le a diák-specifikus extra beviteli mezők a képernyőn.
- **Valós idejű Kapacitás-kalkulátor (GetSzabadSzobak és GetAktivLakokSzam):** Ez a két, egymásra épülő metódus felel a szobaválasztó legördülő menü feltöltéséért.
 1. A GetAktivLakokSzam egy adott szobára vonatkozóan megszámlolja azokat a beosztásokat, ahol még nincs tényleges kiköltözési dátum (TenylegesKikoltozes == null).
 2. A GetSzabadSzobak végigiterál a kollégium összes szobáján. Meghívja a fenti számlálót, és összehasonlítja az eredményt a szoba maximális férőhelyével (FerohelyMax). A listába kizárólag azok a szobák kerülnek be, ahol van még legalább egy szabad ágy. A végeredményt szobaszám szerint növekvő sorrendbe (OrderBy) rendezi, megkönnyítve az adminisztrátor dolgát a legördülő menüben.

- **Biztonságos Kilépés (Megse):** Egy egyszerű, de fontos UX funkció. A Mégse gombra kattintva a NavigationManager azonnal, adatmentés nélkül visszairányítja a felhasználót a központi adminisztrációs műszerfalra (/adminAttekintes).

2.25 Szobak.razor

A Szobak.razor komponens az intézmény férőhelyeinek menedzselésére szolgál. Míg a korábbi űrlapok adatbeviteli funkciókat láttak el, ez az oldal egy nagy teljesítményű, csak olvasható (Read-only) adatrács (Data Grid), amely fókuszált szűrési és keresési lehetőségekkel támogatja a napi adminisztrációs munkát (pl. új hallgatók beköltöztetésekor a szabad helyek gyors megtalálása).

2.25.1 Alapok és Injektált Szolgáltatások

```
@page "/szobak"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@layout AdminLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))

@inject SzobakService Szoba
@inject SzobaBeosztasokService SzobaBeosztasok
@inject FelhasznalokService Felhasznalok
@inject IJSRuntime JS
@inject NavigationManager Nav
```

- A komponens az AdminLayout elrendezést használja.
- A valós idejű kereséshez és lapozáshoz a szerveroldali interaktivitás (InteractiveServer) elengedhetetlen, az előrenderelés (prerender: false) kikapcsolásával együtt a JWT token biztonságos kiolvasása érdekében.
- A rendszer három fő adatforrást injektál: a szobák, a felhasználók és a szobabeosztások hálózati szolgáltatásait (SzobakService, SzobaBeosztasokService, FelhasznalokService).

2.25.2 Dinamikus Szűrő és Keresőfelület

```

@* --- SZŰRŐK, KERESŐ ÉS OLDALMÉRLET VÁLASZTÓ --- *@
<div class="d-flex justify-content-between align-items-center mb-3 flex-wrap gap-3">
  <div class="filter-controls m-0">
    <button class="filter-btn @(mutasdCsakSzabadokat ? "" : "active")"
      @onclick="() => SetSzuro(false)">
      Összes
    </button>
    <button class="filter-btn @(mutasdCsakSzabadokat ? "active" : "")"
      @onclick="() => SetSzuro(true)">
      Szabad ágyak
    </button>
  </div>

  <div class="d-flex align-items-center gap-4">
    <div class="input-group input-group-sm shadow-sm" style="width: 250px;">
      <span class="input-group-text bg-white border-end-0"><i class="bi bi-search text-muted"></i></span>
      <input type="text" class="form-control border-start-0 ps-0" placeholder="Keresés (szoba, lakó)..." value="@keresoSzoveg" @oninput="KeresesValtozas" />
    </div>

    <div class="d-flex align-items-center gap-2">
      <span class="text-muted small fw-bold text-uppercase">Sor/oldal:</span>
      <select class="form-select form-select-sm shadow-sm page-size-select" style="width: 80px; border-radius: 8px; cursor: pointer;" value="@elemekOldalankent"
        @onchange="OldalMeretValtozas">
        <option value="5">5</option>
        <option value="10">10</option>
        <option value="25">25</option>
        <option value="50">50</option>
      </select>
    </div>
  </div>
</div>

```

Ez a szekció biztosítja az adatok gyors strukturálását az adminisztrátor számára.

- **Gyorsszűrő Gombok:** Két állapot (Összes / Szabad ágyak) közötti váltást tesz lehetővé. A gombok CSS osztálya (active) dinamikusan változik a háttérben lévő @mutasdCsakSzabadokat logikai változó alapján.
- **Komplex Keresőmező:** A valós idejű kereső (@oninput="KeresesValtozas") nemcsak a szobaszámok között keres, hanem képes a szobához rendelt **lakók nevei** alapján is szűrni.

2.25.3 Az Adattábla és az Állapotjelzők

```

@* --- TÁBLÁZAT --- *@
<div class="card shadow-sm">
  <div class="table-scroll-container">
    <table class="data-table">
      <thead>
        <tr>
          <th>Szoba</th>
          <th>Emelet</th>
          <th>Kapacitás</th>
          <th>Lakók</th>
          <th>Telítettség</th>
        </tr>
      </thead>
      <tbody>
        @if (megjelenítettSzobak != null && megjelenítettSzobak.Any())
        {
          @foreach (var szoba in megjelenítettSzobak)
          {
            // Kiszámoljuk az adott szoba adatait
            int aktivLakok = GetAktivLakokSzam(szoba.ID);
            string lakonevek = GetLakoNevek(szoba.ID);
            double szazalek = szoba.FerohelyMax > 0 ? ((double)aktivLakok / szoba.FerohelyMax) * 100 : 0;
            string badgeCss = GetStatuszCss(aktivLakok, szoba.Fe

            <tr>
              <td class="fw-bold">@szoba.Szobaszam</td>
              <td>@szoba.Emelet. Emelet</td>
              <td>@szoba.FerohelyMax fő</td>
              <td>@LakoNevek</td>
              <td>
                <span class="status-badge @badgeCss">
                  @(Math.Round(szazalek))% (@aktivLakok/@szoba.FerohelyMax)
                </span>
              </td>
            </tr>
          }
        }
        else
        {
          <tr>
            <td colspan="5" class="text-center text-muted p-5">Nincs a szűrésnek megfelelő szoba.</td>
          </tr>
        }
      </tbody>
    </table>
  </div>
</div>

```

A táblázat (Grid) HTML struktúrája és adatkötése.

- A `@foreach` ciklus végigiterál a már megszűrt és lapozott `megjelenítettSzobak` listán.
- **Dinamikus Telítettség Számítás:** A táblázat minden sora futásidőben kiszámolja a szoba telítettségi arányát egy százalékos képlettel $((\text{aktivLakok} / \text{szoba.FerohelyMax}) * 100)$. Az értéket a `Math.Round` egész számra kerekíti.
- **Színkódolt Státusz (Badge):** A kiszámolt érték és a kapacitás alapján a HTML egy vizuális állapotjelzőt rajzol ki, amelynek színeit a `GetStatuszCss` C# metódus vezérli (pl. zöld az üres, piros a tele szobáknál).

2.25.4 Biztonság és Aszinkron Adatbetöltés

```
@code {
    private string token = "";
    private bool betoltesAlatt = true;

    private bool mutasdCsakSzabadokat = false;
    private string keresoSzoveg = "";

    // Alapadatok
    private List<KoliPortal.Lib.MODEL.Szobak> szobakLista = new();
    private List<SzobaBeosztasok> aktivBeosztasok = new();
    private List<KoliPortal.Lib.MODEL.Felhasznalok> felhasznalokLista = new();

    // Listák a szűréshez és lapozáshoz
    private List<KoliPortal.Lib.MODEL.Szobak> szurtSzobak = new();
    private List<KoliPortal.Lib.MODEL.Szobak> megjelenítettSzobak = new();

    // LAPOZÁS VÁLTOZÓI
    private int jelenlegiOldal = 1;
    private int elemekOldalankent = 10;
    private int osszesOldal = 1;
}
```

```
protected override async Task OnAfterRenderAsync(bool elsoBetoletes)
{
    if (elsoBetoletes)
    {
        try
        {
            // Kérjük el a token a böngészőtől
            token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

            if (string.IsNullOrEmpty(token))
            {
                Nav.NavigateTo("/login", forceLoad: true);
                return; // Ha nincs token, visszadob a bejelentkező oldalra
            }

            // SZIGORÚ SZEREPKÖR ELLENŐRZÉS CSAK ADMIN!
            var handler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
            var jwtToken = handler.ReadJwtToken(token);
            var szerepkor = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

            // Ha a szerepkör NEM 1-es és NEM admin, akkor login oldal!
            if (szerepkor != "1" && szerepkor?.ToLower() != "admin")
            {
                // Mindenki más repül a loginra
                Nav.NavigateTo("/login");
                return;
            }

            // Adatok letöltése a tokennel
            szobakLista = await Szoba.GetSzobak(token) ?? new();
            felhasznalokLista = await Felhasznalok.GetFelhasznalok(token) ?? new();
            var beosztasok = await SzobaBeosztasok.GetSzobaBeosztasok(token) ?? new();

            // Csak az aktív lakókat tartjuk meg
            aktivBeosztasok = beosztasok.Where(x => x.TenylegesKikoltozes == null).ToList();

            // Kezdetben minden szobát mutatunk, szobaszám szerint sorba rendezve
            FrissitSzurtListat();
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine($"Hiba az adatok lekérésekor: {ex.Message}");
        }
        finally
        {
            betoltesAlatt = false;
            StateHasChanged();
        }
    }
}
```

Az állapotváltozók deklarálása után a rendszer az OnAfterRenderAsync metódusban építi fel az oldal logikáját:

- **Token Validáció:** A kód explicit módon ellenőrzi a JWT tokenet, garantálva, hogy kizárólag "admin" szerepkörű felhasználók láthassák a kollégium teljes szobabeosztását.
- **Memória Optimalizálás:** Miután az API-tól letölti a három fő adatlistát, a rendszer azonnal végez egy előszűrést: az aktivBeosztasok listába csak azokat a

szobabeosztásokat menti el, amelyeknél a `TenylegesKikoltozes` mező még null (azaz a diák jelenleg is ott lakik). Ezzel jelentősen felgyorsítja a későbbi listázó algoritmusokat.

2.25.5 A Szűrési Algoritmus Motorja

```
// Gombnyomásra fut le: beállítja a szűrőt és frissíti a listát
private void SetSzuro(bool csakSzabad)
{
    mutasdCsakSzabadokat = csakSzabad;
    jelenlegiOldal = 1; // Szűrőkor ugrás az 1. oldalra
    FrissitSzurtListat();
}

// Gépelésre fut le
private void KeresesValtozas(EventArgs e)
{
    keresoSzoveg = e.Value?.ToString() ?? "";
    jelenlegiOldal = 1;
    FrissitSzurtListat();
}

// Frissíti a szűrt listát a gombok és a kereső alapján
private void FrissitSzurtListat()
{
    // 1. Alap szűrés (Gombok: Összes / Szabad ágyak)
    if (mutasdCsakSzabadokat)
    {
        // Csak azok a szobák kelljenek, ahol kevesebb az aktív lakó, mint a férőhely
        szurtSzobak = szobakLista
            .Where(x => GetAktivLakokSzam(x.ID) < x.FerohelyMax)
            .OrderBy(x => x.Szobaszam)
            .ToList();
    }
    else
    {
        // Minden szoba
        szurtSzobak = szobakLista.OrderBy(x => x.Szobaszam).ToList();
    }

    // 2. Szöveges keresés (Szobaszám vagy Lakók neve alapján)
    if (!string.IsNullOrEmpty(keresoSzoveg))
    {
        var kereso = keresoSzoveg.ToLower();
        szurtSzobak = szurtSzobak.Where(x =>
            x.Szobaszam.ToLower().Contains(kereso) ||
            GetLakoNevek(x.ID).ToLower().Contains(kereso)
        ).ToList();
    }

    LapozasFrissitese();
}
```

A `FrissitSzurtListat()` metódus a komponens legfontosabb üzleti logikája, amely két lépcsőben szűri a memóriában lévő listákat (LINQ használatával):

1. **Kapacitás szerinti szűrés:** Ha a "Szabad ágyak" gomb aktív, a rendszer a `.Where` feltétellel csak azokat a szobákat engedi tovább, ahol az aktív lakók száma kisebb, mint a szoba maximális férőhelye (`GetAktivLakokSzam(x.ID) < x.FerohelyMax`).
2. **Szöveges relációs keresés:** Ha a keresőmező nem üres, az algoritmus a szobaszám mellett a `GetLakoNevek` segédmetódust is meghívja. Ez utóbbi kikeresi a szobához tartozó diákokat, összefűzi a nevüket, és abban is elvégzi a szövegkeresést (`.Contains`).

2.25.6 Matematikai Lapozás

```
@* --- LAPOZÓ SÁV --- *@
@if (összesOldal > 1)
{
    <div class="d-flex justify-content-center align-items-center gap-3 mt-4 mb-5">
        <button class="btn btn-outline-secondary rounded-pill px-4 fw-bold shadow-sm" @onclick="ElozoOldal" disabled="@(!jelenlegiOldal == 1)">
            <i class="bi bi-chevron-left me-1"></i> Előző
        </button>

        <span class="text-muted fw-bolder bg-white border px-4 py-2 rounded-pill shadow-sm">@jelenlegiOldal / @összesOldal</span>

        <button class="btn btn-outline-secondary rounded-pill px-4 fw-bold shadow-sm" @onclick="KovetkezoOldal" disabled="@(!jelenlegiOldal == összesOldal)">
            Következő <i class="bi bi-chevron-right ms-1"></i>
        </button>
    </div>
}
```

```
// --- LAPOZÁS LOGIKÁJA ---
private void LapozasFrissitese()
{
    összesOldal = (int)Math.Ceiling((double)szurtSzobak.Count / elemekOldalankent);
    if (összesOldal == 0) összesOldal = 1;

    if (jelenlegiOldal > összesOldal) jelenlegiOldal = összesOldal;

    megjelenítettSzobak = szurtSzobak
        .Skip((jelenlegiOldal - 1) * elemekOldalankent)
        .Take(elemekOldalankent)
        .ToList();
}

private void ElozoOldal()
{
    if (jelenlegiOldal > 1)
    {
        jelenlegiOldal--;
        LapozasFrissitese();
    }
}

private void KovetkezoOldal()
{
    if (jelenlegiOldal < összesOldal)
    {
        jelenlegiOldal++;
        LapozasFrissitese();
    }
}

private void OldalMeretValtozas(ChangeEventArgs e)
{
    if (int.TryParse(e.Value?.ToString(), out int ujMeret))
    {
        elemekOldalankent = ujMeret;
        jelenlegiOldal = 1;
        LapozasFrissitese();
    }
}

// --- LAPOZÁS VÉGE ---
```

Mivel egy nagyobb intézményben több száz szoba is lehet, a kód itt is implementálja a szabványos, kliensoldali matematikai lapozómotort. Az összesOldal kiszámítása után a .Skip() és .Take() metódusok vágják ki a memóriából az aktuális oldalméretnek (pl. 10 sor) megfelelő szobákat, drasztikusan javítva a böngésző renderelési teljesítményét.

2.25.7 Intelligens Segédmetódusok

```
// Kiszámolja, hányan laknak az adott szobában
private int GetAktivLakokSzam(int roomId)
{
    return aktivBeosztasok.Count(x => x.RoomID == roomId);
}

// Összefűzi az adott szobában lakók neveit vesszővel elválasztva
private string GetLakoNevek(int roomId)
{
    // Kikeressük a szobában lakó user ID-kat
    var lakoIdk = aktivBeosztasok.Where(x => x.RoomID == roomId).Select(x => x.UserID).ToList();

    if (!lakoIdk.Any()) return "-";

    // Megkeressük a neveket az ID-k alapján
    var nevek = felhasznalokLista
        .Where(x => lakoIdk.Contains(x.ID))
        .Select(x => x.Nev);

    return string.Join(", ", nevek);
}

// Visszaadja a megfelelő CSS osztályt a telítettség alapján (empty, partial, full)
private string GetStatuszCss(int lakokSzama, int ferohelyMax)
{
    if (lakokSzama == 0)
    {
        return "empty";
    }
    else if (lakokSzama >= ferohelyMax)
    {
        return "full";
    }
    else
    {
        return "partial";
    }
}
```

Ezek a metódusok kötik össze a különböző adatbázistáblák adatait a C# memóriájában, kiváltva a lassú és drága SQL JOIN műveleteket:

- **GetAktivLakokSzam:** Visszaadja egy adott szoba aktuális lakóinak számát a memóriában lévő beosztások listájából.
- **GetLakoNevek:** Egy rendkívül elegáns relációs lekérdezés. Először megkeresi a szobában lakó diákok UserID-jait az aktív beosztások között. Ezután átlép a felhasznalokLista memóriatárolóba, kikeresi az egyező azonosítójú profilokat, kivonja belőlük a neveket, majd a string.Join(", ", nevek) függvényvel egyetlen, vesszővel elválasztott olvasható szöveggé fűzi őket össze a táblázat számára.
- **GetStatuszCss:** A szoba telítettsége alapján három lehetséges CSS osztálynevet ad vissza ("empty" = üres, "full" = tele, "partial" = részben teli), automatizálva a felület vizuális formázását.

2.26 Karbantartas.razor

A **Karbantartas.razor** komponens a kollégiumi infrastruktúra zavartalan működését biztosítja. Ez a felület listázza a hallgatók által leadott hibajelentéseket (pl. kiégett

villanykörte, csöpögő csap), és lehetőséget ad az üzemeltetőknek a feladatok delegálására és az állapotok (Státuszok) módosítására.

2.26.1 Függőségek és Alapkonfiguráció

```
@page "/karbantartas"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@layout AdminLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject KarbantartasiKeresekService KarbantartasiKeresek
@inject KarbantartasStatuszokService KarbantartasStatuszok
@inject FelhasznalokService Felhasznalok
@inject SzobaBeosztasokService SzobaBeosztasok
@inject SzobakService Szobak
@inject IJSRuntime JS
@inject NavigationManager Nav
```

- A felület az AdminLayout keretrendszerbe ágyazódik, garantálva az egységes navigációt.
- A rendszer egyidejűleg öt különböző API szolgáltatást (KarbantartasiKeresek, Statuszok, Felhasznalok, SzobaBeosztasok, Szobak) injektál. Erre azért van szükség, mert egy hibajegy önmagában csak számokat (ID-kat) tartalmaz, és a kliensoldal feladata összerakni ebből a teljes képet (Ki jelentette be? Melyik szobából?).

2.26.2 Kétpaneles Adatmegjelenítés

```

@* AKTIV FELADATOK (Nem lezártak) *@
<section class="maintenance-section">
  <div class="section-header">
    <h2>Aktuális Feladatok</h2>
    <p>Itt látod a folyamatban lévő vagy új hibajegyeket.</p>
  </div>
  <div class="card">
    <div class="table-scroll-container">
      <table class="data-table">
        <thead>
          <tr>
            <th style="width: 80px; min-width: 80px;">Azonosító</th>
            <th style="min-width: 160px;">Bejelentő / Helyszín</th>
            <th style="min-width: 250px;">Leírás</th>
            <th class="text-center" style="width: 120px; min-width: 120px;">Állapot</th>
            <th class="text-center" style="width: 80px; min-width: 80px;">Művelet</th>
          </tr>
        </thead>
        <tbody>
          @if (aktivHibak.Any())
          {
            @foreach (var hiba in aktivHibak)
            {
              <tr>
                <td>@hiba.ID</td>
                <td>
                  <strong>@GetBejelentoNev(hiba.BejelentoID)</strong><br />
                  <small class="text-muted">@GetHelyszin(hiba.BejelentoID)</small>
                </td>
                <td style="min-width: 250px; max-width: 350px;">
                  <div class="task-desc">
                    <span class="desc-text text-break d-block" style="white-space: normal; overflow-wrap: break-word;">@hiba.Leiras</span>
                    <span class="desc-date d-block text-muted mt-1">@hiba.LetrehozasDatum.ToShortDateString()</span>
                  </div>
                </td>
                <td class="text-center">
                  <span class="priority-badge @GetStatuszCss(hiba.StatuszID)">
                    @GetStatuszNev(hiba.StatuszID)
                  </span>
                </td>
                <td class="text-center">
                  <button class="action-btn" title="Művelet" @onclick="() => MegnyitSzerkesztes(hiba.ID)">
                    <i class="bi bi-pencil"></i>
                  </button>
                </td>
              </tr>
            }
          }
          else
          {
            <tr>
              <td colspan="5" class="text-center text-muted p-4">Jelenleg nincs aktiv hibajegy! Hurrá!</td>
            </tr>
          }
        </tbody>
      </table>
    </div>
  </div>
</section>

```

```
@* ELŐZMÉNYEK (Lezárt feladatok) *@
<section class="maintenance-section mt-5">
  <div class="section-header">
    <h2>Előzmények</h2>
    <p>Lezárt feladatok listája.</p>
  </div>

  @if (LezartHibak.Any())
  {
    <div class="card">
      <div class="table-scroll-container">
        <table class="data-table">
          <thead>
            <tr>
              <th style="width: 80px; min-width: 80px;">Azonosító</th>
              <th style="min-width: 160px;">Bejelentő</th>
              <th style="min-width: 250px;">Leírás</th>
              <th class="text-center" style="width: 120px; min-width: 120px;">Állapot</th>
              <th class="text-center" style="width: 80px; min-width: 80px;">Művelet</th>
            </tr>
          </thead>
          <tbody>
            @foreach (var hiba in lezartHibak)
            {
              <tr class="text-muted">
                <td>@hiba.ID</td>
                <td>
                  <strong>@GetBejelentoNev(hiba.BejelentoID)</strong><br />
                  <small class="text-muted">@GetHelyszin(hiba.BejelentoID)</small>
                </td>
                <td style="min-width: 250px; max-width: 350px;">
                  <div class="task-desc">
                    <span class="desc-text text-break d-block" style="white-space: normal; overflow-wrap: break-word;">@hiba.Leiras</span>
                    <span class="desc-date d-block text-muted mt-1">@hiba.LetrehozasiDatum.ToShortDateString()</span>
                  </div>
                </td>
                <td class="text-center">
                  <span class="priority-badge p-done">@GetStatuszNev(hiba.StatuszID)</span>
                </td>
                <td class="text-center">
                  <button class="action-btn" title="Kezelés" @onclick="() => MegnyitSzerkesztes(hiba.ID)">
                    <i class="bi bi-pencil"></i>
                  </button>
                </td>
              </tr>
            }
          </tbody>
        </table>
      </div>
    </div>
  }
  else
  {
    <div class="card empty-state-card text-center p-4 text-muted">
      Még nincs megjeleníthető előzmény.
    </div>
  }
</section>
```

A felhasználói élmény (UX) és az átláthatóság érdekében a táblázatok két külön szekcióra vannak bontva:

- **Aktív Feladatok:** Ez a blokk jeleníti meg a folyamatban lévő vagy még érintetlen hibákat. A táblázat HTML kódja (break-word, white-space: normal) fel van készítve a hosszú, többsoros hibaleírások esztétikus, tördeléses megjelenítésére is.
- **Előzmények (Lezárt feladatok):** Egy vizuálisan elkülönülő, visszafogottabb (text-muted) stílusú táblázat, amely archívumként szolgál a már megoldott problémákhoz.

2.26.3 Állapotkezelő Modális Ablak

```

@* MODAL ABLAK A SZERKESZTÉSHEZ *@
@if (isModalOpen)
{
    <div class="custom-modal-backdrop">
    <div class="custom-modal">
        <div class="modal-header">
            <h3>Hibajegy Kezelése (@szerkesztettHiba.ID)</h3>
            <button type="button" class="btn-close" @onclick="BezarModal"></button>
        </div>

        <div class="modal-body">
            @if (!string.IsNullOrEmpty(hibaUzenet))
            {
                <div class="alert alert-danger">@hibaUzenet</div>
            }

            <div class="form-group mb-3">
                <label class="text-muted small">Bejelentő / Helyszín</label>
                <div class="fw-bold">@GetBejelentoNeve(szerkesztettHiba.BejelentoID) (@GetHelysin(szerkesztettHiba.BejelentoID))</div>
            </div>

            <div class="form-group mb-3">
                <label class="text-muted small">Hiba leírása</label>
                <div class="p-2 bg-light border rounded text-break" style="overflow-wrap: break-word;">@szerkesztettHiba.Leiras</div>
            </div>

            <div class="form-group mb-3">
                <label>Státusz megváltoztatása</label>
                <select class="form-select fw-bold" @bind="szerkesztettHiba.StatuszID">
                    @foreach (var stat in statuszok)
                    {
                        <option value="@stat.ID">@stat.Nev</option>
                    }
                </select>
            </div>

            <div class="modal-footer">
                <button class="btn btn-danger me-auto" @onclick="TorlesGomb">Törlés</button>
                <button class="btn btn-secondary" @onclick="BezarModal">Mégse</button>
                <button class="btn btn-success" @onclick="MentesGomb">Mentés</button>
            </div>
        </div>
    </div>
}

```

Amikor az adminisztrátor rákattint a "Kezelés" gombra, egy felugró ablak (Modal) jelenik meg.

- **Adatbiztonság a UI-on:** A modális ablakban a bejelentő neve, a helyszín és a hiba leírása szándékosan **csak olvasható** (Read-only) HTML <div>-ekben jelenik meg. Ez megakadályozza, hogy az adminisztrátor véletlenül vagy szándékosan átírja a diák eredeti panaszát.
- **Státuszváltó:** Az egyetlen szerkeszthető elem a szerkesztettHiba.StatuszID-hez kötött legördülő menü, amely a szerverről kapott lehetséges állapotokból (pl. "Folyamatban", "Kész") építkezik (@foreach).

2.26.4 Adatbetöltés és LINQ Alapú Szortírozás

```
@code {
    private string token = "";
    private bool betoltesAlatt = true;

    // Üzenetek és modal
    private bool isModalOpen = false;
    private string hibaUzenet = "";
    private bool sikeresMentes = false;
    private string sikerUzenet = "";

    // Itt tároljuk az éppen szerkesztett hibajegyet
    private KarbantartasiKeresek szerkesztettHiba = new();

    // Adattároló listák az API-ból
    private List<KarbantartasiKeresek> osszesHiba = new();
    private List<KarbantartasStatuszok> statuszok = new();
    private List<KoliPortal.Lib.MODEL.Felhasznalok> felhasznalok = new();
    private List<SzobaBeosztasok> beosztasok = new();
    private List<KoliPortal.Lib.MODEL.Szobak> szobak = new();

    // Szűrt listák a megjelenítéshez
    private List<KarbantartasiKeresek> aktivHibak = new();
    private List<KarbantartasiKeresek> lezartHibak = new();

    // A Lezárt státusz ID-je
    private readonly int LEZART_STATUSZ_ID = 3;
}
```

```
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        // Kérjük el a tokenet a böngészőtől
        token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

        if (string.IsNullOrEmpty(token))
        {
            Nav.NavigateTo("/");
            return;
        }

        // SZIGORÚ SZEREPKÖR ELLENŐRZÉS
        var handler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
        var jwtToken = handler.ReadJwtToken(token);
        var szerepkor = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

        // Ha NEM Admin, akkor kidobjuk!
        if (szerepkor != "1" && szerepkor?.ToLower() != "admin")
        {
            Nav.NavigateTo("/");
            return;
        }

        await AdatokBetoltese();
        StateHasChanged();
    }
}
```

```
private async Task AdatokBetoltese()
{
    try
    {
        betoltesAlatt = true;
        osszesHiba = await KarbantartasiKeresek.GetKarbantartasiKeresek(token) ?? new();
        statuszok = await KarbantartasStatuszok.GetKarbantartasStatuszok(token) ?? new();
        felhasznalok = await Felhasznalok.GetFelhasznalok(token) ?? new();
        beosztasok = await SzobaBeosztasok.GetSzobaBeosztasok(token) ?? new();
        szobak = await Szobak.GetSzobak(token) ?? new();

        // Szétválogatjuk a hibákat aktívra és lezártra (rendezve dátum szerint)
        aktivHibak = osszesHiba
            .Where(x => x.StatuszID != LEZART_STATUSZ_ID)
            .OrderByDescending(x => x.LetrehozasDatum)
            .ToList();

        lezartHibak = osszesHiba
            .Where(x => x.StatuszID == LEZART_STATUSZ_ID)
            .OrderByDescending(x => x.LetrehozasDatum)
            .ToList();
    }
    catch (Exception ex)
    {
        Console.Error.WriteLine($"Hiba az adatok lekérésekor: {ex.Message}");
    }
    finally
    {
        betoltesAlatt = false;
    }
}

private void MegnyitSzerkesztes(int hibaId)
{
    var hiba = osszesHiba.FirstOrDefault(x => x.ID == hibaId);
    if (hiba != null)
    {
        szerkesztettHiba = new KarbantartasiKeresek
        {
            ID = hiba.ID,
            BejelentoID = hiba.BejelentoID,
            StatuszID = hiba.StatuszID,
            Leiras = hiba.Leiras,
            LetrehozasDatum = hiba.LetrehozasDatum
        };

        hibaUzenet = "";
        isModalOpen = true;
    }
}

private void BezarModal()
{
    isModalOpen = false;
}
```

- **Biztonsági kapu:** Az OnAfterRenderAsync itt is garantálja, hogy kizárólag adminisztrátor férhessen a rendszerhez.
- **Adatok szétválasztása (Split Data Pattern):** A AdatokBetoltese metódus lehúzza a szerverről az összes hibajegyet egyetlen nagy listába (osszesHiba). Ezután C# LINQ lekérdezések segítségével a memóriában választja ketté a listát az előre definiált LEZART_STATUSZ_ID konstans (3-as érték) alapján.
- Mindkét generált listát (aktivHibak, lezartHibak) csökkenő sorrendbe rendezi a létrehozás dátuma szerint (OrderByDescending), így a legfrissebb feladatok mindig a táblázat legtetejére kerülnek.

2.26.5 Aszinkron Mentés, Törlés és UX Finomhangolások

```
private async Task MentisGomb()
{
    try
    {
        await KarbantartasiKeresek.UpdateKarbantartasiKeresek(szerkesztettHiba, token);
        isModalOpen = false;

        await AdatokBetoltese();

        sikerUzenet = "Hibajegy sikeresen frissítve!";
        sikeresMentes = true;

        // 3 másodperc múlva eltüntetjük a zöld üzenetet
        _ = Task.Run(async () =>
        {
            await Task.Delay(3000);
            sikeresMentes = false;
            await InvokeAsync(StateHasChanged);
        });
    }
    catch (Exception ex)
    {
        hibaUzenet = $"Hiba a mentés során: {ex.Message}";
    }
}

private async Task TorlesGomb()
{
    bool isConfirmed = await JS.InvokeAsync<bool>("confirm", $"Biztosan véglegesen törölni szeretnéd a(z) #{szerkesztettHiba.ID} azonosítójú hibajegyet?");

    if (isConfirmed)
    {
        try
        {
            await KarbantartasiKeresek.DeleteKarbantartasiKeresek(szerkesztettHiba.ID, token);

            isModalOpen = false;
            await AdatokBetoltese();

            sikerUzenet = "Hibajegy véglegesen törölve!";
            sikeresMentes = true;

            _ = Task.Run(async () =>
            {
                await Task.Delay(3000);
                sikeresMentes = false;
                await InvokeAsync(StateHasChanged);
            });
        }
        catch (Exception ex)
        {
            hibaUzenet = $"Hiba a törlés során: {ex.Message}";
        }
    }
}
```

A CRUD (Create, Read, Update, Delete) műveletek implementációja:

- **Állapotfrissítés (MentisGomb):** A kiválasztott új státuszt az API-n keresztül menti a rendszer, majd egy `Task.Delay(3000)` hívással egy modern UX trükköt alkalmaz: a sikeres mentést jelző zöld üzenet 3 másodperc után magától eltűnik a képernyőről, így nem zavarja tovább a felhasználót.
- **Biztonságos Törlés (TorlesGomb):** JS Interop (confirm) használatával kér megerősítést a végleges törlés előtt, majd hívja a `DeleteKarbantartasiKeresek` API végpontot.

2.26.6 Dinamikus Segédmetódusok

```
private string GetBejelentoNev(int userId)
{
    var user = felhasznalok.FirstOrDefault(x => x.ID == userId);
    return user != null ? user.Nev : "Ismeretlen felhasználó";
}

// Kinyomozza, hogy a bejelentő diák melyik szobában lakik
private string GetHelyszin(int userId)
{
    var aktivBeosztas = beosztasok.FirstOrDefault(x => x.UserID == userId && x.TenylegesKikoltozes == null);
    if (aktivBeosztas != null)
    {
        var szoba = szobak.FirstOrDefault(x => x.ID == aktivBeosztas.RoomID);
        return szoba != null ? $"{szoba.Szobaszam} szoba" : "Ismeretlen szoba";
    }
    return "Egyéb"; // Ha nem egy diák saját szobájából jött a hiba
}

private string GetStatuszNev(int statuszId)
{
    var statusz = statuszok.FirstOrDefault(x => x.ID == statuszId);
    return statusz != null ? statusz.Nev : "Ismeretlen";
}

// Színezi a jelvényt a státusz alapján
private string GetStatuszCss(int statuszId)
{
    if (statuszId == 1 || statuszId == 2)
    {
        return "p-active"; // Új, még nem kezdődött el vagy folyamatban van
    }
    else
    {
        return "p-done"; // Lezárt
    }
}
```

- **GetHelyszin:** Egy rendkívül intelligens metódus. Mivel a hibajegyek nincsenek közvetlenül szobához rendelve (hiszen a diák bárholnan jelezhet hibát), a rendszer a bejelentő azonosítója alapján megkeresi a diák **aktuális szobabeosztását**, és onnan olvassa ki a szobaszámot. Ha a diák nem rendelkezik szobával, az "Egyéb" feliratot adja vissza.
- **GetStatuszCss:** A státusz azonosítója alapján dinamikusan adja vissza a CSS osztályokat (p-active vagy p-done), amelyek a jelvények (Badges) megfelelő színvilágáért felelnek.

2.27 Penzugyek.razor

A **Penzugyek.razor** komponens az intézmény gazdasági központja. Az adminisztrátorok ezen a felületen követhetik nyomon a diákok befizetéseit, a hátralékokat, illetve itt generálhatnak új fizetési kötelezettségeket. A modul különlegessége a tömeges adatkezelési (Mass Action) képessége, amellyel jelentős adminisztrációs terhet vesz le az üzemeltetők válláról.

2.27.1 Komponens Direktívák és Szolgáltatások

```
@page "/penzugyek"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@using System.IdentityModel.Tokens.Jwt
@layout AdminLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject PenzugyekService Penzugy
@inject FelhasznalokService Felhasznalo
@inject FizetesTipusokService FizetesTipusok
@inject SzobaBeosztasokService SzobaBeosztasok
@inject IJSRuntime JS
@inject NavigationManager Nav
```

- A felület az AdminLayout elrendezést használja, serveroldali interaktivitással (InteractiveServer), előrenderelés nélkül.
- Mivel a pénzügyi tranzakciók szorosan kapcsolódnak a lakókhoz és a szobákhoz, a kód injektálja a PenzugyekService mellett a Felhasznalok, FizetesTipusok és SzobaBeosztasok hálózati szolgáltatásokat is.

2.27.2 Valós Idejű Statisztikák és Szűrőfelület

```
@if (!string.IsNullOrEmpty(hibaUzenet))
{
    <div class="alert alert-danger mb-4">
        <i class="bi bi-exclamation-triangle-fill"></i> @hibaUzenet
    </div>
}
@if (sikeresMentes)
{
    <div class="alert alert-success mb-4">
        <i class="bi bi-check-circle-fill"></i> <strong>Siker:</strong> @sikerUzenet
    </div>
}

@* STATISZTIKA KÁRTYA *@
<div class="card shadow-sm p-4 mb-4 d-flex flex-column flex-md-row justify-content-between align-items-start align-items-md-center gap-3">
    <div>
        <h6 class="stat-label">Összes függőben lévő (Tartozás)</h6>
        <h3 class="stat-value danger-text m-0">@összesTartozas.ToString("N0") Ft</h3>
    </div>
    <div class="text-md-end">
        <h6 class="stat-label">Befizetett összegek</h6>
        <h3 class="stat-value success-text m-0">@összesBefizetett.ToString("N0") Ft</h3>
    </div>
</div>

@* SZÜRŐ GOMBOK ÉS ÚJ DÍJ GOMB *@
<div class="d-flex flex-column flex-md-row justify-content-between align-items-stretch align-items-md-center mb-4 gap-3">
    <div class="d-flex flex-wrap gap-2">
        <button class="filter-btn @(szuresiMod == 0 ? "active" : "")" @onclick="() => Szures(0)">Összes</button>
        <button class="filter-btn @(szuresiMod == 2 ? "active" : "")" @onclick="() => Szures(2)">Tartozások</button>
        <button class="filter-btn @(szuresiMod == 1 ? "active" : "")" @onclick="() => Szures(1)">Befizetve</button>
    </div>
    <div class="d-flex flex-column flex-md-row align-items-stretch align-items-md-center gap-3">
        <button class="btn-submit w-100" @onclick="MegnyitKifirasModal">
            <i class="bi bi-plus-circle me-1"></i> Új díj kifírása
        </button>
        <div class="d-flex justify-content-center align-items-center border-start-md ps-md-3">
            <span class="page-label me-2">Sor/oldal:</span>
            <select class="form-select form-select-sm" style="width: 80px; border-radius: 8px;" value="@elemekOldalankent" @onchange="OldalMeretValtozas">
                <option value="5">5</option>
                <option value="10">10</option>
                <option value="25">25</option>
                <option value="50">50</option>
            </select>
        </div>
    </div>
</div>
```

A felhasználói élményt (UX) a képernyő tetején található statisztikai kártyák és a gyorszűrők határozzák meg:

- **KPI Kártyák:** A rendszer azonnal, nagy betűkkel mutatja az "Összes függőben lévő tartozást" (piros színnel) és a "Befizetett összegeket" (zöld színnel). A kód a

.ToString("N0") formázást használja, amely ezres tagolással, tizedesjegyek nélkül jeleníti meg a forintösszegeket, jelentősen javítva az olvashatóságot.

- **Gyorsszűrők és Kiírás Gomb:** A három állapotkapcsoló ("Összes", "Tartozások", "Befizetve") mellett egy kiemelt, teljes szélességű gomb (btn-submit) kapott helyet, amely a tömeges díjkiíró modális ablakot nyitja meg.

2.27.3 Dinamikus Adattábla és Formázás

```
@* TÁBLÁZAT *@
<div class="card shadow-sm mb-3">
  <div class="table-scroll-container">
    <table class="data-table">
      <thead>
        <tr>
          <th>Diák neve</th>
          <th>Jogcím</th>
          <th>Összeg</th>
          <th>Dátum</th>
          <th class="text-center">Státusz</th>
        </tr>
      </thead>
      <tbody>
        @if (megjelenítettPenzugyek != null && megjelenítettPenzugyek.Any())
        {
          @foreach (var tetel in megjelenítettPenzugyek)
          {
            <tr>
              <td class="fw-bold text-dark">@GetFelhasznaloNev(tetel.UserID)</td>
              <td>@GetTipusNev(tetel.TipusID)</td>
              <td class="fw-bold">@tetel.Osszeg.ToString("N0") Ft</td>
              <td>
                @if (tetel.BefizetesDatum.HasValue)
                {
                  <span class="date-sublabel">Fizetve</span>
                  <strong>@tetel.BefizetesDatum.Value.ToShortDateString()</strong>
                }
                else
                {
                  <span class="date-sublabel">Határidő</span>
                  <strong class="@((tetel.Esedekesseg < DateTime.Now ? "text-danger" : "")">
                    @tetel.Esedekesseg.ToShortDateString()
                  </strong>
                }
              </td>
              <td class="text-center">
                @if (tetel.BefizetesDatum.HasValue || (tetel.Statusz ?? "").ToLower() == "fizetve")
                {
                  <span class="status-badge fizetve">Fizetve</span>
                }
                else
                {
                  <span class="status-badge tartozas">Tartozás</span>
                }
              </td>
            </tr>
          }
        }
        else
        {
          <tr>
            <td colspan="5" class="text-center text-muted p-5">Nincs a szűrésnek megfelelő adat.</td>
          </tr>
        }
      </tbody>
    </table>
  </div>
</div>
```

```
@* LAPOZÓ SÁV *@
@if (összesOldal > 1)
{
  <div class="pagination-bar">
    <button class="filter-btn" onclick="ElozoOldal" disabled="@((jelenlegiOldal == 1))">
      <i class="bi bi-chevron-left me-1"></i> Előző
    </button>

    <span class="text-muted fw-bold">@jelenlegiOldal / @összesOldal</span>

    <button class="filter-btn" onclick="KovetkezoOldal" disabled="@((jelenlegiOldal == összesOldal))">
      Következő <i class="bi bi-chevron-right ms-1"></i>
    </button>
  </div>
}
}
```

A táblázat a már megszokott reszponzív, lapozható (Lapozó Sáv) elrendezésre épül, de a tartalom formázása rendkívül intelligens:

- **Határidő-figyelő logika:** A kód megvizsgálja a BefizetesDatum.HasValue tulajdonságot. Ha a tétel be van fizetve, a zöld "Fizetve" jelvényt és a fizetés dátumát mutatja.
- **Késedelem vizualizációja:** Ha a tétel nincs befizetve, a rendszer összehasonlítja a határidőt a mai nappal ($tétel.Esedekesseg < DateTime.Now$). Ha a határidő lejárt, a dátum automatikusan piros színre (text-danger) vált, így az adminisztrátor azonnal látja a problémás eseteket.

2.27.4 A Tömeges Díjkiíró Modális Ablak

```

@* FELUGRÓ ABLAK (MODAL) *@
@if (IsKiiirasModalOpen)
{
    <div class="custom-modal-backdrop">
        <div class="custom-modal">
            <div class="modal-header">
                <h3>Tömeges díj kiírása</h3>
                <button type="button" class="action-btn" @onclick="BezárKiiirasModal"><i class="bi bi-x-lg"></i></button>
            </div>
            <div class="modal-body">
                <div class="form-group mb-3">
                    <label class="fw-bold mb-1">Jogcím (Fizetés típusa)</label>
                    <select class="form-select" @bind="kivalasztottTípusId">
                        @if (fizetesTípusokLista != null && fizetesTípusokLista.Any())
                        {
                            @foreach (var típus in fizetesTípusokLista)
                            {
                                <option value="@típus.ID">@típus.Megnevezes</option>
                            }
                        }
                        else
                        {
                            <option value="1">Kollégiumi díj</option>
                        }
                    </select>
                </div>
                <div class="form-group mb-3">
                    <label class="fw-bold mb-1">Fizetendő összeg (Ft)</label>
                    <input type="number" class="form-control" @bind="haviDij" min="1000" />
                </div>
                <div class="form-group mb-4">
                    <label class="fw-bold mb-1">Befizetési határidő vége</label>
                    <input type="date" class="form-control" @bind="befizetesiHatarido" @bind:format="yyyy-MM-dd" />
                </div>
            </div>
            <div class="modal-footer">
                <button class="filter-btn" @onclick="BezárKiiirasModal" disabled="@mentesFolyamatban">Mégse</button>
                <button class="filter-btn active" @onclick="DijakKiirasaMindenkinek" disabled="@mentesFolyamatban">
                    @if (mentesFolyamatban)
                    {
                        <span class="spinner-border spinner-border-sm me-2" role="status" aria-hidden="true"></span>
                        Generálás...</span>
                    }
                    else
                    {
                        <span>Díjak kiírása aktív lakóknak</span>
                    }
                </button>
            </div>
        </div>
    </div>
}

```

Ez az űrlap teszi lehetővé a tömeges generálást.

- Tartalmaz egy legördülő menüt a fizetés típusának kiválasztásához (pl. "Kollégiumi díj").
- Kétirányú adatkötéssel (@bind) bekéri a fizetendő összeget (haviDij), valamint a fizetési határidőt, ami intelligens módon, alapértelmezetten a mai nap + 10 napra van beállítva a @code blokkban.
- Ha a mentés folyamatban van, a gomb inaktívvá válik (disabled), és egy töltésjelző (spinner) akadályozza meg a többszörös beküldést.

2.27.5 Adatbetöltés és Memória-statisztikák

```
@code {
    private bool betoltesAlatt = true;
    private string token = "";
    private string hibaUzenet = "";
    private string sikerUzenet = "";
    private bool sikeresMentes = false;

    private List<KoliPortal.Lib.MODEL.Penzugyek> osszesPenzugy = new();
    private List<KoliPortal.Lib.MODEL.Penzugyek> szurtPenzugyek = new();
    private List<KoliPortal.Lib.MODEL.Penzugyek> megjelenítettPenzugyek = new();

    private List<KoliPortal.Lib.MODEL.Felhasznalok> felhasznalokLista = new();
    private List<FizetesTipusok> fizetesTipusokLista = new();

    private decimal osszesTartozas = 0;
    private decimal osszesBefizetett = 0;
    private int szuresiMod = 0;

    private int jelenlegiOldal = 1;
    private int elemekOldalankent = 10;
    private int osszesOldal = 1;

    private bool isKirasModalOpen = false;
    private bool mentesFolyamatban = false;
    private int kivlasztottTipusId = 1;
    private decimal haviDij = 15000;
    private DateTime befizetesHatarido = DateTime.Now.AddDays(10);
}
```

```
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        try
        {
            token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

            if (string.IsNullOrEmpty(token))
            {
                Nav.NavigateTo("/");
                return;
            }

            var handler = new JetSecurityTokenHandler();
            var jwtToken = handler.ReadJwtToken(token);
            var szerepkor = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

            if (szerepkor != "1" && szerepkor?.ToLower() != "admin")
            {
                Nav.NavigateTo("/");
                return;
            }

            await AdatokBetoltese();
        }
        catch (Exception ex)
        {
            hibaUzenet = $"Rendszerhiba: {ex.Message}";
        }
        finally
        {
            betoltesAlatt = false;
            StateHasChanged();
        }
    }
}
```



```
private async Task AdatokBetoltese()
{
    try
    {
        osszesPenzugy = await Penzugy.GetPenzugyek(token) ?? new();
        felhasznalokLista = await Felhasznalo.GetFelhasznalok(token) ?? new();
        fizetesTipusokLista = await FizetesTipusok.GetFizetesTipusok(token) ?? new();

        osszesTartozas = osszesPenzugy.Where(p => !p.BefizetesDatum.HasValue && (p.Statusz ?? "").ToLower() != "fizetve").Sum(p => p.Osszeg);
        osszesBefizetett = osszesPenzugy.Where(p => p.BefizetesDatum.HasValue || (p.Statusz ?? "").ToLower() == "fizetve").Sum(p => p.Osszeg);

        Szures(szuresiMod);
    }
    catch (Exception ex)
    {
        hibaUzenet = $"Hiba az adatok letöltésekor: {ex.Message}";
    }
}

private void MegnyitKiirasModal()
{
    isKiirasModalOpen = true;
    hibaUzenet = "";
    haviDij = 15000;
    befizetesMatarido = DateTime.Now.AddDays(10);
}

private void BezarKiirasModal()
{
    isKiirasModalOpen = false;
}
```

- **Szigorú Biztonság:** A szokásos Token és Role (Szerepkör = Admin) validáció itt is az OnAfterRenderAsync metódusban történik.
- **Gyorsított LINQ Matematika:** Az AdatokBetoltese() metódus a letöltés után azonnal kiszámolja a KPI kártyák értékeit. A .Where(p => !p.BefizetesDatum.HasValue).Sum(p => p.Osszeg) LINQ lekérdezés másodpercek töredéke alatt összegzi az összes kifizetetlen számlát a memóriában, elkerülve a lassú adatbázis-szintű aggregációkat.

2.27.6 Állapotfüggő Rendezés

```
private void Szures(int mod)
{
    szuresiMod = mod;
    jelenlegiOldal = 1;

    if (mod == 1)
    {
        szurtPenzugyek = osszesPenzugy
            .Where(p => p.BefizetesDatum.HasValue || (p.Statusz ?? "").ToLower() == "fizetve")
            .OrderByDescending(p => p.BefizetesDatum)
            .ToList();
    }
    else if (mod == 2)
    {
        szurtPenzugyek = osszesPenzugy
            .Where(p => !p.BefizetesDatum.HasValue && (p.Statusz ?? "").ToLower() != "fizetve")
            .OrderBy(p => p.Esedekesseg)
            .ToList();
    }
    else
    {
        szurtPenzugyek = osszesPenzugy
            .OrderBy(p => p.BefizetesDatum.HasValue ? 1 : 0)
            .ThenByDescending(p => p.Esedekesseg)
            .ToList();
    }

    LapozasFrissitese();
}

private void LapozasFrissitese()
{
    osszesOldal = (int)Math.Ceiling((double)szurtPenzugyek.Count / elemekOldalankent);
    if (osszesOldal == 0) osszesOldal = 1;

    if (jelenlegiOldal > osszesOldal) jelenlegiOldal = osszesOldal;

    megjelenitettPenzugyek = szurtPenzugyek
        .Skip((jelenlegiOldal - 1) * elemekOldalankent)
        .Take(elemekOldalankent)
        .ToList();
}

private void ElozoOldal()
{
    if (jelenlegiOldal > 1)
    {
        jelenlegiOldal--;
        LapozasFrissitese();
    }
}

private void KovetkezoOldal()
{
    if (jelenlegiOldal < osszesOldal)
    {
        jelenlegiOldal++;
        LapozasFrissitese();
    }
}

private void OldalMeretValtozas(EventArgs e)
{
    if (int.TryParse(e.Value?.ToString(), out int ujMeret))
    {
        elemekOldalankent = ujMeret;
        jelenlegiOldal = 1;
        LapozasFrissitese();
    }
}
```

A Szures metódus egy professzionális táblázatrendezési logikát valósít meg:

- **Ha a "Befizetve" nézet aktív:** A listát a befizetés dátuma szerint csökkenő sorrendbe rendezi (OrderByDescending), így a legfrissebb tranzakciók vannak legfelül.
- **Ha a "Tartozás" nézet aktív:** A kód egy dupla rendezést alkalmaz (.OrderBy().ThenByDescending()). Ennek célja, hogy a lejárt, sürgős határidejű tartozások kerüljenek a lista legtetetejére.

2.27.7 A Tömeges Díjkiírás Algoritmus

```
private async Task DijakKiirasaMindenkinek()
{
    hibaUzenet = "";
    sikeresMentes = false;

    // 1000 nál ne lehessen kevesebbet megadni
    if (haviDij < 1000)
    {
        hibaUzenet = "A fizetendő összeg nem kevesebb mint 1000 Ft!";
        return; // Kilépünk a mentésből
    }

    mentesFolyamatban = true;

    try
    {
        var beosztasok = await SzobaBeosztasok.GetSzobaBeosztasok(token) ?? new();

        var aktivLakoIdk = beosztasok
            .Where(b => b.TenylegesKikoltozes == null)
            .Select(b => b.UserID)
            .Distinct()
            .ToList();

        if (!aktivLakoIdk.Any())
        {
            hibaUzenet = "Nincsenek aktív lakók a rendszerben, nincs kinek kiírni a díjat!";
            mentesFolyamatban = false;
            BezarKiirasModal();
            return;
        }

        int sikeresenLetrehozva = 0;

        foreach (var userId in aktivLakoIdk)
        {
            var ujPenzugy = new KoliPortal.Lib.MODEL.Penzugyek
            {
                UserID = userId,
                TipusID = kivlasztottTipusID,
                Osszeg = haviDij,
                Esedekesseg = befizetesiHatarido,
                BefizetesDatum = null,
                Statusz = "ESEDEKES"
            };

            await Penzugy.CreatePenzugyek(ujPenzugy, token);
            sikeresenLetrehozva++;
        }

        string tipusNev = fizetesTipusokLista?.FirstOrDefault(t => t.ID == kivlasztottTipusID)?.Megnevezes ?? "Egyéb díj";

        sikerUzenet = $"A {tipusNev} {haviDij.ToString("N0")} Ft sikeresen kiírásra került {sikeresenLetrehozva} lakó számára!";
        sikeresMentes = true;

        await AdatokBetoltese();
        BezarKiirasModal();

        _ = Task.Run(async () =>
        {
            await Task.Delay(3000);
            sikeresMentes = false;
            await InvokeAsync(StateHasChanged);
        });
    }
    catch (Exception ex)
    {
        hibaUzenet = $"Hiba a díjak generálásakor: {ex.Message}";
        BezarKiirasModal();
    }
    finally
    {
        mentesFolyamatban = false;
        StateHasChanged();
    }
}
```

A komponens fénypontja a `DijakKiirasaMindenkinek()` metódus. Ez egy rendkívül komplex, mégis optimalizált algoritmus:

1. **Biztonsági Fék:** Ellenőrzi, hogy a megadott díj nagyobb-e 1000 Ft-nál, elkerülve a véletlen elgépeléseket (pl. 0 Ft-os kiírások).
2. **Célközönség meghatározása:** A `.Where(b => b.TenylegesKikoltozes == null).Select(b => b.UserID).Distinct()` LINQ láncolat egy zseniális szűrő. Kiveszi a beosztások közül azokat a diákokat, akik **jelenleg is bent laknak** a

kollégiumban, majd a `Distinct()` metódussal biztosítja, hogy minden hallgató csak egyszer szerepeljen a listában.

3. **Tranzakció Generálás:** A kód végigiterál a szűrt aktív lakókon, és minden egyes hallgatóhoz létrehoz egy új `Penzugyek` objektumot a megadott határidővel és összeggel, amit azonnal be is küld az API-nak.
4. **Visszajelzés:** A folyamat végén a zöld sikerüzenet pontosan kiírja, hogy milyen jogcímen, hány diáknak írt ki sikeresen díjat a rendszer.

2.28 Beallitas.razor

A `Beallitas.razor` komponens az adminisztrátorok személyes profiljának kezelésére szolgál. Jelenlegi fő funkciója a biztonságos jelszóváltoztatás. Mivel itt kiemelten szenzitív adatok cseréje történik, a kód minimalista felhasználói felülettel, de annál szigorúbb kliensoldali validációs és hálózati logikával rendelkezik.

2.28.1 Keretrendszer és Felhasználói Felület

```
@page "/beallitasok"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@using System.IdentityModel.Tokens.Jwt
@layout AdminLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject IJSRuntime JS
@inject NavigationManager Nav
```

- **Függőségek:** A komponens az `AdminLayout` része, és kikapcsolt előrendereléssel (`InteractiveServer`) fut, hogy hozzáférjen a böngésző helyi tárhelyéhez (`Local Storage`). Injektálja a hitelesítési (`AuthService`) és a felhasználókezelő (`UserService`) szolgáltatásokat.
- **Vizuális Visszajelzések:** A UX (User Experience) alapelveit követve a felület tetején dinamikus Bootstrap értesítések (`alert-success`, `alert-danger`) jelennek meg a `@sikerUzenet` és `@hibaUzenet` változók tartalmától függően, azonnali visszacsatolást adva a művelet kimeneteléről.
- **Űrlap és Adatkötés:** A form három jelszómezőt tartalmaz ("Régi jelszó", "Új jelszó", "Új jelszó újra"). A kétirányú adatkötés (`@bind`) segítségével a begépet karakterek azonnal a C# háttérváltozókba kerülnek. Gombnyomáskor a `@toltesAlatt` változó inaktívvá teszi a gombot, megelőzve a többszörös hálózati kéréseket.

2.28.2 Validációs Motor és API Kommunikáció

```

<div class="content-wrapper" style="max-width: 600px; margin: 0 auto;">
  @if (!string.IsNullOrEmpty(hibaUzenet))
  {
    <div class="alert alert-danger mb-4">
      <i class="bi bi-exclamation-triangle-fill"></i> @hibaUzenet
    </div>
  }

  <div class="settings-grid">
    <div class="card settings-card full-width">
      <div class="card-header">
        <h2><i class="bi bi-bell"></i> Értéstitések és Rendszer</h2>
      </div>
      <div class="notification-list">
        <div class="toggle-row">
          <div class="toggle-info">
            <span class="toggle-title">Új hibajegy értesítés</span>
            <span class="toggle-desc">Email küldése, ha egy lakó új karbantartási igényt ad le.</span>
          </div>
          <div class="toggle-switch">
            <input type="checkbox" @bind="ui_HibaErtesites">
            <span class="slider round"></span>
          </div>
        </div>
        <div class="toggle-row">
          <div class="toggle-info">
            <span class="toggle-title">Mésedelmes fizetés riasztás</span>
            <span class="toggle-desc">Automatikus figyelmeztetés a lejárt tartozásokról.</span>
          </div>
          <div class="toggle-switch">
            <input type="checkbox" @bind="ui_MesedelmesRiasztas">
            <span class="slider round"></span>
          </div>
        </div>
        <div class="toggle-row">
          <div class="toggle-info">
            <span class="toggle-title">Karbantartási mód</span>
            <span class="toggle-desc">A diákok nem tudnak bejelentkezni, amíg a rendszer frissül.</span>
          </div>
          <div class="toggle-switch">
            <input type="checkbox" @bind="ui_KarbantartasiMod">
            <span class="slider round"></span>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

A JelszoValtoztatas aszinkron metódus a komponens "motorja", amely többlépcsős védelmet alkalmaz még azelőtt, hogy a hálózathoz fordulna:

1. **Üresség-vizsgálat:** Ellenőrzi, hogy a felhasználó minden mezőt kitöltött-e (string.IsNullOrEmpty).
2. **Hossz-validáció:** Biztosítja, hogy az új jelszó megfeleljen a biztonsági minimumoknak (legalább 6 karakter hosszú legyen).
3. **Egyezés-vizsgálat:** Szintén hálózati kérés nélkül, azonnal kiszűri a tipográfiai hibákat azzal, hogy összehasonlítja az "Új jelszó" és az "Új jelszó újra" mezők tartalmát.
4. **Adattovábbítás (Hálózati réteg):** Ha minden lokális teszt sikeres, a rendszer a bekért adatokat, valamint a bejelentkezett felhasználó e-mail címét egy dedikált

DTO-ba (JelszoValtoztatasDto) csomagolja. Ezt a csomagot küldi el az AuthController.ChangePasswordAsync végpontnak.

5. **Állapot-visszaállítás:** Sikeres válasz esetén a rendszer kiírja a zöld sikerüzenetet, és biztonsági okokból azonnal kiüríti a beviteli mezőket (a változók "" értékre állításával). Bármilyen hálózati hiba esetén a catch blokk elkapja a kivételt, és piros hibaüzenet formájában tájékoztatja a felhasználót.

2.28.3 Biztonsági Kapu és Adatcsomagoló Osztály

```
@code {
    private bool betoltesAlatt = true;
    private string token = "";
    private string hibaUzenet = "";

    // Lokális UI változók a kapcsolódókhoz -> Fejlesztési lehetőségeknél van
    private bool ui_HibaErtesites = true;
    private bool ui_KeszedelmesRiasztas = true;
    private bool ui_KarbantartasiMod = false;

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            try
            {
                token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

                if (string.IsNullOrEmpty(token))
                {
                    Nav.NavigateTo("/", forceLoad: true);
                    return;
                }

                // Szerepkör ellenőrzése csak adminoknak szabad hozzáférniük ehhez az oldalhoz
                var handler = new JwtSecurityTokenHandler();
                var jwtToken = handler.ReadJwtToken(token);
                var szerepkor = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

                if (szerepkor != "1" && szerepkor?.ToLower() != "admin")
                {
                    Nav.NavigateTo("/", forceLoad: true);
                    return;
                }
            }
            catch (Exception ex)
            {
                hibaUzenet = $"Hiba az ellenőrzéskor: {ex.Message}";
            }
            finally
            {
                betoltesAlatt = false;
                StateHasChanged();
            }
        }
    }
}
```

A @code blokk alja tartalmazza az infrastruktúrális és biztonsági elemeket:

- **Útvonalvédelem (OnAfterRenderAsync):** Mint minden védett adminisztrációs oldalon, a kód itt is kiolvassa és dekódolja a JWT tokenet. Szigorúan ellenőrzi a szerepkört, és ha az illető nem "1"-es (Admin) jogosultságú, a NavigationManager segítségével visszadobja a bejelentkezési oldalra. A kód egyúttal a tokenből kinyeri a felhasználó e-mail címét is, amit később a jelszóváltoztatási kérés azonosításához használ fel.
- **Az Adatátviteli Objektum (JelszoValtoztatasDto):** Ez a kis segédosztály a "Clean Architecture" (Tiszta kód architektúra) ékes példája. Ahelyett, hogy a kliens a teljes Felhasznalo objektumot (nevekkel, ID-kkal, szerepkörökkel) átküldené a hálózaton egy egyszerű jelszócsere miatt, ez a DTO kizárólag a

feltétlenül szükséges adatokat (Email, RégiJelszó, ÚjJelszó) tartalmazza, minimalizálva a hálózati forgalmat és a biztonsági kockázatokat.

2.29 Lako/Attekintes.razor

A **Lako/Attekintes.razor** komponens a kollégiumi lakók elsődleges érkezési oldala a rendszerben. Míg az adminisztrátori műszerfal globális intézményi statisztikákat mutat, ez a felület szigorúan személyre szabott: a hallgató kizárólag a saját adataival (szobabeosztás, hátralékok, hibajegyek) találkozhat. A felület fókuszában a letisztultság és a legfontosabb teendők azonnali vizualizálása áll.

2.29.1 Keretrendszer és Moduláris Függőségek

```
@page "/LakoAttekintes"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@using System.Security.Claims
@using System.IdentityModel.Tokens.Jwt
@layout LakoLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject SzobakService Szobak
@inject KarbantartasiKeresekService KarbantartasiKeresek
@inject SzobaBeosztasokService SzobaBeosztasok
@inject PenzugyekService Penzugyek
@inject FizetesTipusokService FizetesTipusok
@inject IJSRuntime JS
@inject NavigationManager Nav
```

- **Elrendezés és Képernyőmód:** A `@layout LakoLayout` utasítás garantálja, hogy a hallgató a dedikált, biztonságosan elszeparált bal oldali menüsávot lássa. A `@rendermode InteractiveServerRenderMode(prerender: false)` a JWT token helyi memóriából történő biztonságos kiolvasásához szükséges.
- **Injektálások:** Mivel egy áttekintő oldalról van szó, a rendszer szinte az összes elérhető adatszolgáltatást injektálja (SzobakService, KarbantartasiKeresekService, PenzugyekService stb.), hogy a háttérben össze tudja fűzni a különböző modulok adatait.

2.29.2 Személyre Szabott UI és Dinamikus KPI Kártyák

```

<div class="stats-grid">
  <div class="stat-card">
    <div class="stat-icon purple">
      <i class="bi bi-door-open-fill icon-large"></i>
    </div>
    <div class="stat-details">
      <h3>@jelenlegiSzobaszam</h3>
      <p>Jelenlegi szobád</p>
    </div>
  </div>

  <div class="stat-card">
    <div class="stat-icon blue">
      <i class="bi bi-cash-stack icon-large"></i>
    </div>
    <div class="stat-details">
      <h3>@(FizetendoOsszeg > 0 ? FizetendoOsszeg.ToString("N0") + " Ft" : "Nincs tartozás")</h3>
      <p>Fizetendő összeg</p>
    </div>
  </div>

  <div class="stat-card">
    <div class="stat-icon orange">
      <i class="bi bi-tools icon-large"></i>
    </div>
    <div class="stat-details">
      <h3>@sajatNytottHibakSzama db</h3>
      <p>Folyamatban lévő hibajelentéseid</p>
    </div>
  </div>
</div>

<div class="content-wrapper mt-5">
  <div class="card recent-residents-card shadow-sm border-0">
    <div class="card-header bg-white border-bottom-0 pt-4 pb-0">
      <h2 class="fs-4 fw-bold">Legutóbbi Befizetéseid</h2>
    </div>
    <div class="card-body p-0">
      <div class="table-scroll-container">
        <table class="data-table">
          <thead class="bg-light">
            <tr>
              <th>Befizetés dátuma</th>
              <th>Jogcím</th>
              <th>Összeg</th>
              <th class="text-center">Státusz</th>
            </tr>
          </thead>
          <tbody>
            <@if (sajatBefizetesek != null && sajatBefizetesek.Any())>
              <@foreach (var tetel in sajatBefizetesek)>
                <tr>
                  <td>
                    <strong>@(tetel.BefizetesDatum?.ToShortDateString() ?? tetel.Esedekesseg.ToShortDateString())</strong>
                  </td>
                  <td style="font-weight: 500;">@GetTipusNev(tetel.TipusID)</td>
                  <td style="font-weight: 700;">@tetel.Osszeg.ToString("N0") Ft</td>
                  <td class="text-center">
                    <span class="badge badge-success">Fizetve</span>
                  </td>
                </tr>
              </foreach>
            </tbody>
            <@else>
              <tr>
                <td colspan="4" class="text-center text-muted p-5">Még nincsenek rögzített befizetéseid.</td>
              </tr>
            </else>
          </tbody>
        </table>
      </div>
    </div>
  </div>
</div>

```

A vizuális felület (UI) felső része három reszponzív statisztikai kártyából (Card) áll, amelyek azonnali információt adnak a diák státuszáról:

- **Szoba státusz (Lila ikon):** Kiírja a hallgató aktuális szobaszámát (@jelenlegiSzobaszam). Ha a diák még nem kapott beosztást, intelligensen a "Nincs" feliratot mutatja.

- **Pénzügyi riasztás (Feltételes UI - Kék ikon):** Egy kiváló UX megoldás! A kártya egy egyszerű inline feltétellel (@if (fizetendoOsszeg > 0)) dönti el a megjelenítést. Ha a diáknak tartozása van, pontosan kiírja a fizetendő összeget (.ToString("N0") formázással). Ha mindent befizetett, a megnyugtató "Nincs tartozás" szöveg jelenik meg.
- **Aktív hibajegyek (Narancssárga ikon):** Egy számláló, amely a diák által bejelentett, de még meg nem oldott műszaki problémák számát mutatja.

A statisztikai kártyák alatt egy letisztult Bootstrap adattábla (table-scroll-container) kapott helyet. Ez a táblázat a diák legutóbbi pénzügyi tranzakcióit (pl. havi díjak) listázza. Kiemelendő a feltételes dátum-megjelenítés: (tetel.BefizetesDatum?.ToShortDateString() ?? tetel.Esedekesseg.ToShortDateString()), amely befizetett tétel esetén a fizetés napját, függőben lévónél pedig a határidőt mutatja. A jogcímek szöveges feloldását a GetTipusNev() segédmetódus végzi el.

2.29.3 Szigorú Biztonság és Kliensoldali Identitás

```
protected override async Task OnAfterRenderAsync(bool elsőBetöltés)
{
    if (elsőBetöltés)
    {
        try
        {
            // Várjuk el a token-t a böngészőtől
            token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

            if (string.IsNullOrEmpty(token))
            {
                Nav.NavigateTo("/");
                return;
            }

            var handler = new JwtSecurityTokenHandler();
            var jwtToken = handler.ReadJwtToken(token);
            var szerepkör = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

            // Ha a szerepkör nem Lako (2), repül a loginra
            if (szerepkör != "2" && szerepkör?.ToLower() != "lako")
            {
                Nav.NavigateTo("/");
                return;
            }

            // Kinyerjük a saját azonosítónkat
            var idClaim = jwtToken.Claims.FirstOrDefault(x =>
                x.Type == ClaimTypes.NameIdentifier ||
                x.Type == "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier");

            if (idClaim != null && int.TryParse(idClaim.Value, out int parsedId))
            {
                sajátId = parsedId;
            }

            // Adatok lekérése az API-ból id alapján

            // Saját szoba megkeresése
            var beosztások = await SzobaBeosztások.GetSzobaBeosztások(token) ?? new();
            var aktívBeosztás = beosztások.FirstOrDefault(b => b.UserID == sajátId && b.TemlyegesIkoltozes == null);

            if (aktívBeosztás != null)
            {
                var összesSzoba = await Szobák.GetSzobák(token) ?? new();
                var szoba = összesSzoba.FirstOrDefault(s => s.ID == aktívBeosztás.RoomID);
                if (szoba != null)
                {
                    jelenlegiSzobaszám = szoba.Szobaszám;
                }
            }

            // Saját nyitott hibáin ahol én vagyok a bejelentő, és a státusz nem lezárt
            var hibák = await KarbantartásiKeresések.GetKarbantartásiKeresések(token) ?? new();
            sajátNyitottHibákSzama = hibák.Count(h => h.BejelentőID == sajátId && h.StatuszID != 3);

            // Saját pénzügyeim lekérése
            var összesPenzugy = await Penzugyek.GetPenzugyek(token) ?? new();

            // Összes tartozás kiszámítása a fenti kártyához ami még nincs fizetve
            fizetendoÖsszeg = összesPenzugy
                .Where(p => p.UserID == sajátId && !p.BefizetesDatum.HasValue && p.Statusz?.ToLower() != "fizetve")
                .Sum(p => p.Összeg);

            // Legújabb 5 tétel a táblázathoz csak ami be van fizetve vagy van befizetés dátuma, rendezve a dátus szerint csökkenő sorrendben
            sajátBefizetések = összesPenzugy
                .Where(p => p.UserID == sajátId && (p.BefizetesDatum.HasValue || p.Statusz?.ToLower() == "fizetve"))
                .OrderByDescending(p => p.BefizetesDatum ?? p.Esedekesseg)
                .Take(5)
                .ToList();

            // Típusok lekérése az API-ból
            fizetesTípusokLista = await FizetesTípusok.GetFizetesTípusok(token) ?? new();
        }
        catch (Exception ex)
        {
            hibaÜzenet = $"Hiba az adatok lekérésekor: {ex.Message}";
        }
        finally
        {
            betöltésAlatt = false;
            StateHasChanged();
        }
    }
}
```

Az `OnAfterRenderAsync` metódus tartalmazza a szoftver egyik legkritikusabb biztonsági mechanizmusát:

1. **Szerepkör-ellenőrzés:** Dekódolja a token-t, és ha a felhasználó nem "2"-es ID-jú vagy nem "lako" szerepkörű, azonnal visszairányítja a `/login` oldalra.
2. **Identitás Kinyerése (Security Claim Extraction):** A kód a `ClaimTypes.NameIdentifier` segítségével magából a kriptográfiailag hitelesített tokenből nyeri ki a bejelentkezett felhasználó azonosítóját (`sajátId`).

Szoftvertervezési szempontból ez létfontosságú: megakadályozza, hogy egy diák manipulálja az URL-t vagy a helyi változókat, hogy mások adataihoz férjen hozzá. A rendszer kizárólag a szerver által igazolt ID alapján hajlandó adatokat lekérdezni.

Az adatok betöltését villámgyors memóriaszűrések (LINQ) végzik a lekérdezett listákon:

- **Aktív szoba:** Megkeresi azt az egyetlen beosztást, ahol `b.UserID == sajátId` és a tényleges kiköltözés még null. Ha talál ilyet, átadja a szobaszámot a UI-nak.
- **Hibajegy számláló:** A `.Count()` módszerrel megszámolja azokat a karbantartási kéréseket, amelyeket ez a diák jelentett be (`h.BejelentőID == sajátId`) és státuszuk nem lezárt (`!= 3`).
- **Pénzügyi aggregáció:** A teljes pénzügyi listából kiszűri a diák saját tételeit. Az összes kifizetetlen tartozást a `.Where(...).Sum(p => p.Osszeg)` algoritmussal azonnal összeadja. Végül a táblázat számára egy csökkenő sorrendbe (`OrderByDescending`) rendezett listát készít a dátumok alapján, amelyből a `.Take(5)` módszerrel csak az 5 legfrissebb tételt jeleníti meg, elkerülve a felület túlzsúfolását.

2.29.4 Segédmetódusok

```
// Objektum lista alapján visszaadja a megnevezést egy adott IDnek
private string GetTipusNev(int tipusId)
{
    var tipus = fizetesTipusokLista.FirstOrDefault(t => t.ID == tipusId);

    if (tipus != null && !string.IsNullOrEmpty(tipus.Megnevezes))
    {
        return tipus.Megnevezes;
    }

    return $"Egyéb ({tipusId})";
}
```

A fájl végén található `GetTipusNev(int tipusId)` feladata a relációs kapcsolat feloldása a kliens memóriájában. Ahelyett, hogy a befizetések táblázatában a fizetés típusának ID-ja (pl. "1") jelenne meg, ez a metódus kikeresi a típusok listájából az egyező azonosítót, és annak szöveges megnevezését (pl. "Kollégiumi díj") adja vissza a felhasználónak.

2.30 Adataim.razor

Az `Adataim.razor` komponens a diákok dedikált profiloldala. Ezen a felületen a hallgatók megtekinthetik a rendszerben róluk tárolt legfontosabb információkat (név, e-mail,

aktuális szobaszám), illetve frissíthetik az elérhetőségüket. A komponens legfőbb sajátossága a szigorúan korlátozott adatmódosítási lehetőség, amely megvédi az adatbázis integritását az illetéktelen (vagy véletlen) felhasználói módosításoktól.

2.30.1 Függőségek és Felhasználói Környezet

```
@page "/adataim"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@using System.Security.Claims
@using System.IdentityModel.Tokens.Jwt
@layout LakoLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject SzobakService Szobak
@inject SzobaBeosztasokService SzobaBeosztasok
@inject FelhasznalokService FelhasznalokService
@inject DiakAdatokService DiakAdatok
@inject IJSRuntime JS
@inject NavigationManager Nav
```

- A komponens a `@layout LakoLayout` elrendezést használja, így vizuálisan és navigációs szempontból is a diák-specifikus felületbe ágyazódik.
- A valós idejű űrlapvalidációhoz és mentéshez az `InteractiveServer` renderelési módot alkalmazza. Injektálja a felhasználói (`FelhasznalokService`), a szoba- és beosztáskezelő, valamint az identitáskezeléshez szükséges szolgáltatásokat.

2.30.2 Adatvédelem a Felhasználói Felületen

```
@if (sajatUser != null)
{
<div id="view-profile" class="view-section">
<div class="section-header text-center mb-4">
<h2>Saját Adataim</h2>
<p class="text-muted">Személyes információk és szoba beosztás.</p>
</div>

<div class="content-panel mx-auto" style="max-width: 580px; background: white; padding: 28px; border-radius: 18px; box-shadow: 0 4px 6px rgba(0,0,0,0.1);">

@if (sikeresMentes)
{
<div class="alert alert-success mb-3">
<div class="bi bi-check-circle-fill"></div> Sikeresen elmentve!
</div>
}

<div class="mb-3">
<label class="form-label fw-bold">Teljes Név</label>
<input type="text" class="form-control" value="@sajatUser.Nev" readonly style="background-color: #f8f9fa; cursor: not-allowed;">
</div>

<div class="mb-3">
<label class="form-label fw-bold">Szoba száma</label>
<input type="text" class="form-control" value="@sajatSzobaSzam" readonly style="background-color: #f8f9fa; cursor: not-allowed;">
</div>

@* E-MAIL MEZŐ ÚJRA CSAK OLVASHATÓ *@
<div class="mb-3">
<label class="form-label fw-bold">E-mail cím</label>
<input type="email" class="form-control" value="@sajatUser.Email" readonly style="background-color: #f8f9fa; cursor: not-allowed;">
</div>

<div class="mb-4">
<label class="form-label fw-bold">Telefonszám</label>
<input type="text" class="form-control" @bind="sajatUser.Telefonszam" placeholder="Pl: +36301234567" maxlength="12">
<div class="form-text">Formátum: + és max. 11 számjegy (pl. +36301234567).</div>
</div>

<div class="d-grid">
<button class="btn btn-primary py-2 fw-bold" @onclick="Mentes" disabled="@mentesFolyamatban">
@if (mentesFolyamatban)
{
<span class="spinner-border spinner-border-sm me-2" role="status" aria-hidden="true"></span>
Mentés...</span>
}
else
{
<span><div class="bi bi-save me-2"></div>Adatok mentése</span>
}
</button>
</div>
</div>
}
```

A vizuális felület (UI) egy letisztult Bootstrap kártyába (content-panel) van csomagolva, amely egy rendkívül fontos biztonsági és üzleti szabályt tükröz:

- **Korlátozott szerkeszthetőség:** A "Teljes Név", "Szoba száma" és "E-mail cím" beviteli mezők explicit módon readonly attribútummal vannak ellátva. CSS segítségével (background-color: #f8f9fa; cursor: not-allowed;) a felület vizuálisan is egyértelműsíti, hogy ezek az adatok zároltak. **Ennek oka:** Egy diák nem írhatja át a saját nevét, e-mail címét vagy szobaszámát kénye-kedve szerint, mert az felborítaná a kollégium adminisztrációs és pénzügyi rendjét. Ezeket csak adminisztrátor módosíthatja.
- **Az egyetlen szerkeszthető mező:** A hallgató kizárólag a "Telefonszám" mezőt módosíthatja, amely kétirányú adatkötéssel (@bind="sajatUser.Telefonszam") kapcsolódik a memóriához.
- **Aszinkron UX:** A "Mentés" gomb a kattintás után azonnal inaktívvá válik (disabled="@mentesFolyamatban"), és egy töltésjelző (spinner) jelenik meg rajta, elkerülve a duplikált mentési kéréseket.

2.30.3 Identitás Alapú Adatlekérés

```

@code {
    private string token = "";
    private bool betoltesAlatt = true;
    private bool mentesFolyamatban = false;
    private bool sikeresMentes = false;
    private string hibaUzenet = "";
    private int sajátId = 0;

    private MoliPortal.Lib.MODEL.Felhasznalok sajátUser;
    private string sajátSzobaSzam = "Nincs szobába beosztva";

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            try
            {
                token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

                if (string.IsNullOrEmpty(token))
                {
                    Nav.NavigateTo("/");
                    return;
                }

                var handler = new JwtSecurityTokenHandler();
                var jwtToken = handler.ReadJwtToken(token);

                var idClaim = jwtToken.Claims.FirstOrDefault(x =>
                    x.Type == ClaimTypes.NameIdentifier ||
                    x.Type == "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier");

                if (idClaim != null && int.TryParse(idClaim.Value, out int parsedId))
                {
                    sajátId = parsedId;
                }
                else
                {
                    hibaUzenet = "Nem sikerült azonosítani a felhasználót a tokenből.";
                    betoltesAlatt = false;
                    StateHasChanged();
                    return;
                }

                var osszesUser = await FelhasznalokService.GetFelhasznalok(token) ?? new();
                sajátUser = osszesUser.FirstOrDefault(u => u.ID == sajátId);

                if (sajátUser == null)
                {
                    hibaUzenet = "A felhasználói fiókod nem található az adatbázisban.";
                    return;
                }

                var osszesBeosztas = await SzobaBeosztasok.GetSzobaBeosztasok(token) ?? new();
                var aktivBeosztas = osszesBeosztas.FirstOrDefault(b => b.UserID == sajátId && b.TenylegesKikoltozes == null);

                if (aktivBeosztas != null)
                {
                    var osszesSzoba = await Szobak.GetSzobak(token) ?? new();
                    var szoba = osszesSzoba.FirstOrDefault(sz => sz.ID == aktivBeosztas.RoomID);
                    if (szoba != null)
                    {
                        sajátSzobaSzam = $"{szoba.Szobaszam}";
                    }
                }
            }
            catch (Exception ex)
            {
                hibaUzenet = $"Hiba az adatok betöltésekor: {ex.Message}";
            }
            finally
            {
                betoltesAlatt = false;
                StateHasChanged();
            }
        }
    }
}

```

Az `OnAfterRenderAsync` metódusban található algoritmus garantálja, hogy a diák *kizárólag* a saját adatait láthassa:

- A JWT tokenből a `ClaimTypes.NameIdentifier` segítségével nyeri ki a kód a bejelentkezett felhasználó hitelesített azonosítóját (`sajátId`). Ha ez a folyamat sérül vagy az ID érvénytelen, a kód azonnal leáll egy hibaüzenettel.

- Ezt követően a memóriában, LINQ lekérdezésekkel (`.FirstOrDefault(u => u.ID == sajátId)`) megkeresi a felhasználó alapadatait, valamint a hozzá tartozó, jelenleg is aktív (`TenylegesKikoltozes == null`) szobabeosztást és szobaszámot, hogy megjelenítse azokat a zárolt mezőkben.

2.30.4 Szigorú Kliensoldali Validáció és Mentési Algoritmus

```
private async Task Mentes()
{
    try
    {
        mentesFolyamatban = true;
        sikeresMentes = false;
        hibaUzenet = "";

        string tel = sajátUser.Telefonszam?.Trim() ?? "";

        if (string.IsNullOrEmpty(tel))
        {
            hibaUzenet = "A telefonszám nem lehet üres!";
            mentesFolyamatban = false;
            return;
        }

        if (!tel.StartsWith("+"))
        {
            hibaUzenet = "A telefonszámnak '+' jellel kell kezdődnie!";
            mentesFolyamatban = false;
            return;
        }

        if (tel.Length > 12)
        {
            hibaUzenet = "A telefonszám maximum 11 számjegyet tartalmazhat a '+' jel után!";
            mentesFolyamatban = false;
            return;
        }

        if (tel.Length < 2)
        {
            hibaUzenet = "A '+' jel után legalább egy számjegyet meg kell adni!";
            mentesFolyamatban = false;
            return;
        }

        for (int i = 1; i < tel.Length; i++)
        {
            if (!char.IsDigit(tel[i]))
            {
                hibaUzenet = "Érvénytelen formátum! A '+' után csak számokat írhatasz be.";
                mentesFolyamatban = false;
                return;
            }
        }

        // MENTÉS AZ ADATBÁZISBA
        await FelhasznalokService.UpdateFelhasznalok(sajátUser, token);

        sikeresMentes = true;

        _ = Task.Run(async () =>
        {
            await Task.Delay(3000);
            sikeresMentes = false;
            await InvokeAsync(StateHasChanged);
        });
    }
    catch (Exception ex)
    {
        hibaUzenet = "Nem sikerült menteni a változtatásokat: " + ex.Message;
    }
    finally
    {
        mentesFolyamatban = false;
        StateHasChanged();
    }
}
```

A Mentés aszinkron metódus a telefonszám ellenőrzésére egy manuális, többlépcsős validációs logikát használ, amely megvédi az adatbázist a hibás formátumoktól:

1. **Üresség és Formátum kezdete:** Ellenőrzi, hogy a mező nem üres-e, és szigorúan megköveteli, hogy a megadott érték a nemzetközi formátumra utaló + jellel kezdődjön (!tel.StartsWith("+")).
2. **Hossz-korlátok:** A karakterlánc hossza nem lehet kevesebb 2-nél és nem haladhatja meg a 12-t, ami lefedi a szabványos mobiltelefonszámokat.
3. **Karakterenkénti elemzés:** Egy for ciklussal a kód a második karaktertől (a + jel után) végigiterál a stringen, és a !char.IsDigit(tel[i]) vizsgálattal garantálja, hogy kizárólag számjegyek kerüljenek mentésre (kizárva a szóközőket, kötőjeleket és betűket).

Ha a validáció minden ponton átmegy, a kód meghívja a UpdateFelhasznalok API végpontot, beállítja a sikeres mentést jelző változót, majd egy elegáns UI/UX megoldással (Task.Delay(3000)) gondoskodik róla, hogy a zöld sikerüzenet 3 másodperc elteltével automatikusan eltűnjön a képernyőről.

2.31 Befizetések.razor

A BefizetesekLako.razor komponens a hallgatók személyes pénzügyi központja. Ezen a felületen a lakók nyomon követhetik a kollégiumi díjaikat és egyéb tartozásaikat, áttekinthetik a korábbi befizetéseiket, és ami a legfontosabb: a felületen keresztül közvetlenül rendezhetik az aktív hátralékaikat. A szoftver egy beépített, biztonságos fizetési szimulációs modult alkalmaz a folyamat demonstrálására.

2.31.1 Keretrendszer és Felhasználói Felület

```
@page "/befizetesekLako"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@using System.Security.Claims
@using System.IdentityModel.Tokens.Jwt
@layout LakoLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject PenzugyekService Penzugyek
@inject FizetesTipusokService FizetesTipusok
@inject IJSRuntime JS
@inject NavigationManager Nav
```

```

<div class="content-panel" style="max-width: 800px;">
  @* OLDALMERET VÁLASZTÓ *@
  <div class="d-flex justify-content-end mb-3">
    <div class="d-flex align-items-center gap-2">
      <span class="text-muted small fw-bold text-uppercase">5or/oldal:</span>
      <select class="form-select form-select-sm shadow-sm" style="width: 80px; border-radius: 8px; cursor: pointer;" value="@elemekOldalankent" @onchange="OldalMeretValtozas">
        <option value="5">5</option>
        <option value="10">10</option>
        <option value="25">25</option>
      </select>
    </div>
  </div>

  <div class="table-container">
    <div class="table-scroll-container">
      <table>
        <thead>
          <tr>
            <th>Határidő / Fizetve</th>
            <th>Jogcím</th>
            <th>Összeg</th>
            <th class="text-center">Státusz</th>
            <th class="text-center">Művelet</th>
          </tr>
        </thead>
        <tbody>
          @if (megjelenítettPenzugyek != null && megjelenítettPenzugyek.Any())
          {
            @foreach (var tetel in megjelenítettPenzugyek)
            {
              <tr>
                <td>
                  @if (tetel.BefizetesDatum.HasValue)
                  {
                    <span class="text-muted small d-block">Fizetve:</span>
                    <strong>@tetel.BefizetesDatum.Value.ToShortDateString()</strong>
                  }
                  else
                  {
                    <span class="text-muted small d-block">Határidő:</span>
                    <strong class="@{tetel.Esedekezesseg < DateTime.Now ? "text-danger" : ""}">
                      @tetel.Esedekezesseg.ToShortDateString()
                    </strong>
                  }
                </td>
                <td style="font-weight: 500;">@GetTipusNev(tetel.TipusID)</td>
                <td style="font-weight: 700;">@tetel.Osszeg.ToString("N0") Ft</td>
                <td class="text-center">
                  @if (tetel.BefizetesDatum.HasValue)
                  {
                    <span class="badge badge-success">Fizetve</span>
                  }
                  else
                  {
                    <span class="badge badge-danger">Tartozás</span>
                  }
                </td>
                <td class="text-center">
                  @if (!tetel.BefizetesDatum.HasValue)
                  {
                    <button class="btn btn-sm btn-primary fw-bold px-3 py-1" style="border-radius: 6px;" @onclick="() => TetelBefizetese(tetel)">
                      <i class="bi bi-credit-card"></i> Fizetés
                    </button>
                  }
                  else
                  {
                    <i class="bi bi-check2-all text-success fs-4"></i>
                  }
                </td>
              </tr>
            }
          }
          else
          {
            <tr>
              <td colspan="5" class="text-center text-muted p-5">Jelenleg nincsenek számláid.</td>
            </tr>
          }
        </tbody>
      </table>
    </div>
  </div>

```

- **Környezet:** Az oldal a @layout LakoLayout elrendezésben jelenik meg. Injektálja a PenzugyekService és FizetesTipusokService hálózati osztályokat az adatok eléréséhez.
- **Valós Idejű KPI Kártyák:** A felület tetején két kiemelt statisztikai kártya fogadja a diákat. A kód egy nagyon intelligens vizuális visszajelzést (Feltételes UI) használ a tartozásoknál: ha az @osszesTartozas > 0, a kártya piros színre és figyelmeztető ikonra vált, míg ha a diák mindent befizetett, megnyugtató zöld pipával jelzi a rendezett státuszt. Az összegek.ToString("N0") metódussal, ezres tagolással jelennek meg a könnyebb olvashatóság érdekében.

2.31.2 Intelligens Adattábla és Gyorsszűrők

```

@* LAPOZÓ GOMBOK *@
@if (összesOldal > 1)
{
    <div class="d-flex justify-content-center align-items-center gap-3 mt-4 mb-2">
        <button class="btn btn-outline-secondary rounded-pill px-4 fw-bold shadow-sm" @onclick="ElőzoOldal" disabled="@(!jelenlegiOldal == 1)">
            <i class="bi bi-chevron-left me-1"></i> Előző
        </button>

        <span class="text-muted fw-bolder bg-white border px-4 py-2 rounded-pill shadow-sm">@jelenlegiOldal / @összesOldal</span>

        <button class="btn btn-outline-secondary rounded-pill px-4 fw-bold shadow-sm" @onclick="KövetkezoOldal" disabled="@(!jelenlegiOldal == összesOldal)">
            Következő <i class="bi bi-chevron-right me-1"></i>
        </button>
    </div>
}

```

```

@code {
    private bool betoltesAlatt = true;
    private string token = "";
    private int sajátId = 0;

    private string hibaUzenet = "";
    private bool sikeresFizetes = false;

    // Listák az API-ból és a lapozáshoz
    private List<Penzugyek> sajátPenzugyek = new();
    private List<Penzugyek> megjelenítettPenzugyek = new();
    private List<FizetesTipusok> fizetesTipusokLista = new();

    // LAPOZÁS VÁLTOZÓI
    private int jelenlegiOldal = 1;
    private int elemekOldalakent = 5; // 5-ös alapbeállítás
    private int összesOldal = 1;

    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
        {
            try
            {
                token = await JS.InvokeAsync<string>("localStorage.getItem", "authToken");

                if (string.IsNullOrEmpty(token))
                {
                    Nav.NavigateTo("/");
                    return;
                }

                var handler = new JwtSecurityTokenHandler();
                var jwtToken = handler.ReadJwtToken(token);
                var szerepkor = jwtToken.Claims.FirstOrDefault(x => x.Type == "role" || x.Type == "http://schemas.microsoft.com/ws/2008/05/identity/claims/role")?.Value;

                if (szerepkor != "2" && szerepkor?.ToLower() != "lako")
                {
                    Nav.NavigateTo("/");
                    return;
                }

                var idClaim = jwtToken.Claims.FirstOrDefault(x =>
                    x.Type == ClaimTypes.NameIdentifier ||
                    x.Type == "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier");

                if (idClaim != null && int.TryParse(idClaim.Value, out int parsedId))
                {
                    sajátId = parsedId;
                    await AdatokBetoletese();
                }
                else
                {
                    hibaUzenet = "Nem sikerült azonosítani a felhasználót.";
                }
            }
            catch (Exception ex)
            {
                hibaUzenet = $"Rendszerhiba: {ex.Message}";
            }
            finally
            {
                betoltesAlatt = false;
                StateHasChanged();
            }
        }
    }
}

```

Az adatokat egy modern, lapozható Bootstrap táblázat listázza.

- **Szűrőfelület:** A diák három gomb ("Összes", "Tartozások", "Befizetve") segítségével tudja szűkíteni a listát a @szuresiMod változón keresztül.
- **Határidő-figyelő (Késedelem vizualizációja):** A táblázat soraiban egy beágyazott C# logika ellenőrzi a határidőket: @(tétel.Esedekesseg < DateTime.Now ? "text-danger" : ""). Ha egy tétel fizetetlen és a határidő már lejárt, a dátum automatikusan pirosra változik.

- **Cselekvésre Ösztönzés (Call to Action):** A "Befizetés" gomb szigorúan csak azoknál a tételeknél jelenik meg, ahol a BefizetesDatum még üres. Erre kattintva a MegnyitFizetesModal(tetel) metódus hívódik meg.

2.31.3 Fizetési Szimuláció

```
private async Task AdatokBetoltese()
{
    try
    {
        // Pénzügyek lekérése
        var osszesPenzugy = await Penzugyek.GetPenzugyek(token) ?? new();

        sajátPenzugyek = osszesPenzugy
            .Where(p => p.UserID == sajátId)
            .OrderBy(p => p.BefizetesDatum.HasValue ? 1 : 0)
            .ThenByDescending(p => p.Esedekesseg)
            .ToList();

        // Fizetési típusok lekérése
        fizetesTipusokLista = await FizetesTipusok.GetFizetesTipusok(token) ?? new();

        LapozasFrissitse();
    }
    catch (Exception ex)
    {
        hibaUzenet = $"Hiba az adatok letöltésekor: {ex.Message}";
    }
}

// LAPOZÁS LOGIKÁJA
private void LapozasFrissitse()
{
    if (sajátPenzugyek == null) return;

    osszesOldal = (int)Math.Ceiling((double)sajátPenzugyek.Count / elemekOldalankent);
    if (osszesOldal == 0) osszesOldal = 1;

    if (jelenlegiOldal > osszesOldal) jelenlegiOldal = osszesOldal;

    megjelenítettPenzugyek = sajátPenzugyek
        .Skip((jelenlegiOldal - 1) * elemekOldalankent)
        .Take(elemekOldalankent)
        .ToList();
}

private void ElozoOldal()
{
    if (jelenlegiOldal > 1)
    {
        jelenlegiOldal--;
        LapozasFrissitse();
    }
}

private void KovetkezoOldal()
{
    if (jelenlegiOldal < osszesOldal)
    {
        jelenlegiOldal++;
        LapozasFrissitse();
    }
}

private void OldalMeretValtozas(ChangeEventArgs e)
{
    if (int.TryParse(e.Value?.ToString(), out int ujMeret))
    {
        elemekOldalankent = ujMeret;
        jelenlegiOldal = 1;
        LapozasFrissitse();
    }
}
```

Ez a szoftver egyik legizgalmasabb része! A "Befizetés" gombra kattintva nem egy külső oldalra navigálunk, hanem a képernyőn megnyílik egy elegáns felugró ablak (Modal).

- **Tranzakciós Adatok:** Az ablak tetején a rendszer összefoglalja a fizetendő tétel nevét (pl. Kollégiumi díj) és pontos összegét.

- **Bankkártya Űrlap:** A kód egy komplett virtuális bankkártya-bekérő űrlapot szimulál (Kártyaszám, Lejárat dátum, CVC kód). Bár az adatok itt nincsenek külső banki API-hoz kötve (mivel ez egy vizsgamunka), az architektúra 100%-ban fel van készítve egy Stripe vagy SimplePay integrációra.

2.31.4 Biztonság és Kliensoldali Identitás

```
private async Task TetelBefizetese(Penzugyek kifizetendoTetel)
{
    hibaUzenet = "";
    sikeresFizetes = false;

    try
    {
        kifizetendoTetel.BefizetesDatum = DateTime.Now;
        kifizetendoTetel.Statusz = "Fizetve";

        await Penzugyek.UpdatePenzugyek(kifizetendoTetel, token);
        await AdatokBetoltese();

        sikeresFizetes = true;

        = Task.Run(async () =>
        {
            await Task.Delay(4000);
            sikeresFizetes = false;
            await InvokeAsync(StateHasChanged);
        });
    }
    catch (Exception ex)
    {
        hibaUzenet = $"Hiba a fizetés során: {ex.Message}";
    }
}

// A TípusID alapján kikeressük az objektumot a listából
private string GetTípusNev(int típusId)
{
    var típus = fizetesTípusokLista.FirstOrDefault(t => t.ID == típusId);
    if (típus != null && !string.IsNullOrEmpty(típus.Megnevezes))
    {
        return típus.Megnevezes;
    }
    return $"Egyéb ({típusId})";
}
```

Az OnAfterRenderAsync garantálja az adatvédelmet:

- Szigorúan ellenőrzi, hogy a felhasználó "Lakó" jogosultságú-e.
- **Identitás Alapú Adatlekérés:** A tokenből kiolvasott sajátId alapján a AdatokBetoltese metódus kizárólag a diák saját pénzügyi tranzakcióit tölti le a memóriába (osszesPenzugy = awaitWhere(p => p.UserID == sajátId).ToList()). Ezzel fizikailag lehetetlenné teszi, hogy egy diák mások számláihoz vagy fizetési adataihoz férjen hozzá.

Amikor a hallgató a "Fizetés szimulálása" gombra kattint, egy aszinkron mentési folyamat indul el a try-catch blokkban:

1. A kiválasztott pénzügyi tétel (kiválasztottTetel) állapotát a rendszer a memóriában frissíti: a BefizetesDatum megkapja a jelenlegi időbélyeget (DateTime.Now), a Statusz pedig átvált "FIZETVE" értékre.

2. Az `await Penzugy.UpdatePenzugyek(...)` parancs ezt a frissített objektumot elküldi a backend API-nak, hogy az adatbázisban is rögzüljön a fizetés ténye.
3. A folyamat legvégén az űrlap bezáródik (`isFizetesModalOpen = false`), az adatok a háttérben újra letöltődnek az azonnali statisztika-frissítés (KPI kártyák frissítése) érdekében, a felhasználó pedig egy zöld sikerüzenetet kap.

2.32 Hibajelentes.razor

A **Hibajelentes.razor** komponens a kollégium üzemeltetési moduljának lakói felülete. A hallgatók itt rögzíthetik az észlelt infrastrukturális problémákat (pl. elromlott zár, kiégett izzó), amelyeket a rendszer azonnal továbbít az adminisztrátori/karbantartói műszerfalra. A felület egyúttal személyes archívumként is szolgál, ahol a diák ellenőrizheti a saját korábbi bejelentéseinek aktuális állapotát.

2.32.1 Keretrendszer és Moduláris Függőségek

```
@page "/hibajelentes"
@using KoliPortal.Lib.MODEL
@using KoliPortal.Lib.SERVICE
@using System.Security.Claims
@using System.IdentityModel.Tokens.Jwt
@layout LakoLayout
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject KarbantartasiKeresekService KarbantartasiKeresek
@inject KarbantartasStatuszokService Statuszok
@inject IJSRuntime JS
@inject NavigationManager Nav
```

- **Környezet:** Az oldal a `@layout LakoLayout` elrendezésbe töltődik be, fenntartva a hallgatói profil szeparációját. A dinamikus űrlapkezeléshez az `InteractiveServer` mód fut.
- **Injektálások:** A működéshez két fő hálózati szolgáltatás szükséges: a `KarbantartasiKeresekService` a hibajegyek küldéséhez és lekéréséhez, valamint a `KarbantartasStatuszokService`, amely a nyers állapot-azonosítókat (pl. 1, 2, 3) olvasható szöveggé ("Új", "Folyamatban") alakítja.

2.32.2 Hibabejelentő Űrlap és Visszajelzések

```
@* ÚJ HIBA BEJELENTÉSE (ŰRLAP KÁRTYA) *@
<div class="content-panel w-100">
  <h4 class="mb-3"><i class="bi bi-tools me-2"></i>Új hiba bejelentése</h4>

  @if (sikeresKudles)
  {
    <div class="alert alert-success mb-4">
      <i class="bi bi-check-circle-fill"></i> <strong>Siker:</strong> @sikerUzenet
    </div>
  }

  @if (!string.IsNullOrEmpty(hibaUzenet))
  {
    <div class="alert alert-danger mb-4">
      <i class="bi bi-exclamation-triangle-fill"></i> @hibaUzenet
    </div>
  }

  <form @onsubmit="HibajelentesKuldese" @onsubmit:preventDefault="true">
    <div class="input-group mb-3">
      <label class="form-label fw-bold">Típus</label>
      <select class="form-select input-field" @bind="valasztottTípus">
        <option value="" disabled selected>Válassz...</option>
        <option value="Vízvezeték és szaniter">Vízvezeték és szaniter</option>
        <option value="Elektromosság és világítás">Elektromosság és világítás</option>
        <option value="Fűtés">Fűtés</option>
        <option value="Nyílászárók és záruk">Nyílászárók és záruk</option>
        <option value="Bűtorzat és asztalosmunka">Bűtorzat és asztalosmunka</option>
        <option value="Közös helyiségek gépei">Közös helyiségek gépei</option>
        <option value="Internet és hálózat">Internet és hálózat</option>
        <option value="Mártevek és higiénia">Mártevek és higiénia</option>
        <option value="Egyéb műszaki hiba">Egyéb műszaki hiba</option>
      </select>
    </div>
    <div class="input-group mb-3">
      <label class="form-label fw-bold">Leírás</label>
      <textarea class="form-control input-field" rows="4" @bind="leiras"
        placeholder="Részletezd a problémát pontosan (pl. 204-es szoba, csöpög a csap a fürdőben)..."></textarea>
    </div>
    <button type="submit" class="btn btn-primary w-100 mt-2 fw-bold py-2">
      <i class="bi bi-send me-2"></i> Jelentés küldése
    </button>
  </form>
</div>
```

```
@* SAJÁT HIBAJEGYEK TÁBLÁZAT KÁRTYA *@
<div class="content-panel w-100">
  <h4 class="mb-3"><i class="bi bi-card-checklist me-2"></i>Nyitott hibajegyeim</h4>

  <div class="table-container">
    <div class="table-scroll-container">
      <table>
        <thead>
          <tr>
            <th style="width: 120px; min-width: 100px;">Dátum</th>
            <th style="min-width: 250px;">Leírás</th>
            <th class="text-center" style="width: 130px; min-width: 120px;">Státusz</th>
          </tr>
        </thead>
        <tbody>
          @if (nyitottHibajegyek.Any())
          {
            @foreach (var hiba in nyitottHibajegyek)
            {
              <tr>
                <td class="text-muted small">@hiba.LetrehozasDatum.ToString("yyyy. MM. dd.")</td>
                <td class="text-break">@hiba.Leiras</td>
                <td class="text-center">
                  <span class="badge @GetStatuszCssClass(hiba.StatuszID) rounded-pill px-3 py-2">
                    @GetStatuszNev(hiba.StatuszID)
                  </span>
                </td>
              </tr>
            }
          }
          else
          {
            <tr>
              <td colspan="3" class="text-center text-muted p-4">Nincsenek folyamatban lévő hibajegyeid.</td>
            </tr>
          }
        </tbody>
      </table>
    </div>
  </div>
</div>
```

- **Vizuális állapotjelzők:** A kód az @if direktívákkal szabályozza egy Bootstrap Alert megjelenését. Ha a mentés sikeres (sikerMentes), zöld sikerdobozt mutat a szerver válaszával (@sikerUzenet); hiba esetén pirosat.
- **Adatkötés:** A <textarea> elem kétirányú adatkötéssel (@bind="ujHibaLeiras") kapcsolódik a C# változóhoz, így a beírt szöveg azonnal a memóriába kerül.
- **Gomb logika:** A "Bejelentés küldése" gomb inaktívvá válik (disabled="@mentesFolyamatban"), amíg a hálózati kérés le nem fut. Ezalatt egy HTML pörgettyű (spinner) ad vizuális visszajelzést a feldolgozásról. A gomb az on_click eseményre a HibaKuldes metódust indítja el.

2.32.3 Személyes Archívum

```
private string token = "";
private bool betoltesAlatt = true;
private int sajátId = 0;

// Üzenetek
private string hibaUzenet = "";
private string sikerUzenet = "";
private bool sikeresKudles = false;

// Űrlap mezők
private string választottTípus = "";
private string leiras = "";

// Listák a táblázathoz
private List<KarbantartasiKeresek> nyitottHibajegyek = new();
private List<KarbantartasStatuszok> statuszokLista = new();
```

```
private async Task AdatokBetoltese()
{
    try
    {
        statuszokLista = await Statuszok.GetKarbantartasStatuszok(token) ?? new();

        var osszesHiba = await KarbantartasiKeresek.GetKarbantartasiKeresek(token) ?? new();

        // Itt szűrjük le egyből a nyitott (nem 3-as státuszú) hibákat
        nyitottHibajegyek = osszesHiba
            .Where(h => h.BejelentoID == sajátId && h.StatuszID != 3)
            .OrderByDescending(h => h.LetrehozasDatum)
            .ToList();
    }
    catch (Exception ex)
    {
        hibaUzenet = $"Hiba az adatok letöltésekor: {ex.Message}";
    }
}
```

Az űrlap alatt egy Bootstrap adattábla (table-scroll-container) listázza a diák saját korábbi bejelentéseit:

- A táblázat a @foreach (var hiba in sajátHibak) ciklussal iterál a szerverről letöltött adatokon.

- **Dinamikus Állapotjelzők:** A UI egyik legfontosabb eleme az "Állapot" oszlop. Itt a kód meghívja a `GetStatuszCss` és `GetStatuszNev` segédmetódusokat. Ezek az aktuális `StatuszID` alapján generálnak egy színes jelvényt (Badge) – például zöldet a "Kész" vagy kéket a "Folyamatban" lévő munkákhoz –, így a diák egyetlen pillantással átlátja a folyamatokat.

2.32.4 Biztonság és Kliensoldali Identitás

```
private async Task HibajelentesKuldese()
{
    hibaUzenet = "";
    sikeresKudles = false;

    if (string.IsNullOrEmpty(valasztottTipus))
    {
        hibaUzenet = "Kérlek, válassz egy hiba típust a listából!";
        return;
    }

    if (string.IsNullOrEmpty(leiras) || leiras.Length < 10)
    {
        hibaUzenet = "Kérlek, részletezd a problémát! (Legalább 10 karakter)";
        return;
    }

    try
    {
        string veglegesLeiras = $"{[valasztottTipus]} {leiras}";

        var ujHiba = new KarbantartasiKeresek
        {
            BejelentoID = sajátId,
            StatuszID = 1,
            Leiras = veglegesLeiras,
            LetrehozasiDatum = DateTime.Now
        };

        await KarbantartasiKeresek.CreateKarbantartasiKeresek(ujHiba, token);

        választottTipus = "";
        leiras = "";
        await AdatokBetoltese();

        sikerUzenet = "Hibajelentésedet sikeresen rögzítettük! A karbantartók hamarosan intézkednek.";
        sikeresKudles = true;

        _ = Task.Run(async () =>
        {
            await Task.Delay(5000);
            sikeresKudles = false;
            await InvokeAsync(StateHasChanged);
        });
    }
    catch (Exception ex)
    {
        hibaUzenet = $"Hiba történt a küldés során: {ex.Message}";
    }
}
```

A `@code` blokk elején történik a memóriaváltozók deklarálása (listák, állapotjelzők). Az `OnAfterRenderAsync` metódus a biztonsági kapu:

- Kiolvassa a JWT token-t a böngészőből.

- **Kétlépcsős validáció:** Ellenőrzi, hogy a tokenből dekódolt szerepkör "lako" vagy "2"-es azonosítójú-e. Ha nem, visszairányít a /login oldalra.
- **Személyes azonosító kinyerése:** A token "Claims" adatai közül megkeresi a ClaimTypes.NameIdentifier értéket. Ez a bejelentkezett diák biztonságos, hitelesített ID-ja (sajatId), amelyet a továbbiakban minden szűréshez és adatmentéshez használni fog a rendszer.

2.32.5 Mentési Logika és Adatfeldolgozás

```
private string GetStatuszNev(int statuszId)
{
    var statusz = statuszokLista.FirstOrDefault(s => s.ID == statuszId);
    return statusz != null ? statusz.Nev : $"Ismeretlen ({statuszId})";
}

private string GetStatuszCssClass(int statuszId)
{
    if (statuszId == 1)
    {
        return "bg-danger";
    }
    else
    {
        return "bg-warning text-dark";
    }
}
```

- **AdatokBetoltese():** Letölti az összes hibajegyet és státuszt a szerverről. A kulcsfontosságú lépés a LINQ szűrés: `sajatHibak = osszesHiba.Where(h => h.Bejelentoid == sajátId)`. Ez garantálja, hogy a memóriába csak azok a rekordok kerülnek be, amelyek a hitelesített diákhoz tartoznak. Végül időrendben csökkenő sorrendbe teszi őket (`OrderByDescending`).
- **HibaKuldes():** 1. Ellenőrzi, hogy a leírás nem üres-e (`string.IsNullOrEmpty`). 2. Felépít egy új `KarbantartasiKeresek` objektumot (DTO). Fixen hozzárendeli a bejelentő ID-ját (`sajatId`) és az induló állapotkódot (`StatuszID = 1`). A dátumot a szerver aktuális idejére állítja. 3. A `CreateKarbantartasiKeresek` metódussal elküldi a csomagot az API-nak. 4. Sikeresetén kiírja a szövegdobozt, újra letölti az adatokat a táblázat frissítéséhez, majd egy 3 másodperces aszinkron késleltetéssel (`Task.Delay(3000)`) automatikusan eltünteti a sikerüzenetet.
- **Állapot-feloldók (`GetStatuszNev`, `GetStatuszCss`):** A táblázat UI-t támogató metódusok. Kikeresik a memóriából (`statuszok.FirstOrDefault`) az adott

azonosítóhoz (ID) tartozó szöveget (pl. "Kész"), illetve meghatározzák a megjelenítéshez szükséges CSS osztályt (p-active vagy p-done).

2.33 Web/Program.cs

```
builder.Services.AddScoped(_ => new HttpClient { BaseAddress = new Uri("https://localhost:44331/") });
builder.Services.AddScoped<DiakAdatokService>();
builder.Services.AddScoped<SzobakService>();
builder.Services.AddScoped<KarbantartasiKeresekService>();
builder.Services.AddScoped<SzobaBeosztasokService>();
builder.Services.AddScoped<FelhasznalokService>();
builder.Services.AddScoped<SzerepkorokService>();
builder.Services.AddScoped<KarbantartasStatuszokService>();
builder.Services.AddScoped<PenzugyekService>();
builder.Services.AddScoped<AuthControllerService>();
builder.Services.AddScoped<FizetesTipusokService>();
builder.Services.AddScoped<KollegiumService>();
```

Szolgáltatások és Függőséginjektálás (Dependency Injection) konfigurálása (Program.cs)

A képen látható kódrészlet a webalkalmazás indítási konfigurációjának (startup) "lelke". Célja a rendszer hálózati kommunikációjának beállítása, valamint az üzleti logikát kezelő szolgáltatások (Services) beregisztrálása a beépített Függőséginjektáló (Dependency Injection - DI) konténerbe.

A kód pontos működése:

- **API Végpont csatlakoztatása (HttpClient):** Az első sor (`builder.Services.AddScoped(_ => new HttpClient...`) létrehoz egy globális HTTP kliens példányt a hálózati kommunikációhoz. A `BaseAddress` beállítása (`https://localhost:44331/`) kritikus jelentőségű: ez definiálja, hogy a frontend alkalmazás hol találja meg a Backend API-t. Az összes későbbi adatkérés (GET, POST, PUT, DELETE) ehhez az alap URL-hez fog csatlakozni.
- **Adatszolgáltatások regisztrálása (AddScoped<T>):** A kód további 11 sora a KoliPortál saját fejlesztésű szolgáltatásait (`DiakAdatokService`, `SzobakService`, `PenzugyekService`, stb.) regisztrálja a DI konténerbe.
- **Élettartam-kezelés (Scoped):** A szolgáltatások `.AddScoped()` módszerrel történő regisztrálása meghatározza az objektumok élettartamát a memóriában. A "Scoped" (hatókör szerinti) beállítás azt jelenti, hogy a keretrendszer egy adott felhasználó böngészős munkamenetén (kapcsolatán) belül egyetlen, közös példányt hoz létre ezekből az osztályokból, és ezt a példányt használja újra minden egyes lapletöltéskor vagy komponens-híváskor.

Szerepe az architektúrában: Ez a blokk teremti meg a technikai alapját annak, hogy a Razor komponensek (UI) adatokat tudjanak fogadni és küldeni. Amikor egy .razor fájl elején az @inject SzobakService Szobak direktíva szerepel, a rendszer ebbe a regisztrációs listába néz bele. Itt találja meg az utasítást arra, hogyan kell a memóriában példányosítani a kért szolgáltatást. Ezen konfigurációs sorok nélkül az injektálások hibára futnának, és a frontend nem tudna kommunikálni a backenddel.

2.34 Fejlesztési lehetőségek

1. Több kollégium elérhetőségének és kezelésének kialakítása

Jelenleg a szoftver egyetlen intézmény (Karinthy Frigyes Kollégium) adminisztrációjára van felkészítve. A jövőbeli cél egy úgynevezett *több-bérlős* architektúra bevezetése, amellyel a szoftver mint szolgáltatás működhetne. Ehhez egy nagyobb rangú adminisztrátor szint bevezetése szükséges, aki új kollégiumokat regisztrálhat a rendszerbe. Ennek eredményeképpen a hallgatók a bejelentkezéskor vagy a regisztráció során választhatnák ki az intézményüket, az adminisztrátorok pedig szigorúan csak a saját kollégiumuk adataihoz (lakók, pénzügyek, szobák) férnének hozzá, logikailag elkülönített adatbázis-szegmensekben.

2. Egyéni pénzügyi befizetések teljeskörű kezelése a diákoknál

Bár a jelenlegi rendszer képes a tartozások tömeges kiírására és a fizetési folyamat szimulációjára, az éles üzemhez elengedhetetlen a valós banki szolgáltatók (pl. SimplePay, Barion vagy Stripe) API-jának integrálása. Továbbá fejleszteni kell az egyéni, részletfizetési konstrukciók kezelését is. Szükséges egy olyan adminisztrátori felület kialakítása is, ahol a helyszínen (kézpénzben vagy terminálon) történő befizetéseket a rendszergazda manuálisan jóváírhatja, majd arról a rendszer automatikusan elektronikus nyugtát (PDF) generál és küld el a hallgató e-mail címére.

3. Pénzügyek fölül a statisztikák és összegek kiterjesztett megjelenítése

A pénzügyi modul áttekinthetőségének növelése érdekében a nyers adatsorokat vizuális adatábrázolási elemekkel (Data Visualization) érdemes bővíteni. Tervben van interaktív diagramok (pl. tortadiagramok a kintlévőségekről és a befizetett arányokról) beépítése. Az adminisztrátorok számára elengedhetetlen lenne egy komplex szűrőrendszer kialakítása, amellyel hónapokra, félévekre vagy konkrét szintekre/szobákra bontva láthatják a pénzügyi mérleget, valamint be kell vezetni egy egygombos exportálási funkciót (Excel/CSV formátumban) a könyvelés megkönnyítésére.

4. A „Beállítások” modul funkcionális továbbfejlesztése

A képen látható "Rendszerbeállítások" kártya jelenleg egy vizuálisan elkészült felhasználói felület, amelynek backend logikája a szoftver egy későbbi fejlesztési fázisát képezi. A felületen látható három specifikus kapcsoló mögé a következő üzleti és hálózati logikákat kell implementálni az éles üzemhez:

- **Új hibajegy értesítés**

Egy külső levélküldő szolgáltatás bekötése a backendbe. A kódnak figyelnie kell ezt az állapotváltozót: ha be van kapcsolva, a rendszer automatikusan e-mailt generál és küld a karbantartóknak/adminisztrátoroknak abban a másodpercben, amikor egy diák a saját felületén elment egy új hibajegyet az adatbázisba.

- **Késedelmes fizetés riasztás**

Egy ütemezett, szerveroldali háttérfolyamat leprogramozása. Bekapcsolt állapot esetén a szervernek (például minden éjfélnél) végig kell iterálnia a teljes pénzügyi adatbázison. Ahol a befizetés határideje lejárt, ott a rendszer automatikus figyelmeztetést (riasztást) generál a késedelmes diáknak.

- **Karbantartási mód**

Egy biztonsági szűrő bevezetése a hálózati rétegben. Ha az adminisztrátor aktiválja ezt a gombot, a bejelentkezési végpontnak (Login) meg kell változtatnia a viselkedését: a "Lakó" (diák) szerepkörrel rendelkező fiókok hitelesítését meg kell tagadnia, és kizárólag az adminisztrátorokat szabad beengednie, amíg a karbantartási mód aktív.

5. Automatizált E-mail és Értesítési Rendszer (Push Notifications)

A rendszer életszerűségét nagyban növelné egy háttérben futó értesítő szolgáltatás. Ez a modul automatikus e-mailt küldene a diákoknak a fizetési határidő lejáta előtt 3 nappal, elkerülve a késedelmeket. Ugyanígy a hallgató azonnali e-mail vagy push értesítést kapna a telefonjára, amint egy adminisztrátor módosítja a leadott hibajegyének státuszát (pl. "A csöpögő csap javítása folyamatban").

3 FELHASZNÁLÓI DOKUMENTÁCIÓ

3.1 A program célja és rövid ismertetése

A KoliPortál egy modern, felhőalapú kollégiumi adminisztrációs szoftver, amelynek elsődleges célja a hagyományos, papíralapú és táblázatkezelőkre épülő intézményi ügyintézés teljes körű digitalizációja. A rendszer egyetlen közös, interaktív felületen köti össze a kollégium vezetőségét (adminisztrátorait) és a bent lakó hallgatókat.

A szoftver funkciói közé tartozik a hallgatók pénzügyi egyenlegének nyomon követése, a szobabeosztások adminisztrációja, valamint egy valós idejű hibabejelentő modul biztosítása. A vezetőség egy letisztult statisztikai kezelőfelületen keresztül menedzselheti az épület kapacitását, a diákok által leadott műszaki problémákat. A program "Mobile-First" (mobilra tervezett) vizuális szemlélettel épült fel, ezáltal asztali számítógépen és okostelefonon egyaránt optimális felhasználói élményt nyújt.

3.2 Rendszerkövetelmények

A szoftver webes alkalmazásként működik, így a végfelhasználók számára nem szükséges telepítőfájlok futtatása. A stabil működéshez a következő minimális hardver és szoftverfeltételeknek kell teljesülniük.

3.2.1 Hardverkövetelmények

A program futtatása nem igényel nagy teljesítményű célhardvert:

- **Asztali számítógép vagy Laptop:** Bármilyen eszköz, amely képes egy modern operációs rendszer futtatására. Külön dedikált videokártya vagy kiemelkedő RAM nem szükséges.
- **Mobileszközök:** Okostelefon vagy Tablet. A responszív felépítésnek köszönhetően a rendszer a képernyő méretéhez igazodva automatikusan rendezi át az elemeket (pl. táblázatokat és a navigációs menüt).
- **Hálózat:** Stabil, folyamatos internetkapcsolat elengedhetetlen.

3.2.2 Szoftverkövetelmények

Nincs szükség külső kiegészítő programok telepítésére. A rendszer eléréséhez mindössze egy modern, naprakész webböngésző szükséges.

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

3.3 Telepítés és indítás lépéseinek ismertetése

A felhasználók számára: A felhasználóknak semmilyen szoftvert nem kell letölteniük a számítógépükre.

Használatba vétel lépései:

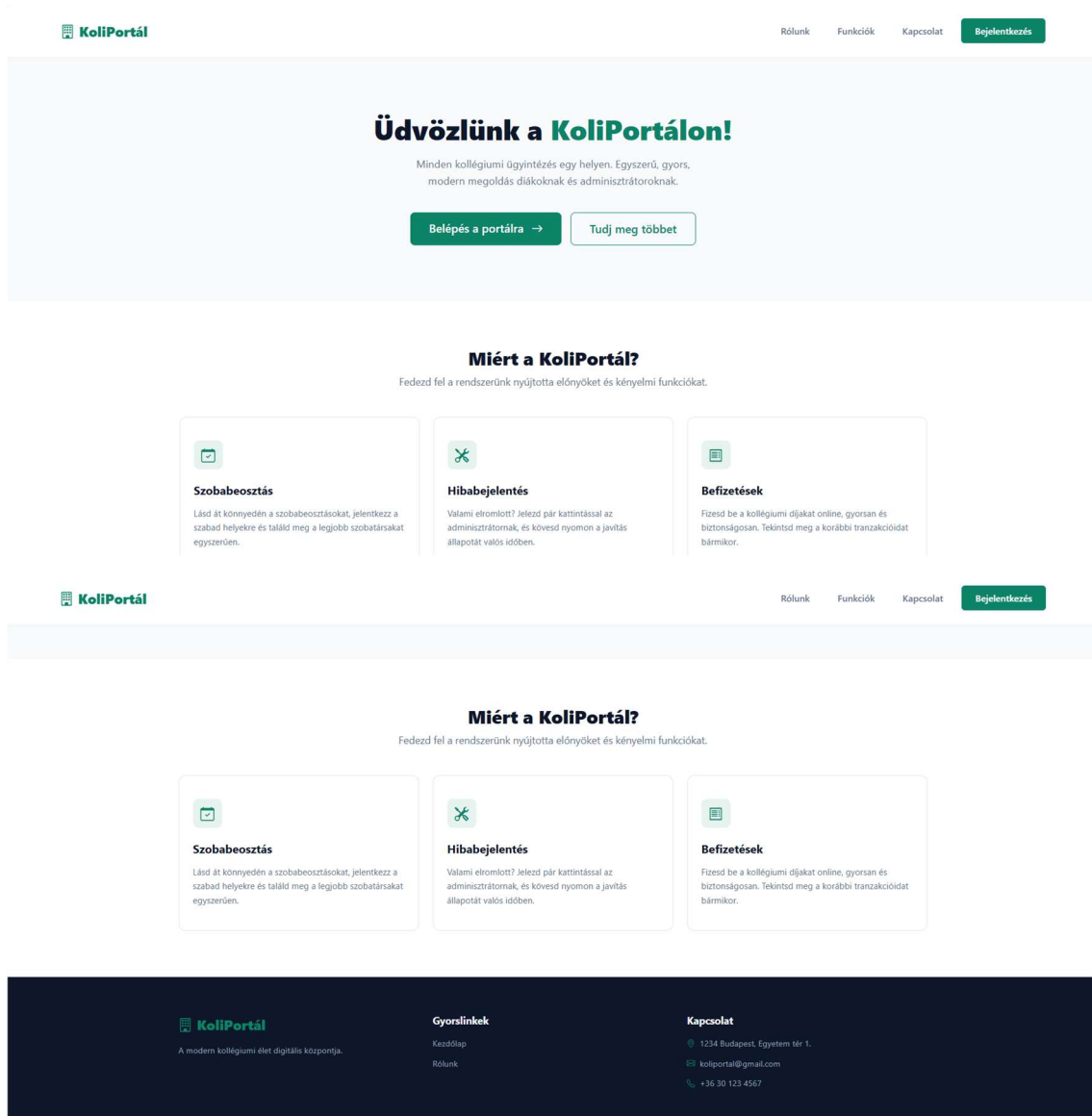
- Nyissa meg az internetböngészőt (Google Chrome, MozillaFirefox, Microsoft Edge)
- A címsorába írja be a weboldal címét: <https://www.koliportal.hu>
- Az oldal betöltődése után megjelenik a kezdőlap.

3.4 Regisztráció nélkül elérhető funkciók (Kezdőlap)

A KoliPortál szigorúan zárt intézményi rendszer, azonban rendelkezik egy publikus, mindenki számára elérhető információs kezdőlappal. Érvényes felhasználói fiók és bejelentkezés hiányában a látogatók kizárólag ezen a felületen navigálhatnak.

A kezdőlap strukturális elemei:

- **Felső navigációs sáv:** Tartalmazza a KoliPortál logót, valamint a gyorslinkeket (Rólunk, Funkciók, Kapcsolat), amelyek az oldalon belüli folyamatos görgetést teszik lehetővé. A jobb felső sarokban található a kiemelt "Bejelentkezés" gomb.
- **Üdvözlő (Hero) szekció:** Egy rövid összefoglaló a portál céljáról, két akciógombbal ("Belépés a portálra" és "Tudj meg többet").
- **Funkciók bemutatása ("Miért a KoliPortál?"):** Három információs kártya, amely a Szobabeosztás, a Hibabejelentés és a Befizetések modulok működési logikáját foglalja össze a jövőbeli felhasználók számára.
- **Lábléc (Footer):** Sötét tónusú sáv az oldal alján, amely az üzemeltető hivatalos elérhetőségeit (fizikai cím, e-mail, telefonszám), a gyorslinkeket, valamint az Adatkezelési tájékoztató hivatkozását tartalmazza.



A regisztráció korlátozásai:

A szoftver nem rendelkezik nyílt regisztrációs űrlappal. A tényleges funkciók (hibabejelentés, egyenleg lekérdezése, szobabeosztás megtekintése) regisztráció nélkül nem érhetők el. A felhasználói fiókokat kizárólag a kollégium adminisztrátorai hozhatják létre a hivatalos felvételi listák alapján, ezzel kizárva az illetéktelenek hozzáférését.

3.5 Bejelentkezés a rendszerbe és beviteli szabályok

A publikus felület "Bejelentkezés" gombjára kattintva a felhasználó az autentikációs oldalra navigál. A rendszer ezen a felületen azonosítja a személyt, és a hitelesítési adatok alapján irányítja át a megfelelő (Diák vagy Adminisztrátor) jogosultsági szintre.

A belépési űrlap beviteli szabályai és elvárásai:

- **Kollégium kiválasztása:** Legördülő menü, amely jelenleg egyetlen (alapértelmezett) intézményre van korlátozva és a felhasználó által nem módosítható. A funkció a szoftver jövőbeli, hálózatbővítési (több épület kezelése) céljait szolgálja.
- **Email cím mező:** A rendszer kizárólag érvényes, szabványos e-mail formátumot fogad el (pl. diak@koliportal.hu). Az e-mail címek esetében a szoftver szigorúan kis- és nagybetű érzékeny (Case-sensitive), tehát a rendszer élesen megkülönbözteti a kisbetűket a nagybetűktől. Például a Diak@koliportal.hu és a diak@koliportal.hu két teljesen különböző címnek minősül. Ékezetes karakterek és speciális írásjelek (a ponton, kötőjelen, alulvonáson és a @ jelen kívül) használata az e-mail címben nem megengedett.

- **Jelszó mező:** A beírt karaktereket a rendszer vizuálisan pontokkal fedi el. A jelszó mező szintén szigorúan kis- és nagybetű érzékeny. A jelszó hosszára vonatkozóan a rendszer szigorú biztonsági házirendet alkalmaz: a jelszónak kötelezően minimum 6 karakter hosszúnak kell lennie. Ennél rövidebb karakterlánc beírása esetén a rendszer nem kezdeményezi a belépést. A mező nem maradhat üresen.

Hitelesítési hibajelzések:

A szoftver vizuális visszajelzést ad a felhasználónak a sikertelen belépési kísérletekről:

- **„Kérlek tölts ki minden mezőt!”:** A hibaüzenet piros színnel jelenik meg az űrlap felett, amennyiben a felhasználó az e-mail vagy a jelszó mező kitöltése nélkül kezdeményezi a bejelentkezést.

Kérlek tölts ki minden mezőt!

- **„Hibás email vagy jelszó!”:** Abban az esetben jelenik meg a felhasználónak, ha a megadott e-mail cím nem szerepel az adatbázisban (esetleg eltér a kis/nagybetű), vagy a jelszó nem egyezik.

Hibás email vagy jelszó!

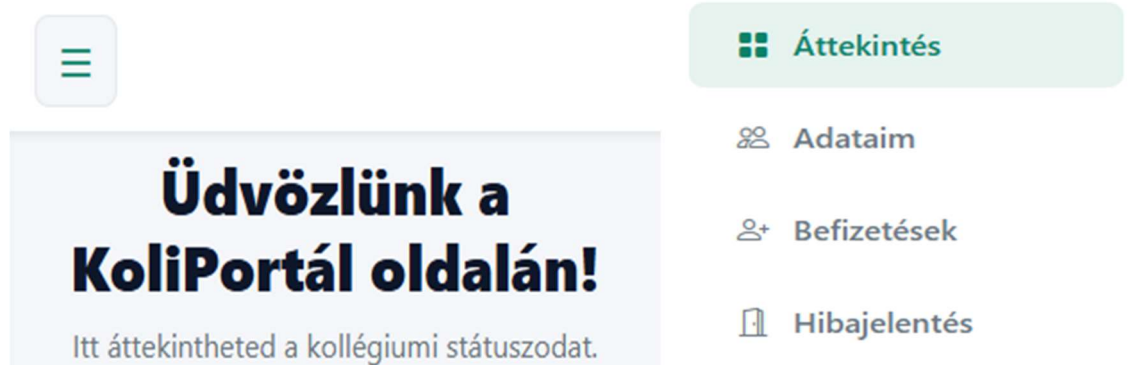
- **„Hiba történt a szerverhez való kapcsolódáskor.”:** Kliensoldali hibaüzenet, amely akkor lép fel, ha az eszköz hálózati probléma miatt nem éri el az API szerveret.

Hiba történt a szerverhez való
kapcsolódáskor.

3.6 A Diák (Lakó) felhasználói felület részletes bemutatása

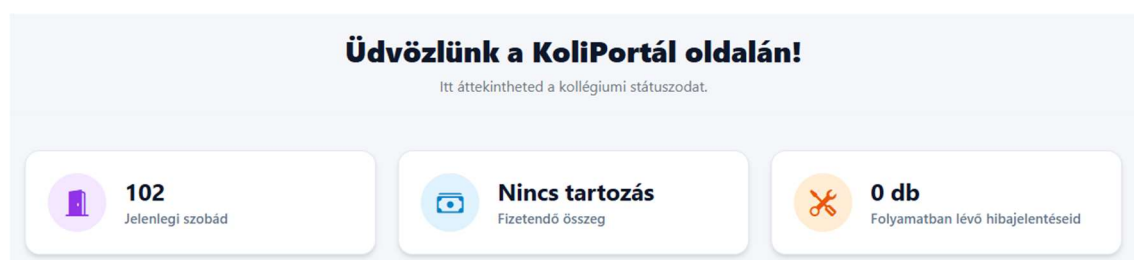
Sikeres hitelesítés esetén a Lakó jogosultsággal rendelkező felhasználó a saját személyes felületére kerül.

Az oldal navigációját egy oldalsáv biztosítja. Mobileszközön a bal felső „hamburger” ikonnal nyitható le. Innen három fő modult érhető el.



3.6.1 Áttekintés

A nyitóoldalon (Áttekintés) a diák megtekintheti az adminisztrátor által rögzített hivatalos szobabeosztását, valamint egy kiemelt információs kártyán a fennálló, aktív pénzügyi tartozásainak összesített összegét és a jelenleg beküldött hibajegyeinek számát.



3.6.2 Pénzügyek

A Pénzügyek menüpontra navigálva a rendszer egy táblázatos listát jelenít meg a hallgató rendelt összes múltbéli és jelenlegi fizetendő tételről. Mivel ez a lista a tanulmányok során rendkívül hasznúvá válhat, a szoftver dinamikusan lapozható felületet alkalmaz, amely oldalaként 5, 10 vagy 25 tételt jelenít meg, elkerülve a memóriátúlterhelést. A vizuális elkülönítés érdekében a „Fizetve” státuszú tételek zöld, míg a tartozást jelentő

„Fizetendő” tételek piros jelvénnel szerepelnek.

Saját Befizetések

Kollégiumi díjak és egyéb tranzakciók áttekintése, rendezése.

SOR/OLDAL: 5 ▼

Határidő / Fizetve	Jogcím	Összeg	Státusz	Művelet
Határidő: 2026. 04. 10.	Féléves DÖK/Kulturális Hozzájárulás	5 000 Ft	Tartozás	Fizetés
Fizetve: 2026. 03. 31.	Féléves Hálózatüzemeltetés	25 000 Ft	Fizetve	

3.6.3 Hibajelentés modul és beviteli szabályok

A „Hibajelentés” menüpont a műszaki problémák közvetlen, papírmentes leadására szolgál. A felületi felső részén az adatbeviteli űrlap, alsó részén pedig a korábban leadott, folyamatban lévő jegyek táblázata található.

Hibabejelentés

Műszaki probléma jelzése a karbantartóknak és a saját jegyeid követése.

✂ Új hiba bejelentése

Típus

Válassz... ▼

Leírás

Részletezd a problémát pontosan (pl. 204-es szoba, csöpög a csap a fürdőben)...

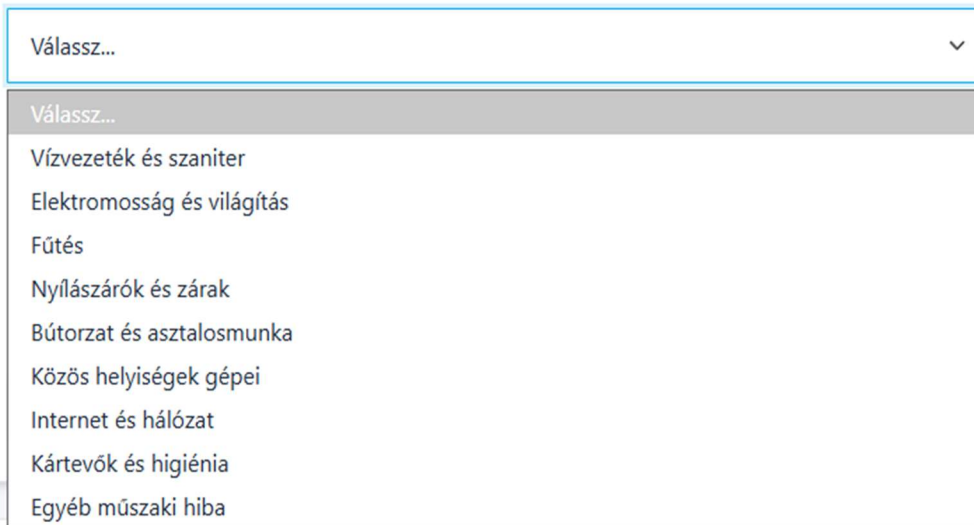
Jelentés küldése

📋 Nyitott hibajegyeim

Dátum	Leírás	Státusz
Nincsenek folyamatban lévő hibajegyeid.		

Az űrlap kitöltési szabályai és validációs folyamatai:

- **Típus mező (Legördülő menü):** A hiba kategóriájának kiválasztása kötelező. Ez a funkció az adatok strukturálását és a karbantartók feladatainak prioritizálását szolgálja.

✕ Új hiba bejelentése**Típus**

The screenshot shows a web form titled 'Új hiba bejelentése' (New error report). Below the title is a dropdown menu labeled 'Típus' (Type). The dropdown is open, showing a list of error categories. The first item is 'Válassz...' (Select...). The other items are: 'Vízvezeték és szaniter' (Plumbing and sanitation), 'Elektromosság és világítás' (Electrical and lighting), 'Fűtés' (Heating), 'Nyílászárók és zárok' (Windows and doors), 'Bútorzat és asztalosmunka' (Furniture and carpentry), 'Közös helyiségek gépei' (Machines in common areas), 'Internet és hálózat' (Internet and network), 'Kártevők és higiénia' (Pests and hygiene), and 'Egyéb műszaki hiba' (Other technical error).

- **Leírás mező (Textarea):** Szabadon kitölthető szöveges felület a probléma részletezésére.
 - **Karakterhosszúság:** A szoftver szigorúan megköveteli a minimum 10 karakteres hosszúságot. Ennek célja a „Rossz” vagy „Nem működik” típusú, elégtelen információt tartalmazó bejelentések szűrése. Maximális karakterkorlát nincs definiálva.
 - **Engedélyezett formátumok:** A mező feldolgozza és adatbázisba menti a teljes magyar karakterkészletet (ékezetes betűket), a számokat, a speciális írásjeleket és a sortöréseket is. A rendszer megőrzi a felhasználók által alkalmazott kis és nagybetűk formátumát.

Hibakezelés az űrlap beküldésekor: Ha a felhasználó a „Jelentés küldése” gombra kattint, de a Leírás mező kevesebb mint 10 karaktert tartalmaz, vagy elfelejtett kategóriát választani, a szoftver biztonsági okokból megszakítja a mentési folyamatot. A felületen a következő hibaüzenetek egyike jelenik meg azonnal, egy figyelemfelkeltő piros dobozban: "Kérlek, részletezd a problémát! (Legalább 10 karakter)" vagy "Kérlek,

válassz egy hiba típust a listából!".

✕ Új hiba bejelentése

⚠ Kérlek, válassz egy hiba típust a listából!

✕ Új hiba bejelentése

⚠ Kérlek, részletezd a problémát! (Legalább 10 karakter)

A leadott hibajegyek nyomon követése: Sikeres adatbevitel esetén a képernyő tetején zöld színű megerősítő üzenet jelenik meg.

✕ Új hiba bejelentése

✓ **Siker:** Hibajelentésedet sikeresen rögzítettük! A karbantartók hamarosan intézkednek.

Típus

Válassz...

Leírás

Részletezd a problémát pontosan (pl. 204-es szoba, csöpög a csap a fürdőben)...

📤 Jelentés küldése

A beküldött hiba pedig oldalfrissítés nélkül, azonnal listázódik az űrlap alatti "Nyitott bejelentéseim" táblázatban. Ez az átlátható táblázat három oszlopra tagolódik:

- **Dátum:** A bejelentés rögzítésének pontos napja.
- **Leírás:** A leírás tartalmazza a kiválasztott típust és a leírásban megadott problémát.
- **Státusz:** Egy színkódolt információs jelvény mutatja a probléma elhárításának aktuális szakaszát. A frissen felvitt hibák piros színű "Új" státuszt kapnak. Ha

megkezdődik a folyamat akkor a felvitt hibának a státusza sárga színű "Folyamatban" jelzésre vált.

Amikor az adminisztrátor a munkát "Kész" állapotra módosítja, a bejegyzés automatikusan kikerül a hallgató aktív feladatokat listázó nézetéből.

Nyitott hibajegyeim

Dátum	Leírás	Státusz
2026. 03. 31.	[Vízvezeték és szaniter] Csöpög a csap a fürdőben.	

Nyitott hibajegyeim

Dátum	Leírás	Státusz
2026. 03. 31.	[Vízvezeték és szaniter] Csöpög a csap a fürdőben.	

3.7 Az Adminisztrátor felhasználói felület részletes bemutatása

Sikeres hitelesítés esetén az Admin jogosultsággal rendelkező felhasználó az Adminisztrátor felületére kerül.

3.7.1 Adminisztrátori Navigációs Oldalsáv

A képen a rendszer adminisztrátorai számára elérhető bal oldali fő navigációs menü látható. Ez a sáv biztosítja az átjárást a KoliPortál különböző moduljai között. A felület felépítése letisztult, minden menüpontot egy beszédes ikon és egy egyértelmű felirat azonosít.

Adminisztráció

Áttekintés

Felhasználók

Regisztrálás

Szobák

Karbantartás

Pénzügyek

Beállítások

A felhasználó az alábbi elemeket látja és használhatja a felületen:

- **Fejléc ("Adminisztráció"):** A menü tetején található felirat egyértelműen vizuálisan is megerősíti a felhasználó számára, hogy a kiemelt jogosultságú, adminisztrátori felületen tartózkodik.
- **Áttekintés:** A fő műszerfal (Dashboard) elérésére szolgál. A képen látható zöldes háttérű kiemelés jelzi, hogy a felhasználó jelenleg ezen az oldalon tartózkodik (aktív menüpont).
- **Felhasználók:** Erre a menüpontra kattintva érhető el a kollégiumi lakók, karbantartók és adminisztrátorok teljes adatbázisa, ahol módosítani és keresni lehet az adataik között.
- **Regisztrálás:** Ez a gomb ahhoz az űrlaphoz vezet, ahol az adminisztrátor új felhasználókat (például új beköltöző diákokat) tud hozzáadni a rendszerhez.
- **Szobák:** A kollégiumi férőhelyek, a szobák telítettségének és a lakók beosztásának áttekintésére szolgáló felület.

- **Karbantartás:** A diákok által leadott műszaki hibajelentések (aktív és lezárt feladatok) kezelésére és nyomon követésére szolgáló modul.
- **Pénzügyek:** Itt kezelhetők a kollégiumi díjak, a diákok tartozásai, az eddigi befizetések, valamint itt indítható el a tömeges díjkiírás.
- **Beállítások:** A rendszer globális funkcióinak (pl. értesítések, karbantartási mód) konfigurálására szolgáló felület.

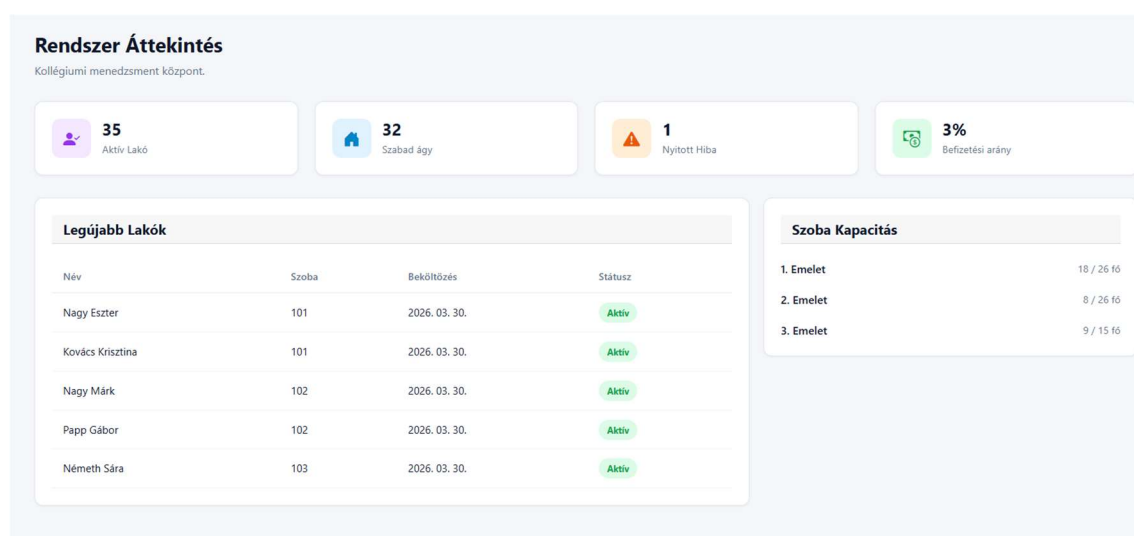
 Kijelentkezés

Alsó szekció:

- **Kijelentkezés:** Az oldalsáv legalsó részén, a többi menüponttól vizuálisan is elkülönítve, piros kerettel és szöveggel kapott helyet a biztonságos kilépést szolgáló gomb. Erre kattintva a felhasználó lezárja a munkamenetét, és a rendszer visszairányítja a bejelentkezési oldalra.

3.7.2 Áttekintés

A képen az adminisztrátorok központi érkezői oldala, a "Rendszer Áttekintés" (Dashboard) látható. Ennek a felületnek a célja, hogy bejelentkezés után azonnal, egyetlen pillantással átfogó képet adjon a kollégium aktuális üzemeltetési és kihasználtsági állapotáról.



A felhasználó a következő elemeket látja és értelmezheti a felületen:

- **Fejléc:** A "Rendszer Áttekintés" cím egyértelműsíti a funkciót, alatta a "Kollégiumi menedzsment központ" leírás olvasható.
- **Kiemelt statisztikai kártyák (Felső sáv):** Négy darab vizuális, ikonokkal ellátott kártya mutatja a rendszer legfontosabb globális mutatóit (KPI-okat):
 - **Aktív Lakó (Lila):** Megmutatja a kollégiumban jelenleg aktív beosztással rendelkező, bent lakó diákok pontos számát (a képen 35 fő).
 - **Szabad ágy (Kék):** Kijelzi a teljes intézményben aktuálisan elérhető, kiadható férőhelyek számát (a képen 32).
 - **Nyitott Hiba (Narancssárga):** Figyelmeztet a diákok által leadott, de még meg nem oldott (folyamatban lévő vagy új) karbantartási feladatokra (a képen 1 db).
 - **Befizetési arány (Zöld):** Százalékosan mutatja a kiírt pénzügyi tételek és a sikeresen teljesített befizetések arányát, gyors visszajelzést adva a kintlévőségekről (a képen 3%).
- **Legújabb Lakók (Bal oldali panel):** Egy gyorsnézeti táblázat, amely a legutóbb a rendszerbe rögzített (vagy frissen beköltözött) hallgatókat listázza. A táblázat oszlopai egyértelműsítik a diák nevét, a hozzárendelt szobaszámot, a beköltözés pontos dátumát, valamint egy zöld "Aktív" jelvénnel a jelenlegi státuszát. Ez segíti az adminisztrátort a friss beiratkozások gyors ellenőrzésében.
- **Szoba Kapacitás (Jobb oldali panel):** Egy szintekre (emeletekre) bontott statisztikai összesítő. Megmutatja, hogy az egyes emeleteken a maximális kapacitáshoz képest jelenleg hány diák lakik (pl. az 1. emeleten 18 fő a lehetséges 26-ból). Ez a nézet rendkívül hasznos az új diákok szobabeosztásának megtervezésekor, mivel azonnal látható, melyik szinten van a legtöbb üres hely.

3.7.3 Felhasználók

Felhasználók Kezelése
Kollégisták adatbázisa.

Név	Kollégium	Szoba	Email	Szerepkör	Művelet
Balázs Botond	Karinthy Frigyes Kollégium	303	balazs.botond@koliportal.hu	LAKO	
Balogh Márton	Karinthy Frigyes Kollégium	202	balogh.marton@koliportal.hu	LAKO	
Farkas Zsófia	Karinthy Frigyes Kollégium	Kiköltözött	farkas.zsofia@koliportal.hu	LAKO	
Fekete András	Karinthy Frigyes Kollégium	105	fekete.andras@koliportal.hu	LAKO	
Fekete Noémi	Karinthy Frigyes Kollégium	Kiköltözött	fekete.noemi@koliportal.hu	LAKO	

Alapnézet és az "Összes" szűrő

- **Mit lát a felhasználó:** A rendszer alapértelmezett állapotát. A bal felső sarokban az "Összes" gomb aktív (sötétzöld színnel kiemelve).
- **Működés:** A táblázat a kollégium teljes felhasználói adatbázisát listázza, vegyesen mutatva az aktív, szobával rendelkező diákokat (zöld szobaszám jelvény, pl. "303") és a már távozott hallgatókat (piros "Kiköltözött" jelvény). A táblázat alján a lapozósáv jelzi, hogy a nagy adatmennyiség miatt a lista több oldalra van bontva (a képen 1/8 oldal látható).

Felhasználók Kezelése

Kollégisták adatbázisa.

Összes **Bent lakók** Kiköltözöttek

Keresés (név, email, szoba)...

SOR/OLDAL: 5

Név	Kollégium	Szoba	Email	Szerepkör	Művelet
Balázs Botond	Karinthy Frigyes Kollégium	303	balazs.botond@koliportal.hu	LAKO	
Balogh Márton	Karinthy Frigyes Kollégium	202	balogh.marton@koliportal.hu	LAKO	
Fekete András	Karinthy Frigyes Kollégium	105	fekete.andras@koliportal.hu	LAKO	
Hajdu Melinda	Karinthy Frigyes Kollégium	109	hajdu.melinda@koliportal.hu	LAKO	
Horváth Ákos	Karinthy Frigyes Kollégium	304	horvath.akos@koliportal.hu	LAKO	

< Előző 1 / 7 Következő >

Felhasználók Kezelése

Kollégisták adatbázisa.

Összes Bent lakók **Kiköltözöttek**

Keresés (név, email, szoba)...

SOR/OLDAL: 5

Név	Kollégium	Szoba	Email	Szerepkör	Művelet
Farkas Zsófia	Karinthy Frigyes Kollégium	Kiköltözött	farkas.zsofia@koliportal.hu	LAKO	
Fekete Noémi	Karinthy Frigyes Kollégium	Kiköltözött	fekete.noemi@koliportal.hu	LAKO	

Állapotszűrők használata

- **Mit lát a felhasználó:** A gyorsművelet gombok használatát és azok azonnali hatását a táblázatra.
- **Működés (Bent lakók):** Amikor a felhasználó a **"Bent lakók"** gombra kattint, a lista dinamikusan leszűkül. A képen látható, hogy eltűntek a **"Kiköltözött"** státuszú diákok, és a táblázat kizárólag azokat a sorokat mutatja, ahol aktív szobabeosztás van (zöld jelvények). A lapozósáv is automatikusan frissült a kevesebb találatához (1/7 oldal).
- **Működés (Kiköltözöttek):** A **"Kiköltözöttek"** gombra kattintva az ellenkezője történik: a rendszer csak az archív, kollégiumi szobával már nem rendelkező (piros jelvényes) profilokat jeleníti meg.

Felhasználók Kezelése

Kollégisták adatbázisa.

Összes
Bent lakók
Kiköltözöttek

SOR/OLDAL: 5

Név	Kollégium	Szoba	Email	Szerepkör	Művelet
Nagy Eszter	Karinthy Frigyes Kollégium	101	nagy.eszter@koliportal.hu	LAKO	

Valós idejű szöveges keresés

- **Mit lát a felhasználó:** A képernyő jobb felső részén található keresőmező működését, ami név, email és szobaszám szerint szűr.
- **Működés:** A felhasználó elkezdte begépelni a "Nagy Eszter" nevet a mezőbe. A rendszer a gépelés pillanatában (Enter lenyomása nélkül) azonnal leszűrte a többoldalas adatbázist erre az egyetlen egyező sorra.

SOR/OLDAL:

5

5
10

Oldalméret és megjelenítés beállítása

- **Mit lát a felhasználó:** Egy legördülő menüt a keresőmező mellett, "SOR/OLDAL" felirattal.
- **Működés:** A felhasználó itt testreszabhatja a képernyőn egyszerre megjelenő adatok mennyiségét. A legördülő listából kiválasztható, hogy a táblázat 5 vagy 10 sort jelenítsen meg egyetlen oldalon, ami segít a kisebb monitorokon való átláthatóságban.

Felhasználó Szerkesztése és Szobabeosztás Kezelése

Az adminisztrátori felületen lehetőség van a már regisztrált felhasználók adatainak utólagos módosítására és a kollégiumi lakók szobabeosztásának kezelésére. Ezt a funkciót a listában a felhasználó sorában található (ceruza

Művelet

ikon) szerkesztés gombra kattintva lehet elérni, ami egy felugró (modális) ablakot nyit meg.

A szerkesztő űrlap dinamikusan alkalmazkodik a felhasználó szerepköréhez. A képeken egy "Diák" (lakó) adatlapjának szerkesztése látható.

Felhasználó Szerkesztése

×

Név

Kollégium

Email

Szerepkör

Diák adatok

Évfolyam

Lakcím

Szoba beosztás

Jelenlegi szoba

Törlés

Mégse

Mentés

Szerepkör

ADMIN
LAKO

Alapadatok Módosítása

Az ablak felső szekciójában az adminisztrátor az alapvető profiladatokat tudja frissíteni:

- **Név és Email:** A felhasználó teljes neve és e-mail címe szabadon átírható a szövegdobozokban.
- **Kollégium:** Egy legördülő listából kiválasztható, hogy a felhasználó melyik intézményhez tartozik (jelenleg a Karinthy Frigyes Kollégium látható).
- **Szerepkör:** Szintén egy legördülő menü, ahol a jogosultsági szint módosítható (pl. Diákból Adminisztrátor vagy Karbantartó lehet).

Szemeszter

6 szemeszter

Válassz szemesztert...

1 szemeszter

2 szemeszter

3 szemeszter

4 szemeszter

5 szemeszter

6 szemeszter

7 szemeszter

8 szemeszter

9 szemeszter

10 szemeszter

Diák Specifikus Adatok

Mivel a kiválasztott felhasználó "Diák", a rendszer automatikusan megjeleníti az ehhez a szerepkörhöz tartozó extra mezőket:

- **Évfolyam:** Egy legördülő lista segítségével beállítható a hallgató aktuális szemesztere (pl. "2 szemeszter").
- **Lakcím:** Egy egyszerű szöveges mező a diák állandó lakcímének frissítésére.

101 (Jelenlegi szobája)

Kiköltöztetés

104 (Szabad helyek: 2)
105 (Szabad helyek: 1)
108 (Szabad helyek: 2)
109 (Szabad helyek: 2)
110 (Szabad helyek: 1)
201 (Szabad helyek: 1)
204 (Szabad helyek: 1)
205 (Szabad helyek: 2)
206 (Szabad helyek: 3)
207 (Szabad helyek: 2)
208 (Szabad helyek: 2)
209 (Szabad helyek: 4)
210 (Szabad helyek: 4)
301 (Szabad helyek: 1)
302 (Szabad helyek: 2)
305 (Szabad helyek: 3)
306 (Szabad helyek: 1)

101 (Jelenlegi szobája)

Szobabeosztás és Kapacitáskezelés

A szerkesztőablak legalsó része a kollégiumi elhelyezésért felel. A "Jelenlegi szoba" legördülő menü rendkívül sokrétű funkciót lát el:

- **Jelenlegi állapot megtekintése:** Alapértelmezetten mutatja, hogy a diák hol lakik jelenleg (a képen a 303-as szobában).
- **Kiköltöztetés:** Az opciók között szerepel egy piros színnel kiemelt "Kiköltöztetés" gomb. Ha ezt választják, a rendszer automatikusan megszünteti a diák aktív szobabeosztását (anélkül, hogy a profilját törölné).
- **Átköltöztetés új szobába:** A legördülő lista kizárólag azokat a szobákat ajánlja fel, **ahol jelenleg van szabad férőhely**. Az opciók mellett a rendszer automatikusan, zárójelben kiírja az aktuálisan szabad ágyak számát (pl. "101 Szabad helyek: 2"), így az adminisztrátornak nem kell külön listákat ellenőriznie a döntéshez.

- **Archív/Szoba nélküli állapot:** Ha a felhasználó egy olyan diákot szerkeszt, akinek jelenleg nincs aktív beosztása (pl. korábban kiköltöztették, ahogy a képen látható "Kiköltözött" jelvény is mutatja), a menü alapértelmezetten a "Nincs szobába beosztva" állapotot mutatja, ahonnan természetesen szintén választható neki egy új, szabad szoba.



Mentés és Törlés: Az ablak alján található gombokkal menthetők el a módosítások. A "Törlés" gomb (amely piros színű) szintén innen érhető el, amennyiben a felhasználó profilját véglegesen el szeretné távolítani a rendszerből.

Felhasználó Szerkesztése

Név

Nagy Márk

Kollégium

Karinthy Frigyes Kollégium

Email

nagy.mark@koliportal.hu

Szerepkör

LAKO

Diák adatok

Szemeszter

4 szemeszter

Lakcím

2000 Szentendre, Dunakorzó 5.

Szoba beosztás

Jelenlegi szoba

102 (Jelenlegi szobája)

Törlés

Mégse

Mentés

Felhasználó Szerkesztése

Név

Nagy Márk

Kollégium

Karinthy Frigyes Kollégium

Email

nagy.mark@koliportal.hu

Szerepkör

LAKO

Diák adatok

Szemeszter

2 szemeszter

Lakcím

2000 Szentendre, Királyhágó u. 58.

Szoba beosztás

Jelenlegi szoba

104 (Szabad helyek: 2)

Törlés

Mégse

Mentés

Felhasználói Adatok és Szobabeosztás Módosítása

A rendszer rugalmasan kezeli a diákok adatainak változását. Ha egy lakó adatai megváltoznak, vagy át kell költöztetni egy másik szobába, az adminisztrátornak elegendő a "Felhasználó Szerkesztése" ablakban felülírnia a korábbi értékeket.

A képeken látható példa egy valós adatmódosítási folyamatot mutat be lépésről lépésre:

- Adatok átírása az űrlapon:** Az adminisztrátor megnyitotta egy meglévő diák ("Nagy Márk") adatlapját, majd módosította az elavult információkat:
 - Tanulmányi adat frissítése:** A diák aktuális szemesztere "4 szemeszter"-ről "2 szemeszter"-re lett átállítva a legördülő menüben.
 - Lakcímváltozás rögzítése:** A korábbi lakcím (Szentendre, Dunakorzó 5.) átírásra került az új lakcímre (Szentendre, Királyhágó u. 58.).
 - Szoba cseréje (Átköltöztetés):** A hallgatót a rendszer a korábbi, 102-es szobájából egy új beosztásba, a 104-es szobába helyezte át. A rendszer a kiválasztásnál intelligensen jelezte, hogy a 104-es szobában jelenleg még 2 szabad hely áll rendelkezésre.
- Mentés:** A módosítások befejezése után a folyamat a jobb alsó sarokban található, zöld "**Mentés**" gomb megnyomásával véglegesíthető.

A zöld háttérű értesítési sáv, amely a bal oldalon egy zöld ellenőrző jelet tartalmaz, majd a szöveg: "Siker: Sikeres mentés és adatok frissítése!"

- Rendszer-visszajelzés:** A sikeres mentést követően a szerkesztőablak automatikusan bezáródik, és a képernyő tetején egy zöld értesítési sáv jelenik meg ("**Siker: Sikeres mentés és adatok frissítése!**"). Ez vizuálisan is megerősíti az adminisztrátor számára, hogy az új adatok (és az új szobabeosztás) hibátlanul rögzítésre kerültek az adatbázisban, a mögöttes táblázat pedig már a frissített állapotot mutatja.

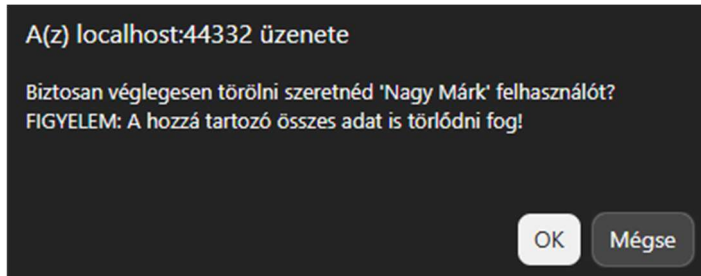
Felhasználó Végleges Törlése és Hibakezelés

A KoliPortál rendszerben egy felhasználó (diák) törlése egy összetett (kaszkádolt) művelet, mivel a hallgatóhoz számos altábla adata kapcsolódik (szobabeosztások,

pénzügyi tranzakciók, hibajelentések). Az adminisztrátori felület ezt a folyamatot biztonságosan és vizuálisan nyomon követhetően hajtja végre.

1. A normál törlési folyamat (Sikeres ág)

A törlés menete a következő lépésekből áll:



- **Biztonsági megerősítés:** Amikor az adminisztrátor a "Törlés" gombra kattint, a művelet nem hajtodik végre azonnal. A böngésző egy natív figyelmeztető ablakot dob fel, amely felhívja a figyelmet arra, hogy a felhasználóval együtt minden hozzá tartozó adat is megsemmisül.

✓ **Siker:** Felhasználó és adatai véglegesen törölve!

- **Sikeres végrehajtás:** Ha az adminisztrátor jóváhagyja a műveletet (OK), a rendszer a háttérben lépésről lépésre kiüríti a kapcsolódó adatbázistáblákat, majd végül magát a felhasználói fiókot is törli. Erről a felület tetején megjelenő zöld értesítési sáv (Alert) ad azonnali, egyértelmű visszajelzést: *"Siker: Felhasználó és adatai véglegesen törölve!"*.

2. Kivételkezelés: Rendszerhibák kommunikációja

Előfordulhatnak olyan hálózati vagy adatbázis-szintű anomáliák, amelyek megakadályozzák a törlést. A rendszer ezeket elkapja, és egy piros hibaüzenet formájában jeleníti meg, elkerülve a szoftver lefagyását. A két leggyakoribb HTTP státuszkódhoz tartozó hibajelenség a következő:

▲ **Hiba:** Hiba a törlés során: Response status code does not indicate success: 404 (Not Found).

- **404 (Not Found) Hiba:**

- **Jelentése:** A kliens (böngésző) olyan erőforrást vagy hálózati végpontot próbált elérni a szerveren (Backend API), amely nem létezik.
- **Mikor fordulhat elő éles üzemben?** Kialakulhat akkor, ha a háttérrendszerből hiányzik egy adott parancs (pl. egy törlési végpont nincs megfelelően konfigurálva). Életszerűbb példa erre a "versenyhelyzet" (Race Condition): ha két adminisztrátor dolgozik egyszerre a rendszerben, és az "A" adminisztrátor rányom a törlésre egy olyan diáknál, akit a "B" adminisztrátor pár másodperccel korábban már sikeresen kitörölt. A szerver ilyenkor 404-es hibát dob, hiszen az adott azonosítójú diák már nem található meg az adatbázisban.

▲ Hiba: Hiba a fő felhasználó törlése során: Response status code does not indicate success: 500 (Internal Server Error).

- **500 (Internal Server Error) Hiba:**

- **Jelentése:** A szerver megkapta és megértette a törlési parancsot, de a végrehajtás közben valamilyen belső, kritikus hiba miatt összeomlott a folyamat.
- **Mikor fordulhat elő éles üzemben?** A leggyakoribb oka a szigorú relációs adatbázis-védelem, az úgynevezett idegenkulcs-ütközés (Foreign Key Constraint Violation). Ha a rendszer megpróbálja törölni a fő felhasználót, de egy másik adatbázistáblában (például egy archivált beléptetési naplóban vagy egy külső logikai táblában) még létezik olyan rekord, amely erre a felhasználói azonosítóra hivatkozik, az SQL szerver biztonsági okokból blokkolja a törlést, ami 500-as hibát eredményez a felületen.

3.7.4 Regisztrálás

Az "Új Felhasználó Regisztrálása" menüpont alatt az adminisztrátorok új kollégistákat, karbantartókat vagy további adminisztrátorokat adhatnak hozzá a rendszerhez. A felület egyik legfontosabb funkciója a **dinamikus űrlapkezelés**: a rendszer csak azokat az adatbekérő mezőket jeleníti meg, amelyek a kiválasztott szerepkörhöz (jogosultsághoz) feltétlenül szükségesek.

A piros csillaggal (*) jelölt mezők kitöltése minden esetben kötelező.

1. Alapadatok (Minden szerepkör esetén)

Új Felhasználó Regisztrálása

Kollégista, adminisztrátor vagy karbantartó hozzáadása a rendszerhez.

Vezetéknév *

Keresztnév *

Email cím *

Jelszó *

Jelszó megerősítés *

Szerepkör *

Válassz szerepkört...

▼

Mégse

Létrehozás

Az űrlap felső szakasza független a felhasználó típusától. Itt kell megadni a belépéshez és azonosításhoz szükséges alapinformációkat:

- **Vezetéknév és Keresztnév:** A felhasználó hivatalos neve.
- **Email cím:** Ez szolgál majd a felhasználó bejelentkezési azonosítójaként (login név) és az értesítések címzettjeként.
- **Jelszó és Jelszó megerősítés:** Biztonsági okokból a jelszónak legalább 6 karakter hosszúságúnak kell lennie, és a hibás gépelés elkerülése végett kétszer kell megadni.
- **Szerepkör:** Egy legördülő lista, ahol kiválasztható a felhasználó jogosultsági szintje (pl. ADMIN, LAKO). Ennek kiválasztása határozza meg az űrlap további viselkedését.

2. Adminisztrátor Regisztrációja

Vezetéknév *	Keresztnév *
Holló	Tamás
Email cím *	
hollo.tamas@koliportal.hu	
Jelszó *	Jelszó megerősítés *
.....	
Szerepkör *	
ADMIN	

Amikor a Szerepkör mezőben az "ADMIN" opciót választják ki, az űrlap véget ér az alapadatoknál. Mivel a személyzet tagjai nem kapnak kollégiumi szobát, és nincsenek tanulmányi adataik, a rendszer nem kér be tőlük felesleges információkat. A fiók a zöld "Létrehozás" gombbal azonnal elmenthető.

3. Lakó (Diák) Regisztrációja és Extra Adatok

Vezetéknév *	Keresztnév *
Holló	Tamás
Email cím *	
hollo.tamas@koliportal.hu	
Jelszó *	Jelszó megerősítés *
.....	
Szerepkör *	
LAKO	

Diák extra adatok

Szemeszter *	Szoba beosztás
2 szemeszter	204 (Szabad hely: 1)

Lakcím

Telefonszám

Formátum: + és max. 11 számjegy (szóköz nélkül).

Ha a Szerepkör mezőben a **"LAKO"** opciót választják, a felület intelligensen lenyílik, és megjelenik a **"Diák extra adatok"** szekció. Itt a kollégiumi elhelyezéshez és adminisztrációhoz szükséges specifikus adatokat kell rögzíteni:

- **Szemeszter (Kötelező):** Legördülő listából választható ki a hallgató aktuális féléve (pl. "2 szemeszter").
- **Szoba beosztás:** Egy rendkívül hasznos funkció az azonnali beosztáshoz. A rendszer itt **kizárólag a szabad férőhellyel rendelkező szobákat** listázza ki, és zárójelben mutatja a fennmaradó ágyak számát (pl. *204 (Szabad hely: 1)*). Így az

adminisztrátor már a regisztráció pillanatában el tudja helyezni a diákot anélkül, hogy külön menüpontot kellene megnyitnia.

- **Lakcím (Opcionális):** A hallgató állandó lakcíme (pl. számlázási vagy értesítési célokra).
- **Telefonszám (Opcionális):** Szigorú formátumellenőrzéssel ellátott mező (kötelező + jel és maximum 11 számjegy), amely biztosítja a gyors kapcsolattartást a diákkal.

A rögzítés mindkét esetben a **Létrehozás** gombbal történik, a **Mégse** gomb pedig üríti az űrlapot és megszakítja a folyamatot.

Űrlap Validáció és Hibakezelés (Regisztráció)

A rendszer a hibás, hiányos vagy nem biztonságos adatrögzítés elkerülése érdekében beépített formai és logikai ellenőrzéseket (validációkat) alkalmaz. Ha az adminisztrátor valamilyen szabályt megsért, a "Létrehozás" gomb megnyomásakor a folyamat megszakad, és a rendszer vizuális hibaüzenetekkel mutatja meg, mit kell javítani.

A képeken látható leggyakoribb validációs hibajelzések és kiváltó okaik:

- **"Kérlek, válassz egy szerepkört!"** (Központi piros figyelmeztetés)
 - *Mikor jelenik meg?* Akkor fordul elő, ha az űrlap beküldésekor a "Szerepkör" legördülő menü az alapértelmezett üres állapoton maradt. Mivel a jogosultsági szint a legfontosabb adat egy fióknál, enélkül a rendszer blokkolja a regisztrációt.

- **"Válasszon ki egy elemet a listából."** (Mezőhöz kötött buborék / Tooltip)
 - *Mikor jelenik meg?* Ez egy közvetlenül a mezőre mutató figyelemfelhívás. Akkor jelentkezik, ha egy kötelezően kitöltendő (piros csillaggal jelölt) specifikus mező, például a diákoknál a "Szemeszter", üresen lett hagyva mentéskor. A felület automatikusan a hiányzó adat mezőjéhez görget.

⚠ A jelszónak legalább 6 karakter hosszúnak kell lennie!

- **"A jelszónak legalább 6 karakter hosszúnak kell lennie!"** (Központi piros figyelmeztetés)
 - *Mikor jelenik meg?* A beépített biztonsági házirend (Security Policy) érvényesítésekor. Ha az adminisztrátor által megadott kezdeti jelszó nem éri el a minimális 6 karakteres hosszt, a rendszer megtagadja a fiók létrehozását a biztonsági kockázatok elkerülése végett.

⚠ A telefonszámnak '+' jellel kell kezdődnie!

- **"A telefonszámnak '+' jellel kell kezdődnie!"** (Központi piros figyelmeztetés)
 - *Mikor jelenik meg?* A lakók esetében a telefonszám megadása opcionális, de ha kitöltik, szigorú formátumi követelményeknek kell megfelelnie a nemzetközi hívhatóság érdekében. Ha a felhasználó a számot nem a + karakterrel kezdi (például "0630..." formátumban gépele be), a rendszer hibát jelez az űrlap alján lévő segítő szövegnek megfelelően.

3.7.5 Szobák

A "Szobák" menüpont egy interaktív és valós idejű műszerfal a kollégiumi férőhelyek menedzselésére. Az adminisztrátorok itt egyetlen felületen ellenőrizhetik az épület telítettségét, a szabad ágyak számát, és azt, hogy melyik diák pontosan melyik szobában lakik.

Az Adattábla Oszlopai

Szobák Állapota
Kapacitás és lakók áttekintése.

Összes Szabad ágyak

Keresés (szoba, lakó)...

SOR/OLDAL: 5

Szoba	Emelet	Kapacitás	Lakók	Telítettség
101	1. Emelet	2 fő	Nagy Eszter, Kovács Krisztina	100% (2/2)
102	1. Emelet	2 fő	Papp Gábor	50% (1/2)
103	1. Emelet	2 fő	Németh Sára, Maros Lili	100% (2/2)
104	1. Emelet	2 fő	-	0% (0/2)
105	1. Emelet	3 fő	Fekete András, Juhász Péter	67% (2/3)

< Előző 1 / 6 Következő >

A fő táblázat minden fontos szobaadatot logikusan, egy sorban jelenít meg:

- **Szoba és Emelet:** A szoba egyedi azonosítója és épületen belüli elhelyezkedése (pl. 101, 1. Emelet).
- **Kapacitás:** A szoba maximális ágyszáma (pl. 2 fő, 3 fő, 4 fő).
- **Lakók:** A szobába jelenleg beosztott diákok teljes nevei. Ha egy szoba teljesen üres, itt egy kötőjel (-) látható.
- **Telítettség (Színkódolt vizualizáció):** A rendszer automatikusan kiszámolja a foglaltságot, és százalékos, illetve törtes formában is megmutatja (pl. 2/3). A gyorsabb átláthatóságot beszédes színkódok segítik:
 - **Zöld (0%):** A szoba teljesen üres, a maximális kapacitás kiadható.
 - **Sárga (pl. 33%, 50%, 67%):** A szoba részben foglalt, de még van benne szabad férőhely új beköltöző számára.
 - **Piros (100%):** A szoba teljesen megtelt.

Gyorsszűrők a kapacitás ellenőrzésére

A táblázat bal felső sarkában két gomb segíti a nézetek közötti gyors váltást:

- **Összes:** Ez az alapértelmezett nézet, amely a teljes intézmény összes szobáját listázza, függetlenül attól, hogy tele vannak-e vagy üresek.

Szobák Állapota				
Kapacitás és lakók áttekintése.				
Összes Szabad ágyak		Keresés (szoba, lakó)...		SOR/OLDAL: 5
Szoba	Emelet	Kapacitás	Lakók	Telítettség
102	1. Emelet	2 fő	Papp Gábor	50% (1/2)
104	1. Emelet	2 fő	-	0% (0/2)
105	1. Emelet	3 fő	Fekete András, Juhász Péter	67% (2/3)
108	1. Emelet	2 fő	-	0% (0/2)
109	1. Emelet	4 fő	Szűcs Emese, Hajdu Melinda	50% (2/4)

- **Szabad ágyak:** Erre a gombra kattintva a rendszer azonnal elrejt a 100%-ig telt (piros) szobákat. Ez a funkció rendkívül hasznos a tanév eleji beköltözéseknél, mert az adminisztrátor egyetlen kattintással listázni tudja azokat a helyeket, ahová még oszthat be diákokat.

Intelligens Kereső (Valós idejű szűrés)

A jobb felső sarokban található Keresés (szoba, lakó)... mező kettős funkciót lát el, és azonnal, Enter lenyomása nélkül szűri a táblázatot:

Összes Szabad ágyak		Kovács Flóra		SOR/OLDAL: 5
Szoba	Emelet	Kapacitás	Lakók	Telítettség
205	2. Emelet	3 fő	Kovács Flóra	33% (1/3)

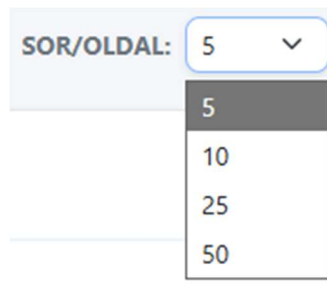
- **Keresés lakó alapján:** Ha az adminisztrátor beírja egy diák nevét (pl. "Kovács Flóra"), a rendszer azonnal megmutatja azt az egyetlen sort (szobát), ahová a diák be van osztva.

Összes Szabad ágyak		205		SOR/OLDAL: 5
Szoba	Emelet	Kapacitás	Lakók	Telítettség
205	2. Emelet	3 fő	Kovács Flóra	33% (1/3)

- **Keresés szobaszám alapján:** Ha egy konkrét szobára kíváncsi (pl. "205"), a szám begépelésével azonnal le tudja kérni az adott szoba adatait és jelenlegi lakóit.

Oldalméret és Lapozás

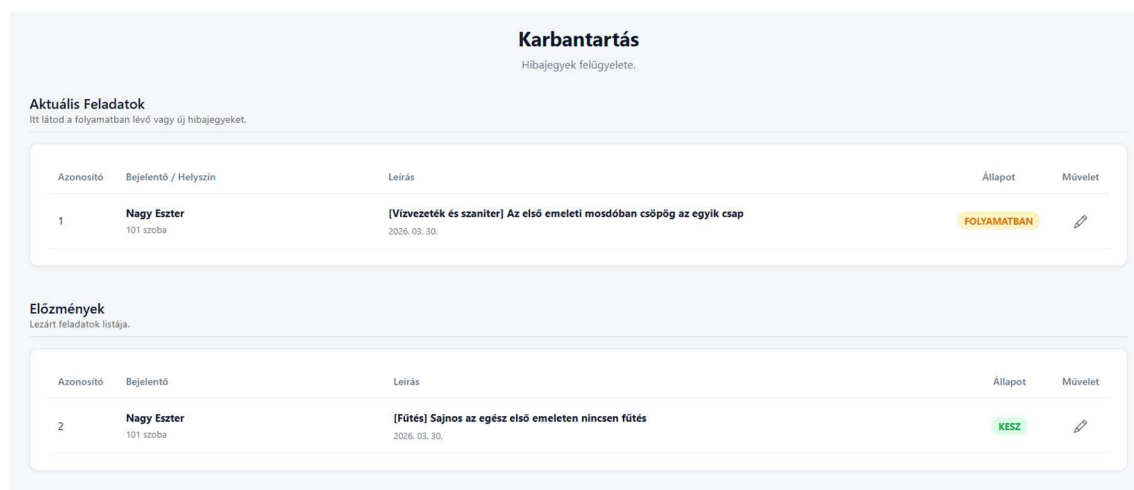
A nagy mennyiségű adat átláthatósága érdekében a táblázat lapozható. A jobb felső sarokban lévő **"SOR/OLDAL"** legördülő menüben a felhasználó saját preferenciája (vagy monitorjának mérete) alapján testreszabhatja, hogy egy oldalon egyszerre 5, 10, 25 vagy 50 szobát szeretne látni. A táblázat alján lévő navigációs gombokkal (Előző / Következő) lehet az oldalak között lépkedni.



3.7.6 Karbantartás

A "Karbantartás" modul szolgál a kollégium lakói által leadott műszaki, üzemeltetési vagy egyéb javítási igények (hibajegyek) központi nyomon követésére és menedzselésére. Az adminisztrátorok és a karbantartók itt kezelhetik a beérkező feladatokat a bejelentéstől egészen a megoldásig.

A Főképernyő és a Feladatlisták



- **Aktuális Feladatok:** Ez a felső táblázat listázza a még megoldásra váró, aktív problémákat. Ide tartoznak a frissen beérkezett, illetve a már folyamatban lévő munkák (például vízvezeték vagy fűtés problémák).
- **Előzmények:** Az alsó táblázat egy archívum. Ide kerülnek át automatikusan a már megoldott, lezárt feladatok, így a korábbi javítások bármikor visszakereshetők maradnak.
- **Adatmezők:** A listákban minden sor egyértelműen mutatja a hibajegy egyedi azonosítóját (ID), a bejelentő nevét és szobaszámát, a probléma leírását,

valamint a jelenlegi **állapotát** egy színkódolt címkével (pl. narancssárga "FOLYAMATBAN", zöld "KESZ").

Hibajegy Állapotának Frissítése

Hibajegy Kezelése (#1)

×

Bejelentő / Helyszín

Nagy Eszter (101 szoba)

Hiba leírása

[Vízvezeték és szaniter] Az első emeleti mosdóban csöpög az egyik csap

Státusz megváltoztatása

FOLYAMATBAN

▼

Törlés

Mégse

Mentés

A feladatok menedzselése a táblázat sorának végén található ceruza ikonra (Művelet) kattintva lehetséges. Ekkor megnyílik a "**Hibajegy Kezelése**" felugró ablak.

Státusz megváltoztatása

FOLYAMATBAN

▼

FOLYAMATBAN

KESZ

UJ

- **Állapotok:** Ebben az ablakban a "Státusz megváltoztatása" legördülő menü segítségével lehet léptetni a feladatot a munkafolyamatban. Három opció választható:
 - **UJ:** Frissen beérkezett, feldolgozatlan igény.
 - **FOLYAMATBAN:** A karbantartó látta a jegyet és már dolgozik a hiba elhárításán.
 - **KESZ:** A javítás megtörtént, a feladat lezárható (ekkor a jegy átkerül az Előzmények közé).

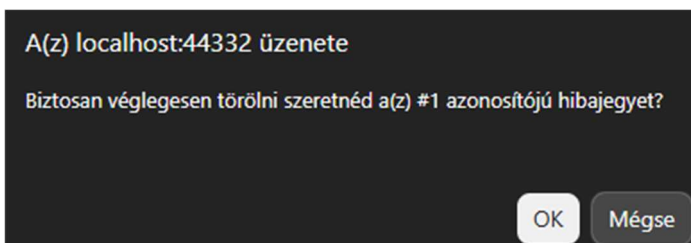
✓ **Siker:** Hibajegy sikeresen frissítve!

- **Mentés:** Az új státusz kiválasztása után a zöld **"Mentés"** gombbal véglegesíthető a folyamat. A rendszer ezt a képernyő tetején egy zöld értesítősávval nyugtázza: *"Siker: Hibajegy sikeresen frissítve!"*.

Hibajegy Végleges Törlése

Amennyiben egy hibajegy tévesen lett leadva, értelmetlen, vagy véletlen duplikáció történt, lehetőség van az eltávolítására.

- **Törlés indítása:** A "Hibajegy Kezelése" ablak bal alsó sarkában található piros **"Törlés"** gombra kell kattintani.



- **Biztonsági megerősítés:** A véletlen törlések elkerülése érdekében a böngésző egy felugró ablakban megerősítést kér, egyértelműen azonosítva a törölni kívánt jegyet (*"Biztosan véglegesen törölni szeretnéd a(z) #1 azonosítójú hibajegyet?"*).

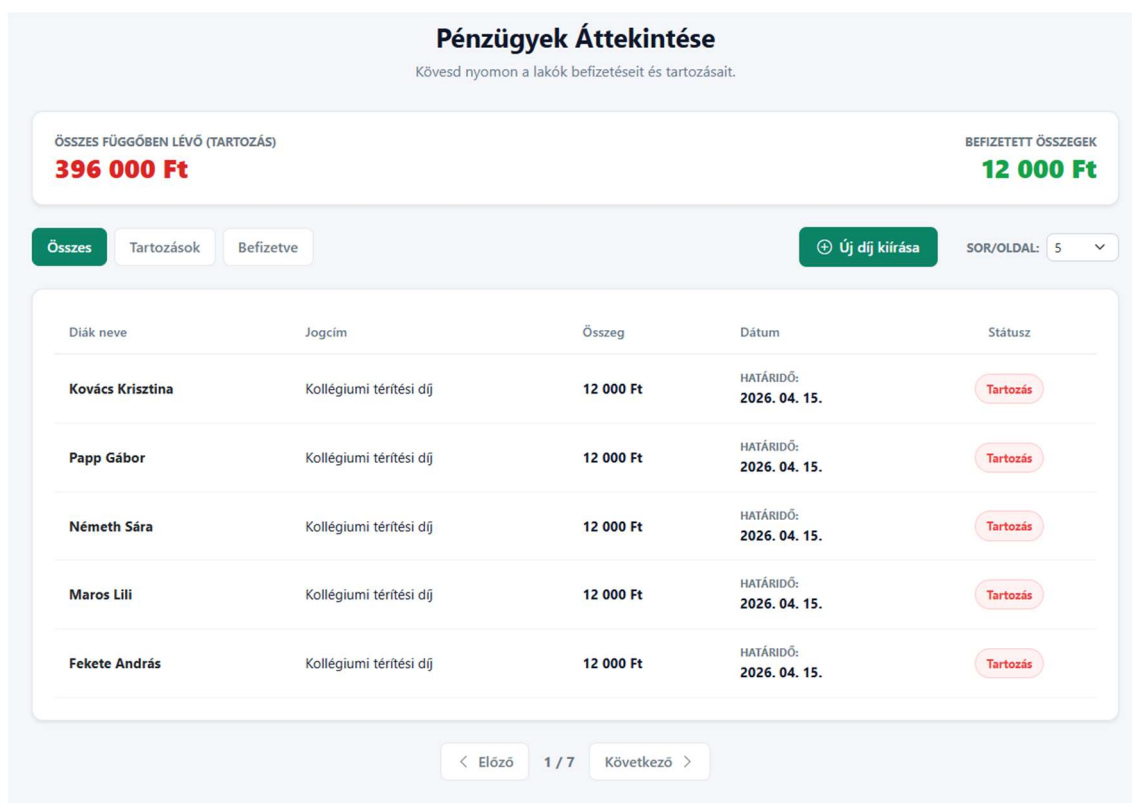
✓ **Siker:** Hibajegy véglegesen törölve!

- **Sikeres törlés:** Az "OK" gomb megnyomása után a jegy véglegesen eltűnik az adatbázisból, amit egy újabb zöld értesítési sáv erősít meg a felületen: *"Siker: Hibajegy véglegesen törölve!"*.

3.7.7 Pénzügyek

A "Pénzügyek" modul egy központi pénzügyi műszerfal, amely lehetővé teszi az adminisztrátorok számára a kollégiumi térítési díjak, büntetések és egyéb fizetési kötelezettségek átlátható nyomon követését. A felület azonnali visszajelzést ad az intézmény kintlévőségeiről és a már sikeresen beszedett összegekről.

Pénzügyi Összesítő Kártyák (Felső sáv)



A képernyő tetején két kiemelt, dinamikus mutató (KPI) ad azonnali képet a kollégium pénzügyi helyzetéről:

- **Összes függőben lévő (Tartozás) – Piros színnel:** Összegzi a diákok által még be nem fizetett, aktív tartozások teljes értékét (a képen 396 000 Ft). A figyelemfelhívó piros szín jelzi a hiányzó bevételeket.
- **Befizetett összegek – Zöld színnel:** Megmutatja az adott időszakban (vagy globálisan) már sikeresen teljesített, lezárt befizetések együttes összegét (a képen 12 000 Ft).

Állapotszűrők (Nézetek közötti váltás)

A táblázat felett bal oldalon három gyorszűrő gomb található, amellyel az adminisztrátor célzottan listázhatja a tranzakciókat:

- **Összes:** A teljes pénzügyi adatbázist mutatja, vegyesen listázva a tartozásokat és a teljesített befizetéseket.

Összes	Tartozások	Befizetve	Új díj kiírása	SOR/OLDAL: 5
Diák neve	Jogcím	Összeg	Dátum	Státusz
Kovács Krisztina	Kollégiumi térítési díj	12 000 Ft	HATÁRIDŐ: 2026. 04. 15.	Tartozás
Papp Gábor	Kollégiumi térítési díj	12 000 Ft	HATÁRIDŐ: 2026. 04. 15.	Tartozás
Németh Sára	Kollégiumi térítési díj	12 000 Ft	HATÁRIDŐ: 2026. 04. 15.	Tartozás
Maros Lili	Kollégiumi térítési díj	12 000 Ft	HATÁRIDŐ: 2026. 04. 15.	Tartozás
Fekete András	Kollégiumi térítési díj	12 000 Ft	HATÁRIDŐ: 2026. 04. 15.	Tartozás

- **Tartozások:** Erre kattintva a rendszer leszűri a listát kizárólag a még nyitott, befizetésre váró tételekre. Ez a nézet elengedhetetlen a késedelmes fizetők ellenőrzéséhez.

Összes	Tartozások	Befizetve	Új díj kiírása	SOR/OLDAL: 5
Diák neve	Jogcím	Összeg	Dátum	Státusz
Nagy Eszter	Kollégiumi térítési díj	12 000 Ft	FIZETVE: 2026. 03. 30.	Fizetve

- **Befizetve:** Csak a már sikeresen lezárt, kifizetett tételeket jeleníti meg.

Az Adattábla Felépítése és Vizuális Jelzések

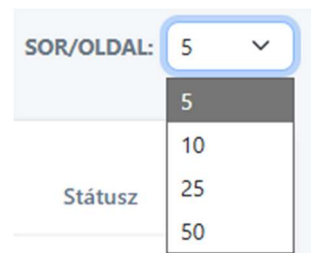
A pénzügyi tételeket listázó táblázat logikusan felépített oszlopokból áll:

- **Diák neve és Jogcím:** Egyértelműsíti, hogy ki és milyen címen tartozik (pl. "Kollégiumi térítési díj").
- **Összeg:** A fizetendő vagy már kifizetett érték (pl. 12 000 Ft).
- **Dátum (Dinamikus adat):** A rendszer intelligensen cseréli az itt megjelenő információt a tétel státuszától függően. Ha a diák még tartozik, a fizetési **HATÁRIDŐT** mutatja. Ha a diák már fizetett, a sikeres tranzakció pontos dátumát (**FIZETVE:**) írja ki.

- **Státusz:** Beszédes színekkel ellátott jelvények segítik a gyors áttekintést (Piros "Tartozás" vagy halványzöld "Fizetve")

Műveletek és Megjelenítés

- **Új díj kiírása gomb:** A jobb felső sarokban található zöld gombbal indítható el a folyamat, amellyel az adminisztrátor új fizetési kötelezettséget (pl. havi díjat) rendelhet a lakókhoz.
- **Oldalméret és Lapozás:** Ahogy a többi listanézetnél, itt is testreszabható a képernyőn egyszerre megjelenő sorok száma a "SOR/OLDAL" legördülő menüvel (5, 10, 25 vagy 50 elem/oldal), a táblázat alatti gombokkal pedig a többoldalas listákban lehet navigálni.



Tömeges Díjkiírás és a Diákok Pénzügyi Felülete

A KoliPortál rendszer lehetőséget biztosít arra, hogy az adminisztrátorok egyetlen kattintással, tömegesen írjanak ki fizetési kötelezettségeket (pl. havi kollégiumi díjat) az összes jelenleg aktív, bent lakó diák számára. Ezt az "Új díj kiírása" gombbal lehet elindítani, amely egy dedikált űrlapot nyit meg.

A Díjkiíró Űrlap Felépítése

Az űrlapon három egyszerű, de kritikus adatot kell megadni a tartozás generálásához:

Jogcím (Fizetés típusa)

Kollégiumi térítési díj

Féléves DÖK/Kulturális Hozzájárulás

Féléves Hálózatüzemeltetés

Kollégiumi térítési díj

- **Jogcím (Fizetés típusa):** Egy legördülő menüből választható ki a kötelezettség kategóriája. A rendszer előre definiált opciókat kínál fel a standardizált könyvelés érdekében (pl. *Kollégiumi térítési díj*, *Féléves DÖK/Kulturális Hozzájárulás*, *Féléves Hálózatüzemeltetés*).
- **Fizetendő Összeg (Ft):** Egy numerikus mező, ahová a kötelezően befizetendő pontos összeget kell beírni (pl. 12 000).

Befizetési határidő vége

2026. 04. 14. 📅

2026. április ▾
↑ ↓

H	K	Sze	Cs	P	Szo	V
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

Törlés
Ma

Díjak kiírása aktív lakóknak

12 000 Ft

< Előző
1 / 7
Következő >

- **Befizetési határidő vége:** Egy interaktív, beépített naptár (Datepicker) segítségével vizuálisan és pontosan kiválasztható a fizetés utolsó megengedett napja.

Adatvalidáció és Hibakezelés

⚠️ A fizetendő összeg nem kevesebb mint 1000 Ft!

- **"A fizetendő összeg nem kevesebb mint 1000 Ft!"** (Piros figyelmeztetés) A rendszer beépített védelmet tartalmaz az elgépelések (tipográfiai hibák) ellen. Ha az adminisztrátor véletlenül irreálisan alacsony összeget ad meg (például 12000 helyett csak 120-at ír be), a rendszer blokkolja a kiírást, és ezzel a hibaüzenettel kéri a javítást.

Sikeres Végrehajtás

- Miután az adatok helyesek, a **"Díjak kiírása aktív lakóknak"** zöld gomb megnyomásával indul el a folyamat.
- A rendszer ekkor a háttérben végigiterál a felhasználói adatbázison, megkeresi az összes olyan diákot, akinek *jelenleg van aktív szobabeosztása*, és mindegyikük profiljához generál egy új, egyedi tartozást.

✅ **Siker:** A Kollégiumi térítési díj 12 000 Ft sikeresen kiírásra került 33 lakó számára!

- A sikeres lefutást egy zöld értesítési sáv nyugtázza a képernyő tetején: *"Siker: A Kollégiumi térítési díj 12 000 Ft sikeresen kiírásra került 33 lakó számára!"*. Ez az üzenet pontos visszajelzést ad az érintett diákok számáról is.

A Diák Nézete (A kiírt díj megjelenése)

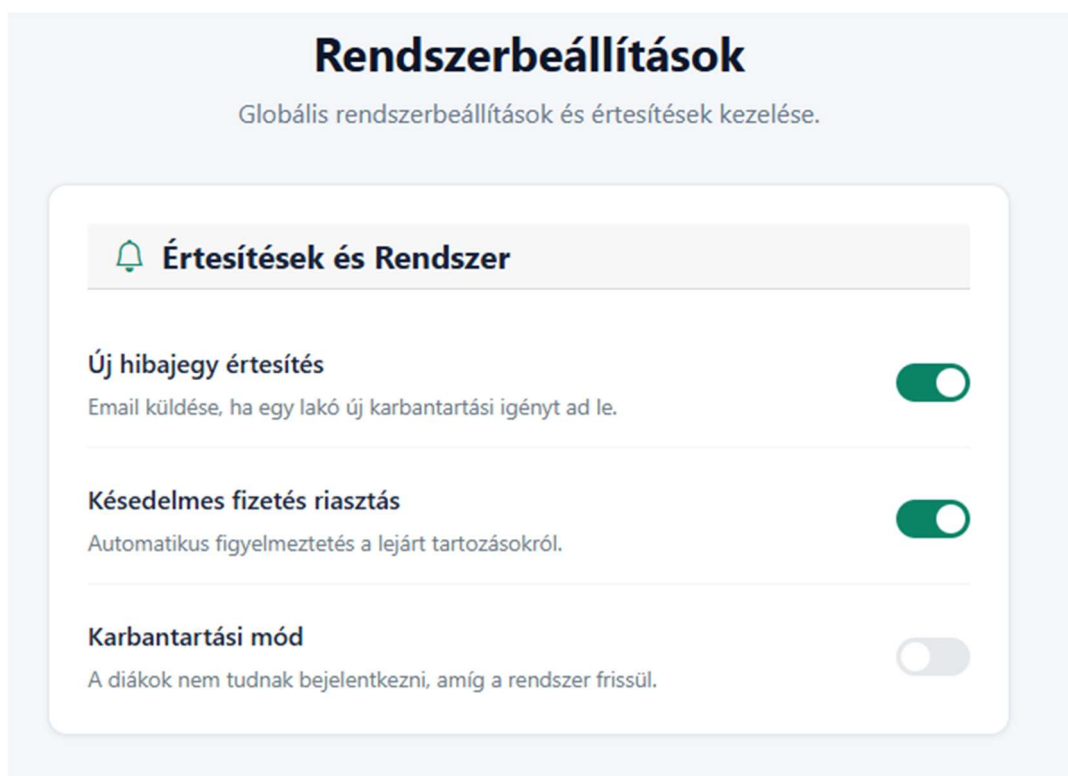
Határidő / Fizetve	Jogcím	Összeg	Státusz	Művelet
Határidő: 2026. 06. 15.	Kollégiumi térítési díj	12 000 Ft	Tartozás	
Fizetve: 2026. 03. 30.	Kollégiumi térítési díj	12 000 Ft	Fizetve	

Abban a pillanatban, hogy az adminisztrátor sikeresen kiírta a díjat, az valós időben megjelenik a diákok saját, személyes felületén.

- A diák a bejelentkezés után látni fogja a friss tételt, amely **"Tartozás"** (piros) státusszal és a megadott határidővel szerepel a listájában.
- A tartozás mellett egy feltűnő, kék színű **"Fizetés"** gomb jelenik meg. Erre kattintva a hallgató elindíthatja a tartozás kiegyenlítését (bankkártyás fizetési folyamatát).
- Ugyanezen a felületen a diák transzparensen látja a korábbi, már sikeresen teljesített tranzakcióit is, amelyek zöld **"Fizetve"** jelvénnel és egy zöld pipával szerepelnek.

3.7.8 Beállítások

A felületen látható "Rendszerbeállítások" kártya a KoliPortál egy tervezett, globális konfigurációs központját ábrázolja. A jövőbeli fejlesztés célja, hogy az adminisztrátorok programozói beavatkozás nélkül, egy egyszerű, vizuális felületen (kapcsolók segítségével) tudják szabályozni a rendszer háttérfolyamatait és automatizmusait.



A felületen szereplő tervezett funkciók:

- **Új hibajegy értesítés:** Egy külső e-mail küldő szolgáltatás beépítése a rendszerbe. Ha a kapcsoló aktív, a rendszer a háttérben azonnali automatikus e-mailt generál a karbantartóknak, amint egy diák új műszaki hibát jelent be, jelentősen gyorsítva ezzel a reakcióidőt.
- **Késedelmes fizetés riasztás:** Egy ütemezett, szerveroldali automatizáció (háttérfolyamat). Bekapcsolt állapotban a rendszer napi szinten ellenőrzi a pénzügyi modul adatait, és emberi beavatkozás nélkül küld figyelmeztetést (e-mailt vagy rendszerüzenetet) azoknak a lakóknak, akik túllépték a befizetési határidőt.
- **Karbantartási mód (Biztonsági rendszerzár):** Egy globális "vészsleállító" kapcsoló a hálózati rétegben. Aktiválása esetén a bejelentkezési felület megváltoztatja a viselkedését: a diákok számára letiltja a belépést (egy tájékoztató üzenet kíséretében), és kizárólag az adminisztrátori jogosultsággal rendelkező fiókokat engedi be, amíg a szoftverfrissítési vagy adatbázis-karbantartási munkálatok zajlanak.

4 ÖSSZEFOGLALÁS

4.1 Elkészült projektünk értékelése

A KoliPortál projekt fejlesztése egy rendkívül izgalmas és összetett folyamat volt, amelynek végéhez érve érdemes visszatekinteni a megtett útra. Ha saját megítélésünk szerint kellene értékelni a munkánkat, kijelenthetjük: az első fejezetben megfogalmazott szakmai és funkcionális célokat sikeresen megvalósítottuk. Egy olyan modern, átlátható és felhasználóbarát kollégiumi menedzsment rendszert alkottunk meg, amely képes valós problémákra (pl. szobabeosztások káosza, pénzügyi kintlévőségek nyomon követése, hibajegyek kezelése) hatékony, digitális választ adni.

Kész projektként tekintve a szoftverre, büszkék vagyunk az elért eredményre, ugyanakkor az út idáig nem volt akadálymentes.

4.1.1 Kihívások és tanulságok

A fejlesztés során a legnagyobb kihívást a komplex adatbázis-architektúra és a relációs kapcsolatok biztonságos kezelése jelentette. Számos alkalommal szembesültünk azzal, hogy egy-egy egyszerűnek tűnő művelet – mint például egy felhasználó végleges törlése – a háttérben komoly idegenkulcs-ütközéseket és kaszkádolt adatbázis-problémákat vethet fel. Szintén komoly feladatot jelentett a frontend (Blazor) és a backend (API) közötti aszinkron kommunikáció hibatűrővé tétele.

Ezekből a nehézségekből tanultuk azonban a legtöbbet. Elmélyítettük a tudásunkat a C# keretrendszerekben, megértettük a precíz kivételkezelés (try-catch blokkok) fontosságát, és megtanultuk, hogy a szoftverfejlesztés nem csupán a funkciók megírásáról szól, hanem arról is, hogy a rendszer a hibás állapotokat is elegánsan, a felhasználó számára érthetően kommunikálja.

4.1.2 Erős alapok és fejlesztési lehetőségek

Tisztában vagyunk vele, hogy egy szoftver sosem tekinthető 100%-ig "késznek", és a KoliPortálban is van még fejlesztési lehetőség bőven. Ahogy a dokumentációban is felvázoltuk, a jövőbeli tervek között szerepel a "Rendszerbeállítások" modul élesítése, az automatizált e-mail értesítések bevezetése, vagy akár a felület teljes mobilalkalmazássá alakítása. Ennek ellenére az elsődleges célunkat elértük: **egy technológiailag nagyon erős, biztonságos és kiválóan skálázható alapot állítottunk fel.** A kód felépítése és az adatbázis szerkezete is olyan, hogy a jövőben bármilyen új modul (pl. raktárkészlet-kezelő, portaszolgálati modul) könnyedén és fájdalommentesen ráépíthető legyen.

4.1.3 A projekt utóélete

A programunk utóéletét tekintve több irányt is elképzelhetőnek tartunk. Elsősorban ez a projekt egy rendkívül erős referenciamunka, amely jól demonstrálja a modern webfejlesztési és adatbázis-tervezési képességeinket. Távlati céljaink között szerepel, hogy a rendszert egy kisebb létszámú, valós kollégiumi vagy diákszállói környezetben is teszteljük (pilot projekt). Látni a saját kódunkat "élesben" működni, és tapasztalni, ahogy valódi adminisztrátorok mindennapi munkáját könnyíti meg – ez lenne a KoliPortál fejlesztésének legszebb és legmértőbb folytatása.

4.2 Köszönetnyilvánítás

Ezúton szeretnénk kifejezni őszinte hálánkat és nagyrabecsülésünket mindazoknak, akik szakmai tudásukkal, iránymutatásukkal és töretlen támogatásukkal hozzájárultak vizsgaremekünk sikeres elkészítéséhez és tanulmányaink eredményes befejezéséhez.

Külön köszönettel tartozunk **Méhes József** tanár úrnak. Az ő átfogó szakmai felkészültsége és inspiráló pedagógiai hozzáállása nélkülözhetetlen alapot biztosított számunkra. Hálásak vagyunk a komplex adatbázis-szerkezetek tervezésének, valamint az alkalmazásprogramozási interfészek (API-k) különféle formáinak alapos és gyakorlatias átadásáért. Folyamatos, építő jellegű és rendkívül részletes visszajelzései nemcsak a projektünk minőségét emelték, hanem a rendszerszintű szakmai látásmódunkat is nagymértékben formálták.

Kiemelt hálával gondolunk **Csukárdi Rajmund** tanár úrra, aki a webprogramozás mélyebb összefüggéseibe avatott be minket, és ezen a téren tudásának hatalmas részét adta át a tanév során. Szakmai iránymutatása mellett emberséges, közvetlen és támogató személyisége mindvégig kiemelkedő motivációt jelentett számunkra. Az általa teremtett nyitott és inspiráló légkör erőt adott a mindennapi kihívások leküzdéséhez és a folyamatos fejlődéshez.

Őszinte tisztelettel és köszönettel adózunk **Szalai Edit** tanárnőnek, akinek végtelen türelme, útmutatása és értékes tanácsai végigkísérték munkánkat. Köszönjük azt a rendkívüli mennyiségű és elmélyült tudást, amelyet a konzolos alkalmazások és a WPF technológia terén megosztott velünk. Külön hálásak vagyunk azért a kiemelkedő és áldozatos vizsgafelkészítő munkáért, amellyel felvértezett minket a vizsga során felmerülő akadályok és váratlan helyzetek magabiztos, sikeres leküzdésére.

Megtiszteltetés és kiváltság volt számunkra ilyen elhivatott, nagy tudású és hallgatóközpontú pedagógusoktól tanulni. Az általuk átadott elméleti és gyakorlati ismeretek, az odaadásuk, valamint a belénk vetett bizalmuk nélkül ez a munka nem jöhetett volna létre. Köszönjük, hogy iránymutatásukkal és példamutatásukkal elindítottak minket ezen a szakmai úton.

5 IRODALOMJEGYZÉK

- Záródolgozat témák, szempontok, ötletek:
<http://infojegyzet.hu/webszerkesztes/zarodolgozat/>

5.1 Szakmai Fórumok és Problémamegoldó Közösségek (StackOverflow)

- A felhasználói jogosultságok frontend-oldali dekódolásának megvalósításához:
<https://stackoverflow.com/questions/60302496/how-to-parse-jwt-token-in-blazor-webassembly>
- Az adatbázis-törlési anomáliák és az idegenkulcs-ütközések [HTTP 500] hibakeresésekor felhasznált szakmai diskurzus:
<https://stackoverflow.com/questions/54243026/the-delete-statement-conflicted-with-the-reference-constraint-entity-framework>
- A felhasználói felület aszinkron frissítésének és a StateHasChanged metódus helyes hívásának elsajátításához:
<https://stackoverflow.com/questions/66380632/blazor-component-not-updating-after-data-change>
- A natív böngészős funkciók, például a confirm ablakok és a LocalStorage C# kódból történő meghívásához:
<https://blazor-university.com/javascript-interop/calling-javascript-from-dotnet/>

5.2 Frontend, UI Design és Webes technológiák (W3Schools & Bootstrap)

- A reszponzív beviteli mezők és az űrlapellenőrzések kialakításához:
<https://getbootstrap.com/docs/5.3/forms/validation/>
- A felugró szerkesztő- és fizetési ablakok felépítéséhez:
<https://getbootstrap.com/docs/5.3/components/modal/>
- A modern, eszközfüggetlen képernyőelrendezés megalkotásához:
https://www.w3schools.com/css/css3_flexbox.asp
- A bejelentkezési tokenek kliensoldali tárolásának megértéséhez:
https://www.w3schools.com/jsref/prop_win_localstorage.asp

5.3 Hivatalos Microsoft Dokumentációk

- ASP.NET Core Blazor adatfolyam (Data binding) és komponensek:
<https://learn.microsoft.com/hu-hu/aspnet/core/blazor/components/data-binding?view=aspnetcore-8.0>
- ASP.NET Core Blazor hitelesítés és engedélyezés (Authentication & Authorization):
<https://learn.microsoft.com/hu-hu/aspnet/core/blazor/security/?view=aspnetcore-8.0>
- Entity Framework Core – Adatbázis-migrációk kezelése:
<https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/?tabs=dotnet-core-cli>
- Kivételkezelés (Exception Handling) a C# nyelvben:
<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/>
- A HttpClient Factory használata és HTTP kérések küldése:
<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/http-requests?view=aspnetcore-8.0>

5.4 C# Backend és ASP.NET Core Web API

- ASP.NET Core Web API Features You Need to Know
<https://www.youtube.com/watch?v=s1bk-68aB1U>
- ASP.NET Web API .NET 8 Tutorial:
<https://www.youtube.com/playlist?list=PL82C6-04XrHfrGOCPmKmwTO7M0avXyQKc>

5.5 Adatbázis-kezelés: Entity Framework Core

- Getting Started with Entity Framework Core for Beginners:
<https://www.youtube.com/watch?v=SryQxUeChMc>
- Getting Started with Entity Framework Core in .NET:
<https://www.youtube.com/watch?v=2t88FOeQ898>

5.6 Frontend Logika és Blazor WebAssembly/Server

- **Blazor Fundamentals Tutorial** – *Learn Blazor Step-by-Step*
<https://www.youtube.com/watch?v=CpbRAWgFBRQ>
- **CuriousDrive: Blazor WebAssembly Tutorials:**
https://www.youtube.com/playlist?list=PL4WEkbdagHIRjjBJvK_TSfddJSvEEA_tnt

5.7 Blazor tesztelés (bUnit, xUnit, Moq)

- Getting started with Mocking using Moq in .NET (Core, Framework, Standard):
https://www.youtube.com/watch?v=9ZvDBSQa_so
- How to Write Blazor Unit Tests With bUnit and xUnit:
<https://www.youtube.com/watch?v=EXFHM5wonlM>
- C# Unit Test Mocking with Moq:
<https://www.youtube.com/watch?v=IFN4-YrgBEI>